

TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG
ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÀI GIẢNG HỌC PHẦN LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Nhóm tác giả:

Trịnh Tuấn Đạt
Nguyễn Mạnh Tuấn
Trịnh Thành Trung
Trần Nhật Hóa
Nguyễn Tuấn Dũng
Nguyễn Tiến Thành
Lê Đức Hậu

Lời nói đầu

Sau nhiều năm tham gia giảng dạy môn Lập trình hướng đối tượng, chúng tôi đã tích lũy được những kinh nghiệm quý báu từ thực tiễn. Đồng thời, với sự đóng góp ý kiến từ các đồng nghiệp và người học, chúng tôi nhận thấy nhu cầu cấp thiết về một tài liệu học thuật toàn diện, có thể hỗ trợ hiệu quả cho cả giảng viên và sinh viên trong quá trình giảng dạy và học tập. Từ đó, chúng tôi tiến hành biên soạn cuốn sách này với mục tiêu cung cấp một nguồn tài liệu chất lượng, giúp nâng cao hiệu quả học tập và giảng dạy môn Lập trình hướng đối tượng.

Cuốn sách được xây dựng dựa trên nền tảng các slide bài giảng môn Lập trình hướng đối tượng, được sử dụng rộng rãi trong các khóa học. Tuy nhiên, để đáp ứng nhu cầu tìm hiểu sâu hơn và toàn diện hơn, chúng tôi đã bổ sung và mở rộng các nội dung, bao gồm các ví dụ minh họa, bài tập thực hành và giải thích chi tiết. Mục tiêu là giúp người đọc không chỉ nắm vững các khái niệm cơ bản mà còn hiểu sâu sắc hơn về cách áp dụng chúng trong thực tế, từ đó xây dựng nền tảng vững chắc để phát triển các ứng dụng phần mềm phức tạp.

Chúng tôi hy vọng rằng cuốn sách này sẽ trở thành một tài liệu hữu ích, đồng hành cùng các giảng viên, sinh viên và những ai đam mê tìm hiểu về lập trình hướng đối tượng. Mọi ý kiến đóng góp để hoàn thiện cuốn sách đều được trân trọng đón nhận.

Hà Nội, ngày 28 tháng 08 năm 2025

Trịnh Tuấn Đạt

Mục lục

Lời nói đầu	3
Mục lục	4
Chương 1. Tổng quan về Lập trình hướng đối tượng	11
1.1. Sơ lược về kỹ thuật lập trình	11
1.2. Kỹ thuật hướng đối tượng	14
1.3. Các khái niệm	15
1.3.1. Phương thức và thuộc tính	15
1.3.2. Lớp và thể hiện	16
1.4. Các nguyên lý trong lập trình hướng đối tượng	17
1.5. Phân tích và thiết kế hệ thống hướng đối tượng	19
Chương 2. Cú pháp Java cơ bản	21
2.1. Các khái niệm cơ bản	21
2.2. Biến	23
2.3. Các kiểu dữ liệu trong Java	24
2.4. Toán tử (Operators)	30
2.5. Cấu trúc điều khiển	32
2.6. Mảng	38
2.7. Giới thiệu UML	39
2.7.1. Khái niệm UML	40
2.7.2. Các biểu đồ UML cơ bản	41
Chương 3. Trừu tượng hóa và đóng gói	46
3.1. Trừu tượng hóa	46
3.1.1. Trừu tượng hóa điều khiển	47
3.1.2. Trừu tượng hóa dữ liệu	47

3.2. Lớp và Đối tượng	47
3.2.1. Khái niệm lớp	47
3.2.2. So sánh lớp và đối tượng	48
3.3. Đóng gói và Che giấu Dữ liệu	48
3.3.1. Đóng gói	48
3.3.2. Che giấu dữ liệu	50
3.3.3. Phân biệt trừu tượng hóa và đóng gói	50
3.4. Quản lý Lớp với Package	50
3.5. Xây dựng Lớp trong Java	53
3.6. Chỉ định Truy cập cho Thành viên của Lớp	57
3.6.1. Bảng So sánh Khả năng Truy cập	58
3.6.2. Ví dụ Minh họa	58
3.6.3. Lợi ích và Lưu ý	60
3.7. Phương thức getter và setter	61
3.8. Biểu diễn Đối tượng, Lớp, Gói trong UML	62
3.9. Kết luận	63
3.10. Bài tập	63
3.11. Lời giải bài tập chương 3	63

Chương 4. Khởi Tạo và Sử Dụng Đối Tượng 65

4.1. Phương Thức Khởi Tạo (Constructor)	65
4.2. Các Loại Phương Thức Khởi Tạo	67
4.2.1. Phương Thức Khởi Tạo Mặc Định (Default Constructor)	67
4.2.2. Phương Thức Khởi Tạo Có Tham Số (Parameterized Constructor)	68
4.2.3. Nạp Chồng Constructor	70
4.3. Khai Báo và Khởi Tạo Đối Tượng	71
4.3.1. Khai Báo Đối Tượng	71
4.3.2. Khởi Tạo Đối Tượng	71
4.3.3. Kết Hợp Khai Báo và Khởi Tạo	72
4.3.4. Mảng Đối Tượng	72
4.3.5. Quản Lý Đối Tượng	72

4.4. Sử Dụng Đối Tượng	73
4.5. Quản Lý Bộ Nhớ và So Sánh Đối Tượng	76
4.6. Hủy Bỏ Đối Tượng	78
4.7. Ví Dụ và Bài Tập	81
Chương 5. Các kỹ thuật xây dựng lớp và sử dụng đối tượng	83
5.1. Chồng phương thức	83
5.1.1. Định nghĩa chồng phương thức	84
5.1.2. Chồng phương thức khởi tạo	88
5.1.3. Từ khoá this	89
5.2. Thành viên đối tượng và thành viên lớp	91
5.2.1. Thành viên static	91
5.2.2. Thành viên hằng	96
5.3. Truyền tham số cho phương thức	98
5.3.1. Với kiểu dữ liệu tham trị	99
5.3.2. Với kiểu dữ liệu tham chiếu	100
5.3.3. Truyền số lượng tham số tùy ý	104
5.4. Một số lớp tiện ích trong Java	106
5.4.1. Package trong Java	106
5.4.2. Các gói trong Java	108
5.4.3. Các lớp bao - Wrapper class	109
5.4.4. Xâu	113
5.4.5. StringBuffer	119
5.4.6. Math	123
5.4.7. Lớp System	124
5.5. Ví dụ và bài tập	127
Chương 6. Kết tập và kế thừa	128
6.1. Tái sử dụng mã nguồn	128
6.2. Kết tập (Aggregation)	131
6.2.1. Bản chất của kết tập	131
6.2.2. Biểu diễn kết tập bằng UML	132

6.2.3.	Minh họa kết tập trên Java	136
6.2.4.	Thứ tự khởi tạo trong kết tập	139
6.3.	Kế thừa (Inheritance)	140
6.3.1.	Tổng quan về kế thừa	140
6.3.2.	Nguyên lý kế thừa	146
6.3.3.	Cú pháp kế thừa trong Java	148
6.3.4.	Khởi tạo và hủy bỏ đối tượng trong kế thừa	151
6.4.	Ví dụ và bài tập	155
Chương 7. Một số kỹ thuật trong kế thừa		157
7.1.	Ghi đè (Overriding)	157
7.2.	Lớp trừu tượng (abstract class)	163
7.3.	Giao diện (interface)	166
Chương 8. Đa hình		171
8.1.	Upcasting và Downcasting	171
8.1.1.	Chuyển đổi kiểu dữ liệu	171
8.1.2.	Upcasting	174
8.1.3.	Downcasting	178
8.2.	Liên kết tĩnh và Liên kết động	180
8.2.1.	Liên kết lời gọi hàm	180
8.2.2.	Liên kết tĩnh	181
8.2.3.	Liên kết động	183
8.3.	Đa hình (Polymorphism)	184
8.4.	Ví dụ và bài tập	187
Chương 9. Lập trình tổng quát		191
9.1.	Giới thiệu về lập trình tổng quát	191
9.2.	Định nghĩa và sử dụng Template	193
9.2.1.	Lớp tổng quát	193
9.2.2.	Phương thức tổng quát	194
9.2.3.	Giới hạn kiểu dữ liệu tổng quát	195
9.2.4.	Lập trình tổng quát trong Java collections framework	196

9.2.5. Ký tự đại diện (Wildcard)	209
9.2.6. Bài tập và Lời giải	211
Chương 10. Ngoại lệ và xử lý ngoại lệ	218
10.1. Ngoại lệ	218
10.2. Bắt và xử lý ngoại lệ	219
10.2.1. Mục đích của xử lý ngoại lệ	219
10.2.2. Mô hình xử lý ngoại lệ	219
10.2.3. Xử lý ngoại lệ trong Java	220
10.3. Ủy nhiệm ngoại lệ	229
10.4. Tạo ngoại lệ tự định nghĩa	233
10.5. Tổng kết	235
10.6. Bài tập và Lời giải	235
Chương 11. Lập trình giao diện GUI với JavaFX	242
11.1. Giới thiệu về JavaFX	242
11.1.1. Tổng quan về JavaFX và lập trình GUI	242
11.1.2. Lịch sử phát triển JavaFX	243
11.1.3. Các tính năng nổi bật của JavaFX	243
11.1.4. So sánh JavaFX với các thư viện AWT và Swing	244
11.1.5. Lợi ích khi sử dụng JavaFX trong phát triển GUI	245
11.2. Cài đặt JavaFX	245
11.2.1. Giới thiệu về cài đặt JavaFX	245
11.2.2. Cấu hình JavaFX trong Eclipse	245
11.2.3. Cài đặt JavaFX Scene Builder	246
11.2.4. Cấu hình JavaFX trong các IDE khác	247
11.2.5. Kiểm tra cài đặt với ứng dụng Hello World	247
11.3. Các thành phần giao diện JavaFX	248
11.3.1. Stage	248
11.3.2. Scene	250
11.3.3. Node	251
11.3.4. Scene Graph	252

11.3.5. Cách tạo ứng dụng JavaFX	252
11.3.6. Vòng đời ứng dụng JavaFX	253
11.3.7. Cách xây dựng Scene Graph	255
11.4. UI Controls trong JavaFX	256
11.4.1. Button	257
11.4.2. Label	258
11.4.3. TextField	258
11.5. Layout Panes trong JavaFX	260
11.5.1. HBox	260
11.5.2. VBox	261
11.5.3. BorderPane	262
11.5.4. GridPane	264
11.5.5. FlowPane	265
11.5.6. StackPane	267
11.6. Mô hình xử lý sự kiện trong JavaFX	268
11.6.1. Các loại sự kiện trong JavaFX	268
11.6.2. Xử lý sự kiện	269
11.6.3. Các giai đoạn xử lý sự kiện trong JavaFX	269
11.6.4. Trình xử lý và bộ lọc sự kiện	270
11.6.5. Tạo và Đăng ký Trình Xử Lý Sự Kiện	270
11.6.6. Đăng ký Trình Xử Lý Sự Kiện Thông Qua Thuộc Tính	272
11.6.7. Event Filters trong JavaFX	273
11.6.8. Event Handlers trong JavaFX	277
11.7. Kéo thả giao diện với SceneBuilder	282
11.7.1. Giao diện chính của SceneBuilder	282
11.7.2. Thiết kế giao diện bằng cách kéo-thả	282
11.7.3. Sử dụng tệp FXML trong JavaFX	283
11.7.4. Ưu điểm của việc sử dụng SceneBuilder	286
11.8. Bài tập	286
11.9. Lời giải bài tập chương 11	286

12.1. Phân tích thiết kế hướng đối tượng	288
12.1.1. Phân tích hướng đối tượng (Object-Oriented Analysis – OOA):	289
12.1.2. Thiết kế hướng đối tượng (Object-Oriented Design – OOD) .	292
12.2. Biểu đồ lớp (Class diagram)	296
12.3. Quan hệ giữa các lớp	299
12.4. Bài tập	306
Tài liệu tham khảo	309

Chương 1

Tổng quan về Lập trình hướng đối tượng

1.1. Sơ lược về kỹ thuật lập trình	11
1.2. Kỹ thuật hướng đối tượng	14
1.3. Các khái niệm	15
1.4. Các nguyên lý trong lập trình hướng đối tượng	17
1.5. Phân tích và thiết kế hệ thống hướng đối tượng	19

Nội dung

- Tổng quan về kỹ thuật lập trình, sự phát triển của các ngôn ngữ lập trình
- Tư tưởng hướng đối tượng và các khái niệm cơ bản
- Các nguyên lý cơ bản của lập trình hướng đối tượng
- Giới thiệu về phân tích và thiết kế hướng đối tượng và UML

1.1. Sơ lược về kỹ thuật lập trình

Lập trình hướng đối tượng (LTHĐT), tiếng Anh là Object-oriented Programming (viết tắt là OOP) được coi là một kỹ thuật lập trình. Do đó, để hiểu về LTHĐT, trước hết ta cần biết kỹ thuật lập trình là gì.

Kỹ thuật lập trình là: kỹ thuật thực thi một giải pháp phần mềm (cấu trúc dữ liệu + giải thuật) dựa trên nền tảng một phương pháp luận (methodology) và một hoặc nhiều ngôn ngữ lập trình phù hợp với yêu cầu đặc thù của ứng dụng.

Thứ nhất, chúng ta cần lưu ý đến khái niệm *phương pháp luận*. Phương pháp luận bao gồm các mô thức lập trình (programming paradigms), các ý tưởng, các thuật toán (algorithms) để giải quyết một vấn đề. Phương pháp luận cũng bao gồm các phong cách trình bày trong lập trình (programming style), bao gồm việc đặt tên biến, thụt dòng, cách lề, cách đoạn, và nó cũng bao gồm cả những cái gọi là văn hóa

lập trình (programming culture). Mỗi một ngôn ngữ lập trình khác nhau lại có một văn hóa lập trình khác nhau. Ví dụ, những người dùng C++ thường viết hết dòng rồi mới xuống dòng và mở ngoặc nhọn, còn những người dùng Java thì mở ngoặc nhọn trên cùng một dòng với phương thức. Tuy rằng nếu không áp dụng các nguyên tắc này, thực ra vẫn dịch tốt do các trình dịch (compiler) vẫn có thể biên dịch các đoạn mã đó thành mã nguồn, nhưng chúng ta vẫn nên tuân thủ các nguyên tắc này nhằm đảm bảo mã nguồn dễ đọc, dễ bảo trì, sửa lỗi. Như vậy, lưu ý là học về kỹ thuật lập trình là học về các nguyên tắc, các quy ước, các lý thuyết trong lập trình, không phải là học cách sử dụng một phần mềm hay cách ứng dụng một công nghệ mới trong lập trình.

Thứ hai, khái niệm chúng ta cần quan tâm đến là *ngôn ngữ lập trình*, là ngôn ngữ để chúng ta áp dụng các kỹ thuật lập trình ở trên để có thể lập trình. Ngôn ngữ lập trình định nghĩa các mô thức hay các nguyên tắc chung cơ bản để có thể lập trình được. Nó bao gồm các cú pháp (syntax) xác định cái gì là hợp lệ trong mã nguồn, có thể coi là các từ ngữ. Ngôn ngữ lập trình cũng bao gồm các ngữ nghĩa (semantics), có thể coi như ngữ pháp của các ngôn ngữ lập trình.

Ngôn ngữ lập trình, bản chất cũng là ngôn ngữ, là một phương tiện để giao tiếp, như tiếng Nhật, tiếng Pháp, hay tiếng Việt... Chẳng hạn như tiếng Nhật là phương tiện để một người Việt Nam có thể giao tiếp được với người Nhật. Ngôn ngữ cũng có thể là một hệ thống ký hiệu để diễn đạt, trao đổi với nhau, ví dụ như từ thời tiền sử, ta có thể thấy các ký hiệu trên vách đá, cũng có thể coi như là một loại ngôn ngữ. Thậm chí, chó sủa gâu gâu hay mèo kêu meo meo cũng là dạng ngôn ngữ (chỉ là loài người chưa hiểu được).

Ngôn ngữ lập trình là một loại ngôn ngữ để giao tiếp giữa người với máy tính. Con người sử dụng ngôn ngữ của con người, hay còn gọi là ngôn ngữ tự nhiên, không được chuẩn hóa và có rất nhiều bất quy tắc. Do đó, rõ ràng là ngôn ngữ của con người không phù hợp để trao đổi với máy tính, vốn cần được các chỉ thị rõ ràng (Tuy nhiên điều này đang dần thay đổi nhờ sự phát triển mạnh mẽ của Trí tuệ nhân tạo Tạo sinh như ChatGPT, Gemini...). Máy tính, ngược lại, sử dụng ngôn ngữ máy, chỉ bao gồm các ký tự 0 và 1.

Ngày xưa, thời các giảng viên Công nghệ thông tin "đời đầu" của Đại học Bách khoa Hà Nội, các thầy đã phải lập trình sử dụng ngôn ngữ máy để giao tiếp với máy. Máy chỉ hiểu 1 và 0. Để lập trình, người ta đục lỗ trên giấy: cái nào đục lỗ là 1, không đục là 0. Nếu đục sai thì cắt một mảnh giấy dán (tiếng Anh là "patch") đè lên và đục lại. Hiện nay, khái niệm patch, bản vá, được sử dụng cho các bản sửa lỗi phần mềm, không còn là miếng giấy như thời đầu nữa.

Ngôn ngữ lập trình có thể coi như một ngôn ngữ trung gian, tạo ra một chuẩn hóa để giao tiếp giữa người và máy tính. Ngôn ngữ lập trình cần phải tương đối thân thiện với con người, do đó hầu hết các ngôn ngữ lập trình hiện nay thường dùng tiếng Anh. Ngôn ngữ lập trình cũng cần phải chuẩn tắc, để dễ dàng chuyển đổi về mã máy.

Lưu ý là ngôn ngữ lập trình vẫn là ngôn ngữ, tức là tập hợp các quy tắc và quy ước. Để biến các mã lệnh sử dụng các quy tắc và quy ước này thành chương trình, cần có một phần mềm đặc biệt gọi là chương trình dịch (compiler) để chuyển đổi sang ngôn ngữ máy. Có nghĩa là, để cho máy tính có thể hiểu được ngôn ngữ lập trình, cần phải có các compiler để dịch các mã lệnh sang các ký tự 1 và 0. Ở đây, chúng ta cần phân biệt rõ ràng giữa ngôn ngữ lập trình và công cụ lập trình. Ngôn ngữ lập trình là quy tắc, quy ước. Ví dụ: "if (điều kiện) { làm cái này }" là một quy ước. Nếu chúng ta viết mã này một file txt, hay hét vào màn hình máy tính, thì sẽ không có điều gì xảy ra cả. Còn công cụ lập trình là phần mềm để làm việc đó. Ví dụ, Turbo Pascal, Visual C#... không phải là ngôn ngữ lập trình.

Các ngôn ngữ lập trình đầu tiên theo mô thức lập trình tuần tự, chẳng hạn như Assembly, đã sử dụng các ký hiệu, các từ viết tắt mang tính gợi nhớ hơn để biểu diễn các thao tác cơ bản. Ví dụ, thay vì một chuỗi các số 0 và 1 phức tạp, người ta có thể định nghĩa một dãy bit nhất định là **MOV**, có nghĩa là di chuyển (move). Có thể coi đây là một hình thức **trừu tượng hóa**, chúng ta định nghĩa một cái tên đơn giản cho một chuỗi các ký tự mã máy phức tạp.

Tiếp theo, các ngôn ngữ lập trình tiếp tục phát triển theo mô thức lập trình hướng thủ tục, hay lập trình **cấu trúc**. Ở mức này, chúng ta tiếp tục **trừu tượng hóa** các điều khiển. Cụ thể, các chương trình sẽ được xây dựng dựa trên các hàm/thủ tục/chương trình con. Đây là một bước tiến trong trừu tượng hóa, giúp người lập trình tập trung vào logic của chương trình hơn là các thao tác ở mức thấp của máy tính. Ví dụ, chúng ta định nghĩa một tập các chỉ thị là *sort*, sắp xếp. Khi đó, mỗi khi muốn thực hiện việc sắp xếp, chúng ta chỉ cần quan tâm đến việc gọi hàm *sort* thay vì việc gọi lại từng lệnh, chỉ thị của hàm này.

Có thể thấy, sự phát triển của các ngôn ngữ lập trình chính là sự phát triển của quá trình trừu tượng hóa (abstraction). Đối với các ngôn ngữ lập trình tuần tự, đây là trừu tượng hóa kiểu dữ liệu/chỉ thị cơ bản. Còn đối với các ngôn ngữ lập trình cấu trúc, đây là trừu tượng hóa điều khiển (control abstraction), và trừu tượng hóa chức năng (functional abstraction).

Lập trình hướng đối tượng có thể coi là bước tiếp theo của sự phát triển của quá trình trừu tượng hóa, cụ thể là trừu tượng hóa dữ liệu. Cụ thể là, trong lập trình

hướng đối tượng, các dữ liệu trong chương trình sẽ được trừu tượng hóa thành các *đối tượng*.

1.2. Kỹ thuật hướng đối tượng

Theo định nghĩa của Alan Kay, cha đẻ của lập trình hướng đối tượng:

"Tất cả mọi thứ đều là đối tượng"

Định nghĩa này có thể được hiểu như sau: Khi nhìn vào các mọi thứ trong thực tế, có thể thấy tất cả đều là đối tượng. Ví dụ: cái bàn, cái ghế, sinh viên. Hoặc thậm chí những thứ khó cảm nhận hơn như màu sắc, cơn gió, nhiệt độ, độ ẩm... cũng có thể coi là các đối tượng.

Mục tiêu của lập trình hướng đối tượng là mô hình hóa mọi thứ trong phần mềm thành các đối tượng. Cũng tương tự như các đối tượng trong thực tế, các đối tượng phần mềm cũng có thể tương tác được với nhau. Ví dụ: trong thực tế có ngôi nhà, ông Trung, xe ô tô, bông hoa. Khi đưa vào phần mềm, chúng sẽ trở thành các đối tượng phần mềm: đối tượng ngôi nhà, đối tượng Trung, đối tượng ô tô, đối tượng bông hoa. Các đối tượng phần mềm này cũng tương tác với nhau như trong thế giới thực. Ví dụ: ông Trung sống trong ngôi nhà, ông Trung lái ô tô, ông Trung ngắt bông hoa.

Ví dụ: bài toán quản lý tài khoản ngân hàng:

Trong lập trình truyền thống: chúng ta sẽ làm việc với các dữ liệu được lưu trong các *biến*, có thể là các mảng float, mảng char*... Các dữ liệu này không được đóng gói và có thể nằm rải rác ở nhiều vị trí khác nhau trong bộ nhớ máy tính. Chẳng hạn như chúng ta có một mảng các số dư (ví dụ: `int[] balance;`), một mảng các tên (ví dụ: `String[] account;`). Khi chúng ta cần gửi tiền, chúng ta có thể dùng một hàm gửi tiền, ví dụ:

```
void deposit(int accountID, int amount) {  
    balance[accountID] += amount;  
}
```

Việc này có thể hình dung giống như có một thực thể quyền tối cao (có thể coi như "Chúa" của chương trình đó) can thiệp vào dữ liệu số dư tài khoản của tài khoản.

Trong lập trình hướng đối tượng: Chúng ta có thể xây dựng một kiểu dữ liệu mới tên là "tài khoản ngân hàng". Trong lớp này, có các đối tượng cụ thể của tài khoản ngân hàng (tài khoản 1, tài khoản 2, tài khoản 3,...). Mỗi đối tượng có các thuộc tính (tên, số dư) và các hành động (gửi tiền, rút tiền, kiểm tra số dư).

Ví dụ, nếu có một tài khoản ngân hàng `account1`, ta chỉ cần gọi phương thức gửi tiền của tài khoản đó, ví dụ `account1.deposit(100000)`. Cách tư duy này tự nhiên, tương tự như trong cuộc sống hàng ngày, tài khoản ngân hàng đó tự gửi 100.000 vào tài khoản của mình chứ không phải có một "ai đó" gửi 100.000 vào tài khoản như trong lập trình truyền thống.

Các đối tượng trong kỹ thuật hướng đối tượng có các đặc điểm sau:

- **Mỗi đối tượng có dữ liệu độc lập riêng và chiếm một bộ nhớ riêng.**
Ví dụ: hai tài khoản ngân hàng khác nhau thì mỗi tài khoản là một thực thể riêng biệt, không chia sẻ dữ liệu. Dữ liệu của tài khoản A (lãi suất, số dư, ngày gửi) là riêng, không liên quan đến dữ liệu của tài khoản B.
- **Mỗi đối tượng có dạng đặc trưng của lớp đối tượng đó.** Ví dụ: các đối tượng sinh viên khác nhau có những điểm chung. Ta trừu tượng hóa các đối tượng đó thành lớp "sinh viên". Ví dụ, sinh viên có các đặc điểm chung như họ tên, mã số sinh viên, giới tính, ngày sinh. Một em học sinh cấp 3 ở cạnh đó có thể trông giống sinh viên, nhưng không phải là đối tượng của lớp "sinh viên" vì không có các đặc điểm của sinh viên.
- **Tất cả các đối tượng thuộc cùng một lớp đều có các hành vi giống nhau.** Ví dụ: sinh viên có thể đi học, đi thi, ăn, ngủ, nghỉ. Bất kỳ sinh viên nào cũng có thể thực hiện các hành vi này.

Do đó, có thể nói lập trình hướng đối tượng là bước phát triển tiếp theo của quá trình trừu tượng hóa. Trong lập trình hướng đối tượng, ta trừu tượng hóa các kiểu dữ liệu thô sơ thành các đối tượng. Trong lập trình truyền thống, các kiểu dữ liệu chúng ta có chỉ có thể là số, ký tự..., chứ không có khái niệm "sinh viên" hay "tài khoản ngân hàng".

1.3. Các khái niệm

1.3.1. Phương thức và thuộc tính

Nhìn vào một đối tượng bất kỳ, chúng ta có thể thấy tất cả các đối tượng đều có:

- Các thông tin, đặc điểm của đối tượng, được trừu tượng hóa thành các "thuộc tính" trong đối tượng phần mềm. Về ngữ nghĩa, đây là các danh từ, như là "tên, tuổi, quê quán, vận tốc, số dư...".

- Các hành động mà đối tượng có thể thực hiện, được trừu tượng hóa thành các "phương thức". Về ngữ nghĩa, đây là các động từ, ví dụ "đi học, đi thi, rút tiền, gửi tiền..."

Do đó, khi phân tích một bài toán, ta cần trích xuất các thông tin (danh từ) và hành vi (động từ) để mô hình hóa thành các đối tượng.

Đối tượng là duy nhất. Không bao giờ có hai đối tượng giống nhau, dù trùng tên, trùng tuổi, trùng ngày sinh. Ví dụ: có nhiều thầy tên Trung, nhưng đó là những người khác nhau. Hai sinh viên trùng tên, trùng tuổi, trùng ngày sinh vẫn là hai người khác nhau.

Trong lập trình các đối tượng được mô hình hóa bằng cách đóng gói thành các **trạng thái (state)** và **hành vi (behavior)**. Trạng thái được biểu diễn bởi các *thuộc tính (attributes)* và các mối quan hệ (*relationships*), còn hành vi được biểu diễn bởi các thao tác (*operations*) hay *phương thức (methods)*. Về mặt lập trình, thuộc tính tương tự như các biến và phương thức là các hàm. Tuy nhiên, điều quan trọng trong LTHDT là chúng ta cần xác định các phương thức, các thuộc tính đó như thế nào, không phải chỉ đơn giản là đóng gói các biến và các hàm lại với nhau. Các phần tiếp theo sẽ nói rõ hơn về việc xác định các phương thức hay thuộc tính này.

Trạng thái của đối tượng chỉ là một trong các điều kiện mà đối tượng tồn tại, và nó có thể thay đổi theo thời gian. Ví dụ: ô tô có thuộc tính vận tốc, và giá trị này thay đổi liên tục. Sinh viên có thuộc tính tuổi, và nó cũng thay đổi. Trạng thái giống như biến (*variable*), có thể thay đổi được.

Hành vi quyết định đối tượng hành động và đáp trả như thế nào đối với bên ngoài. Do đó, khi vẽ một đối tượng, thuộc tính vẽ bên trong, hành vi vẽ ra bên ngoài. Vì cái giao tiếp ra bên ngoài mới là cái ta thấy được. Ví dụ: ta không biết vận tốc của ô tô trừ khi nhìn vào đồng hồ hoặc thấy nó phóng nhanh. Thuộc tính thường là cái ta không thấy được, còn hành vi là cái ta thấy được. Ví dụ: tài khoản ngân hàng có các phương thức rút tiền, gửi tiền, kiểm tra số dư. Có thể thấy các thao tác này ở trên máy ATM, và chúng ta sẽ thay đổi giá trị số dư của tài khoản thông qua các thao tác trên máy này.

1.3.2. Lớp và thể hiện

Lớp (*class*) là một thiết kế (*blueprint*) hay mẫu (*prototype*) cho các đối tượng cùng kiểu. Nó định nghĩa các thuộc tính chung và các phương thức chung mà tất cả các đối tượng của loại đó đều có. Một đối tượng là một thể hiện cụ thể của một lớp. Mỗi thể hiện có thể có những thuộc tính thể hiện khác nhau.

Đối tượng (object) là một thực thể phần mềm bao bọc các thuộc tính và các phương thức liên quan. Các đối tượng cụ thể có thể còn được gọi là các *thể hiện* (*instance*). Ví dụ: một thể hiện của lớp sinh viên là một bạn sinh viên cụ thể, đang ngồi đọc tài liệu này.

- Khi nói đến lớp chung chung (ví dụ: lớp sinh viên), ta không thể biết tên hay tuổi của sinh viên là gì.
- Khi có một đối tượng cụ thể, ta mới có thể hỏi tên, tuổi.
- Các thuộc tính cụ thể gọi là thuộc tính thể hiện (instance attributes).
- Các đối tượng cụ thể mới có thể gọi được các phương thức cụ thể.

Lớp thực chất là một kiểu đối tượng. Giống như đối với kiểu dữ liệu nguyên thủy, *int*, *float*, *double* là kiểu biến, thì lớp là kiểu đối tượng. Lớp định nghĩa các thuộc tính chung và phương thức chung mà **tất cả** đối tượng của lớp đó đều có. Ví dụ, lớp "Sinh viên" có thể có có "mã số sinh viên". Nếu có một đối tượng trông giống như sinh viên, nhưng không có mã số sinh viên, thì đối tượng này không thuộc lớp sinh viên mà chúng ta đang mô hình hóa ở đây.

Lớp định nghĩa các thuộc tính cơ bản, nhưng không có giá trị cụ thể. Ví dụ, lớp xe đạp có thể có có thuộc tính "tốc độ hiện tại", nhưng cụ thể giá trị là bao nhiêu thì không thể xác định được. Khi có đối tượng cụ thể, chúng ta mới có thể có thuộc tính và phương thức cụ thể.

Mối quan hệ giữa đối tượng và lớp là mối quan hệ "LÀ" ("IS-A"). Ví dụ, đối tượng sinh viên đang đọc tài liệu này LÀ một Sinh viên. Cái laptop bạn đang dùng để xem LÀ một Máy tính.

Các đối tượng phần mềm, tương tự đối tượng trong thế giới thực, cũng có thể tương tác được với nhau. Các đối tượng phần mềm giao tiếp với nhau bằng cách **gửi thông điệp** (**message**). Việc gửi thông điệp tương tự như gọi hàm; tuy nhiên, khác với gọi hàm chỉ ra chính xác đoạn mã nào được thực hiện, gửi thông điệp yêu cầu một dịch vụ từ một đối tượng, và đối tượng sẽ quyết định cần phải làm gì. Các đối tượng khác nhau có thể có các cách thực thi các thông điệp khác nhau (đa hình).

1.4. Các nguyên lý trong lập trình hướng đối tượng

1. Trừu tượng hóa (Abstraction)

Trừu tượng hóa có nghĩa là **chung chung, khái quát hoặc định nghĩa**. Khi nói về một đối tượng bất kỳ, có rất nhiều thông tin liên quan đến nó. Ví dụ: khi nói

về sinh viên, có tên, tuổi, ngày sinh, năm sinh, mã số sinh viên, CPA, GPA, thậm chí cả các chỉ số men gan, mỡ máu, hay họ tên người yêu cũ. Các thông tin liên quan đến một đối tượng thì có vô cùng nhiều, nhưng không thể mô hình hóa hết vài đối tượng phần mềm. Do đó, ta phải bỏ **bớt đi, chỉ giữ lại những đặc điểm cơ bản để phân biệt thực thể đó với các thực thể khác**. Đây chính là quá trình "trừu tượng hóa"

Việc xác định cái gì là quan trọng nhất lại phụ thuộc vào góc nhìn. Ví dụ: cũng là đối tượng sinh viên đang nói ở trên, nếu để xây dựng hệ thống Quản lý sinh viên cho trường đại học thì cần tên, tuổi, ngày sinh, mã số sinh viên... Nhưng nếu xây dựng hệ thống Quản lý sức khỏe cho sinh viên thì cần chỉ số men gan, mỡ máu... Nếu xây dựng hệ thống làm quen kết bạn cho sinh viên thì ảnh đại diện, sở thích còn quan trọng hơn CPA, GPA.

Quá trình trừu tượng hóa chính là quá trình xây dựng lớp, tức là xác định lớp có những phương thức và thuộc tính gì.

2. Đóng gói (Encapsulation)

Đóng gói có hai ý: Đưa tất cả vào thành một gói, và **Che giấu dữ liệu**.

Trong lập trình hướng đối tượng, ta có thể chỉ định những thành phần nào là công khai - public, những thành phần nào là ẩn - private. Những thành phần private sẽ được ẩn đi, không cho phép truy cập từ bên ngoài. Người dùng chỉ cần quan tâm đến những gì là public, không cần quan tâm đến cách nó hoạt động bên trong.

Trong các nguyên lý của lập trình hướng đối tượng (Object-Oriented Programming - OOP), nguyên lý đóng gói (encapsulation) được giảng viên giải thích bằng ví dụ về lò vi sóng.

Ví dụ về việc sử dụng **lò vi sóng** để hâm bánh mì, liệu chúng ta có cần phải quan tâm xem các cơ cấu linh kiện điện tử, mạch hay cơ chế vật lý của việc này như thế nào không? Hầu hết chúng ta chỉ quan tâm nó là một cái hộp có các nút. Khi cần hâm nóng một cái bánh, chúng ta ấn một cái nút để mở nắp ra, cho đồ ăn vào, đóng lại. Sau đó chúng ta ấn thêm một cái nút nữa (hoặc vặn núm). Khi nào có âm thanh "ding" thì chúng ta lấy đồ ăn để sử dụng. Trong lập trình hướng đối tượng cũng vậy. Trong một lớp, có thể có nhiều phương thức, thuộc tính, nhưng không có nghĩa là tất cả mọi người đều cần phải biết phương thức đấy là gì. Có rất nhiều phương thức sẽ dùng cho việc tính toán nội bộ bên trong lớp đấy thôi. Đối với bên ngoài lớp đó, chúng ta chỉ cần quan tâm đến phương thức công khai (public), được hiển thị ra bên ngoài thôi.

Có thể, một số người muốn hỏi, nếu như bạn là người viết lớp này, hiểu rõ về các cấu trúc dữ liệu và giải thuật, thì có cần phải che giấu dữ liệu không? Câu hỏi

này cũng tương tự như nếu hỏi: Nếu một kỹ sư về lò vi sóng, hay một nhà vật lý, thì sẽ làm gì để hâm một cái bánh mì? Chắc các bạn cũng có câu trả lời.

Trong lập trình, nguyên lý đóng gói được thực hiện thông qua các chỉ định truy cập. Tối thiểu, các ngôn ngữ hỗ trợ hướng đối tượng phải có public và private. Ví dụ: thông tin số dư tài khoản ngân hàng nên là private. Ta chỉ có thể thay đổi số dư thông qua các thao tác rút tiền, gửi tiền.

Quá trình đóng gói hỗ trợ cho nguyên lý "chia để trị" (divide and conquer). Ta có thể làm việc với các đối tượng như những hộp đen, không cần biết bên trong nó như thế nào.

3. Thừa kế (Inheritance)

Thừa kế cho phép định nghĩa một lớp mới dựa trên một lớp đã có. Lớp mới sẽ kế thừa các thuộc tính và phương thức của lớp cũ. Bản chất của thừa kế vẫn là trừu tượng hóa. Khi đi lên trên cây phân cấp kế thừa, mức độ trừu tượng hóa tăng lên ("trừu tượng hơn", "chung chung hơn"). Ngược lại, đi xuống dưới cây phân cấp kế thừa, mức độ trừu tượng hóa giảm ("chi tiết hơn", "cụ thể hơn")

4. Đa hình (Polymorphism)

Đa hình, "Polymorphism" có nghĩa là "Nhiều hình thức thực hiện, nhiều kiểu tồn tại". (Poly: đa, nhiều; Morph: mặt). Trong lập trình truyền thống, đa hình có thể sử dụng trong phương thức:

Các phương thức có thể **trùng tên**, nhưng chúng được phân biệt bởi danh sách các tham số khác nhau (hay còn gọi là nạp chồng). Còn trong lập trình hướng đối tượng, đa hình đối tượng được hiểu là: Một đối tượng có thể được nhìn nhận theo nhiều kiểu khác nhau. Các đối tượng khác nhau có thể cùng đáp ứng chung một danh sách các thông điệp, nhưng chúng sẽ giải nghĩa và thực hiện thông điệp đó theo những cách thức khác nhau.

1.5. Phân tích và thiết kế hệ thống hướng đối tượng

Phân tích thiết kế hướng đối tượng (OOAD) là một **phương pháp luận** trong phân tích và thiết kế phần mềm áp dụng cho các hệ thống hướng đối tượng, được định nghĩa như là một tập các quá trình và thao tác để tìm và khám phá cách có thể giải quyết được bài toán phần mềm. Thông thường quá trình phát triển phần mềm thường có sáu giai đoạn:

- Giai đoạn 0: Lập kế hoạch
- Giai đoạn 1: Xác định mục tiêu - chúng ta cần phải làm cái gì (what)

- Giai đoạn 2: Xác định cách làm như thế nào (how)
- Giai đoạn 3: Xây dựng phần lõi, các tính năng cơ bản
- Giai đoạn 4: Lặp lại, hiệu chỉnh các trường hợp sử dụng (use case)
- Giai đoạn 5: Phát triển

Trong giai đoạn xác định mục tiêu (Giai đoạn 1), chúng ta xác định cụ thể các mục tiêu, chức năng và nhiệm vụ mà phần mềm cần đáp ứng. Trong OOAD, người ta sử dụng các ký pháp và kỹ thuật **Use case** để mô tả các công việc này.

Kết quả của quá trình thiết kế là các **biểu đồ**. Có nhiều loại biểu đồ trong OODA, tuy nhiên trong môn học này, chúng ta quan tâm đến các biểu đồ chính như:

- **Biểu đồ use case:** Chỉ ra các tác nhân (người sử dụng) và các use case (chức năng) mà hệ thống cung cấp cho các tác nhân đó.
- **Biểu đồ lớp:** Minh họa các lớp được thiết kế, các thuộc tính và phương thức của mỗi lớp, và mối quan hệ giữa các lớp. Đây là một trong những biểu đồ quan trọng nhất trong môn học này.

Cần lưu ý một điểm là các biểu đồ này là minh họa cho thiết kế. Không có gì đảm bảo việc sử dụng các biểu đồ thiết kế này sẽ giúp cho các thiết kế là chính xác. Trong các bài tiếp theo, chúng ta sẽ tìm hiểu cách thiết kế các lớp làm sao cho phù hợp. Ngoài ra, chúng ta cũng sẽ có bài 12 tập trung vào OOAD cũng như các biểu đồ UML, là một loại ngôn ngữ mô hình hóa sử dụng trong OOAD.

Chương 2

Cú pháp Java cơ bản

2.1. Các khái niệm cơ bản	21
2.2. Biến	23
2.3. Các kiểu dữ liệu trong Java	24
2.4. Toán tử (Operators)	30
2.5. Cấu trúc điều khiển	32
2.6. Mảng	38
2.7. Giới thiệu UML	39

Nội dung

- Nắm được quy định cơ bản về định danh, câu lệnh, chú thích, và biến trong Java
- Sử dụng thành thạo các kiểu dữ liệu nguyên thủy trong Java
- Nắm được các loại toán tử, các cấu trúc điều khiển, và cấu trúc dữ liệu kiểu mảng trong Java
- Hiểu được ý nghĩa ngôn ngữ mô hình hóa thống nhất UML, biết các loại biểu đồ thông dụng

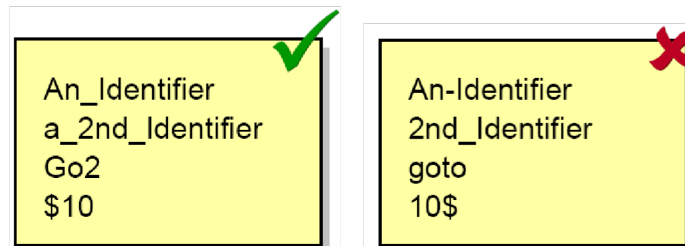
2.1. Các khái niệm cơ bản

a. Định danh:

Trong ngôn ngữ lập trình Java, định danh (identifier) là tên dùng để đặt cho các thành phần như: Biến (variable), Phương thức (method), Lớp (class), Giao diện (interface), Mảng (array), Nhãn (label) trong lệnh break hoặc continue. Mỗi định danh đóng vai trò như một cách để lập trình viên tham chiếu đến các thực thể trong chương trình. Định danh phải duy nhất trong phạm vi sử dụng để tránh xung đột.

Định danh là một xâu ký tự thể hiện tên các biến, các phương thức, các lớp và nhãn là duy nhất trong chương trình. Quy định với định danh hợp lệ (bắt buộc tuân

thủ). Định danh bao gồm các ký tự có thể là chữ cái, chữ số, ký tự '\$' hoặc '_'. Các định danh không được phép: Bắt đầu bởi một chữ số, trùng với từ khóa, có chứa dấu cách. Lưu ý: Định danh phân biệt chữ hoa chữ thường. Ví dụ: Yourname, yourname, YourName và yourName là 4 định danh khác nhau.



Hình 2.1 Định danh.

Đối với định danh, ta có một số quy ước sau đây - naming convention (Quy ước: không bắt buộc, nhưng nên làm theo)

- Phải mang tính gợi nhớ. Ví dụ: nên dùng định danh “bookPrice” hơn là “bp” để lưu thông tin về giá 1 quyển sách.
- Bắt đầu bằng chữ cái
- Gói (package): tất cả sử dụng chữ thường. Ví dụ: theexample
- Lớp (Class): viết hoa chữ cái đầu tiên trong các từ ghép lại. Ví dụ: TheExample
- Phương thức/thuộc tính (method/field): Bắt đầu bằng chữ thường, viết hoa chữ cái đầu tiên trong các từ còn lại. Ví dụ: theExample
- Hằng (constants): Tất cả viết hoa. Ví dụ: THE_EXAMPLE

b. Các từ khóa

- Người lập trình không được phép sử dụng các từ khóa như một định danh
- Literals: null true false
- Từ khóa (keyword): abstract assert boolean break byte case catch char class continue default do double else extends final finally float for if implements import instanceof int interface long native new package private protected public return short static strictfp super switch synchronized this throw throws transient try void volatile while

- Từ dành riêng (reserved word): byvalue cast const future generic goto inner operator outer rest var volatile

c. Câu lệnh

Trong Java, mỗi câu lệnh kết thúc bằng dấu ; (semicolon). Dấu ; báo cho trình biên dịch biết kết thúc của một câu lệnh, không phụ thuộc vào việc nó nằm trên dòng nào. Nhiều lệnh có thể viết trên một dòng. Một câu lệnh có thể được chia thành nhiều dòng để tăng khả năng đọc hiểu, đặc biệt với những câu dài. Java cho phép điều này miễn là cú pháp hợp lệ và không chia tách không hợp lý (ví dụ: không tách giữa tên biến và toán tử theo kiểu không rõ ràng).

Ví dụ:

```
System.out.println(  
    "This is part of the same line");
```

```
a=0; b=1; c=2;
```

d. Chú thích trong Java

Java hỗ trợ ba kiểu chú thích như sau:

- // Chú thích trên một dòng
- // Không xuống dòng
- /* Chú thích một đoạn */
- /** Javadoc * chú thích dạng Javadoc */

Chú thích dùng để mô tả thêm về mã nguồn (source code). Trình thông dịch sẽ bỏ qua các chú thích này.

2.2. Biến

Trong Java, biến (variable) giống như một chiếc hộp trong bộ nhớ dùng để lưu trữ giá trị của một đại lượng nào đó trong quá trình chương trình chạy. Mỗi biến đều được gán một tên cố định để lập trình viên dùng nó khi cần truy cập hoặc thao tác với giá trị chứa trong biến. Tên biến không thể thay đổi trong suốt thời gian chạy. Mỗi biến khi được khai báo sẽ có một kiểu dữ liệu (data type) xác định, dùng để chỉ định loại giá trị mà biến có thể chứa. Biến có thể chứa các giá trị kiểu số, ký tự, văn bản, hay đối tượng và kiểu giá trị này của biến cũng không thay đổi.

```
/* This is a simple Java program.  
   FileName : "HelloWorld.java". */  
class HelloWorld  
{  
    // Your program begins with a call to main().  
    // Prints "Hello, World" to the terminal window.  
    public static void main(String args[])  
    {  
        System.out.println("Hello, World");  
    }  
}
```

Hình 2.2 Chú thích trong Java.

Hãy tưởng tượng bộ nhớ máy tính như một chiếc tủ lớn, còn mỗi biến là một chiếc hộp nhỏ trong tủ đó: Tên biến là nhãn dán trên hộp. Kiểu dữ liệu quyết định loại vật có thể bỏ vào hộp (số, chữ, đối tượng...). Giá trị là thứ mà chiếc hộp đang chứa. Ta có thể thay đổi giá trị bên trong hộp, nhưng không thể thay đổi loại vật mà hộp được thiết kế để chứa.

Cách khai báo biến: Biến khi dùng phải được khai báo tên (định danh) và gán cho một kiểu dữ liệu (số, ký tự, văn bản, đối tượng, v.v.). Các biến đơn cần phải được khởi tạo trước khi sử dụng. Có thể kết hợp khai báo và khởi tạo cùng một lúc. Lúc đó, ta sử dụng toán tử = để gán (bao gồm cả khởi tạo). Ví dụ:

```
int price = 0;  
int speed = 100;  
int stockPrice = 75;
```

Phạm vi của biến: là vùng chương trình mà trong đó biến có thể được tham chiếu đến, có thể sử dụng được. Phạm vi hoạt động (scope) của các biến cho phép xác định các nguyên lý của tạo biến, sử dụng biến và giải phóng biến.

Phân loại:

- Biến toàn cục: phạm vi trong cả chương trình
- Biến cục bộ: được khai báo trong một phương thức/khối lệnh thì chỉ có thể truy cập trong phương thức/khối lệnh đó.

2.3. Các kiểu dữ liệu trong Java

Trong Java kiểu dữ liệu được chia thành hai loại:

- Kiểu dữ liệu nguyên thủy (primitive): Số nguyên (integer), Số thực (float), Ký tự (char), Giá trị logic (boolean).
- Kiểu dữ liệu tham chiếu (reference): Mảng (array), Đối tượng (object).

Trong Java, kích thước của các kiểu dữ liệu nguyên thủy được định nghĩa rõ ràng và cố định bởi JVM (Java Virtual Machine), nhờ đó chúng luôn giống nhau trên mọi nền tảng và hệ điều hành. Điều này giúp Java đảm bảo tính nhất quán và khả năng chạy chương trình độc lập với phần cứng, phù hợp với triết lý "viết một lần, chạy mọi nơi". Các kiểu dữ liệu như `int`, `byte`, `long`, `float`... đều có kích thước cụ thể, ví dụ `int` luôn chiếm 32 bit, `long` là 64 bit, còn `char` là 16 bit để hỗ trợ bộ mã Unicode. Khi lập trình, việc lựa chọn kiểu dữ liệu phù hợp cần cân nhắc giữa nhu cầu lưu trữ giá trị và việc tiết kiệm bộ nhớ. Nếu chọn kiểu dữ liệu quá lớn cho những giá trị nhỏ, chương trình sẽ lãng phí bộ nhớ không cần thiết; ngược lại, nếu chọn kiểu quá nhỏ, giá trị có thể bị tràn (overflow). Vì vậy, cần cân bằng giữa độ lớn cần thiết của giá trị và khả năng sử dụng bộ nhớ hiệu quả để tối ưu hóa chương trình.

Số nguyên là các giá trị không có phần thập phân, ví dụ như -10, 0, 25, hoặc 1000. Java cung cấp bốn kiểu dữ liệu nguyên thủy để biểu diễn số nguyên, bao gồm `byte`, `short`, `int` và `long`. Mỗi kiểu có kích thước bộ nhớ và phạm vi giá trị khác nhau, giúp lập trình viên linh hoạt lựa chọn để tối ưu hóa việc sử dụng bộ nhớ.

Kiểu	Kích thước	Phạm vi giá trị	Ví dụ
<code>byte</code>	8 bit	-128 đến 127	<code>byte a = 10;</code>
<code>short</code>	16 bit	-32,768 đến 32,767	<code>short b = 300;</code>
<code>int</code>	32 bit	-2,147,483,648 đến 2,147,483,647	<code>int c = 10000;</code>
<code>long</code>	64 bit	$\pm 9,223,372,036,854,775,807$	<code>long d = 123456789L;</code>

Bảng 2.1 Các kiểu số nguyên trong Java

Kích thước của các kiểu dữ liệu nguyên thủy trong Java được định nghĩa bởi **JVM (Java Virtual Machine)** và luôn đồng nhất trên mọi nền tảng phần cứng và hệ điều hành. Điều này giúp Java đảm bảo tính nhất quán và hỗ trợ tính năng "write once, run anywhere".

Java mặc định sử dụng kiểu `int` cho các giá trị số nguyên. Nếu muốn khai báo một số thuộc kiểu `long`, lập trình viên cần thêm hậu tố `L` hoặc `l` vào cuối giá trị. Ví dụ:

```
int a = 123456;           // Mặc định là int
long b = 123456789L;     // Phải có hậu tố L để là long
```

Khi khai báo biến số nguyên, cần cân nhắc giữa phạm vi giá trị có thể chứa và mức độ sử dụng bộ nhớ. Nếu sử dụng kiểu dữ liệu quá lớn cho các giá trị nhỏ sẽ dẫn đến lãng phí bộ nhớ. Ngược lại, nếu sử dụng kiểu quá nhỏ sẽ gây lỗi tràn số (*overflow*).

Số thực là các giá trị có phần thập phân, ví dụ như 3.14, -0.5, 2.71828, hoặc 100.0. Để biểu diễn số thực, Java cung cấp hai kiểu dữ liệu nguyên thủy là `float` và `double`. Đây là các kiểu số dấu phẩy động, sử dụng chuẩn IEEE 754 để biểu diễn trong bộ nhớ.

Kiểu	Kích thước	Độ chính xác	Ví dụ
<code>float</code>	32 bit	xấp xỉ 6–7 chữ số thập phân	<code>float x = 3.14f;</code>
<code>double</code>	64 bit	xấp xỉ 15 chữ số thập phân	<code>double y = 2.71828;</code>

Bảng 2.2 Các kiểu số thực trong Java

Theo mặc định, khi khai báo một hằng số số thực trong Java (ví dụ: 3.14), trình biên dịch sẽ coi nó là kiểu `double`. Nếu muốn khai báo số thực dưới dạng `float`, cần thêm hậu tố `f` hoặc `F` sau giá trị.

```
float pi = 3.14f;           // float cần hậu tố f
double e = 2.718281828;    // double mặc định
```

Kiểu `float` thường được sử dụng khi cần tiết kiệm bộ nhớ trong các ứng dụng yêu cầu độ chính xác vừa phải, như đồ họa hoặc xử lý tín hiệu. Trong khi đó, kiểu `double` thường được dùng phổ biến hơn trong các phép toán số học do cung cấp độ chính xác cao hơn.

Giống như số nguyên, `float` và `double` có kích thước cố định do JVM quy định, giúp đảm bảo tính nhất quán và khả năng chạy đồng nhất trên mọi nền tảng. Tuy nhiên, do bản chất sử dụng biểu diễn nhị phân, các phép toán trên số thực có thể gặp lỗi làm tròn hoặc sai số rất nhỏ trong quá trình tính toán.

Kiểu ký tự trong Java Trong Java, kiểu dữ liệu `char` được sử dụng để lưu trữ một ký tự đơn lẻ. Mỗi giá trị `char` được biểu diễn bằng **16 bit (2 byte)** theo chuẩn **Unicode**, cho phép biểu diễn 65,536 ký tự khác nhau, bao gồm cả chữ cái Latin, ký tự đặc biệt, ký hiệu toán học và các ký tự từ nhiều bảng mã ngôn ngữ khác trên thế giới.

- Kiểu `char` dùng để biểu diễn một ký tự Unicode.
- Giá trị của biến `char` được đặt trong dấu nháy đơn (' '), ví dụ: `'A'`, `'9'`, `'#'`.

- Do được biểu diễn bằng mã Unicode, mỗi ký tự có thể được ánh xạ sang một số nguyên không âm tương ứng.

Ví dụ khai báo biến char:

```
char letter = 'A';
char digit = '7';
char symbol = '#';
char unicodeChar = '\u03A9'; // Ký tự Unicode Omega ()
```

Java cho phép gán giá trị kiểu số nguyên (trong phạm vi 0 đến 65535) cho biến `char` vì bản chất `char` là một số nguyên không âm. Tuy nhiên, khi in ra màn hình, nó sẽ được hiển thị dưới dạng ký tự.

Ví dụ:

```
char ch = 65;
System.out.println(ch); // In ra 'A' vì mã Unicode của 'A' là 65
```

Kiểu `char` trong Java là một kiểu dữ liệu nguyên thủy đặc biệt được sử dụng để lưu trữ ký tự Unicode, giúp Java hỗ trợ đa ngôn ngữ và làm việc hiệu quả với văn bản toàn cầu. Việc sử dụng chuẩn Unicode thay vì ASCII giới hạn là một trong những điểm mạnh của Java khi làm việc với dữ liệu ngôn ngữ.

Giá trị logic trong Java. Trong Java, giá trị logic được biểu diễn bằng kiểu dữ liệu nguyên thủy boolean, và chỉ có hai giá trị hợp lệ là `true` (đúng) và `false` (sai). Đây là kiểu dữ liệu rất quan trọng, thường được sử dụng trong các biểu thức điều kiện và các cấu trúc điều khiển như `if`, `while`, `for`, hay các phép so sánh và toán tử logic.

Kiểu boolean có thể được sử dụng để lưu trữ kết quả của một phép so sánh (ví dụ: `a > b`), hoặc làm biến điều kiện trong quá trình xử lý logic của chương trình. Ví dụ, ta có thể khai báo boolean `isPassed = score >= 50`;, trong đó biến `isPassed` sẽ nhận giá trị `true` nếu `score` lớn hơn hoặc bằng 50, và `false` trong trường hợp ngược lại.

Mặc dù Java không quy định chính xác kích thước bộ nhớ của kiểu boolean, nhưng trong hầu hết các hệ thống thực thi JVM, một biến boolean thường chiếm 1 byte trong bộ nhớ.

Giá trị boolean được xác định rõ ràng trong Java. Lưu ý: Một giá trị `int` không thể sử dụng thay cho giá trị boolean. Giá trị boolean có thể lưu trữ giá trị hoặc `true` hoặc `false`. Biến boolean được khởi tạo là `false`.

Giá trị hằng (literal) Literal là một giá trị của các kiểu dữ liệu nguyên thủy và xâu ký tự. Gồm 5 loại: integer, floating point, boolean, character, String

Hằng số nguyên

- Hệ cơ số 8 (Octals) bắt đầu với chữ số 0: $032 = 011\ 010(2) = 16 + 8 + 2 = 26(10)$
- Hệ cơ số 16 (Hexadecimals) bắt đầu với 0 và ký tự x: $0x1A = 0001\ 1010(2) = 16 + 8 + 2 = 26(10)$
- Kết thúc bởi ký tự “L” thể hiện kiểu dữ liệu long: 26L
- Ký tự hoa, thường cho giá trị bằng nhau: 0x1a , 0x1A , 0X1a , 0X1A đều có giá trị 26 trong hệ decimal

Hằng số thực

- float kết thúc bằng ký tự f (hoặc F). Ví dụ: 7.1f
- double kết thúc bằng ký tự d (hoặc D). Ví dụ: 7.1D
- e (hoặc E) được sử dụng trong dạng biểu diễn khoa học. Ví dụ: 7.1e2
- Một giá trị thực mà không có ký tự kết thúc đi kèm sẽ có kiểu là double. Ví dụ: 7.1 giống như 7.1d

Hằng boolean, ký tự và xâu ký tự

- boolean: true, false
- Ký tự: Được đặt giữa 2 dấu nháy đơn. Ví dụ: ‘a’, ‘A’ hoặc ‘\uffff’
- Xâu ký tự: Được đặt giữa hai dấu nháy kép. Ví dụ: “Hello world”, “Xin chào bạn”,...

Các ký tự điều khiển nhấn phím:

- \b backspace
- \f form feed
- \n newline
- \r return (về đầu dòng)

- `\t` tab

Hiển thị các ký tự đặc biệt trong xâu

- `\"` quotation mark
- `\'` apostrophe
- `\\` backslash
-

Chuyển đổi kiểu dữ liệu (Casting) Java là ngôn ngữ định kiểu chặt. Có nghĩa là gán sai kiểu giá trị cho một biến có thể dẫn đến các lỗi biên dịch hoặc các ngoại lệ của JVM. JVM có thể ngầm định chuyển từ một kiểu dữ liệu hẹp sang một kiểu rộng hơn. Tuy nhiên, để chuyển sang một kiểu dữ liệu hẹp hơn, cần phải định kiểu rõ ràng. Ví dụ:

```
int a, b;
short c;
a = b + c;

int d;
short e;
e = (short)d;

double f;
long g;
f = g;
g = f; //error
```

Chuyển đổi kiểu sẽ được thực hiện tự động nếu không xảy ra mất mát thông tin (byte -> short -> int -> long -> float -> double) Lưu ý: ép kiểu từ short về char, từ byte về char và ngược lại đều phải ép kiểu tường minh Ép kiểu trực tiếp (explicit cast) được yêu cầu nếu có “nguy cơ” giảm độ chính xác

Ví dụ:

```
long p = (long) 12345.56; // p sẽ nhận giá trị 12345
int g = p; // không hợp lệ dù kiểu int có thể lưu giá trị 12345
char c = 't';
```



```

int j = c; // hợp lệ, tự động chuyển đổi
short k = c; // không hợp lệ, phải ép kiểu tường minh
short k = (short) c; // hợp lệ
float f = 12.35; // Báo lỗi do 12.35 là hằng double
float f = 0.0; // Báo lỗi do 0.0 là hằng double
float f = 0;
long l = 999999999999; //Báo lỗi: The literal 999999999999 of type int is out of range
short k = 99999999; // Báo lỗi: Type mismatch: cannot convert from int to short

```

Bài tập: Kiểm tra với ví dụ sau

```

short i = 6, j=7;
i = i + j;
i += j;

short i, j = 5;
int n = 6;
i = (short)n + j;

```

2.4. Toán tử (Operators)

Kết hợp các giá trị đơn hoặc các biểu thức con thành những biểu thức mới, phức tạp hơn và có thể trả về giá trị. Java cung cấp nhiều dạng toán tử sau:

- Toán tử số học
- Toán tử bit, toán tử quan hệ
- Toán tử logic
- Toán tử gán
- Toán tử một ngôi
- Toán tử số học: +, -, *, /, %
- Toán tử bit: AND: &, OR: |, XOR: ^, NOT: ~, Dịch bit: <<, >>
- Toán tử quan hệ: ==, !=, >, <, >=, <=
- Toán tử logic: &&, ||, !

- Toán tử một ngôi: Đảo dấu: +, -; Tăng giảm 1 đơn vị: ++, --; Phủ định một biểu thức logic: !
- Toán tử gán: =, +=, -=, %= tương tự với >>, <<, &, ..., ^

```
int i = 10/3;
float f0 = 10;
float f1 = (float) 10/3;
float f2 = 10/3;
float f3 = f0/3;

System.out.println(i); //3
System.out.println(f1); //3.3333333
System.out.println(f2); //3.0
System.out.println(f3); //3.3333333
```

Thứ tự ưu tiên của toán tử được xác định bởi các dấu ngoặc đơn hoặc theo ngầm định như sau (ưu tiên từ trên xuống thực hiện trước):

1. Toán tử [] . ()
2. Toán tử x++ x--
3. Toán tử một ngôi: ++x --x +x -x !
4. Toán tử khởi tạo, toán tử chuyển kiểu: new (type)x
5. Nhân, chia: * /
6. Cộng, trừ: + -
7. Dịch bit: « » >> (unsigned shift)
8. So sánh: < > <= >= instanceof
9. So sánh bằng == !=
10. Toán tử bit AND: &
11. Toán tử bit OR: ^
12. Toán tử bit XOR: |

13. Toán tử logic AND: &&
14. Toán tử logic OR: ||
15. Toán tử điều kiện: (ternary) ?:
16. Toán tử gán: = *= /=

Ví dụ: Thứ tự ưu tiên các toán tử.

```
double paid = 10;
double change = 3.25;
double tip = (paid-change)*0.2;
```

```
int i;
System.out.println(i=5); //5
System.out.println(i+=4); //9
System.out.println(i++); //9
System.out.println(--i); //9
```

2.5. Cấu trúc điều khiển

Là các cấu trúc lệnh nhằm chỉ định cho chương trình thực hiện các câu lệnh/đoạn lệnh khác nhau, tùy theo từng điều kiện nào đó. Có 2 loại cấu trúc điều khiển:

- Câu lệnh điều kiện: Lệnh if – else, Lệnh switch – case
- Câu lệnh lặp: Vòng lặp for, Vòng lặp while, Vòng lặp do – while

a. Lệnh if

Cú pháp

```
if (dieu_kien){
    cac_cau_lenh;
}
```

Nếu biểu thức điều kiện dieu_kien (có kiểu boolean) nhận giá trị true thì thực hiện khối lệnh cac_cau_lenh;

```
if (dieu_kien){
    cac_cau_lenh_1;
```

```

} else {
    cac_cau_lenh_2;
}

```

Nếu biểu thức điều kiện `dieu_kien` (có kiểu boolean) nhận giá trị true thì thực hiện khối lệnh `cac_cau_lenh_1`, là false thì thực hiện khối lệnh `cac_cau_lenh_2`;

```

if (dieu_kien_1){
    cac_cau_lenh_1;
} else if (dieu_kien_2){
    cac_cau_lenh_2;
} else if (dieu_kien_3){
    cac_cau_lenh_3;
} else {
    cac_cau_lenh_n;
}

```

Có thể có nhiều else-if, chỉ có 1 else tối đa

Ví dụ kiểm tra số chẵn - lẻ

```

class CheckNumber
{
    public static void main(String args[])
    {
        int num =10;
        if (num %2 == 0)
            System.out.println (num+ "la so chan");
        else
            System.out.println (num + "la so le");
    }
}

```

b. Lệnh switch-case

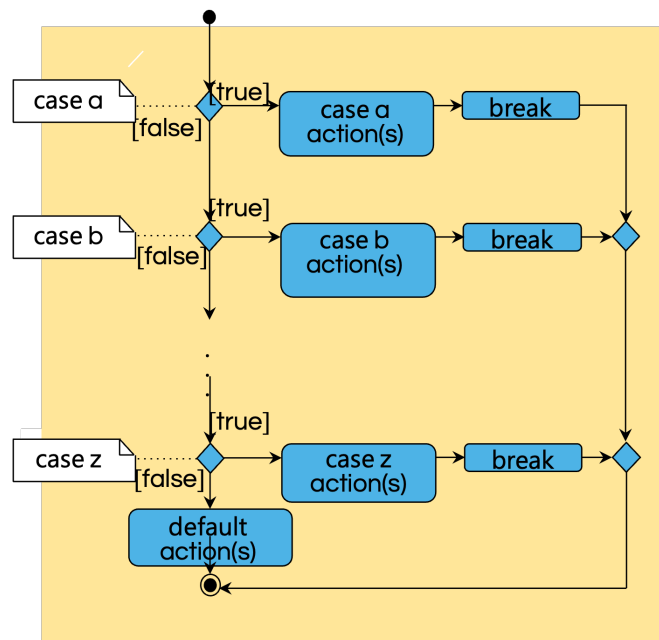
Kiểm tra một biến đơn với nhiều giá trị khác nhau và thực hiện trường hợp tương ứng. Trong đó lệnh `break` để thoát khỏi lệnh `switch-case` và `default` để kiểm soát các giá trị nằm ngoài các giá trị case.

Ví dụ 1:

```

class CheckNumber
{

```



Hình 2.3 Cấu trúc lệnh switch-case.

```

public static void main(String args[])
{
    int num =10;
    if (num %2 == 0)
        System.out.println (num+ "la so chan");
    else
        System.out.println (num + "la so le");
}
}

```

Ví dụ 2:

```

switch (day) {
    case 0:
    case 1:
        rule = "weekend";
        break;
    case 2:
    case 3:
    case 4:
    case 5:

```

```
case 6:
    rule = "weekday";
    break;
default:
    rule = "error";
}
```

Ví dụ 3:

```
if (day == 0 || day == 1) {
    rule = "weekend";
} else if (day > 1 && day < 7) {
    rule = "weekday";
} else {
    rule = error;
}
```

Bài tập: tính số ngày trong tháng

- Input: Năm, tháng
- Output: số ngày trong tháng của năm đó

Yêu cầu: sử dụng lệnh switch-case

Gợi ý:

- Tháng 1, 3, 5, 7, 8, 10, 12: 31 ngày
- Tháng 4, 6, 9, 11: 30 ngày

Riêng tháng 2:

- Năm thường: 28 ngày
- Năm nhuận: 29 ngày (năm nhuận là “năm chia hết cho 4 và không chia hết cho 100”, hoặc là “năm chia hết cho 400”)

c. Vòng lặp while và do while

Thực hiện một câu lệnh hoặc một khối lệnh khi điều kiện vẫn nhận giá trị true.
Trong đó:

- while() thực hiện 0 hoặc nhiều lần
- do...while() thực hiện ít nhất một lần

```
while(condition) {
    // code block to be executed
}
```

```
do {
    // code block to be executed
} while (condition);
```

Ví dụ:

```
class WhileDemo{
    public static void main(String args[]){
        int a = 5, fact = 1;
        while (a >= 1){
            fact *=a;
            a--;
        }
        System.out.println("The Factorial of 5 is "+fact);
    }
}
```

Kết quả: “The factorial of 5 is 120” được hiển thị. Hãy viết thay lệnh while bằng lệnh do-while ?

d. Vòng lặp for

Cú pháp

```
for (start_expr; test_expr; increment_expr){
    // code to execute repeatedly
}
```

Lưu ý: Có thể khai báo biến trong câu lệnh for. Thường sử dụng để khai báo một biến đếm. Thường khai báo trong biểu thức “start”. Phạm vi của biến giới hạn trong vòng lặp

Ví dụ

```
class ForDemo {
    public static void main(String args[]){
        int i=1, sum=0;
        for (i=1;i<=10;i+=2){
            sum+=i;
            System.out.println("Sum of first five odd numbers is" + sum);
        }
    }
}
```

Kết quả: “Sum of first five odd numbers is 25” được hiển thị.

Vòng lặp for và while: Các câu lệnh for và while cung cấp chức năng tương đương nhau. Các cấu trúc lặp thường được sử dụng trong các tình huống khác nhau. Trong đó: while được sử dụng cho lặp từ đầu đến cuối; for được sử dụng để lặp với số vòng lặp xác định

```
int sum = 0;
for (int index = 1; index <= 10; index++) {
    sum += index;
}
```

...

```
int sum = 0;
int index = 1;
while (index <= 10) {
    sum += index;
    index++;
}
```

Các lệnh thay đổi cấu trúc điều khiển

1. break: Có thể được sử dụng để thoát ra ngoài câu lệnh switch; Kết thúc vòng lặp for, while hoặc do...while; Có hai dạng:

- Gắn nhãn: Tiếp tục thực hiện câu lệnh tiếp theo sau vòng lặp được gắn nhãn
- Không gắn nhãn: Thực hiện câu lệnh tiếp theo bên ngoài vòng lặp

2. continue

- Có thể được sử dụng cho vòng lặp for, while hoặc do...while
- Bỏ qua các câu lệnh còn lại của vòng lặp hiện thời và chuyển sang thực hiện vòng lặp tiếp theo.

Ví dụ:

```
public int myMethod(int x) {
    int sum = 0;
    outer: for (int i=0; i<x; i++) {
        inner: for (int j=i; j<x; j++){
            sum++;
            if (j==1) continue;
            if (j==2) continue outer;
            if (i==3) break;
            if (j==4) break outer;
        }
    }
    return sum;
}
```

2.6. Mảng

Dùng để lưu một tập hợp hữu hạn các phần tử cùng kiểu (nguyên thủy hoặc đối tượng), liên kề nhau trong bộ nhớ. Mỗi mảng có 1 tên gọi. Các phần tử được đánh số thứ tự, bắt đầu từ 0. Mỗi phần tử mảng hoạt động như 1 biến đơn. Cách thức khai báo và khởi tạo mảng phải khai báo trước khi sử dụng. Kích thước của một mảng sau khai báo sẽ không thể thay đổi Cú pháp:

```
kieu_dulieu[KichThuoc_MANG];
kieu_dulieu ten_mang[] = new kieu_dulieu[KichThuoc_MANG];
kieu_dl[] ten_mang = {ds_gia_tri_cac_ptu};
```

Lưu ý: Nếu không khởi tạo thì tất cả các phần tử của mảng nhận giá trị mặc định tùy thuộc vào kiểu dữ liệu.

Cách khai báo mảng:

- Chỉ đơn thuần khai báo mảng: Datatype identifier[]. Ví dụ: char ch[]; //khai báo mảng ký tự có tên ch

- Khai báo và cấp phát bộ nhớ cho các phần tử mảng sử dụng toán tử new: Datatype identifier[] = new datatype [size]. Ví dụ: char ch[] = new char [10]; //Khai báo một mảng ch và lưu trữ 10 ký tự
- Khai báo mảng, cấp phát bộ nhớ cho nó và gán các giá trị ban đầu cho các phần tử của mảng
- Datatype identifier[] = value1, value2 ... valueN; Ví dụ: char ch [] = 'A','B','C','D'; //khai báo mảng ch và lưu 4 chữ cái kiểu ký tự

Làm việc với mảng:

- Dùng thuộc tính .length để lấy kích thước của một mảng
- Lưu ý không truy cập vào các chỉ số không thuộc mảng, ví dụ chỉ số âm, chỉ số $\geq$ kích thước mảng.
- Duyệt tất cả các phần tử trong mảng: dùng vòng lặp.

Với mảng 2 chiều: Có thể khai báo và sử dụng một mảng nhiều chiều. Mảng 2 chiều giống một bảng với các dòng và cột. Mảng 2 chiều: coi như 1 mảng của các phần tử A, mỗi phần tử A lại là 1 mảng các phần tử B. Khai báo mảng 2 chiều

```
kieu_dulieu[][] ten_mang;
```

Truy cập phần tử trong mảng thông qua các chỉ số hàng và cột sử dụng các vòng lặp lồng nhau: ten_mang[chi_so_hang][chi_so_cot]

2.7. Giới thiệu UML

- **Hướng tiếp cận không có phân tích – thiết kế:** Bắt đầu lập trình ngay khi có được yêu cầu. Cách tiếp cận này thường dẫn đến việc mất rất nhiều thời gian để chỉnh sửa và viết lại mã nguồn nhiều lần. Hệ thống không có bất kỳ kiến trúc rõ ràng nào, khiến việc bảo trì và mở rộng trở nên khó khăn. Lập trình viên sẽ phải đối mặt với nhiều lỗi phát sinh do không có định hướng thiết kế ban đầu.
- **Hướng tiếp cận có phân tích – thiết kế:** Trước khi lập trình, các yêu cầu của bài toán được chuyển thành một bản thiết kế rõ ràng. Cách tiếp cận này tập trung vào việc phân tích các **YÊU CẦU** và thiết kế các **MÔ HÌNH** cho hệ thống. Việc lập trình chỉ bắt đầu sau khi đã có một kiến trúc tổng thể, giúp quá trình phát triển phần mềm trở nên hiệu quả, dễ bảo trì và giảm thiểu lỗi.

Ưu điểm của việc PTTK hệ thống là: Việc phân tích và thiết kế (PTTK) hệ thống mang lại nhiều lợi ích quan trọng trong quá trình phát triển phần mềm. Các ưu điểm cụ thể bao gồm:

- **Đơn giản hóa thế giới thực thông qua các mô hình:** Giúp các khái niệm phức tạp trở nên dễ hiểu và dễ quản lý hơn bằng cách trừu tượng hóa các đối tượng, hành vi và mối quan hệ của hệ thống.
- **Mô tả đúng và đồng nhất cấu trúc, hành vi của hệ thống:** Đảm bảo hệ thống được hiểu và xây dựng nhất quán trong suốt quá trình phát triển.
- **Đảm bảo thỏa mãn mục tiêu và yêu cầu của hệ thống:** Giúp phát hiện và xử lý các mâu thuẫn trong yêu cầu trước khi tiến hành xây dựng thực tế.
- **Cung cấp nhiều góc nhìn khác nhau về hệ thống:** Cho phép người dùng, khách hàng, kỹ sư phân tích, thiết kế và lập trình hiểu hệ thống theo các cách khác nhau tùy theo vai trò, từ đó tăng khả năng cộng tác và hiểu đúng mục tiêu.
- **Ghi lại các quyết định quan trọng của nhà phát triển:** Tài liệu hóa quá trình ra quyết định giúp việc bảo trì, mở rộng hoặc chuyển giao hệ thống trở nên thuận tiện và hiệu quả hơn trong tương lai.

2.7.1. Khái niệm UML

UML (viết tắt của Unified Modeling Language) là một ngôn ngữ mô hình hóa được thống nhất, đóng vai trò quan trọng trong phát triển phần mềm hiện đại. UML không phải là một phương pháp phát triển phần mềm, mà là một công cụ dùng để trực quan hóa, đặc tả, xây dựng và tài liệu hóa các thành phần của một hệ thống phần mềm. Việc trực quan hóa (visualizing) giúp các thành phần hệ thống được thể hiện bằng sơ đồ dễ hiểu, hỗ trợ lập trình viên và các bên liên quan dễ hình dung cấu trúc và hành vi của hệ thống. Đặc tả (specifying) cho phép mô tả rõ ràng các yêu cầu và mối quan hệ giữa các đối tượng trong hệ thống. Quá trình xây dựng (constructing) giúp chuyển đổi mô hình thành hiện thực phần mềm, trong khi tài liệu hóa (documenting) ghi lại toàn bộ thiết kế và quyết định trong quá trình phát triển để phục vụ cho bảo trì và nâng cấp sau này. Việc sử dụng UML giúp quá trình phát triển phần mềm trở nên nhất quán, giảm thiểu lỗi phát sinh, đồng thời tăng cường hiệu quả trong việc trao đổi, giao tiếp giữa các thành viên trong nhóm phát triển và giữa nhóm phát triển với khách hàng.

2.7.2. Các biểu đồ UML cơ bản

Biểu đồ trong UML là các hình vẽ sử dụng các ký hiệu mô hình hóa để minh họa một thành phần cụ thể hoặc một khía cạnh cụ thể của hệ thống. Mỗi biểu đồ đóng vai trò là một phần của hướng nhìn (viewpoint) đối với hệ thống, giúp người phát triển và các bên liên quan hiểu rõ hơn về cấu trúc hoặc hành vi của phần mềm. Một mô hình hệ thống thường bao gồm nhiều loại biểu đồ khác nhau, và mỗi loại có thể chứa nhiều biểu đồ riêng biệt để diễn tả các phần khác nhau của hệ thống. Đáng chú ý, một số loại biểu đồ có thể xuất hiện trong nhiều hướng nhìn khác nhau, tùy thuộc vào mục tiêu sử dụng. Trong UML thế hệ 2 (UML 2.x), có khoảng 13 đến 14 loại biểu đồ được tiêu chuẩn hóa. Tuy nhiên, trong thực tế phát triển phần mềm, không nhất thiết phải sử dụng tất cả các biểu đồ này mà chỉ chọn những biểu đồ phù hợp và cần thiết nhất cho dự án cụ thể.

Các biểu đồ trong UML được chia thành hai nhóm chính.

- Thứ nhất là biểu đồ cấu trúc (structural diagrams), dùng để mô tả các thành phần tĩnh, luôn tồn tại trong hệ thống và mối quan hệ giữa chúng, chẳng hạn như biểu đồ lớp (Class Diagram), biểu đồ thành phần (Component Diagram) hay biểu đồ triển khai (Deployment Diagram).
- Thứ hai là biểu đồ hành vi (behavioral diagrams), dùng để mô tả cách hệ thống vận hành theo thời gian, phản ánh hành vi và tương tác giữa các đối tượng, chẳng hạn như biểu đồ hoạt động (Activity Diagram), biểu đồ trạng thái (State Machine Diagram), hay biểu đồ trình tự (Sequence Diagram).

Việc lựa chọn và sử dụng biểu đồ một cách hợp lý giúp tăng hiệu quả trong phân tích, thiết kế và truyền đạt ý tưởng giữa các thành viên trong nhóm phát triển cũng như với khách hàng.

Biểu đồ cấu trúc trong UML là nhóm các biểu đồ dùng để mô tả các thành phần tĩnh của hệ thống, tức là những phần không thay đổi trong suốt thời gian thực thi chương trình. Các biểu đồ này thể hiện cấu trúc logic hoặc vật lý của hệ thống phần mềm, cũng như mối quan hệ giữa các thành phần với nhau.

Biểu đồ cấu trúc bao gồm ba nhóm chính.

- Thứ nhất là biểu đồ cấu trúc tĩnh, gồm các biểu đồ như: biểu đồ lớp (Class Diagram) – biểu diễn các lớp và mối quan hệ giữa chúng; biểu đồ đối tượng (Object Diagram) – mô tả các thể hiện (instance) của lớp tại một thời điểm cụ thể; và biểu đồ gói (Package Diagram) – tổ chức các lớp và thành phần trong các gói (package) để quản lý tốt hơn.

- Thứ hai là biểu đồ thực thi, bao gồm: biểu đồ thành phần (Component Diagram) – mô tả cấu trúc các module phần mềm độc lập và mối liên kết giữa chúng; biểu đồ triển khai (Deployment Diagram) – biểu diễn sự phân bố của phần mềm trên phần cứng (máy chủ, thiết bị); và biểu đồ cấu thành (Composite Structure Diagram) – thể hiện cấu trúc bên trong của một lớp hoặc một thành phần, cũng như sự cộng tác giữa các phần tử cấu thành.
- Cuối cùng là biểu đồ profile (Profile Diagram), dùng để mở rộng ngôn ngữ UML thông qua các cơ chế như stereotype, tagged value và constraint. Loại biểu đồ này rất hữu ích trong việc tùy biến UML cho các miền ứng dụng cụ thể.

Việc sử dụng đúng loại biểu đồ cấu trúc giúp hệ thống được thiết kế rõ ràng, logic và dễ quản lý trong suốt quá trình phát triển phần mềm.

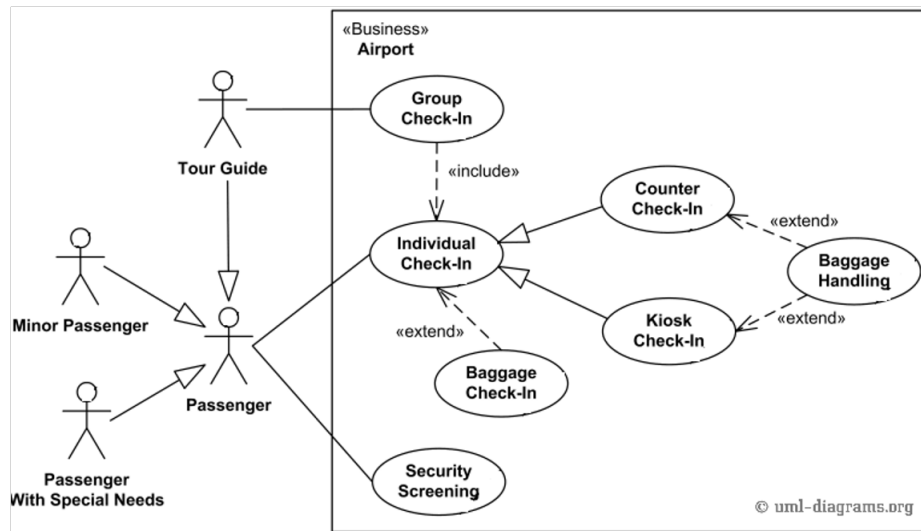
Biểu đồ hành vi trong UML là nhóm các biểu đồ dùng để mô tả cách hệ thống vận hành theo thời gian, tức là thể hiện hành vi động (dynamic behavior) của các thành phần trong hệ thống. Những biểu đồ này phản ánh cách hệ thống phản ứng lại với các sự kiện bên ngoài, cách các đối tượng tương tác với nhau và cách các quá trình bên trong được thực hiện.

- Một trong những biểu đồ hành vi quan trọng là biểu đồ use case (Use Case Diagram), dùng để mô tả các chức năng chính mà hệ thống cung cấp cho người dùng hoặc các hệ thống khác, đồng thời xác định các tác nhân (actor) tương tác với hệ thống.
- Tiếp theo là biểu đồ hoạt động (Activity Diagram), biểu diễn luồng công việc hoặc các bước xử lý nghiệp vụ theo trình tự logic, thường được sử dụng để mô hình hóa quy trình nghiệp vụ hoặc thuật toán.
- Trong nhóm biểu đồ tương tác (Interaction Diagrams), UML cung cấp nhiều loại như: biểu đồ tổng quát (Interaction Overview Diagram) – cho phép mô tả tổng thể luồng tương tác ở mức cao; biểu đồ trình tự (Sequence Diagram) – thể hiện thứ tự gửi và nhận thông điệp giữa các đối tượng theo trục thời gian; và biểu đồ giao tiếp hoặc cộng tác (Communication/Collaboration Diagram) – tập trung vào mối quan hệ và trao đổi thông điệp giữa các đối tượng trong quá trình tương tác.
- Ngoài ra, biểu đồ trạng thái (State Diagram) mô tả các trạng thái khác nhau của một đối tượng trong suốt vòng đời của nó và các sự kiện khiến đối tượng chuyển từ trạng thái này sang trạng thái khác.

- Cuối cùng là biểu đồ thời gian (Timing Diagram), dùng để mô tả sự thay đổi trạng thái hoặc giá trị của một hoặc nhiều đối tượng theo thời gian, rất hữu ích trong các hệ thống thời gian thực hoặc nhúng.

Nhìn chung, các biểu đồ hành vi giúp nhà phát triển hiểu rõ hơn về cách hệ thống vận hành, tương tác và phản ứng, từ đó hỗ trợ quá trình phân tích, thiết kế và triển khai phần mềm một cách chính xác và hiệu quả hơn.

Biểu đồ use case: Mô hình chức năng hệ thống với các tác nhân và use case. Ví dụ:



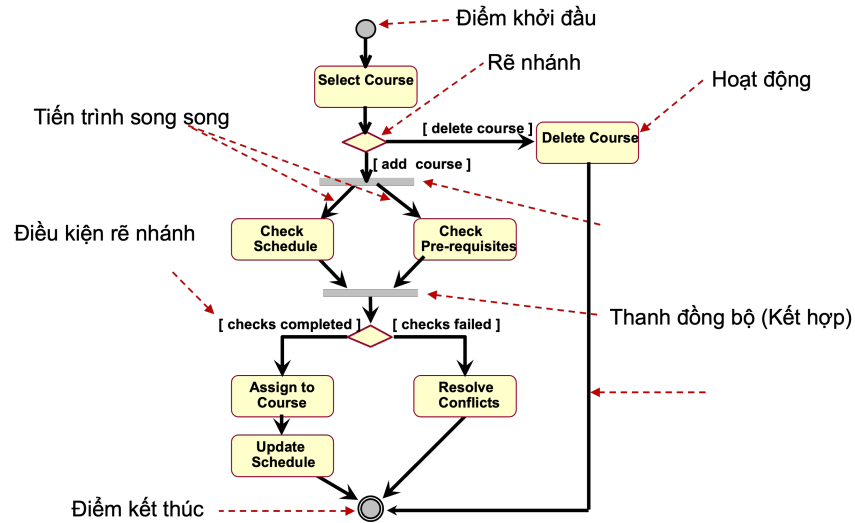
Hình 2.4 Biểu đồ use case.

Biểu đồ hoạt động: Biểu đồ hoạt động biểu diễn chuỗi các hoạt động hoặc luồng điều khiển có thứ tự của hệ thống thực hiện trong một use case. Ví dụ:

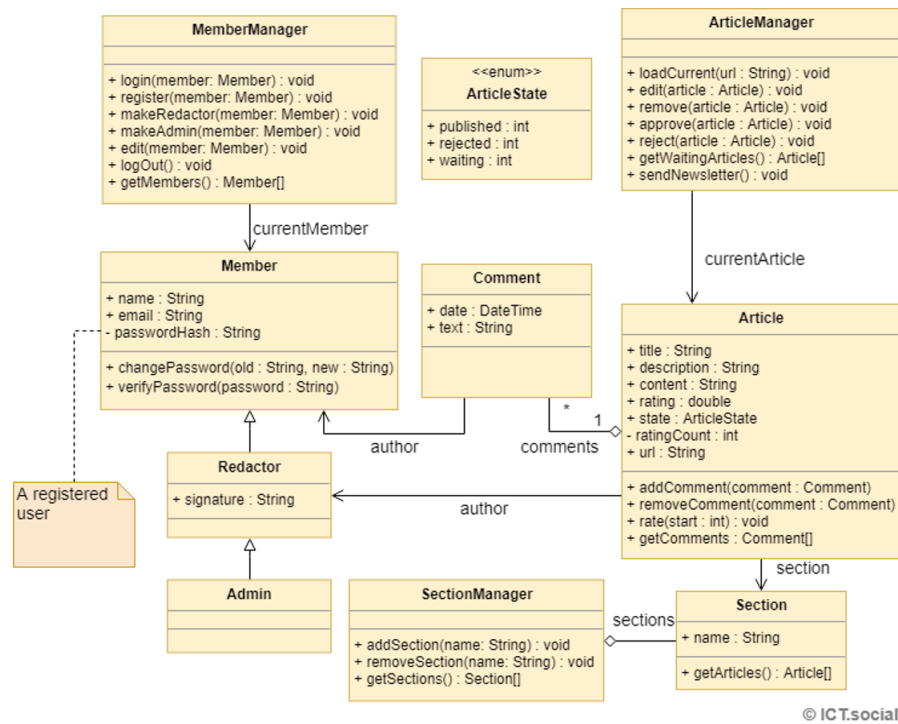
Biểu đồ lớp: Dạng biểu đồ phổ biến nhất, mô tả cấu trúc tĩnh của hệ thống. Biểu diễn các lớp và mối quan hệ giữa các lớp

Biểu đồ tuần tự là một loại biểu đồ tương tác, biểu diễn trình tự trao đổi thông điệp giữa các đối tượng theo thời gian trong một use case.

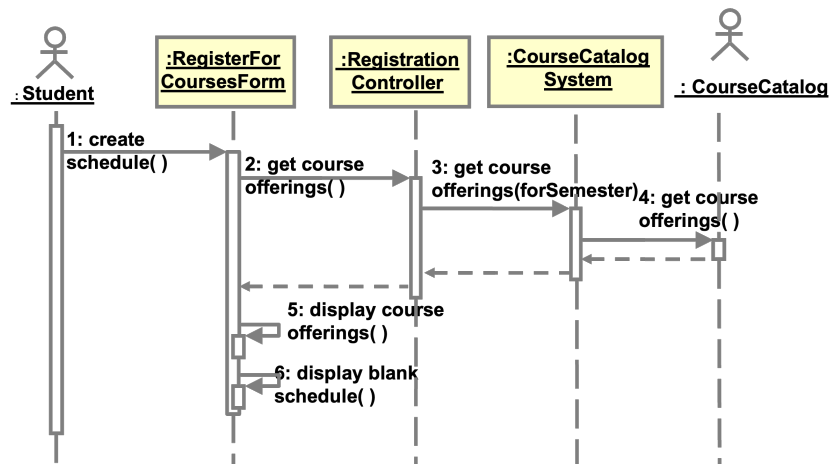
Biểu đồ giao tiếp: Cũng là một loại biểu đồ tương tác, nhưng nhấn mạnh vào việc tổ chức các đối tượng tham gia vào tương tác hơn là trình tự trao đổi thông điệp giữa các đối tượng



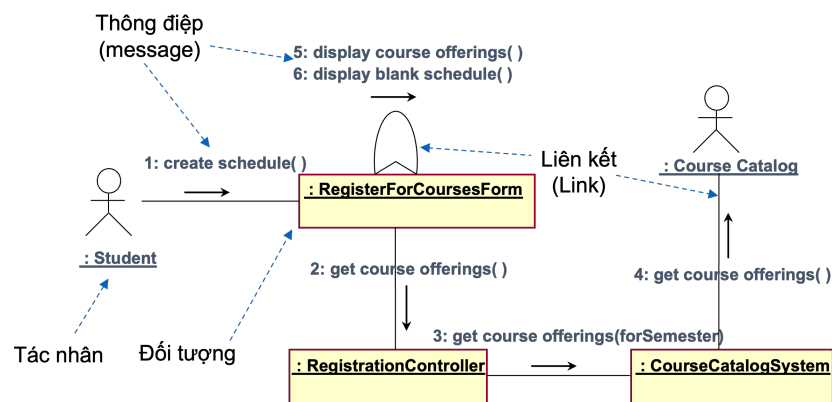
Hình 2.5 Biểu đồ use case.



Hình 2.6 Biểu đồ use case.



Hình 2.7 Biểu đồ tuần tự.



Hình 2.8 Biểu đồ tuần tự.

Chương 3

Trừu tượng hóa và đóng gói

3.1. Trừu tượng hóa	46
3.2. Lớp và Đối tượng	47
3.3. Đóng gói và Che giấu Dữ liệu	48
3.4. Quản lý Lớp với Package	50
3.5. Xây dựng Lớp trong Java	53
3.6. Chỉ định Truy cập cho Thành viên của Lớp	57
3.7. Phương thức getter và setter	61
3.8. Biểu diễn Đối tượng, Lớp, Gói trong UML	62
3.9. Kết luận	63
3.10. Bài tập	63
3.11. Lời giải bài tập chương 3	63

Trong chương này, chúng ta sẽ tìm hiểu về hai khái niệm nền tảng của lập trình hướng đối tượng: trừu tượng hóa và đóng gói. Chương sẽ trình bày bản chất và vai trò của trừu tượng hóa, cách xây dựng lớp và đối tượng trong Java, khái niệm đóng gói cùng với cơ chế che giấu dữ liệu, cũng như cách quản lý lớp bằng package và biểu diễn chúng trong UML. Các ví dụ minh họa và đoạn mã nguồn sẽ được cung cấp để làm rõ các khái niệm này.

3.1. Trừu tượng hóa

Trừu tượng hóa là một trong bốn nguyên lý cơ bản của lập trình hướng đối tượng, đóng vai trò quan trọng trong việc đơn giản hóa các hệ thống phức tạp. Đây là quá trình loại bỏ những thông tin không cần thiết và chỉ giữ lại những thông tin quan trọng, phù hợp với ngữ cảnh cụ thể của bài toán. Nhờ trừu tượng hóa, lập trình viên có thể tập trung vào logic chính của chương trình mà không bị phân tâm bởi các chi tiết kỹ thuật không liên quan. Có hai loại trừu tượng hóa chính: trừu tượng hóa điều khiển và trừu tượng hóa dữ liệu.

3.1.1. Trừu tượng hóa điều khiển

Trừu tượng hóa điều khiển liên quan đến việc sử dụng các chương trình con (subprogram) và luồng điều khiển (control flow) để ẩn đi các chi tiết phức tạp trong quá trình thực thi mã máy. Ví dụ, với câu lệnh đơn giản $a = (1 + 2) * 5$, lập trình viên không cần phải quan tâm đến các bước xử lý mức thấp như chuyển đổi sang dạng nhị phân, thao tác trên các thanh ghi, hay lưu trữ giá trị vào bộ nhớ. Nếu không có trừu tượng hóa điều khiển, lập trình viên sẽ phải chỉ định chi tiết từng bước này mỗi khi thực hiện một phép toán, dẫn đến việc lập trình trở nên cực kỳ phức tạp và phụ thuộc vào phần cứng cụ thể. Trừu tượng hóa điều khiển giúp giảm thiểu sự lặp lại và tăng tính trừu tượng, cho phép lập trình viên tập trung vào ý nghĩa của chương trình thay vì cách thức thực hiện.

3.1.2. Trừu tượng hóa dữ liệu

Trừu tượng hóa dữ liệu cho phép tạo ra các kiểu dữ liệu trừu tượng, tách biệt giữa các đặc tính trừu tượng (giao diện) và chi tiết thực thi cụ thể. Điều này giúp che giấu cách thức triển khai bên trong khỏi người dùng, đồng thời cho phép thay đổi cách thực hiện mà không ảnh hưởng đến mã khách hàng. Chẳng hạn, một kiểu dữ liệu trừu tượng như bảng tra cứu (*lookup table*) có thể được triển khai bằng nhiều cách khác nhau như bảng băm (*hash table*), cây tìm kiếm nhị phân (*binary search tree*), hoặc danh sách tuyến tính (*linear list*). Người dùng chỉ cần biết giao diện của kiểu dữ liệu này (ví dụ: cung cấp một khóa để nhận giá trị tương ứng) mà không cần quan tâm đến cách nó được thực hiện bên trong.

Ví dụ, trong thực tế, khi mô tả một chiếc điện thoại Nokia, tùy thuộc vào bài toán, chúng ta có thể chọn lọc các thuộc tính khác nhau để trừu tượng hóa. Nếu bài toán liên quan đến công việc, ta có thể quan tâm đến các tính năng như email hoặc lịch làm việc; nếu liên quan đến giải trí, ta có thể tập trung vào khả năng chơi nhạc hoặc trò chơi. Như vậy, trừu tượng hóa dữ liệu giúp định hình các đặc điểm quan trọng của thực thể trong một ngữ cảnh cụ thể, từ đó dẫn đến khái niệm lớp trong lập trình hướng đối tượng.

3.2. Lớp và Đối tượng

3.2.1. Khái niệm lớp

Lớp (*class*) là kết quả của quá trình trừu tượng hóa dữ liệu, được sử dụng để phân loại các đối tượng dựa trên những đặc điểm chung. Lớp định nghĩa một kiểu dữ liệu mới, bao gồm các thuộc tính (dữ liệu) và phương thức (hành vi) mà các đối tượng thuộc lớp đó sở hữu. Mỗi đối tượng là một thể hiện (*instance*) cụ thể của lớp,

mang bản sao riêng của các thuộc tính và có khả năng thực hiện các phương thức được định nghĩa trong lớp.

Ví dụ, lớp **Car** có thể bao gồm các thuộc tính như *màu sắc* và *vận tốc*, cùng với các phương thức như *tăng tốc* và *giảm tốc*. Một đối tượng cụ thể của lớp **Car** có thể có giá trị *màu sắc* = *đen* và *vận tốc* = *60 km/h*, đồng thời có thể thực hiện hành vi *tăng tốc* để thay đổi trạng thái *vận tốc* của nó.

3.2.2. So sánh lớp và đối tượng

Lớp và đối tượng có mối quan hệ chặt chẽ nhưng khác biệt rõ ràng. **Lớp** là một mô hình khái niệm, đóng vai trò như bản mẫu định nghĩa các thuộc tính và phương thức chung của các đối tượng. Nó mang tính trừu tượng, đại diện cho một tập hợp các đối tượng có đặc điểm tương tự. **Đối tượng** là một thực thể cụ thể, được tạo ra từ lớp, sở hữu dữ liệu riêng biệt và có thể thực hiện các phương thức của lớp. Mỗi đối tượng duy trì bản sao riêng của các thuộc tính, và giá trị của chúng có thể khác nhau giữa các đối tượng.

Ví dụ, trong lớp **Book**, các thuộc tính có thể là *tên sách* và *số trang*, với các phương thức như *mượn sách* (*borrow*) và *đếm số trang* (*countPages*). Một đối tượng cụ thể, chẳng hạn *myBook*, có thể có giá trị *tên sách* = *"Human Computer Interaction"* và *số trang* = *250*, đồng thời thực hiện các phương thức được định nghĩa trong lớp.

3.3. Đóng gói và Che giấu Dữ liệu

3.3.1. Đóng gói

Đóng gói là một trong bốn nguyên lý cơ bản khác của lập trình hướng đối tượng, trong đó dữ liệu (thuộc tính) và hành vi (phương thức) được nhóm lại trong một đơn vị duy nhất là lớp. Mục đích của đóng gói là cung cấp một tập hợp các dịch vụ cụ thể cho bên ngoài, đồng thời ẩn đi các chi tiết triển khai bên trong. Một đối tượng được đóng gói có thể được xem như một "hộp đen", trong đó người dùng chỉ cần biết đầu vào và đầu ra mà không cần hiểu cách thức hoạt động bên trong. Điều này cho phép thay đổi thiết kế hoặc mã nguồn bên trong mà không làm ảnh hưởng đến giao diện bên ngoài.

Ví dụ, một máy pha cà phê có thể được trừu tượng hóa với hai phương thức chính: *nhận cà phê vào* và *pha chế cà phê*. Trong quá trình triển khai, phương thức *pha chế cà phê* có thể gọi các phương thức phụ trợ (chẳng hạn như *đun nước*, *xay hạt cà phê*, hoặc *lọc cà phê*) được khai báo với phạm vi truy cập **private**, do đó ẩn khỏi người dùng bên ngoài. Đóng gói được thể hiện qua việc nhóm các thuộc tính

(như *lượng cà phê*, *nhiệt độ nước*) và các phương thức này vào trong một lớp duy nhất, ví dụ `CoffeeMachine`. Người dùng chỉ cần tương tác với các phương thức công khai (`public`) như *nhận cà phê vào* và *pha chế cà phê*, mà không cần biết chi tiết cách thức các phương thức phụ trợ hoạt động hoặc cách dữ liệu được quản lý bên trong lớp. Nhờ đóng gói, các chi tiết triển khai có thể được thay đổi (ví dụ, cải tiến thuật toán pha chế) mà không ảnh hưởng đến giao diện bên ngoài, qua đó tăng tính bảo trì và khả năng tái sử dụng của mã nguồn.

```
public class CoffeeMachine {
    private double coffeeAmount; // Thuộc tính ẩn
    private double waterTemperature;

    // Phương thức công khai
    public void addCoffee(double amount) {
        this.coffeeAmount = amount;
    }

    public void brewCoffee() {
        heatWater();
        grindBeans();
        filterCoffee();
        // Kết quả: cà phê được pha chế
    }

    // Các phương thức phụ trợ ẩn
    private void heatWater() {
        waterTemperature = 95.0; // Đun nước đến 95 độ C
    }

    private void grindBeans() {
        // Xay hạt cà phê
    }

    private void filterCoffee() {
        // Lọc cà phê
    }
}
```

3.3.2. Che giấu dữ liệu

Che giấu dữ liệu là một khía cạnh quan trọng của đóng gói, nhằm bảo vệ dữ liệu khỏi sự truy cập hoặc chỉnh sửa trái phép. Trong Java, điều này được thực hiện bằng cách khai báo các thuộc tính của lớp ở chế độ **private**, chỉ cho phép truy cập thông qua các phương thức công khai (**public**) như *getter* và *setter*. Nhờ đó, lập trình viên có thể kiểm soát việc truy cập và đảm bảo tính hợp lệ của dữ liệu.

Ví dụ, trong một lớp **Student**, thuộc tính **name** được khai báo là **private** và chỉ có thể được truy cập hoặc thay đổi thông qua các phương thức **getName** và **setName**. Điều này ngăn chặn việc thay đổi trực tiếp giá trị của **name** từ bên ngoài, đảm bảo rằng mọi thay đổi đều được kiểm soát.

```
class Student {  
    private String name;  
    public String getName() {  
        return this.name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

3.3.3. Phân biệt trừu tượng hóa và đóng gói

Mặc dù cả trừu tượng hóa và đóng gói đều liên quan đến việc ẩn thông tin, chúng khác nhau về mục đích và mức độ. **Trừu tượng hóa** tập trung vào việc ẩn chi tiết ở mức thiết kế, xác định những thuộc tính và phương thức cần thiết cho bài toán mà không quan tâm đến cách triển khai. Ví dụ, khi lái xe, người lái chỉ cần biết cách đạp ga mà không cần hiểu động cơ hoạt động ra sao. **Đóng gói** tập trung vào việc ẩn chi tiết ở mức triển khai, bảo vệ dữ liệu và phương thức bên trong lớp, chỉ cung cấp giao diện cần thiết cho bên ngoài. Ví dụ, các phương thức phụ trợ trong quá trình pha cà phê được ẩn đi để đơn giản hóa việc sử dụng.

3.4. Quản lý Lớp với Package

Package trong Java là cơ chế nhóm các lớp có liên quan logic với nhau, giúp tổ chức mã nguồn hiệu quả và tránh xung đột tên. Package được khai báo bằng từ khóa **package** ở đầu tệp mã nguồn, ví dụ:

```
package oop.cnpm;  
class Student {  
    // Nội dung lớp  
}
```

Package còn gọi là không gian tên (*namespace*) trong một số ngôn ngữ lập trình khác (C#, C++), cho phép các lớp có cùng tên tồn tại trong các package khác nhau mà không gây xung đột. Ví dụ, hai lớp **Student** trong package `oop.cnpm` và package `oop.hust` là hoàn toàn riêng biệt.

Trong lập trình Java, gói (*package*) đóng vai trò như một thư mục trong hệ thống tệp, cung cấp một cơ chế tổ chức và quản lý các lớp (*class*), giao diện (*interface*), và các thành phần khác một cách hiệu quả. Package giúp định vị các lớp một cách rõ ràng, hỗ trợ việc sử dụng chúng một cách hợp lý trong các dự án phần mềm lớn, đồng thời đảm bảo tính cấu trúc và dễ bảo trì của mã nguồn. Một trong những lợi ích quan trọng của package là tránh xung đột tên lớp (*name collision*), một vấn đề phổ biến khi nhiều lớp có cùng tên được định nghĩa trong các ngữ cảnh khác nhau. Nhờ đó, các package khác nhau có thể chứa các lớp mang cùng tên mà không gây nhầm lẫn, miễn là chúng được đặt trong các không gian tên riêng biệt. Ngoài ra, package còn cung cấp một lớp bảo vệ bổ sung cho các lớp, dữ liệu, và phương thức, giới hạn quyền truy cập ở mức rộng hơn so với mối quan hệ trực tiếp giữa các lớp, từ đó tăng cường tính an toàn và kiểm soát trong mã nguồn.

Java cung cấp một số package có sẵn trong thư viện chuẩn (Java Standard Library), được thiết kế để hỗ trợ các chức năng cơ bản và nâng cao. Một số package tiêu biểu bao gồm: (1) `java.lang`: Chứa các lớp cơ bản như **String**, **Object**, và **System**, được tự động nhập (import) vào mọi chương trình Java mà không cần khai báo. (2) `java.io`: Cung cấp các lớp và phương thức để thực hiện thao tác nhập/xuất dữ liệu, chẳng hạn như **FileReader** và **BufferedWriter**. (3) `javax.swing`: Hỗ trợ phát triển giao diện người dùng đồ họa (GUI) với các thành phần như **JFrame** và **JButton**.

Bên cạnh các package có sẵn, lập trình viên có thể tự định nghĩa package theo nhu cầu của dự án. Điều này cho phép tạo ra các cấu trúc tổ chức mã nguồn tùy chỉnh, phù hợp với quy mô và mục đích sử dụng của ứng dụng.

Tên của package được xây dựng bằng cách sử dụng các ký tự thường, tuân theo quy ước đặt tên trong Java để đảm bảo tính nhất quán và dễ đọc. Các thành phần của tên package được phân cách bằng dấu chấm (.), ví dụ: `com.example.myapp`. Quy tắc này không chỉ giúp phân biệt các package mà còn phản ánh cấu trúc phân cấp, trong đó một package có thể chứa các package con (sub-packages). Chẳng hạn,

package `com.example.myapp.utils` có thể chứa các lớp tiện ích nằm trong package cha `com.example.myapp`. Trong mã nguồn, tương ứng ta sẽ có sự phân cấp cấu trúc thư mục, với thư mục gốc là thư mục `com`, trong thư `com` lại có thư mục `example`, và trong thư mục `example` lại chứa thư mục `myapp`, ...

Khai báo package là một bước bắt buộc trong mọi tệp mã nguồn Java, và câu lệnh này phải được đặt ở đầu tệp, trước mọi khai báo khác. Cú pháp cơ bản là:

```
package tenpackage;
```

Một tệp mã nguồn chỉ được phép chứa duy nhất một câu lệnh `package`, và khai báo này sẽ áp dụng cho toàn bộ các lớp, giao diện, hoặc thành phần khác được định nghĩa trong tệp đó. Điều này đảm bảo rằng tất cả các thành phần trong cùng một tệp thuộc về cùng một không gian tên, giúp duy trì tính nhất quán. Ví dụ, trong tệp `Student.java`, khai báo sau sẽ xác định rằng lớp `Student` thuộc package `oop.cnpm`.

```
package oop.cnpm;
```

```
public class Student {
    // Nội dung lớp
}
```

Package trong Java hỗ trợ cấu trúc phân cấp, cho phép một package có thể chứa các package con. Mối quan hệ này được biểu thị bằng dấu chấm, chẳng hạn như `org.apache.commons.math`. Trong trường hợp này, `org.apache.commons` là package cha, và `math` là package con. Cấu trúc này không chỉ giúp tổ chức mã nguồn theo cách logic mà còn cho phép tái sử dụng và mở rộng dễ dàng. Lưu ý, dù có quan hệ phân cấp, các lớp trong package con vẫn không thể truy cập các thành phần `default` của package cha.

Sử dụng `package` mang lại nhiều lợi ích thực tế trong phát triển phần mềm:

- **Tổ chức mã nguồn:** *Package* giúp chia nhỏ dự án thành các mô-đun rõ ràng, chẳng hạn như *package* cho giao diện người dùng, logic nghiệp vụ, và truy cập dữ liệu.
- **Tránh xung đột tên:** Khi làm việc trong các dự án lớn hoặc tích hợp mã từ nhiều nguồn, *package* đảm bảo rằng các lớp như `Student` trong `edu.university` và `Student` trong `com.company` không bị nhầm lẫn.
- **Kiểm soát truy cập:** Chỉ định `public` hoặc `default` của *package* cho phép giới hạn phạm vi truy cập, tăng cường tính bảo mật và đóng gói.

- **Quản lý phụ thuộc:** *Package* hỗ trợ việc quản lý các thư viện bên thứ ba hoặc mã nguồn nội bộ một cách có hệ thống.

Tóm lại, package là một công cụ quan trọng trong lập trình Java, không chỉ giúp tổ chức mã nguồn mà còn đóng vai trò như một lớp bảo vệ và cơ chế phân giải xung đột tên. Việc hiểu và áp dụng đúng quy tắc khai báo và đặt tên package sẽ giúp lập trình viên xây dựng các ứng dụng có cấu trúc rõ ràng, dễ bảo trì và mở rộng. Với sự linh hoạt trong việc tạo package tùy chỉnh và sử dụng các package có sẵn, Java cung cấp một nền tảng mạnh mẽ để phát triển phần mềm chuyên nghiệp.

3.5. Xây dựng Lớp trong Java

Trong Java, lớp được khai báo bằng từ khóa `class`, theo sau là tên lớp và thân lớp chứa các thuộc tính và phương thức. Quy mô sử dụng lớp được kiểm soát bởi **chỉ định truy cập của lớp**, với 2 loại trong Java: `public` và `default` (không có từ khóa chỉ định). Cụ thể, chỉ định `public` cho phép lớp được truy cập và sử dụng từ bất kỳ đâu, kể cả bên ngoài package chứa lớp đó, trong khi chỉ định `default` (mặc định) giới hạn quyền truy cập chỉ trong cùng package. Ví dụ, trong gói `oop.cnpm`, lớp `BankAccount` được khai báo với chỉ định `public` sẽ có thể được sử dụng từ các package khác, trong khi lớp `Student` không có chỉ định sẽ chỉ được sử dụng trong package chứa nó.

```
package oop.cnpm;  
public class BankAccount {  
}
```

```
package oop.cnpm;  
class BankAccount {  
}
```

Trong lập trình hướng đối tượng, thuộc tính và phương thức là hai thành phần cốt lõi của một lớp, đóng vai trò định nghĩa trạng thái và hành vi của các đối tượng. Việc khai báo chúng cần tuân thủ các quy tắc cú pháp cụ thể, bao gồm chỉ định truy cập và kiểu dữ liệu, để đảm bảo tính nhất quán và khả năng tái sử dụng.

Khai báo Thuộc tính

Thuộc tính của một lớp đại diện cho trạng thái của các đối tượng thuộc lớp đó, tương tự như biến trong lập trình. Cú pháp khai báo thuộc tính trong **Java** giống

với khai báo biến, bao gồm:

- **Chỉ định truy cập** (*access modifier*): Xác định phạm vi truy cập của thuộc tính, chẳng hạn như `public`, `private`, hoặc mặc định (không có chỉ định).
- **Kiểu dữ liệu** (*type*): Xác định loại dữ liệu mà thuộc tính có thể lưu trữ, ví dụ: `String`, `int`, `double`.
- **Tên thuộc tính** (*name*): Tên duy nhất của thuộc tính, thường bắt đầu bằng chữ cái thường theo quy ước đặt tên.

Thuộc tính có thể được khởi tạo giá trị mặc định khi khai báo, và nếu không được khởi tạo, Java sẽ tự động gán giá trị mặc định (ví dụ: 0 cho `int`, `null` cho đối tượng). Ví dụ, trong lớp `BankAccount`, các thuộc tính được khai báo như sau:

```
package com.megabank.models;

public class BankAccount {
    private String owner; // Thuộc tính không khởi tạo, mặc định là null
    private double balance = 0.0; // Thuộc tính được khởi tạo với giá trị 0.0
}
```

Trong đó, `private` đảm bảo rằng các thuộc tính này chỉ có thể được truy cập hoặc thay đổi thông qua các phương thức công khai của lớp, thể hiện nguyên lý đóng gói và che giấu dữ liệu (*encapsulation*).

Khai báo Phương thức

Phương thức định nghĩa hành vi của các đối tượng, tương tự như hàm trong lập trình thủ tục. Cú pháp khai báo phương thức bao gồm:

- **Chỉ định truy cập** (*access modifier*): Như `public`, `private`, hoặc mặc định, quyết định phạm vi truy cập của phương thức.
- **Kiểu trả về** (*return type*): Xác định loại dữ liệu mà phương thức trả về, chẳng hạn `void` (không trả về), `boolean`, hoặc `double`. Nếu phương thức trả về giá trị, cần có lệnh `return` để cung cấp kết quả.
- **Tên phương thức** (*method name*): Tên duy nhất, thường là động từ, theo quy ước bắt đầu bằng chữ cái thường.

- **Danh sách tham số** (*parameter list*): Các tham số được truyền vào phương thức, bao gồm kiểu dữ liệu và tên tham số, được đặt trong dấu ngoặc đơn.

Phương thức được khai báo trong thân lớp, và nội dung của nó được viết trong khối mã `{...}`. Ví dụ, trong lớp `BankAccount`, phương thức `debit` được khai báo như sau:

```
public boolean debit(double amount) {  
    if (amount > balance)  
        return false;  
    else {  
        balance -= amount;  
        return true;  
    }  
}
```

Chữ ký phương thức (Signature) là sự kết hợp duy nhất của tên phương thức và danh sách tham số (bao gồm số lượng và kiểu dữ liệu của các tham số), không bao gồm kiểu trả về. Chữ ký này giúp phân biệt các phương thức, đặc biệt trong trường hợp nạp chồng phương thức (*method overloading*). Ví dụ, chữ ký của `debit(double amount)` là `debit(double)`, và nó không trùng với một phương thức khác như `debit(int amount)`.

Lệnh `return` được sử dụng để trả về giá trị từ phương thức và chuyển quyền điều khiển trở lại cho đối tượng gọi phương thức. Có hai trường hợp chính:

- **Phương thức trả về giá trị**: Phải có ít nhất một lệnh `return` với giá trị phù hợp với kiểu trả về đã khai báo. Ví dụ, trong `debit`, lệnh `return true` hoặc `return false` được sử dụng để báo hiệu trạng thái giao dịch.
- **Phương thức không trả về giá trị (phương thức void)**: Không yêu cầu lệnh `return`, nhưng có thể sử dụng `return;` để thoát phương thức sớm nếu cần. Ví dụ, phương thức `credit` không trả về giá trị.

```
public void credit(double amount) {  
    balance += amount;  
    // Không cần return vì kiểu là void  
}
```

Nếu một phương thức có kiểu trả về khác `void`, việc thiếu lệnh `return` hoặc trả về giá trị không phù hợp sẽ gây lỗi biên dịch. Ngoài ra, trong một phương thức, chỉ

lệnh `return` đầu tiên được thực thi sẽ có hiệu lực, và các lệnh `return` sau đó sẽ không được thực hiện.

Ví dụ minh họa

Dưới đây là một ví dụ hoàn chỉnh kết hợp khai báo thuộc tính và phương thức trong lớp `BankAccount`:

```
package com.megabank.models;

public class BankAccount {
    private String owner;
    private double balance = 0.0;

    public boolean debit(double amount) {
        if (amount > balance) {
            return false; // Trả về false nếu số tiền rút vượt quá số dư
        } else {
            balance -= amount;
            return true; // Trả về true nếu giao dịch thành công
        }
    }

    public void credit(double amount) {
        if (amount > 0) {
            balance += amount; // Cộng tiền vào số dư
        }
    }

    public double getBalance() {
        return balance; // Trả về số dư hiện tại
    }
}
```

Trong ví dụ này, thuộc tính `owner` và `balance` được khai báo với chỉ định `private`, trong khi các phương thức `debit`, `credit`, và `getBalance` được khai báo `public` để cung cấp giao diện cho các thao tác trên tài khoản ngân hàng.

Tóm lại, việc hiểu rõ cách khai báo thuộc tính và phương thức là nền tảng để xây dựng các lớp hiệu quả trong Java. Chỉ định truy cập, kiểu dữ liệu, và cấu trúc cú

pháp cần được áp dụng chính xác để đảm bảo tính bảo mật, tính nhất quán, và khả năng mở rộng của mã nguồn. Các ví dụ trên minh họa cách kết hợp các khái niệm này để tạo ra một lớp hoạt động đúng chức năng, đồng thời tuân thủ các nguyên lý cơ bản của lập trình hướng đối tượng.

3.6. Chỉ định Truy cập cho Thành viên của Lớp

Trong lập trình hướng đối tượng, chỉ định truy cập (*access modifier*) là một cơ chế quan trọng để kiểm soát khả năng tiếp cận các thành viên của lớp, bao gồm thuộc tính (*fields* hoặc *attributes*) và phương thức (*methods*). Các chỉ định này giúp thực hiện nguyên lý đóng gói (*encapsulation*) và che giấu dữ liệu (*information hiding*), đảm bảo rằng các phần bên trong lớp chỉ được truy cập một cách hợp lý, tránh sửa đổi trái phép hoặc làm sai lệch dữ liệu. Java cung cấp bốn chỉ định truy cập chính: `public`, `protected`, `private`, và mặc định (`default` hoặc `package-private`, khi không có chỉ định nào). Tuy nhiên, chỉ định `protected` sẽ được học chi tiết trong bài học về kế thừa (*inheritance*) sau, vì nó liên quan đến mối quan hệ giữa lớp cha và lớp con. Trong phần này, chúng ta tập trung vào ba chỉ định: `public`, mặc định (không chỉ định), và `private`.

Ý nghĩa của Các Chỉ định Truy cập như sau:

- **public:** Cho phép thành viên (thuộc tính hoặc phương thức) được truy cập từ bất kỳ đâu trong chương trình, bao gồm cả các lớp khác gói (*package*) hoặc thậm chí từ các gói khác. Đây là mức truy cập rộng nhất, thường được sử dụng cho các phương thức công khai (*public API*) mà lớp cung cấp cho bên ngoài. Tuy nhiên, nên sử dụng cẩn thận để tránh lộ dữ liệu nhạy cảm.
- **default (mặc định/không chỉ định):** Thành viên chỉ có thể được truy cập từ các lớp nằm trong cùng một gói (*package*) với lớp chứa thành viên đó. Nếu không có từ khóa chỉ định nào, đây là mức mặc định. Nó cung cấp mức bảo vệ ở cấp độ gói, phù hợp cho các thành viên chỉ cần chia sẻ nội bộ trong một mô-đun phần mềm.
- **private:** Thành viên chỉ có thể được truy cập từ bên trong chính lớp chứa nó. Không lớp nào khác, kể cả lớp trong cùng gói hoặc lớp con, có thể truy cập trực tiếp. Đây là mức truy cập hạn chế nhất, thường dùng để che giấu dữ liệu và chỉ cho phép truy cập qua các phương thức getter/setter.
- **protected:** Thành viên được truy cập trong cùng lớp, cùng gói, và từ lớp con kế thừa (kể cả ở gói khác), nhưng chúng ta sẽ không đi sâu ở đây vì nó sẽ được trình bày trong bài học về thừa kế.

3.6.1. Bảng So sánh Khả năng Truy cập

Dưới đây là bảng tóm tắt khả năng truy cập của các thành viên lớp theo các chỉ định, dựa trên ba ngữ cảnh (cùng lớp, cùng gói, khác gói). Bảng này giúp hình dung rõ ràng sự khác biệt giữa các mức truy cập. Theo đó:

- **Cùng lớp:** Tất cả các chỉ định đều cho phép truy cập từ bên trong chính lớp chứa thành viên.
- **Cùng gói:** `public` và mặc định cho phép truy cập từ các lớp khác trong cùng gói, nhưng `private` không cho phép.
- **Khác gói:** Chỉ `public` cho phép truy cập từ các lớp ở gói khác; mặc định và `private` đều chặn truy cập này.

	public	Không có	private
Cùng lớp	Yes	Yes	Yes
Cùng gói	Yes	Yes	No
Khác gói	Yes	No	No

3.6.2. Ví dụ Minh họa

Hãy xem xét một lớp `BankAccount` trong gói `com.example.banking`. Chúng ta sẽ khai báo các thuộc tính và phương thức với các chỉ định truy cập khác nhau để minh họa cách chúng hoạt động.

```
package com.example.banking; // Gói chứa lớp

public class BankAccount {
    // Thuộc tính với chỉ định public: Có thể truy cập từ bất kỳ đâu
    public String accountNumber;

    // Thuộc tính với chỉ định default (không chỉ định): Chỉ truy cập trong cùng
    double balance;

    // Thuộc tính với chỉ định private: Chỉ truy cập trong lớp này
    private String ownerName;

    // Phương thức public: Có thể gọi từ bất kỳ đâu
    public void deposit(double amount) {
        balance += amount;
    }
}
```

```
        System.out.println("Đã nạp " + amount + " vào tài khoản.");
    }

    // Phương thức default: Chỉ gọi trong cùng gói
    void checkBalance() {
        System.out.println("Số dư hiện tại: " + balance);
    }

    // Phương thức private: Chỉ gọi từ bên trong lớp này
    private void updateOwner(String newName) {
        ownerName = newName;
    }

    // Phương thức public để minh họa truy cập private (getter/setter)
    public String getOwnerName() {
        return ownerName; // Truy cập private từ bên trong lớp
    }

    public void setOwnerName(String newName) {
        updateOwner(newName); // Gọi phương thức private
    }
}
```

Trong ví dụ về lớp `BankAccount`:

- **public**: Thuộc tính `accountNumber` và phương thức `deposit` có thể được truy cập từ bất kỳ lớp nào, ví dụ từ một lớp ở gói khác như `com.example.app.Main`. Điều này hữu ích cho các tính năng công khai, nhưng nên tránh lạm dụng để không vi phạm encapsulation.
- **default**: Thuộc tính `balance` và phương thức `checkBalance` chỉ có thể được sử dụng từ các lớp khác trong gói `com.example.banking`. Ví dụ, một lớp `SavingsAccount` trong cùng gói có thể truy cập `balance`, nhưng lớp ở gói khác thì không.
- **private**: Thuộc tính `ownerName` và phương thức `updateOwner` chỉ có thể được sử dụng bên trong lớp `BankAccount`. Để truy cập từ bên ngoài, chúng ta sử dụng các phương thức công khai như `getOwnerName` và `setOwnerName` (getter/setter), giúp kiểm soát và xác thực dữ liệu (ví dụ: kiểm tra tên hợp lệ trước

khi cập nhật).

Giả sử chúng ta có một lớp khác `TestAccount` trong cùng gói (`com.example.banking`):

```
package com.example.banking;

public class TestAccount {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        // Truy cập public: OK
        account.accountNumber = "123456";
        account.deposit(1000);

        // Truy cập default: OK (cùng gói)
        account.balance = 500; // Truy cập trực tiếp (vd mang tính minh họa, không
        account.checkBalance();

        // Truy cập private: Lỗi biên dịch!
        // account.ownerName = "John Doe"; // Không cho phép
        // account.updateOwner("Jane Doe"); // Không cho phép

        // Thay vào đó, dùng getter/setter public
        account.setOwnerName("John Doe");
        System.out.println("Chủ tài khoản: " + account.getOwnerName());
    }
}
```

Nếu lớp `TestAccount` nằm ở gói khác (ví dụ: `com.example.app`), thì chỉ các thành viên `public` mới có thể truy cập; `default` và `private` sẽ gây lỗi biên dịch.

3.6.3. Lợi ích và Lưu ý

Sử dụng chỉ định truy cập đúng cách giúp tăng tính bảo mật, dễ bảo trì và tái sử dụng mã nguồn. Ví dụ, bằng cách khai báo thuộc tính là `private` và cung cấp getter/setter, bạn có thể thêm logic kiểm tra (validation) mà không thay đổi giao diện lớp. Lưu ý rằng chỉ định truy cập áp dụng cho cả thuộc tính và phương thức, nhưng không áp dụng cho lớp lồng (*nested class*) theo cách giống nhau (lớp lồng có

thể có `protected` hoặc `private`). Luôn ưu tiên mức truy cập hạn chế nhất có thể để tuân thủ nguyên lý "ít nhất cần thiết" (*principle of least privilege*), kết hợp với việc sử dụng các phương thức getter và setter trình bày trong phần tiếp theo.

3.7. Phương thức getter và setter

Để truy cập và chỉnh sửa các thuộc tính `private`, lớp cần cung cấp các phương thức *getter* (truy vấn) và *setter* (thiết lập). Các phương thức này không chỉ đảm bảo tính bảo mật mà còn cho phép kiểm tra tính hợp lệ của dữ liệu trước khi thay đổi. Ví dụ, trong lớp `NhanVien` dưới đây, phương thức `tangLuong` kiểm tra điều kiện trước khi thay đổi hệ số lương:

```
class NhanVien {
    private String tenNhanVien;
    private double luongCoBan;
    private double heSoLuong;
    public static final double LUONG_MAX = 10000000;

    public double tinhLuong() {
        return luongCoBan * heSoLuong;
    }

    public boolean tangLuong(double amount) {
        double newHeSo = heSoLuong + amount;
        if (luongCoBan * newHeSo > LUONG_MAX) {
            System.out.println("Lương vượt quá mức tối đa cho phép!");
            return false;
        } else {
            heSoLuong = newHeSo;
            return true;
        }
    }

    public void inTTin() {
        System.out.println("Tên: " + tenNhanVien + ", Lương: " + tinhLuong());
    }

    // Getter và Setter
}
```



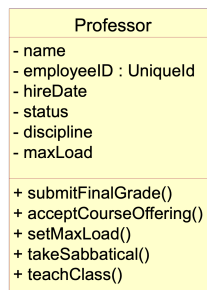
```

public String getTenNhanVien() { return tenNhanVien; }
public void setTenNhanVien(String ten) { this.tenNhanVien = ten; }
public double getLuongCoBan() { return luongCoBan; }
public void setLuongCoBan(double luong) { this.luongCoBan = luong; }
public double getHeSoLuong() { return heSoLuong; }
public void setHeSoLuong(double heSo) { this.heSoLuong = heSo; }
}

```

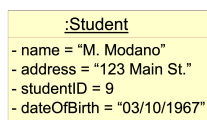
3.8. Biểu diễn Đối tượng, Lớp, Gói trong UML

UML (Unified Modeling Language) là một ngôn ngữ mô hình hóa tiêu chuẩn để mô tả các hệ thống phần mềm. Trong UML, **Lớp**: Được biểu diễn bằng một hình chữ nhật chia thành ba phần: tên lớp, thuộc tính và phương thức, như minh họa trên Hình 3.1.

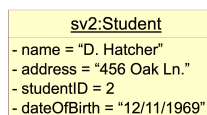


Hình 3.1 Biểu diễn của lớp trong UML

Đối tượng được biểu diễn tương tự lớp nhưng bao gồm tên đối tượng và giá trị cụ thể của các thuộc tính. Ví dụ, Hình 3.2 biểu diễn một đối tượng vô danh và Hình 3.3 biểu diễn một đối tượng có tên.

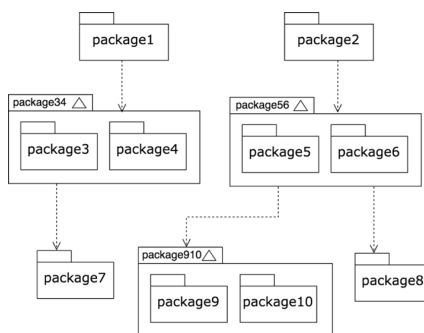


Hình 3.2 Biểu diễn của đối tượng vô danh trong UML



Hình 3.3 Biểu diễn của đối tượng có tên sv2 trong UML

Package được biểu diễn bằng một hình chữ nhật lớn chứa các lớp bên trong, thể hiện mối quan hệ tổ chức giữa các lớp, như minh họa trên Hình 3.4.



Hình 3.4 Biểu diễn gói trong UML

3.9. Kết luận

Chương này đã trình bày chi tiết về trừu tượng hóa và đóng gói – hai nguyên lý cốt lõi của lập trình hướng đối tượng. Thông qua các khái niệm, ví dụ mã nguồn và biểu diễn UML, người đọc có thể hiểu rõ cách áp dụng chúng trong Java để xây dựng các chương trình có tính bảo trì cao và dễ mở rộng. Các bài tập thực hành giúp củng cố kiến thức và kỹ năng lập trình thực tế.

3.10. Bài tập

Bài tập 3.1. Viết khai báo một gói chứa hai lớp: `HinhVuong` và `HinhTron`, cùng các thuộc tính và phương thức *getter/setter* thích hợp.

Bài tập 3.2. Viết khai báo lớp `Vector` với các phương thức cộng, trừ, nhân với hằng số và nhân vô hướng..

3.11. Lời giải bài tập chương 3

3.1 Mã nguồn lớp `HinhVuong` và `HinhTron` như sau:

```

package shapes;

public class HinhVuong {
    private double canh;
    public double tinhDienTich() { return canh * canh; }
    public double getCanh() { return canh; }
    public void setCanh(double canh) { this.canh = canh; }
}

```

```
public class HìnhTron {
    private double banKinh;
    public double tinhDienTich() { return Math.PI * banKinh * banKinh; }
    public double getBanKinh() { return banKinh; }
    public void setBanKinh(double banKinh) { this.banKinh = banKinh; }
}
```

3.2 Mã nguồn lớp Vector như sau:

```
public class Vector {
    private double x, y, z;

    public Vector(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    public Vector cong(Vector v) {
        return new Vector(x + v.x, y + v.y, z + v.z);
    }

    public Vector tru(Vector v) {
        return new Vector(x - v.x, y - v.y, z - v.z);
    }

    public Vector nhanHangSo(double k) {
        return new Vector(x * k, y * k, z * k);
    }

    public double nhanVoHuong(Vector v) {
        return x * v.x + y * v.y + z * v.z;
    }
}
```

Chương 4

Khởi Tạo và Sử Dụng Đối Tượng

4.1. Phương Thức Khởi Tạo (Constructor)	65
4.2. Các Loại Phương Thức Khởi Tạo	67
4.3. Khai Báo và Khởi Tạo Đối Tượng	71
4.4. Sử Dụng Đối Tượng	73
4.5. Quản Lý Bộ Nhớ và So Sánh Đối Tượng	76
4.6. Hủy Bỏ Đối Tượng	78
4.7. Ví Dụ và Bài Tập	81

4.1. Phương Thức Khởi Tạo (Constructor)

Phương thức khởi tạo là một khái niệm cốt lõi trong lập trình hướng đối tượng (OOP), đặc biệt quan trọng trong các ngôn ngữ như Java và C++. Đây là một phương thức đặc biệt được hệ thống gọi tự động ngay khi một đối tượng được tạo ra từ một lớp thông qua từ khóa `new`. Mục đích chính của phương thức khởi tạo là thiết lập các giá trị ban đầu cho các thuộc tính của đối tượng, đảm bảo rằng chúng ở trạng thái hợp lệ và sẵn sàng để sử dụng ngay sau khi được tạo. Điều này giúp ngăn ngừa các lỗi tiềm ẩn, chẳng hạn như `NullPointerException`, khi truy cập dữ liệu chưa được gán giá trị.

Phương thức khởi tạo được định nghĩa với tên trùng hoàn toàn với tên của lớp và không khai báo kiểu trả về, kể cả kiểu `void`. Sự vắng mặt của kiểu trả về là đặc trưng riêng biệt, phân biệt constructor với các phương thức thông thường. Phương thức này có thể được khai báo với các mức truy cập như `public`, `private`, hoặc mặc định (package scope), tùy thuộc vào ý định của lập trình viên về cách đối tượng được tạo. Tuy nhiên, constructor không thể sử dụng các từ khóa như `abstract`, `static`, hay `final`, vì những từ khóa này không phù hợp với vai trò khởi tạo đối tượng. Nếu một lớp không định nghĩa bất kỳ constructor nào, trình biên dịch Java (JVM) sẽ tự động cung cấp một constructor mặc định không tham số, giúp đảm

bảo rằng mọi lớp đều có ít nhất một cách để tạo đối tượng.

Tầm quan trọng của phương thức khởi tạo nằm ở khả năng bảo vệ tính toàn vẹn của dữ liệu. Chẳng hạn, trong một lớp **Student** có thuộc tính **name**, nếu không có constructor để gán giá trị ban đầu, việc truy cập **name** trước khi khởi tạo có thể dẫn đến lỗi. Ngoài việc gán giá trị, constructor còn có thể thực hiện các logic bổ sung như kiểm tra tính hợp lệ của dữ liệu (validation), thiết lập trạng thái mặc định, hoặc gọi các phương thức khởi tạo khác trong cùng lớp thông qua từ khóa **this**.

Dưới đây là một ví dụ minh họa về cách sử dụng constructor trong lớp **Car**, nơi các thuộc tính được khởi tạo với giá trị cụ thể ngay khi đối tượng được tạo:

```
public class Car {
    private String brand;
    private int speed;

    // Constructor với tham số
    public Car(String brand, int speed) {
        this.brand = brand; // Sử dụng this để phân biệt thuộc tính và tham số
        this.speed = speed;
    }

    public void displayInfo() {
        System.out.println("Xe: " + brand + ", Tốc độ: " + speed + " km/h");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car("Toyota", 120);
        myCar.displayInfo(); // Kết quả: Xe: Toyota, Tốc độ: 120 km/h
    }
}
```

Trong ví dụ trên, constructor **Car(String brand, int speed)** đảm bảo rằng mọi đối tượng **Car** được tạo đều có **brand** và **speed** được gán giá trị ngay lập tức. Ngoài ra, từ khóa **this** được sử dụng để tránh nhầm lẫn giữa tham số phương thức và thuộc tính của lớp. Để mở rộng, constructor có thể bao gồm các kiểm tra như sau:

```
public Car(String brand, int speed) {  
    if (brand == null || brand.isEmpty()) {  
        throw new IllegalArgumentException("Thương hiệu không được để trống!");  
    }  
    if (speed < 0) {  
        throw new IllegalArgumentException("Tốc độ không được âm!");  
    }  
    this.brand = brand;  
    this.speed = speed;  
}
```

Phương thức khởi tạo không chỉ đơn thuần gán giá trị mà còn là nơi để thực hiện các tác vụ khởi tạo phức tạp, như kết nối cơ sở dữ liệu hoặc cấu hình ban đầu, tùy thuộc vào yêu cầu của ứng dụng.

4.2. Các Loại Phương Thức Khởi Tạo

Phương thức khởi tạo trong Java được chia thành hai loại chính: **phương thức khởi tạo mặc định** (default constructor) và **phương thức khởi tạo có tham số** (parameterized constructor). Mỗi loại phục vụ các nhu cầu khác nhau trong quá trình tạo và quản lý đối tượng, cung cấp sự linh hoạt cho lập trình viên.

4.2.1. Phương Thức Khởi Tạo Mặc Định (Default Constructor)

Phương thức khởi tạo mặc định là một constructor không nhận bất kỳ tham số nào, được thiết kế để tạo đối tượng với các giá trị mặc định cho các thuộc tính. Khi không có constructor nào được định nghĩa trong lớp, Java tự động cung cấp một constructor mặc định, có mức truy cập tương tự như lớp (thường là **public** nếu lớp là **public**). Constructor này gán các giá trị mặc định như **null** cho các kiểu tham chiếu (reference types), 0 cho các kiểu số nguyên hoặc thực, và **false** cho kiểu **boolean**, tùy thuộc vào kiểu dữ liệu của thuộc tính. Mục đích của nó là đảm bảo rằng đối tượng luôn ở trạng thái hợp lệ ngay cả khi không cung cấp thông tin cụ thể.

Ví dụ về một lớp **Person** với constructor mặc định:

```
public class Person {  
    private String name;  
    private int age;
```

```
// Constructor mặc định
public Person() {
    name = "Unknown"; // Giá trị mặc định
    age = 18;          // Giá trị mặc định
}

public void displayInfo() {
    System.out.println("Tên: " + name + ", Tuổi: " + age);
}
}

public class Main {
    public static void main(String[] args) {
        Person person = new Person();
        person.displayInfo(); // Kết quả: Tên: Unknown, Tuổi: 18
    }
}
```

Constructor mặc định rất hữu ích khi muốn tạo đối tượng với trạng thái ban đầu mặc định mà không cần nhập dữ liệu ngay lập tức, chẳng hạn trong các ứng dụng nơi thông tin chi tiết sẽ được cập nhật sau.

4.2.2. Phương Thức Khởi Tạo Có Tham Số (Parameterized Constructor)

Phương thức khởi tạo có tham số cho phép lập trình viên truyền các giá trị cụ thể vào constructor khi tạo đối tượng, từ đó khởi tạo các thuộc tính với dữ liệu phù hợp ngay từ đầu. Loại constructor này linh hoạt hơn so với constructor mặc định vì nó cho phép tùy chỉnh trạng thái của đối tượng theo yêu cầu nghiệp vụ. Ngoài ra, Java hỗ trợ nạp chồng constructor (constructor overloading), cho phép định nghĩa nhiều constructor với danh sách tham số khác nhau về số lượng hoặc kiểu dữ liệu.

Ví dụ về lớp Book với các constructor có tham số:

```
public class Book {
    private String title;
    private double price;

    // Constructor có hai tham số
```

```
public Book(String title, double price) {
    this.title = title;
    this.price = price;
}

// Constructor nạp chồng với một tham số
public Book(String title) {
    this.title = title;
    this.price = 0.0; // Giá mặc định nếu không cung cấp
}

public void displayInfo() {
    System.out.println("Tên sách: " + title + ", Giá: " + price + " USD");
}
}

public class Main {
    public static void main(String[] args) {
        Book book1 = new Book("Java Programming", 29.99);
        Book book2 = new Book("Python Basics");
        book1.displayInfo(); // Kết quả: Tên sách: Java Programming, Giá: 29.99 USD
        book2.displayInfo(); // Kết quả: Tên sách: Python Basics, Giá: 0.0 USD
    }
}
```

Trong ví dụ trên, nạp chồng constructor cho phép tạo đối tượng Book với hoặc không có giá tiền, tăng tính linh hoạt. Constructor có tham số cũng có thể được sử dụng để kiểm tra đầu vào, ví dụ:

```
public Book(String title, double price) {
    if (title == null || title.isEmpty()) {
        throw new IllegalArgumentException("Tên sách không được để trống!");
    }
    if (price < 0) {
        throw new IllegalArgumentException("Giá không được âm!");
    }
    this.title = title;
    this.price = price;
}
```



```
}
```

4.2.3. Nạp Chồng Constructor

Một lớp có thể định nghĩa nhiều constructor với các danh sách tham số khác nhau, miễn là chúng khác nhau về số lượng hoặc kiểu dữ liệu. Điều này gọi là nạp chồng (overloading) và giúp tối ưu hóa việc tạo đối tượng trong các tình huống khác nhau. Ví dụ với lớp `Rectangle`:

```
public class Rectangle {
    private double width, height;

    // Constructor mặc định
    public Rectangle() {
        width = 1.0;
        height = 1.0;
    }

    // Constructor với một tham số
    public Rectangle(double w) {
        width = w;
        height = 1.0;
    }

    // Constructor với hai tham số
    public Rectangle(double w, double h) {
        width = w;
        height = h;
    }

    public void displayArea() {
        System.out.println("Diện tích: " + (width * height));
    }
}

public class Main {
    public static void main(String[] args) {
        Rectangle rect1 = new Rectangle();        // Diện tích: 1.0
        Rectangle rect2 = new Rectangle(5.0);     // Diện tích: 5.0
    }
}
```

```
        Rectangle rect3 = new Rectangle(4.0, 3.0); // Diện tích: 12.0
        rect1.displayArea();
        rect2.displayArea();
        rect3.displayArea();
    }
}
```

Nạp chồng constructor cung cấp sự linh hoạt, nhưng cần đảm bảo rằng các danh sách tham số đủ khác biệt để tránh lỗi biên dịch do mơ hồ.

4.3. Khai Báo và Khởi Tạo Đối Tượng

Khai báo và khởi tạo đối tượng là hai bước quan trọng trong việc tạo ra các thể hiện của lớp trong Java. Quá trình này liên quan đến việc xác định kiểu dữ liệu và cấp phát bộ nhớ để lưu trữ đối tượng.

4.3.1. Khai Báo Đối Tượng

Khai báo đối tượng là quá trình xác định kiểu của đối tượng thông qua tên lớp mà không thực hiện cấp phát bộ nhớ. Khi khai báo, một biến tham chiếu (reference variable) được tạo ra để đại diện cho đối tượng trong tương lai. Ban đầu, biến này có giá trị mặc định là `null` nếu không được khởi tạo, nghĩa là nó chưa trỏ đến bất kỳ đối tượng nào trong bộ nhớ. Việc khai báo chỉ định nghĩa loại đối tượng mà biến có thể tham chiếu, nhưng không tạo ra đối tượng thực tế.

Cú pháp khai báo đối tượng là: **TênLớp tênBiến;**. Ví dụ:

```
Student student; // Biến student được khai báo, giá trị mặc định là null
```

4.3.2. Khởi Tạo Đối Tượng

Khởi tạo đối tượng là quá trình sử dụng từ khóa `new` kết hợp với constructor của lớp để cấp phát bộ nhớ trong vùng heap và tạo ra một đối tượng cụ thể. Khi `new` được gọi, nó phân bổ không gian bộ nhớ cho đối tượng, sau đó constructor được thực thi để khởi tạo các thuộc tính của đối tượng với giá trị hợp lệ. Quá trình này trả về một tham chiếu (reference) đến đối tượng vừa được tạo, được gán cho biến tham chiếu.

Cú pháp khởi tạo đối tượng là: **tênBiến = new Constructor(tham_số);**. Ví dụ:

```
// Khởi tạo đối tượng với constructor có tham số
student = new Student("Nam", 20);
```

4.3.3. Kết Hợp Khai Báo và Khởi Tạo

Để tiện lợi, khai báo và khởi tạo có thể được thực hiện cùng lúc trong một câu lệnh. Cú pháp là: `TênLớp tênBiến = new Constructor(tham_số);`. Ví dụ:

```
// Khai báo và khởi tạo cùng lúc  
Student student = new Student("Lan", 21);
```

4.3.4. Mảng Đối Tượng

Mảng đối tượng trong Java được khai báo và khởi tạo tương tự như mảng các kiểu nguyên thủy, nhưng mỗi phần tử mặc định là `null` cho đến khi được gán một đối tượng cụ thể. Để sử dụng mảng, cần khởi tạo từng phần tử bằng cách gọi constructor.

Ví dụ về mảng đối tượng:

```
// Khai báo mảng với 3 phần tử, mặc định null  
Student[] students = new Student[3];  
students[0] = new Student("An", 19);  
students[1] = new Student("Bình", 20);  
students[2] = new Student("Cường", 21);  
  
for (Student s : students) {  
    System.out.println("Tên: " + s.getName() + ", Tuổi: " + s.getAge());  
}
```

4.3.5. Quản Lý Đối Tượng

Đối tượng trong Java hoạt động dựa trên cơ chế tham chiếu, không phải giá trị trực tiếp. Điều này có nghĩa là biến tham chiếu chỉ chứa địa chỉ bộ nhớ của đối tượng, không phải dữ liệu của đối tượng. Nếu một biến tham chiếu không được khởi tạo (vẫn là `null`), việc truy cập các phương thức hoặc thuộc tính của nó sẽ gây ra lỗi `NullPointerException`. Ví dụ:

```
Student s; // Chỉ khai báo, chưa khởi tạo  
System.out.println(s.getName()); // Lỗi NullPointerException
```

Để tránh lỗi này, luôn đảm bảo khởi tạo đối tượng trước khi sử dụng. Ngoài ra, Java sử dụng bộ thu gom rác (garbage collector) để tự động giải phóng bộ nhớ khi đối tượng không còn được tham chiếu, giúp tối ưu hóa tài nguyên.

4.4. Sử Dụng Đối Tượng

Sử dụng đối tượng trong lập trình hướng đối tượng đòi hỏi hiểu rõ cách tương tác với các thành viên của nó, bao gồm thuộc tính và phương thức. Quá trình truy cập thành viên của một đối tượng được thực hiện thông qua toán tử dấu chấm (.) sau khi đối tượng đã được khởi tạo. Toán tử này cho phép lập trình viên gọi các phương thức hoặc truy cập các thuộc tính được định nghĩa trong lớp của đối tượng, dựa trên tham chiếu đến đối tượng đó. Việc sử dụng toán tử . đảm bảo rằng các thao tác được áp dụng chính xác trên đối tượng cụ thể, chẳng hạn như thay đổi trạng thái qua phương thức hoặc lấy thông tin từ thuộc tính. Điều này là nền tảng để các đối tượng thực hiện hành động hoặc giao tiếp với nhau trong một ứng dụng.

Ví dụ, trong lớp `Student`, ta có thể truy cập phương thức `setAge` để cập nhật tuổi và phương thức `getName` để lấy tên của đối tượng:

```
public class Student {
    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public String getName() {
        return name;
    }
}

public class Main {
    public static void main(String[] args) {
        Student student = new Student("Hùng", 22);
        student.setAge(23); // Gọi phương thức để cập nhật tuổi
        System.out.println(student.getName()); // Truy cập thông tin: Hùng
    }
}
```

```
}
```

Một khía cạnh quan trọng khác khi sử dụng đối tượng là khái niệm tự tham chiếu với từ khóa `this`. Từ khóa `this` được sử dụng để chỉ đối tượng hiện tại trong phạm vi của một phương thức hoặc constructor, đặc biệt hữu ích khi tên của tham số truyền vào trùng với tên thuộc tính của lớp. Nó giúp phân biệt rõ ràng giữa tham số và thuộc tính, đảm bảo rằng giá trị được gán đúng cho thuộc tính của đối tượng hiện tại. Ngoài ra, `this` còn cho phép gọi các constructor khác trong cùng lớp (constructor chaining) hoặc truy cập các phương thức và thuộc tính khác của đối tượng.

Ví dụ về việc sử dụng `this` trong lớp `Employee` để tránh nhầm lẫn giữa tham số và thuộc tính:

```
public class Employee {
    private String name;

    public Employee(String name) {
        this.name = name; // this.name là thuộc tính, name là tham số
    }

    public void displayName() {
        System.out.println("Tên nhân viên: " + this.name);
    }
}

public class Main {
    public static void main(String[] args) {
        Employee emp = new Employee("Lan");
        emp.displayName(); // Kết quả: Tên nhân viên: Lan
    }
}
```

Tuy nhiên, từ khóa `this` không thể được sử dụng trong các khối hoặc phương thức static, chẳng hạn như khối khởi tạo static hoặc phương thức static. Lý do là các thành phần static thuộc về lớp, không gắn với bất kỳ đối tượng cụ thể nào, trong khi `this` đại diện cho một thể hiện cụ thể của lớp. Khi lớp được tải vào bộ nhớ, các thành phần static được khởi tạo trước khi bất kỳ đối tượng nào được tạo, nên không có đối tượng nào để `this` trỏ tới. Việc sử dụng `this` trong ngữ cảnh static sẽ

gây lỗi biên dịch. Chi tiết về thành viên static sẽ được trình bày trong chương tiếp theo.

Ví dụ minh họa sự khác biệt giữa ngữ cảnh instance và static:

```
public class Example {
    private String instanceVar = "Tôi là biến instance";
    private static String staticVar = "Tôi là biến static";

    static {
        // System.out.println(this.instanceVar); // Lỗi biên dịch: "this" không tồn tại
        System.out.println(staticVar); // Hợp lệ: Truy cập biến static
    }

    public void instanceMethod() {
        System.out.println(this.instanceVar); // Hợp lệ: Truy cập biến instance
    }
}
```

Kết luận, `this` chỉ có ý nghĩa trong các phương thức hoặc khối lệnh gắn với đối tượng (instance), không thể áp dụng trong ngữ cảnh static.

Mở rộng hơn, các đối tượng trong lập trình hướng đối tượng có thể tương tác với nhau thông qua việc gọi các phương thức được định nghĩa trong lớp của chúng. Điều này cho phép trao đổi thông tin, thực hiện các hành động phối hợp, hoặc thay đổi trạng thái lẫn nhau dựa trên logic nghiệp vụ. Ví dụ, một đối tượng có thể sử dụng phương thức của đối tượng khác để truy cập hoặc thao tác dữ liệu, miễn là mức truy cập cho phép (ví dụ: `public`). Sự tương tác này là nền tảng của các hệ thống phức tạp, mô phỏng các mối quan hệ trong thế giới thực như nhóm làm việc hoặc hệ thống giao dịch.

Ví dụ về tương tác giữa hai đối tượng `Student` và `Team`:

```
public class Team {
    private String leader;

    public void assignLeader(Student student) {
        leader = student.getName();
    }
}
```

```

    public void displayLeader() {
        System.out.println("Trưởng nhóm: " + leader);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Minh", 25);
        Team team = new Team();
        team.assignLeader(s); // Gán trưởng nhóm từ đối tượng Student
        team.displayLeader(); // Kết quả: Trưởng nhóm: Minh
    }
}

```

4.5. Quản Lý Bộ Nhớ và So Sánh Đối Tượng

Quản lý bộ nhớ trong Java được thực hiện tự động bởi Máy ảo Java (JVM), khác với các ngôn ngữ như C++ nơi lập trình viên phải quản lý thủ công thông qua con trỏ và toán tử `delete`. Bộ nhớ trong Java được chia thành hai khu vực chính: stack và heap. Vùng stack lưu trữ các biến cục bộ và tham chiếu đến đối tượng, chẳng hạn như biến `s` trong `String s`, trỏ đến địa chỉ của đối tượng trong bộ nhớ. Vùng heap là nơi lưu trữ dữ liệu thực tế của các đối tượng, được cấp phát khi sử dụng từ khóa `new`. JVM sử dụng cơ chế garbage collection (GC) để tự động thu hồi bộ nhớ của các đối tượng không còn được tham chiếu, tối ưu hóa tài nguyên và giảm thiểu rò rỉ bộ nhớ.

Ví dụ minh họa cách hai biến tham chiếu trỏ đến cùng một vùng nhớ trên heap:

```

public class Main {
    public static void main(String[] args) {
        String s = new String("Xin chào");
        String t = s; // t và s cùng trỏ đến cùng một đối tượng trên heap

        System.out.println("s: " + s + ", t: " + t);
        // Kết quả: s: Xin chào, t: Xin chào

        s = null;
        // s không còn tham chiếu, nhưng t vẫn trỏ đến đối tượng
    }
}

```

```
}
```

Về so sánh đối tượng, Java cung cấp hai cách chính: toán tử `==` và phương thức `equals()`. Toán tử `==` được sử dụng để so sánh tham chiếu, tức là kiểm tra xem hai biến có trỏ đến cùng một đối tượng trong bộ nhớ heap hay không. Nó không so sánh nội dung hoặc giá trị của đối tượng, mà chỉ kiểm tra địa chỉ bộ nhớ. Nếu hai biến trỏ đến cùng một đối tượng, `==` trả về `true`; nếu chúng trỏ đến các đối tượng khác nhau, dù nội dung giống nhau, kết quả sẽ là `false`.

Xét ví dụ về sử dụng toán tử `==` sau.

```
public class Main {  
    public static void main(String[] args) {  
        String a = new String("abc");  
        String b = new String("abc");  
        System.out.println(a == b); // false: a và b trỏ đến các đối tượng khác nhau  
        String c = a;  
        System.out.println(a == c); // true: c và a trỏ đến cùng một đối tượng  
    }  
}
```

Ngược lại, phương thức `equals()` được thiết kế để so sánh nội dung hoặc giá trị của hai đối tượng. Mặc định, phương thức `equals()` được kế thừa từ lớp `Object` và hoạt động tương tự toán tử `==`, chỉ so sánh tham chiếu. Tuy nhiên, để so sánh dựa trên ý nghĩa nghiệp vụ (ví dụ: so sánh giá trị thuộc tính), phương thức `equals()` cần được ghi đè (override) trong lớp cụ thể. Lớp `Integer` trong Java đã override `equals()` để so sánh giá trị số thay vì tham chiếu.

Ví dụ minh họa `equals()` trong lớp `Integer`:

```
public class TestInteger {  
    public static void main(String[] args) {  
        Integer n1 = new Integer(42);  
        Integer n2 = new Integer(42);  
        System.out.println(n1 == n2);           // false: khác tham chiếu  
        System.out.println(n1.equals(n2));      // true: cùng giá trị 42  
    }  
}
```

Giải thích, phương thức `equals()` của `Integer` kiểm tra giá trị nguyên (int) bên

trong đối tượng, không phụ thuộc vào việc chúng là hai đối tượng riêng biệt trên heap. Để áp dụng cho lớp tùy chỉnh, lập trình viên cần ghi đè `equals()` như sau:

```
public class Student {
    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Student other = (Student) obj;
        return age == other.age &&
            (name == null ? other.name == null : name.equals(other.name));
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Hùng", 22);
        Student s2 = new Student("Hùng", 22);
        System.out.println(s1.equals(s2)); // true: cùng nội dung
    }
}
```

4.6. Hủy Bỏ Đối Tượng

Hủy bỏ đối tượng là quá trình giải phóng tài nguyên mà đối tượng đang chiếm giữ khi nó không còn cần thiết trong chương trình. Trong ngôn ngữ C++, việc này được thực hiện thủ công thông qua destructor, một phương thức đặc biệt có tên bắt đầu bằng dấu `~` theo sau là tên lớp (ví dụ: `~ClassName()`). Destructor được sử dụng để giải phóng các tài nguyên như tệp, socket, hoặc bộ nhớ động được cấp phát, và nó được gọi tự động khi đối tượng ra khỏi phạm vi (scope) hoặc được xóa

rõ ràng bằng toán tử `delete`. Ngược lại, trong Java, không có khái niệm destructor tương tự. Thay vào đó, việc giải phóng bộ nhớ được giao hoàn toàn cho Garbage Collector (GC), một cơ chế chạy ngầm trong JVM, thu hồi bộ nhớ của các đối tượng không còn được tham chiếu (có giá trị `null`).

Java cung cấp phương thức `finalize()`, được kế thừa từ lớp `Object`, cho phép lập trình viên ghi đè để thực hiện các tác vụ dọn dẹp tài nguyên đặc biệt trước khi đối tượng bị GC thu hồi. Tuy nhiên, phương thức này hiếm khi được sử dụng vì thời điểm gọi `finalize()` không thể kiểm soát, phụ thuộc vào lịch trình của GC. Dù vậy, GC không thể bị ép buộc chạy, mặc dù lập trình viên có thể gợi ý bằng cách gọi `System.gc()`, nhưng điều này không đảm bảo thực thi ngay lập tức. Do đó, để quản lý tài nguyên hiệu quả, nên sử dụng các phương thức như `close()` trong cấu trúc `try-with-resources` thay vì phụ thuộc vào `finalize()`.

Ví dụ minh họa ghi đè `finalize()` trong lớp `Resource`, dù thời điểm thực thi không chắc chắn:

```
public class Resource {
    @Override
    protected void finalize() throws Throwable {
        try {
            System.out.println("Đối tượng bị hủy");
        } finally {
            super.finalize(); // Gọi finalize của lớp cha
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Resource res = new Resource();
        res = null; // Giải phóng tham chiếu, chờ GC
        System.gc(); // Gợi ý GC, nhưng không đảm bảo
    }
}
```

Trong thực tế, việc phụ thuộc vào `finalize()` bị khuyến cáo tránh vì tính không xác định. Thay vào đó, lập trình viên nên sử dụng các cơ chế quản lý tài nguyên rõ ràng, chẳng hạn như giao diện `AutoCloseable` và khối `try-with-resources`, để đảm bảo tài nguyên (như tệp hoặc kết nối mạng) được giải phóng đúng lúc. Kiến

thức về giao diện (interface) và xử lý ngoại lệ với khối lệnh try sẽ được trình bày trong các chương sau của cuốn sách này.

```
public class FileHandler implements AutoCloseable {
    private java.io.File file;

    public FileHandler(String fileName) {
        file = new java.io.File(fileName);
    }

    public void processFile() {
        System.out.println("Xử lý file: " + file.getName());
    }

    @Override
    public void close() {
        System.out.println("Đóng file: " + file.getName());
    }
}

public class Main {
    public static void main(String[] args) {
        try (FileHandler handler = new FileHandler("example.txt")) {
            handler.processFile();
        } // File tự động đóng khi thoát khỏi try
    }
}
```

Nếu không muốn sử dụng giao diện `AutoCloseable`, lập trình viên có thể tự gọi phương thức `close()` một cách thủ công sau khi hoàn thành việc sử dụng tài nguyên. Cách tiếp cận này yêu cầu lập trình viên phải ghi nhớ và thực hiện việc đóng tài nguyên một cách rõ ràng, tránh trường hợp quên giải phóng dẫn đến rò rỉ tài nguyên. Phương pháp này phù hợp trong các ứng dụng đơn giản, nhưng cần cẩn trọng để đảm bảo tính nhất quán.

Ví dụ minh họa cách tự gọi phương thức `close()`:

```
public class FileHandler {
    private java.io.File file;
```

```
public FileHandler(String fileName) {
    file = new java.io.File(fileName);
}

public void processFile() {
    System.out.println("Xử lý file: " + file.getName());
}

public void close() {
    System.out.println("Đóng file: " + file.getName());
}
}

public class Main {
    public static void main(String[] args) {
        FileHandler handler = new FileHandler("example.txt");
        try {
            handler.processFile();
        } finally {
            handler.close(); // Đóng tài nguyên thủ công trong khối finally
        }
    }
}
```

Trong ví dụ trên, khối `try-finally` đảm bảo rằng phương thức `close()` được gọi ngay cả khi xảy ra ngoại lệ, giúp giải phóng tài nguyên một cách đáng tin cậy. Tuy nhiên, phương pháp này đòi hỏi lập trình viên phải triển khai logic quản lý tài nguyên một cách cẩn thận, đặc biệt trong các ứng dụng phức tạp với nhiều đối tượng cần đóng.

4.7. Ví Dụ và Bài Tập

• Bài tập 1:

- Viết lớp `Student` với thuộc tính `name` (`String`), `year` (`int`).
- Tạo constructor `Student(String name, int year)`, getter/setter, đảm bảo đóng gói.

– Viết lớp Test:

- * Nhập số lượng sinh viên.
- * Nhập thông tin từng sinh viên.
- * In danh sách tên và tổng tuổi (giả sử năm hiện tại là 2025).

• **Bài tập 2:**

– Viết lớp Circle với thuộc tính radius (double) và một hằng số $PI = 3.14$.

– Tạo constructor Circle(double radius), các phương thức:

- * `getArea()`: Tính diện tích hình tròn ($PI * radius * radius$).
- * `getCircumference()`: Tính chu vi ($2 * PI * radius$).

– Viết lớp TestCircle:

- * Tạo một mảng gồm 3 hình tròn với bán kính khác nhau (nhập từ người dùng).
- * In diện tích và chu vi của từng hình tròn.
- * Tìm và hiển thị hình tròn có diện tích lớn nhất.

Chương 5

Các kỹ thuật xây dựng lớp và sử dụng đối tượng

5.1. Chồng phương thức	83
5.2. Thành viên đối tượng và thành viên lớp	91
5.3. Truyền tham số cho phương thức	98
5.4. Một số lớp tiện ích trong Java	106
5.5. Ví dụ và bài tập	127

5.1. Chồng phương thức

Trong OOP, phương thức (method) là khối lệnh được định nghĩa bên trong lớp để thực hiện một chức năng cụ thể. Mỗi phương thức phải có một **chữ ký** (signature) riêng để phân biệt với các phương thức khác trong cùng một lớp. Chữ ký của phương thức bao gồm tên phương thức và danh sách các tham số đầu vào cùng kiểu dữ liệu của chúng. Điều này có nghĩa là hai phương thức (trong một lớp) có thể trùng tên nhưng khác nhau về số lượng hoặc kiểu tham số thì vẫn được coi là các phương thức khác nhau (nạp chồng phương thức – method overloading). Nhờ có chữ ký, trình biên dịch có thể xác định chính xác phương thức nào được gọi tại thời điểm chạy.

```
public void credit(double amount) {  
    ...  
}
```

Hình 5.1 Chữ ký phương thức

Trong ví dụ trên slide, chữ ký của phương thức là ‘credit(double amount)’. Ở đây, ‘credit’ là tên phương thức (method name) và ‘(double amount)’ là danh sách

đối số, với ‘double’ là kiểu dữ liệu và ‘amount’ là tên biến tham số. Lưu ý rằng tên biến tham số (‘amount’) không ảnh hưởng đến chữ ký, mà chỉ kiểu dữ liệu và số lượng tham số mới quyết định chữ ký. Ví dụ, ‘credit(int amount)’ và ‘credit(double amount)’ có chữ ký khác nhau dù cùng tên phương thức. Điều này cho phép ta định nghĩa nhiều biến thể của cùng một hành động, phù hợp với các kiểu dữ liệu khác nhau.

Phần khai báo trước chữ ký, như ‘public void’, không nằm trong chữ ký phương thức. Từ khóa ‘public’ là phạm vi truy cập (access modifier) và ‘void’ là kiểu trả về (return type), chúng không được tính để phân biệt các phương thức trong nạp chồng. Việc tách chữ ký ra khỏi các yếu tố này giúp quá trình nhận diện phương thức tập trung vào cách thức truyền dữ liệu vào. Khi thiết kế lớp, người lập trình cần đặt tên phương thức ngắn gọn, dễ hiểu, và lựa chọn tham số đầu vào hợp lý để chữ ký rõ ràng và dễ nhớ. Điều này giúp mã nguồn dễ đọc, dễ bảo trì và hạn chế nhầm lẫn khi mở rộng chức năng.

5.1.1. Định nghĩa chồng phương thức

Chồng phương thức (Method Overloading) là kỹ thuật cho phép định nghĩa nhiều phương thức cùng tên trong một lớp nhưng khác nhau về cách nhận dữ liệu đầu vào. Điểm khác biệt này có thể đến từ số lượng tham số hoặc kiểu dữ liệu của từng tham số. Khi chương trình được biên dịch, hệ thống sẽ tự động chọn phương thức phù hợp dựa trên dữ liệu mà ta truyền vào. Đây là cơ chế giúp các phương thức linh hoạt hơn, đáp ứng được nhiều tình huống sử dụng khác nhau. Điều này đặc biệt hữu ích trong những lớp làm việc với nhiều loại dữ liệu hoặc hình thức xử lý khác nhau nhưng cùng bản chất công việc.

Ví dụ, ta có thể viết nhiều phiên bản của phương thức ‘printData’, một phiên bản nhận chuỗi, một phiên bản nhận số nguyên, và một phiên bản nhận số thực. Khi gọi ‘printData("Hello")’, trình biên dịch sẽ chọn phương thức nhận tham số kiểu chuỗi, còn khi gọi ‘printData(42)’, phương thức nhận số nguyên sẽ được dùng. Lợi ích ở đây là lập trình viên không cần nghĩ ra nhiều tên phương thức khác nhau cho cùng một mục đích. Thay vào đó, chỉ cần nhớ một tên và để cho chữ ký phương thức quyết định cách xử lý. Nhờ đó, mã nguồn trở nên gọn gàng và dễ đọc hơn.

Mục tiêu quan trọng của chồng phương thức là giúp mô tả hành vi một cách tự nhiên và thống nhất. Khi nhiều hành động có bản chất giống nhau, việc đặt chung một tên sẽ phản ánh rõ ý nghĩa nghiệp vụ của chúng. Đồng thời, điều này giúp giảm sự phức tạp khi bảo trì phần mềm, vì thay đổi tên hoặc logic chung sẽ ít phải sửa ở nhiều nơi. Ngoài ra, chồng phương thức còn hỗ trợ khả năng mở rộng, cho phép thêm phiên bản mới khi cần hỗ trợ loại dữ liệu hoặc số lượng tham số khác. Đây là

một trong những yếu tố quan trọng để viết code hướng đối tượng rõ ràng và dễ bảo trì.

Một ví dụ tiêu biểu về chồng phương thức là ‘println()’ trong ‘System.out’ của Java. Phương thức này được định nghĩa với nhiều phiên bản khác nhau, mỗi phiên bản nhận một loại tham số riêng như: ‘boolean’, ‘char[]’, ‘char’, ‘double’, ‘float’, ‘int’, ‘long’, ‘Object’, ‘String’, và một phiên bản không nhận tham số nào. Khi lập trình viên gọi ‘println()’ với dữ liệu thuộc kiểu nào, Java sẽ tự động chọn phương thức tương ứng để xử lý và hiển thị dữ liệu đó. Cơ chế này giúp mã nguồn trở nên linh hoạt và tự nhiên hơn khi làm việc với nhiều kiểu dữ liệu.

Điểm nổi bật ở đây là lập trình viên không cần tạo ra nhiều phương thức với tên khác nhau như ‘printString()’ hay ‘printDouble()’ cho từng loại dữ liệu. Chỉ cần nhớ một tên duy nhất ‘println()’ và truyền tham số phù hợp, chương trình sẽ biết cách xử lý đúng. Điều này giúp giảm sự rườm rà, tăng khả năng tái sử dụng, và làm cho mã dễ đọc hơn. Đây cũng là minh chứng rõ ràng cho lợi ích của chồng phương thức trong việc thiết kế API thân thiện và trực quan.

```
class MyDate {
    int year, month, day;
    public boolean setMonth(int m) { ... }
    public boolean setMonth(String s) { ... }
}

public class Test{
    public static void main(String args[]){
        MyDate d = new MyDate();
        d.setMonth(9);
        d.setMonth("September");
    }
}
```

Chồng phương thức chỉ được áp dụng khi các phương thức nằm trong cùng một lớp. Điều này có nghĩa là nếu hai phương thức có tên giống nhau nhưng thuộc các lớp khác nhau, chúng không được xem là chồng phương thức mà là hai phương thức riêng biệt. Sự ràng buộc này giúp việc xác định và quản lý chữ ký phương thức được rõ ràng và thống nhất. Việc giới hạn phạm vi trong cùng một lớp còn đảm bảo tính nhất quán của hành vi phương thức. Đây là nguyên tắc cơ bản để tránh nhầm lẫn trong lập trình hướng đối tượng.

Khi sử dụng kỹ thuật chồng phương thức, nên áp dụng cho các phương thức có cùng mục đích hoặc chức năng. Ví dụ, các biến thể của một hành động in dữ liệu (print) cho nhiều loại tham số khác nhau vẫn giữ nguyên ý nghĩa “in” nhưng khác cách xử lý đầu vào. Nếu lạm dụng, việc đặt cùng tên cho những hành động không liên quan sẽ khiến mã khó hiểu và khó bảo trì. Vì vậy, chồng phương thức cần được dùng hợp lý để vừa tận dụng sự tiện lợi, vừa giữ được tính rõ ràng của chương trình. Nguyên tắc này đặc biệt quan trọng trong các dự án lớn với nhiều lập trình viên cùng tham gia.

Trong quá trình biên dịch, trình dịch sẽ dựa vào số lượng và kiểu dữ liệu của tham số để quyết định gọi phiên bản phương thức nào. Nếu các thông tin này không đủ rõ ràng hoặc gây mâu thuẫn (ví dụ: hai phương thức đều có thể phù hợp), trình biên dịch sẽ báo lỗi. Ngược lại, nếu không tìm thấy phương thức nào có chữ ký khớp với lời gọi, cũng sẽ xảy ra lỗi biên dịch. Điều này đòi hỏi lập trình viên phải xác định tham số truyền vào một cách rõ ràng. Sự khớp chính xác giữa lời gọi và chữ ký phương thức là yêu cầu bắt buộc để chương trình chạy được.

Tóm lại, chồng phương thức là công cụ mạnh mẽ nhưng cần được sử dụng đúng cách. Chỉ áp dụng khi các phương thức có cùng ý nghĩa, giới hạn trong cùng một lớp, và đảm bảo chữ ký rõ ràng để trình dịch chọn được phiên bản phù hợp. Tránh lạm dụng để giữ mã nguồn dễ hiểu và dễ bảo trì. Việc tuân thủ các nguyên tắc này sẽ giúp tận dụng tối đa lợi ích của chồng phương thức mà không làm giảm chất lượng mã nguồn.

Thảo luận

Cho phương thức sau đây:

```
public double test(String a, int b)
```

Hãy chọn ra các phương thức chồng cho phương thức trên:

1. `void test(String b, int a)`
2. `public double test(String a)`
3. `private int test(int b, String a)`
4. `private int test(String a, int b)`
5. `double test(double a, int b)`
6. `double test(int b)`
7. `public double test(String a, long b)`

Câu trả lời được mô tả ở hình 5.2

```

void prt(String s) { System.out.println(s); }
void f1(char x) { prt("f1(char)"); }
void f1(byte x) { prt("f1(byte)"); }
void f1(short x) { prt("f1(short)"); }
void f1(int x) { prt("f1(int)"); }
void f1(long x) { prt("f1(long)"); }
void f1(float x) { prt("f1(float)"); }
void f1(double x) { prt("f1(double)"); }

```

• Điều gì xảy ra nếu thực hiện:

- `f1(5);`
- `char x='a'; f1(x);`
- `byte y=0; f1(y);`
- `float z = 0; f1(z);...`

5 → int

f1(int)
f1(char)
f1(byte)
f1(float)

Hình 5.2 Chồng phương thức

Dưới đây là một câu hỏi khác: theo bạn vì sao mã nguồn dưới đây bị lỗi?

```

class MyClass {
    public void myMethod(int a, long b) {
    }
    public void myMethod(long a, int b) { //overloading
    }
}

public class Test {
    public static void main(String args[]) {
        MyClass m = new MyClass();
        m.myMethod(); // error do không có method phù hợp
        m.myMethod(9, 10); // error do có 2 phiên bản method phù hợp
    }
}

```

Đoạn mã trên minh họa tình huống chồng phương thức (method overloading) nhưng dẫn đến lỗi biên dịch. Trong lớp MyClass, có hai phương thức myMethod

có cùng tên nhưng khác thứ tự và kiểu dữ liệu tham số: một phiên bản nhận (int, long) và một phiên bản nhận (long, int). Khi gọi `m.myMethod()` không truyền tham số, trình biên dịch báo lỗi vì không có phương thức nào khớp chữ ký. Khi gọi `m.myMethod(9, 10)`, cả hai phương thức đều có thể nhận cặp giá trị (int, int) thông qua cơ chế chuyển đổi kiểu ngầm định, khiến trình biên dịch không biết chọn phiên bản nào, dẫn đến lỗi “gọi mơ hồ” (ambiguous method call). Đây là ví dụ cho thấy chồng phương thức cần được thiết kế sao cho tránh trường hợp gây khó xử lý cho trình biên dịch.

5.1.2. Chồng phương thức khởi tạo

Trong lập trình hướng đối tượng, có nhiều tình huống mà cùng một lớp cần được khởi tạo đối tượng theo những cách khác nhau. Ví dụ, một lớp có thể được tạo với đầy đủ thông tin ban đầu, hoặc chỉ với một phần dữ liệu, hoặc thậm chí không có dữ liệu nào khi khởi tạo. Để đáp ứng sự linh hoạt này, lập trình viên có thể xây dựng nhiều phương thức khởi tạo (constructor) với các chữ ký khác nhau. Mỗi constructor sẽ xử lý cách khởi tạo riêng, phù hợp với dữ liệu được truyền vào. Cách tiếp cận này giúp việc tạo đối tượng trở nên thuận tiện và trực quan hơn.

Kỹ thuật này được gọi là chồng phương thức khởi tạo (constructor overloading), áp dụng nguyên lý giống như chồng phương thức thông thường nhưng dành riêng cho constructors. Khi đối tượng được tạo ra, trình biên dịch sẽ dựa trên số lượng và kiểu dữ liệu của tham số trong lệnh `new` để chọn constructor thích hợp. Điều này cho phép lập trình viên sử dụng cùng một tên lớp nhưng khởi tạo đối tượng theo nhiều kịch bản khác nhau, mà không cần tạo các phương thức khởi tạo với tên khác. Nhờ đó, mã nguồn vừa gọn gàng, vừa dễ sử dụng, lại phản ánh rõ ràng ý định khởi tạo trong từng trường hợp.

```
public class BankAccount{
    private String owner;
    private double balance;
    public BankAccount(){owner = "noname";}
    public BankAccount(String o, double b){
        owner = o; balance = b;
    }
}

public class Test{
    public static void main(String args[]){
        BankAccount acc1 = new BankAccount();
    }
}
```

```
        BankAccount acc2 = new BankAccount("Thuy", 100);
    }
}
```

Đoạn code này minh họa kỹ thuật chồng phương thức khởi tạo (constructor overloading) trong Java. Lớp BankAccount có hai constructor: một constructor không tham số sẽ gán giá trị mặc định cho owner là “noname” và một constructor có hai tham số (String o, double b) cho phép truyền vào tên chủ tài khoản và số dư ban đầu. Trong lớp Test, khi tạo acc1 bằng new BankAccount(), chương trình gọi constructor không tham số và thiết lập owner = “noname”. Khi tạo acc2 bằng new BankAccount(“Thuy”, 100), chương trình gọi constructor có tham số, gán owner = “Thuy” và balance = 100. Điều này cho thấy cách constructor overloading cho phép khởi tạo đối tượng theo nhiều cách khác nhau tùy nhu cầu.

5.1.3. Từ khoá this

Trong Java, this là một biến tham chiếu đặc biệt trỏ tới đối tượng hiện tại – tức là đối tượng đang thực thi phương thức hoặc constructor. Khi bên trong lớp, ta muốn truy cập tới thuộc tính hoặc phương thức của chính đối tượng đó, ta dùng cú pháp this.tên_thuộc_tính hoặc this.tên_phương_thức().

Điều này đặc biệt hữu ích khi tham số truyền vào trùng tên với thuộc tính của lớp. Ví dụ trong phương thức setOwner(String owner), tham số owner và thuộc tính owner có cùng tên. Khi viết this.owner = owner;, phần bên trái là thuộc tính của đối tượng, còn phần bên phải là tham số truyền vào.

Trong ví dụ code:

```
public class BankAccount{
    private String owner;
    public void setOwner(String owner){
        this.owner = owner;
    }
    public BankAccount() {
        this.setOwner("noname");
    }
    ...
}
```

Lớp BankAccount có thuộc tính owner và phương thức setOwner.

Trong constructor không tham số `public BankAccount()`, thay vì gán trực tiếp `owner = "noname"`, ta gọi `this.setOwner("noname")` để tận dụng phương thức đã có, giúp tránh lặp mã và đảm bảo nếu logic trong `setOwner` thay đổi thì constructor cũng được cập nhật theo.

Ngoài ra, từ khóa `this` cũng có thể dùng để gọi một constructor khác trong cùng lớp bằng cú pháp `this(danh_sách_tham_số);`, và lệnh này phải đặt ở dòng đầu tiên của constructor.

Vậy điểm quan trọng cần nắm ở đây:

- `this` luôn trỏ tới chính đối tượng đang hoạt động.
- Dùng `this` để phân biệt rõ thuộc tính và tham số khi trùng tên.
- Có thể dùng `this` để tái sử dụng constructor hoặc phương thức, giúp mã gọn gàng và dễ bảo trì.
- Khi gọi một constructor khác bằng `this(...)`, nó phải là câu lệnh đầu tiên trong constructor hiện tại.

Dưới đây là một ví dụ khác về mã nguồn Java:

```
public class Ship {
    private double x=0.0, y=0.0;
    private double speed=1.0, direction=0.0;
    public String name;

    public Ship(String name) {
        this.name = name;
    }
    public Ship(String name, double x, double y){
        this(name); this.x = x; this.y = y;
    }
    public Ship(String name, double x, double y, double speed, double direction){
        this(name, x, y);
        this.speed = speed;
        this.direction = direction;
    }
    //continue...
    private double degreeToRadian(double degrees) {
```

```

        return(degrees * Math.PI / 180.0);
    }
    public void move() {
        move(1);
    }
    public void move(int steps) {
        double angle = degreesToRadians(direction);
        x = x + (double)steps*speed*Math.cos(angle);
        y = y + (double)steps*speed*Math.sin(angle);
    }
    public void printLocation() {
        System.out.println(name + " is at ("
                               + x + "," + y + ").");
    }
} //end of Ship class

```

5.2. Thành viên đối tượng và thành viên lớp

Trong Java, thành viên bình thường (non-static) là thành viên gắn với từng đối tượng cụ thể. Mỗi khi bạn tạo một đối tượng mới, các thành viên này sẽ được sao chép riêng cho đối tượng đó.

5.2.1. Thành viên static

Ngược lại, thành viên static là thành viên gắn với lớp chứ không thuộc riêng một đối tượng nào. Chỉ có một bản sao duy nhất tồn tại trong toàn bộ chương trình, và tất cả các đối tượng của lớp đều dùng chung.

Cú pháp khai báo thành viên static là: `<chỉ_định_truy_cập> static <kiểu_dữ_liệu> <tên_thành_viên>;`

Ví dụ: `public static long getMillisSinceEpoch()` là một phương thức static.

Trong ví dụ:

```

public class MyDate{
    public static long getMillisSinceEpoch(){
        ...
    }
}

```

```
...
long millis = MyDate.getMillisSinceEpoch();
```

Lớp MyDate định nghĩa phương thức getMillisSinceEpoch() là public static. Vì nó là static, bạn có thể gọi trực tiếp qua tên lớp mà không cần tạo đối tượng:

```
long millis = MyDate.getMillisSinceEpoch();
```

Điều này rất tiện lợi cho các phương thức hoặc biến mang tính chất dùng chung (utility), không phụ thuộc vào dữ liệu của từng đối tượng.

Thành viên static thường được dùng để lưu trữ hằng số chung, bộ đếm số đối tượng đã tạo, hoặc các hàm tiện ích như trong thư viện Math (Math.sqrt(), Math.pow()).

Một ví dụ khác về phương thức static là lớp JOptionPane trong javax.swing:

Thuộc tính

Field Summary	
static int	CANCEL_OPTION Return value from class method if CANCEL is chosen
static int	CLOSED_OPTION Return value from class method if user closes window CANCEL_OPTION or NO_OPTION.
static int	DEFAULT_OPTION Type used for showConfirmDialog.
static int	ERROR_MESSAGE Used for error messages.

static int	WARNING_MESSAGE Used for warning messages.
static int	YES_NO_CANCEL_OPTION Type used for showConfirmDialog.
static int	YES_NO_OPTION Type used for showConfirmDialog.
static int	YES_OPTION Return value from class method if YES is chosen.

• Phương thức:

static void	showMessageDialog(Component parentComponent, Object message)	Brings up an information-message dialog titled "Message".
static void	showMessageDialog(Component parentComponent, Object message, String title, int messageType)	Brings up a dialog that displays a message using a default icon determined by the messageType parameter.
static void	showMessageDialog(Component parentComponent, Object message, String title, int messageType, ...)	Brings up a dialog displaying a message, specifying all parameters.

Hình 5.3 Thành viên lớp

Dưới đây là một ví dụ khác nữa về kết quả của việc gọi đến các thành viên static của lớp JOptionPane này:

```
JOptionPane.showMessageDialog(null,"Ban da thao tac  
loi", "Thong bao loi", JOptionPane.ERROR_MESSAGE);
```



```
JOptionPane.showConfirmDialog(null,"Ban co chac chan  
muon thoat?", "Hay lua chon",  
JOptionPane.YES_NO_OPTION);
```



Hình 5.4 Kết quả của việc truy cập các thành viên lớp của JOptionPane

Hình 5.4 minh họa cách sử dụng thuộc tính và phương thức static của lớp JOptionPane trong Java để hiển thị các hộp thoại tương tác với người dùng. Vì showMessageDialog và showConfirmDialog là các phương thức static, bạn có thể gọi chúng trực tiếp thông qua tên lớp JOptionPane mà không cần tạo đối tượng. Ví dụ, JOptionPane.showMessageDialog(...) được dùng để hiển thị thông báo lỗi, trong đó JOptionPane.ERROR_MESSAGE là một hằng static xác định loại biểu tượng hiển thị.

Hộp thoại thứ hai, JOptionPane.showConfirmDialog(...) tạo một hộp thoại xác nhận, cho phép người dùng chọn giữa các phương án như Yes hoặc No. Tham số JOptionPane.YES_NO_OPTION là một hằng static chỉ định kiểu nút hiển thị trong hộp thoại. Cách sử dụng này giúp lập trình viên nhanh chóng tạo giao diện thông báo hoặc xác nhận mà không phải viết nhiều mã xử lý giao diện thủ công, đồng thời tận dụng sức mạnh của phương thức và hằng static để gọi và cấu hình dễ dàng.



Hình 5.5 Thông báo Warning được tạo bởi JOptionPane

Hình 5.5 minh họa cách sử dụng phương thức `showOptionDialog` của lớp `JOptionPane` để tạo một hộp thoại tùy chọn với các nút bấm được xác định trước. Đầu tiên, một mảng `Object[] options = "OK", "CANCEL"`; được khai báo để định nghĩa các lựa chọn hiển thị. Sau đó, `JOptionPane.showOptionDialog(...)` được gọi với các tham số chỉ định nội dung thông báo, tiêu đề ("Canh bao"), loại hộp thoại (`WARNING_MESSAGE`), các nút bấm (`options`), và lựa chọn mặc định (`options[0]` – tức là "OK"). Kết quả là một hộp thoại cảnh báo xuất hiện, cho phép người dùng chọn giữa "OK" và "CANCEL" theo yêu cầu của lập trình viên.

Một đặc điểm quan trọng của thành viên static trong Java là chúng được chia sẻ giữa tất cả các đối tượng của cùng một lớp. Điều này có nghĩa là nếu bạn thay đổi giá trị của một biến static thông qua bất kỳ đối tượng nào, sự thay đổi này sẽ ảnh hưởng tới tất cả các đối tượng khác của lớp đó. Vì chỉ tồn tại duy nhất một bản sao của biến static trong bộ nhớ, nó hoạt động như một vùng lưu trữ chung cho mọi đối tượng, giúp tiết kiệm tài nguyên và đảm bảo tính đồng nhất dữ liệu chung.

Với các phương thức static, chúng chỉ có thể truy cập trực tiếp tới những biến static khác hoặc gọi các phương thức static trong cùng lớp. Lý do là phương thức static không gắn liền với một đối tượng cụ thể, do đó nó không thể truy cập trực tiếp các thành viên không static vốn thuộc về từng đối tượng riêng. Quy tắc này giúp duy trì tính rõ ràng và tránh lỗi khi lập trình, đặc biệt trong các tình huống cần sử dụng dữ liệu hoặc hành vi chung cho toàn bộ lớp.

Chúng ta hãy cùng phân tích ví dụ dưới đây:

```
class TestStatic{
    public static int iStatic;
    public int iNonStatic;
}

public class TestS {
    public static void main(String[] args) {
        TestStatic obj1 = new TestStatic();
        obj1.iStatic = 10; obj1.iNonStatic = 11;
        System.out.println(obj1.iStatic + "," + obj1.iNonStatic);

        TestStatic obj2 = new TestStatic();
        System.out.println(obj2.iStatic + "," + obj2.iNonStatic);

        obj2.iStatic = 12;
        System.out.println(obj1.iStatic + "," + obj1.iNonStatic);
    }
}
```

Với mã nguồn trên, khi chúng ta thực thi thì chương trình sẽ in ra các kết quả lần lượt như sau:

```
10, 11
10, 0
12, 11
```

Còn với mã nguồn trong hình dưới đây, thì chương trình lại báo lỗi.

```

public class Demo {
    int i = 0;
    void tang(){ i++; }
    public static void main(String[] args) {
        tang();
        System.out.println("Gia tri cua i la" + i);
    }
}

```

non-static method tang() cannot be referenced from a static context
 non-static variable i cannot be referenced from a static context

Hình 5.6 Phương thức static lại gọi đến phương thức non-static

Lỗi trong hình vẽ 5.6 xuất phát từ việc phương thức main là static nhưng lại trực tiếp gọi phương thức tang() và truy cập biến i — cả hai đều là thành viên không static (non-static) của lớp Demo.

Trong Java, một phương thức static thuộc về lớp chứ không gắn với đối tượng cụ thể, do đó nó không thể trực tiếp truy cập các thành viên non-static vốn gắn với từng đối tượng riêng. Khi viết tang(); và i, trình biên dịch không biết bạn muốn thao tác trên đối tượng nào của lớp Demo, vì không có đối tượng nào được tạo ra trong main.

Để sửa lỗi, bạn cần tạo một đối tượng Demo rồi gọi các thành viên non-static thông qua đối tượng đó, ví dụ:

```

Demo d = new Demo();
d.tang();
System.out.println("Gia tri cua i la " + d.i);

```

5.2.2. Thành viên hằng

Khi một biến hoặc phương thức được khai báo với final, giá trị hoặc nội dung của nó không thể thay đổi trong suốt quá trình chương trình chạy. Điều này có nghĩa là nếu là biến nguyên thủy (primitive), giá trị của biến sẽ cố định sau khi gán lần

đầu; nếu là đối tượng hoặc mảng, tham chiếu tới đối tượng đó sẽ cố định, nhưng nội dung bên trong đối tượng hoặc mảng vẫn có thể thay đổi (trừ khi các phần tử cũng được khai báo là final).

Cú pháp khai báo:

```
<chỉ_định_truy_cập> final <kiểu_dữ_liệu> TÊN_HẰNG = giá_trị;
```

Tên hằng thường được viết in hoa và dùng dấu gạch dưới để phân tách từ. Ví dụ:

```
final double PI = 3.141592653589793;
public final int VAL_THREE = 39;
private final int[] A = { 1, 2, 3, 4, 5, 6 };
```

Trong đó, PI và VAL_THREE là các hằng số, còn A là một mảng có tham chiếu cố định, không thể trở sang mảng khác, nhưng giá trị của từng phần tử vẫn có thể thay đổi.

Việc sử dụng final giúp đảm bảo tính bất biến của dữ liệu quan trọng, tránh những thay đổi ngoài ý muốn và làm cho mã dễ đọc, dễ bảo trì hơn.

Trong Java nói riêng và trong các ngôn ngữ lập trình hướng đối tượng nói chung, các hằng số (constant) liên quan đến lớp thường được khai báo với cả hai từ khóa static và final. Việc kết hợp này có hai lợi ích chính:

- final đảm bảo giá trị của hằng số không thể thay đổi sau khi được gán lần đầu, giúp giữ tính toàn vẹn của dữ liệu.
- static cho phép hằng số thuộc về lớp thay vì từng đối tượng, nhờ đó ta có thể truy cập trực tiếp qua tên lớp mà không cần tạo đối tượng.

Ví dụ:

```
public static final double PI = 3.141592653589793;
```

Khi đó, có thể truy cập giá trị PI bằng cách viết TênLớp.PI từ bất kỳ đâu trong chương trình, vừa tiện lợi, vừa rõ ràng về mặt ý nghĩa.

Trước khi so sánh thành viên đối tượng và thành viên lớp, cần hiểu rằng trong lập trình hướng đối tượng, các thuộc tính và phương thức của một lớp có thể được chia thành hai nhóm chính: nhóm gắn với từng đối tượng cụ thể và nhóm gắn với toàn bộ lớp. Thành viên đối tượng được tạo riêng cho mỗi đối tượng và giá trị của chúng có thể khác nhau giữa các đối tượng. Ngược lại, thành viên lớp là dữ liệu hoặc hành vi dùng chung cho tất cả các đối tượng của lớp đó. Bảng 5.1 sẽ tóm tắt sự khác biệt chính giữa hai loại thành viên này.

Thành viên đối tượng	Thành viên lớp
Thuộc tính/phương thức chỉ được truy cập thông qua đối tượng	Thuộc tính/phương thức có thể được truy cập thông qua lớp
Mỗi đối tượng có 1 bản sao riêng của 1 thuộc tính đối tượng	Các đối tượng có chung 1 bản sao của 1 thuộc tính lớp
Giá trị của 1 thuộc tính đối tượng của các đối tượng khác nhau là khác nhau	Giá trị của 1 thuộc tính lớp của các đối tượng khác nhau là giống nhau

Bảng 5.1 So sánh thành viên đối tượng và thành viên lớp

5.3. Truyền tham số cho phương thức

Trong Java (cũng như các ngôn ngữ hướng đối tượng khác), tham số của phương thức hoặc constructor có thể sử dụng bất kỳ kiểu dữ liệu nào tùy theo nhu cầu xử lý. Các kiểu dữ liệu này có thể là kiểu nguyên thủy như int, double, char, boolean... hoặc kiểu tham chiếu như mảng và các đối tượng do lập trình viên tự định nghĩa.

Ví dụ minh họa là phương thức sau:

```
public Polygon polygonFrom(Point[] corners) {
    // method body goes here
}
```

Ở đây, tham số corners là một mảng các đối tượng Point. Khi phương thức được gọi, lập trình viên chỉ cần truyền vào một mảng các điểm, và phương thức sẽ sử dụng dữ liệu này để tạo ra và trả về một đối tượng Polygon. Cách làm này cho thấy sự linh hoạt của Java trong việc thiết kế phương thức: bạn có thể nhận mọi loại dữ liệu cần thiết, từ những giá trị đơn giản đến các cấu trúc phức tạp, để phục vụ cho logic xử lý bên trong.

Tuy vậy, dù là kiểu dữ liệu nào, Java luôn truyền tham số theo giá trị (pass-by-value). Với kiểu nguyên thủy (int, double...), giá trị được sao chép vào tham số hình thức. Vì chỉ làm việc trên bản sao, mọi thay đổi bên trong phương thức không ảnh hưởng tới biến bên ngoài.

Với kiểu tham chiếu (mảng, đối tượng), “giá trị” được sao chép chính là tham chiếu (địa chỉ) tới đối tượng. Do cả hai cùng trỏ tới cùng một đối tượng, nếu bạn thay đổi trạng thái bên trong đối tượng, kết quả sẽ nhìn thấy bên ngoài. Nhưng nếu bạn gán lại tham số hình thức để trỏ sang đối tượng khác, tham chiếu ở bên gọi không đổi.

```

void inc(int x) { x++; }           // KHÔNG ảnh hưởng biến gọi
void fill(int[] a) { a[0] = 99; } // ẢNH HƯỞNG: sửa phần tử mảng
void reassign(int[] a) { a = new int[]{7,7}; } // KHÔNG ảnh hưởng: chỉ đổi bản sao th

```

Khi gọi phương thức:

```

int v = 1;      inc(v);           // v vẫn = 1
int[] arr = {0}; fill(arr);      // arr[0] trở thành 99
reassign(arr);  // arr KHÔNG đổi, vẫn {99}

```

Tóm lại: hãy nhớ ý then chốt sau — thay đổi tham số hình thức không làm đổi tham số thực; chỉ khi bạn biến đổi trạng thái của đối tượng được tham chiếu, sự thay đổi mới “lọt” ra ngoài.

5.3.1. Với kiểu dữ liệu tham trị

Ví dụ dưới đây minh họa một đặc điểm quan trọng của Java: các giá trị nguyên thủy không thể thay đổi khi được truyền làm tham số. Nguyên nhân là Java truyền bản sao giá trị (pass-by-value) vào phương thức. Khi phương thức thực hiện các thao tác, mọi thay đổi chỉ diễn ra trên bản sao, không ảnh hưởng đến biến gốc bên ngoài.

```

public void swap(int var1, int var2) {
    int temp = var1;
    var1 = var2;
    var2 = temp;
}

```

Phương thức `swap(int var1, int var2)` ở trên có ý định hoán đổi giá trị của hai biến nguyên thủy. Mặc dù bên trong phương thức việc đổi chỗ diễn ra thành công với `var1` và `var2` (là bản sao), nhưng khi phương thức kết thúc, các bản sao này bị hủy và giá trị của hai biến ban đầu vẫn giữ nguyên. Nói cách khác, phương thức này không thể hoán đổi giá trị của hai biến nguyên thủy bên ngoài.

Nếu muốn hoán đổi thực sự, cần truyền vào một kiểu tham chiếu như mảng hoặc một đối tượng chứa các biến đó, để phương thức có thể thay đổi dữ liệu bên trong đối tượng chung. Ví dụ: `swap(int[] arr)` và đổi `arr[0]` với `arr[1]` sẽ tác động trực tiếp lên dữ liệu gốc.

5.3.2. Với kiểu dữ liệu tham chiếu

Sau đây chúng ta sẽ tìm hiểu một ví dụ để làm rõ cơ chế truyền tham số trong Java đối với kiểu tham chiếu. Khi truyền một đối tượng vào phương thức, Java không truyền tham chiếu gốc trực tiếp, cũng không sao chép toàn bộ đối tượng. Thay vào đó, nó sao chép giá trị của tham chiếu gốc (tức là địa chỉ của đối tượng) rồi gán cho tham số hình thức.

Điều này dẫn đến việc sau khi truyền, trong chương trình tồn tại ít nhất hai tham chiếu cùng trỏ tới một đối tượng: một là biến gốc (original reference) bên ngoài phương thức, hai là tham số hình thức (method reference) bên trong phương thức. Cả hai tham chiếu này đều có thể truy cập và thay đổi trạng thái của đối tượng chung. Tuy nhiên, nếu bên trong phương thức bạn gán lại tham số hình thức để trỏ sang đối tượng khác, điều đó không làm thay đổi tham chiếu gốc ở bên ngoài.

Đoạn mã nguồn dưới đây cho thấy rõ điều này: cả original reference và method reference đều chỉ về cùng một Object trong bộ nhớ.

```
public class Student {
    private int year;
    private String name;

    public Student(String name, int year) {
        this.year = year;
        this.name = name;
    }

    public int getYear() {
        return year;
    }

    public void setYear(int year) {
        this.year = year;
    }
}

public class TestVersion01 {
    public static void change(Student std){
        std.setYear(2000);
    }

    public static void main(String[] args) {
```

```
        Student std = new Student("Nam", 1990);
        System.out.println(std.getYear());
        change(std);
        System.out.println(std.getYear());
    }
}
```

Dưới đây là một phiên bản khác của việc truyền tham số:

```
public class TestVersion02 {
    public static void change(Student std){
        std = new Student("Hung", 1995);
    }
    public static void main(String[] args) {
        Student std = new Student("Nam", 1990);
        System.out.println(std.getYear());
        change(std);
        System.out.println(std.getYear());
    }
}
```

Người đọc có thể chạy thử hai ví dụ trên, để nhận ra sự khác biệt. Người đọc có thể phân tích được nguyên nhân dẫn đến sự khác biệt giữa hai kết quả này, dựa trên gợi ý về tham chiếu và tham trị ở trên.

Để cung cấp gợi ý cho người đọc, ở đây có một ví dụ khác về mã nguồn Java:

```
public class Point {
    private double x;
    private double y;
    public Point() { }
    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
    public void setX(double x) {
        this.x = x;
    }
}
```



```

    public void setY(double y) {
        this.y = y;
    }

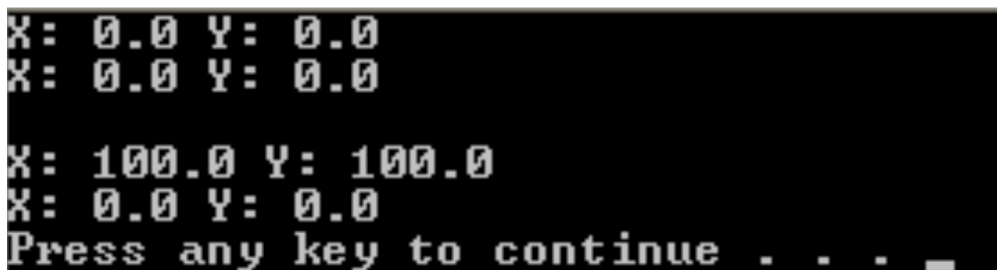
    public void printPoint() {
        System.out.println("X: " + x + " Y: " + y);
    }
}

public class Test {
    public static void tricky(Point arg1, Point arg2) {
        arg1.setX(100); arg1.setY(100);
        Point temp = arg1;
        arg1 = arg2; arg2 = temp;
    }

    public static void main(String [] args) {
        Point pnt1 = new Point(0,0);
        Point pnt2 = new Point(0,0);
        pnt1.printPoint(); pnt2.printPoint();
        System.out.println(); tricky(pnt1, pnt2);
        pnt1.printPoint(); pnt2.printPoint();
    }
}

```

Một khi người đọc chạy thử đoạn mã vừa được cung cấp ở trên, hãy kiểm tra xem kết quả có phải như dưới đây hay không nhé:



```

X: 0.0 Y: 0.0
X: 0.0 Y: 0.0

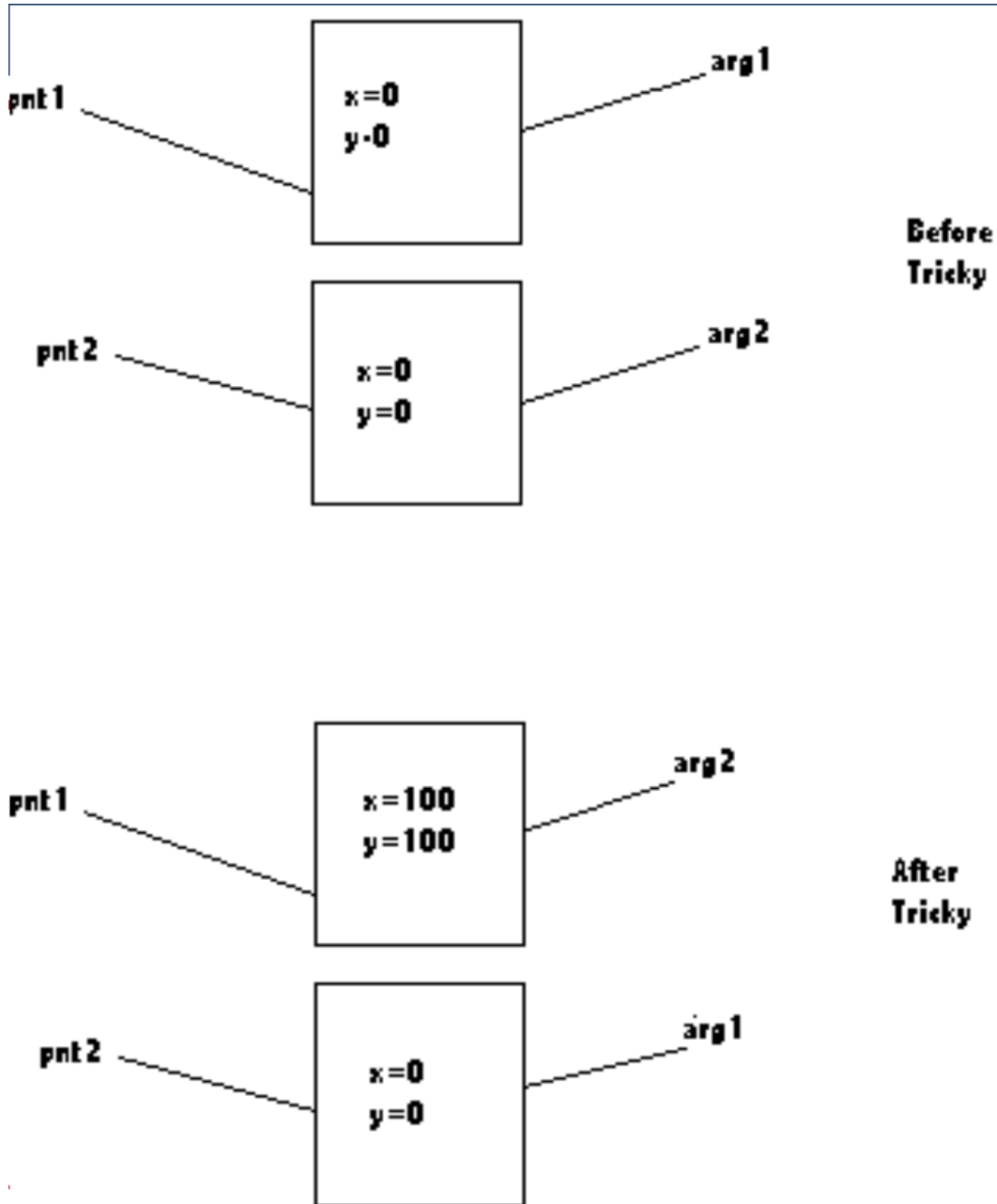
X: 100.0 Y: 100.0
X: 0.0 Y: 0.0
Press any key to continue . . .

```

Hình 5.7 Kết quả của tham chiếu và tham trị

Nếu người đọc vẫn đang gặp khó khăn trong việc giải thích các kết quả trên,

dưới đây là một manh mối cho việc giải thích:



Hình 5.8 Gợi ý giải thích

Hình 5.8 minh họa một tình huống thường gặp khi làm việc với tham số kiểu tham chiếu trong Java: việc trao đổi chỉ diễn ra với bản sao tham chiếu bên trong

phương thức, không ảnh hưởng tới tham chiếu gốc bên ngoài.

Ở phần “Before Tricky”, ta có hai biến gốc `pnt1` và `pnt2` trỏ tới hai đối tượng khác nhau. Khi truyền chúng vào phương thức (thông qua `arg1` và `arg2`), Java tạo ra bản sao giá trị của tham chiếu và gán cho các tham số hình thức này. Trong phương thức, khi ta tráo đổi `arg1` và `arg2`, điều thực sự xảy ra chỉ là đổi chỗ hai bản sao tham chiếu — còn `pnt1` và `pnt2` bên ngoài vẫn trỏ tới đối tượng ban đầu.

Kết quả là, sau khi phương thức kết thúc (“After Tricky”), `pnt1` và `pnt2` không đổi chỗ cho nhau. Điều duy nhất thay đổi là bên trong phương thức, `arg1` trỏ sang đối tượng mà `pnt2` trỏ tới, và `arg2` trỏ sang đối tượng mà `pnt1` trỏ tới, nhưng các tham chiếu gốc bên ngoài vẫn giữ nguyên. Điều này củng cố nguyên tắc: Java truyền tham số theo giá trị, kể cả với tham chiếu — nên thay đổi bản sao tham chiếu không làm thay đổi tham chiếu gốc.

5.3.3. Truyền số lượng tham số tùy ý

Sau đây mục này sẽ giới thiệu về `varargs` trong Java — một tính năng cho phép phương thức nhận số lượng tham số không cố định. Cú pháp khai báo là:

```
tên_phương_thức(kiểu_dữ_liệu... tên_tham_số)
```

Dấu ba chấm (...) cho trình biên dịch biết rằng phương thức có thể nhận 0, 1 hoặc nhiều đối số của cùng một kiểu dữ liệu, và bên trong phương thức các đối số này sẽ được xử lý như một mảng.

Ví dụ sau đây là phương thức `printf` của lớp `PrintStream`, được khai báo:

```
public PrintStream printf(String format, Object... args)
```

Ở đây, `args` là `varargs` kiểu `Object`, cho phép truyền vào bao nhiêu giá trị tùy ý để khớp với các định dạng (

Khi sử dụng, ta có thể gọi:

```
System.out.printf ("%s: %d, %s\n", name, idnum, address);
System.out.printf ("%s: %d, %s, %s, %s\n", name, idnum,
                        address, phone, email);
```

Dù số lượng đối số khác nhau, phương thức vẫn hoạt động nhờ `varargs`. Điều này giúp mã nguồn gọn gàng và linh hoạt hơn, đặc biệt với các tình huống mà số lượng tham số chưa biết trước.

Dưới đây là một ví dụ khác:

```
public Polygon polygonFrom(Point... corners) {
    int numberOfSides = corners.length;
```

```

    double squareOfSide1, lengthOfSide1;
    squareOfSide1 = (corners[1].x - corners[0].x)
        *(corners[1].x - corners[0].x)
        + (corners[1].y - corners[0].y)
        *(corners[1].y - corners[0].y) ;
    lengthOfSide1 = Math.sqrt(squareOfSide1);
    //create & return a polygon connecting the Points
}

```

Ở đây, corners là một tham số varargs kiểu Point, cho phép truyền vào số lượng điểm bất kỳ để tạo đa giác. Bên trong phương thức, corners sẽ được xử lý giống như một mảng Point[]. Do biến corners trong phương thức này được xử lý như một mảng, nên có thể truy cập bằng chỉ số như corners[0], corners[1], ...

Khi gọi phương thức, ta có thể truyền trực tiếp một mảng Point[] hoặc truyền nhiều đối số Point riêng lẻ mà không cần tự tạo mảng trước. Điều này rõ ràng giúp mã nguồn gọn và linh hoạt hơn.

Bài tập - Tính tổng số lượng các số nguyên bất kỳ

Gợi ý:

```

public class Test {
    public static int plus(int... arr) {
        int result = 0;
        for (int i : arr) {
            result += i;
        }
        return result;
    }

    public static void main(String[] args) {
        System.out.println(plus(1, 2, 3, 4, 5));
    }
}

```

5.4. Một số lớp tiện ích trong Java

Trong quá trình lập trình Java, ngoài các lớp cơ bản, thư viện chuẩn của Java còn cung cấp nhiều lớp tiện ích (utility classes) giúp đơn giản hóa công việc lập trình, tăng hiệu quả và giảm thiểu lỗi. Các lớp này thường chứa các phương thức tĩnh (static) hỗ trợ thao tác với dữ liệu, xử lý chuỗi, làm việc với tập hợp, ngày giờ, toán học, và nhiều tác vụ phổ biến khác. Việc nắm vững và sử dụng thành thạo các lớp tiện ích sẽ giúp lập trình viên tiết kiệm thời gian, tận dụng được sức mạnh sẵn có của Java, thay vì phải tự viết lại những đoạn mã đã được tối ưu sẵn.

5.4.1. Package trong Java

Gói được coi như các thư mục, đó là nơi bạn tổ chức các lớp và các giao diện của bạn. Các chương trình Java được tổ chức như những tập của các gói. Mỗi gói gồm có nhiều lớp, và/hoặc các giao diện được coi như là các thành viên của nó. Đó là một phương án thuận lợi để lưu trữ các nhóm của những lớp có liên quan với nhau dưới một cái tên cụ thể. Khi bạn đang làm việc với một chương trình ứng dụng, bạn tạo ra một số lớp. Các lớp đó cần được tổ chức một cách hợp lý. Điều đó trở nên dễ dàng khi ta tổ chức các tập tin lớp thành các gói khác nhau. Hãy tưởng tượng rằng mỗi gói giống như một thư mục con. Tất cả các điều mà bạn cần làm là đặt các lớp và các giao diện có liên quan với nhau vào các thư mục riêng, với một cái tên phản ánh được mục đích của các lớp.

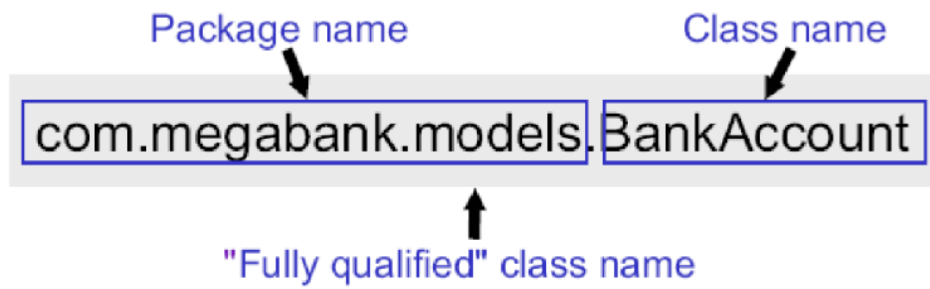
Nói tóm lại, các gói có ích cho các mục đích sau:

- Chúng cho phép bạn tổ chức các lớp thành các đơn vị nhỏ hơn (như là các thư mục), và làm cho việc xác định vị trí trở nên dễ dàng và sử dụng các tập tin của lớp một cách phù hợp.
- Giúp đỡ để tránh cho việc đặt tên bị xung đột (trùng lặp tên). Khi bạn làm việc với một số các lớp bạn sẽ cảm thấy khó để quyết định đặt tên cho các lớp và các phương thức. Đôi lúc bạn muốn sử dụng tên giống nhau mà tên đó liên quan đến lớp khác. Các gói giấu các lớp để tránh việc đặt tên bị xung đột.
- Các gói cho phép bạn bảo vệ các lớp, dữ liệu và phương thức ở mức rộng hơn trên một nền tảng class-to-class.

Các tên của gói có thể được sử dụng để nhận dạng các lớp. Các gói cũng có thể chứa các gói khác.

Hình ảnh 5.9 minh họa khái niệm tên đầy đủ của lớp (fully qualified class name) trong Java. Trong Java, mỗi lớp thường nằm trong một gói để tổ chức mã nguồn

và tránh xung đột tên. Package name: là đường dẫn phân cấp của gói chứa lớp, ví dụ: `com.megabank.models`. Class name: là tên của lớp, ví dụ: `BankAccount`. Khi kết hợp tên gói và tên lớp, ta được tên đầy đủ của lớp.



Hình 5.9 Giải thích cấu trúc tên gói

Tham chiếu giữa các lớp

Đoạn mã dưới đây giải thích cách sử dụng tên lớp khi làm việc với các gói trong Java. Nếu lớp bạn cần sử dụng nằm trong cùng một gói, bạn chỉ cần gọi trực tiếp tên lớp mà không cần chỉ định thêm thông tin gì khác. Tuy nhiên, khi lớp đó thuộc một package khác, bạn bắt buộc phải cung cấp tên đầy đủ (fully qualified class name), bao gồm cả tên gói và tên lớp, để Java có thể xác định chính xác đối tượng mà bạn muốn dùng. Ví dụ trong lớp `HelloNameDialog`, phương thức `main` sử dụng `JOptionPane` của thư viện Swing để hiển thị hộp thoại nhập tên người dùng và sau đó hiện lời chào. Vì `JOptionPane` thuộc package `javax.swing` khác với package của lớp hiện tại, nên trong mã nguồn phải ghi rõ toàn bộ tên `javax.swing.JOptionPane` khi gọi phương thức, thay vì chỉ viết `JOptionPane`. Điều này giúp trình biên dịch biết chính xác lớp cần dùng mà không bị nhầm lẫn với các lớp khác trùng tên ở những gói khác.

```
public class HelloNameDialog {  
    public static void main(String[] args){  
        String result;  
        result = javax.swing.JOptionPane.  
            showInputDialog("Hay nhap ten ban:");  
        javax.swing.JOptionPane.showMessageDialog (null,  
            "Xin chao "+ result + "!");  
    }  
}
```

```

    }
}

```

Khi lập trình, nếu bạn muốn sử dụng các lớp hoặc gói nằm ở nơi khác mà không muốn phải viết tên đầy đủ mỗi lần gọi, bạn có thể dùng lệnh import để khai báo trước. Việc này giúp mã nguồn ngắn gọn và dễ đọc hơn, vì thay vì viết cả tên gói kèm tên lớp, bạn chỉ cần dùng trực tiếp tên lớp. Lệnh import đặc biệt hữu ích khi làm việc với các thư viện lớn như `java.util`, `javax.swing` hoặc các gói do lập trình viên tự định nghĩa, giúp tiết kiệm thời gian và tránh sai sót khi gõ tên đầy đủ nhiều lần.

5.4.2. Các gói trong Java

Trong Java, thư viện chuẩn được tổ chức thành nhiều gói khác nhau, mỗi gói chứa tập hợp các lớp và giao diện phục vụ cho những mục đích lập trình cụ thể. Các gói bắt đầu với tiền tố ‘`java.`’ như ‘`java.io`’ để xử lý nhập xuất dữ liệu, ‘`java.util`’ cung cấp các cấu trúc dữ liệu và tiện ích, ‘`java.sql`’ cho thao tác cơ sở dữ liệu, hay ‘`java.net`’ hỗ trợ lập trình mạng. Bên cạnh đó, tiền tố ‘`javax.`’ thường được dùng cho các API mở rộng như ‘`javax.swing`’ để xây dựng giao diện đồ họa, ‘`javax.crypto`’ cho mã hóa, hoặc ‘`javax.xml`’ để xử lý dữ liệu XML. Ngoài ra, còn có các gói thuộc tổ chức khác như ‘`org.omg`’ cho lập trình phân tán theo chuẩn CORBA, ‘`org.w3c.dom`’ phục vụ xử lý tài liệu XML theo mô hình DOM, hay ‘`org.xml`’ cho các tác vụ liên quan đến XML. Việc phân chia thành các gói này giúp lập trình viên dễ dàng tìm kiếm, sử dụng và quản lý các thành phần của Java, đồng thời giữ cho mã nguồn rõ ràng và có cấu trúc hơn.

• <code>java.applet</code>	• <code>javax.rmi</code>
• <code>java.awt</code>	• <code>javax.security</code>
• <code>java.beans</code>	• <code>javax.sound</code>
• <code>java.io</code>	• <code>javax.sql</code>
• <code>java.lang</code>	• <code>javax.swing</code>
• <code>java.math</code>	• <code>javax.transaction</code>
• <code>java.net</code>	• <code>javax.xml</code>
• <code>java.nio</code>	• <code>org.ietf.jgss</code>
• <code>java.rmi</code>	• <code>org.omg.CORBA</code>
• <code>java.security</code>	• <code>org.omg.CosNaming</code>
• <code>java.sql</code>	• <code>org.omg.Dynamic</code>
• <code>java.text</code>	• <code>org.omg.IOP</code>
• <code>java.util</code>	• <code>org.omg.Messaging</code>
• <code>javax.accessibility</code>	• <code>org.omg.PortableInterceptor</code>
• <code>javax.crypto</code>	• <code>org.omg.PortableServer</code>
• <code>javax.imageio</code>	• <code>org.omg.SendingContext</code>
• <code>javax.naming</code>	• <code>org.omg.stub.java.rmi</code>
• <code>javax.net</code>	• <code>org.w3c.dom</code>
• <code>javax.print</code>	• <code>org.xml</code>

Hình 5.10 Danh sách các gói thường dùng trong Java

Trong Java, một số gói cơ bản đóng vai trò quan trọng trong việc hỗ trợ lập trình viên xây dựng ứng dụng. Trước hết, gói ‘java.lang’ cung cấp các lớp cốt lõi cần thiết cho việc thiết kế ngôn ngữ lập trình Java. Bên trong gói này bao gồm các lớp bao bọc kiểu dữ liệu nguyên thủy (wrapper classes), các lớp xử lý chuỗi như ‘String’ và ‘StringBuffer’, cùng với lớp nền tảng ‘Object’ và nhiều lớp khác. Đặc biệt, ‘java.lang’ được import ngầm định vào tất cả các chương trình Java, giúp lập trình viên sử dụng trực tiếp mà không cần khai báo thêm. Tiếp theo, gói ‘java.util’ chứa các cấu trúc dữ liệu và tiện ích như các framework tập hợp (collections framework), mô hình sự kiện, xử lý thời gian ngày tháng (date time), cùng nhiều công cụ hỗ trợ khác, giúp đơn giản hóa việc quản lý dữ liệu và xử lý logic. Cuối cùng, gói ‘java.io’ đảm nhận việc cung cấp khả năng nhập và xuất dữ liệu, cho phép chương trình làm việc với các luồng dữ liệu (data streams) và hệ thống tệp (file system), từ đó hỗ trợ đọc, ghi, và xử lý thông tin một cách linh hoạt. Nhờ sự kết hợp của các package này, Java tạo nên một nền tảng mạnh mẽ, dễ mở rộng và thuận tiện cho lập trình viên ở mọi cấp độ.

Trong Java, ngoài các gói cơ bản, còn có nhiều gói mở rộng cung cấp các tính năng chuyên biệt cho các lĩnh vực khác nhau. Chẳng hạn, gói ‘java.sql’ hỗ trợ lập trình viên làm việc với cơ sở dữ liệu theo chuẩn JDBC, bao gồm các lớp và interface để kết nối, truy vấn, và xử lý dữ liệu. Gói ‘java.text’ cung cấp các công cụ định dạng và phân tích chuỗi, đặc biệt hữu ích cho việc xử lý số, ngày tháng và văn bản theo các quy tắc ngôn ngữ khác nhau. Với các ứng dụng mạng, gói ‘java.net’ mang đến khả năng giao tiếp qua TCP/IP, xử lý URL, socket, và các giao thức mạng. Nếu cần thao tác với đường dẫn, tập tin và hệ thống tệp theo cách hiện đại và hiệu quả hơn, gói ‘java.nio’ là lựa chọn với các lớp hỗ trợ I/O phi khối (non-blocking I/O) và quản lý bộ nhớ đệm. Bên cạnh đó, gói ‘java.security’ cung cấp các API bảo mật, bao gồm mã hóa, xác thực và quản lý quyền truy cập. Mỗi gói này đóng một vai trò nhất định, giúp Java có khả năng bao quát nhiều nhu cầu phát triển từ cơ bản đến nâng cao.

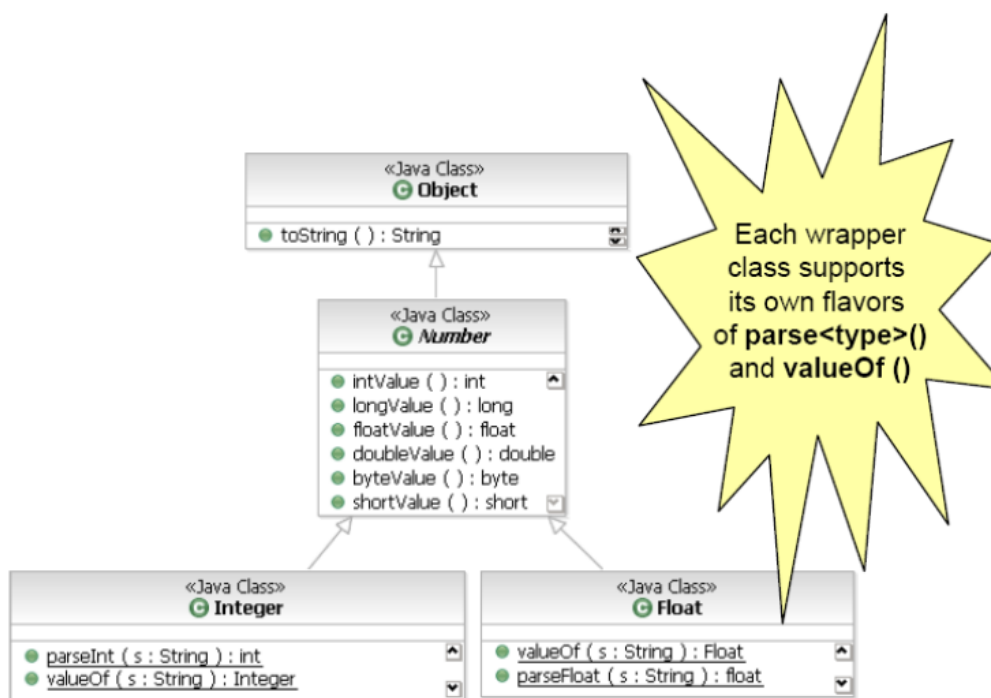
5.4.3. Các lớp bao - Wrapper class

Trong Java, các lớp bao (Wrapper class) là những lớp được sử dụng để “bao bọc” các kiểu dữ liệu nguyên thủy, giúp chúng có thể được xử lý như các đối tượng. Mỗi kiểu dữ liệu nguyên thủy đều có một lớp bao tương ứng, chẳng hạn ‘int’ có lớp ‘Integer’, ‘double’ có lớp ‘Double’, ‘char’ có lớp ‘Character’, và ‘boolean’ có lớp ‘Boolean’. Việc sử dụng các lớp bao cho phép lập trình viên tận dụng các phương thức và thuộc tính có sẵn để thao tác dữ liệu một cách linh hoạt hơn, ví dụ như chuyển đổi kiểu, so sánh, hoặc phân tích từ chuỗi. Các lớp bao đặc biệt hữu ích khi

làm việc với các cấu trúc dữ liệu trong Java như 'ArrayList' hoặc 'HashMap', vì những cấu trúc này chỉ có thể lưu trữ đối tượng chứ không lưu trực tiếp các kiểu nguyên thủy. Ngoài ra, Java cũng hỗ trợ cơ chế tự động đóng gói (autoboxing) và tự động tháo gói (unboxing), giúp việc chuyển đổi giữa kiểu nguyên thủy và lớp bao diễn ra tự động, làm code gọn gàng và dễ đọc hơn.

Primitive Type	Wrapper Class
boolean	Boolean
byte	Byte
char	Character
double	Double
float	Float
int	Integer
long	Long
short	Short

Bảng 5.2 Các kiểu dữ liệu nguyên thủy và lớp bao tương ứng trong Java



Hình 5.11 Một số lớp bao

Chuyển đổi kiểu dữ liệu

Trong Java, phương thức `toString()` được sử dụng để chuyển đổi các giá trị số thành dạng chuỗi, giúp dễ dàng hiển thị hoặc xử lý như một xâu ký tự. Việc này đặc biệt hữu ích khi cần in ra thông tin hoặc kết hợp dữ liệu số với dữ liệu văn bản. Ví dụ, một số nguyên hoặc số thực khi gọi `toString()` sẽ trả về chuỗi biểu diễn giá trị đó.

Ngoài ra, mỗi lớp bao (wrapper class) đều cung cấp các phương thức dạng `<type>Value()` để chuyển từ đối tượng của lớp bao thành giá trị nguyên thủy tương ứng. Chẳng hạn, với đối tượng `Float objF = new Float("4.67")`, ta có thể dùng `objF.floatValue()` để thu được giá trị `float` là `4.67F`, hoặc `objF.intValue()` để lấy giá trị nguyên là `4`. Điều này cho phép linh hoạt thao tác với dữ liệu ở cả dạng đối tượng và dạng nguyên thủy.

Bên cạnh đó, các phương thức `parse<type>()` và `valueOf()` được dùng để chuyển đổi từ xâu ký tự thành giá trị số. Ví dụ, `Integer.parseInt("123")` trả về số nguyên `123`, `Double.parseDouble("1.5")` trả về số thực `1.5`. Với `valueOf()`, ta có thể tạo ra một đối tượng lớp bao từ xâu, như `Double objF2 = Double.valueOf("-36.12")`, sau đó tiếp tục gọi `objF2.longValue()` để lấy giá trị `long` là `-36L`. Nhờ đó, Java hỗ trợ mạnh mẽ việc chuyển đổi qua lại giữa chuỗi và các kiểu dữ liệu số.

Các hằng số

Trong Java, các lớp bao không chỉ cung cấp khả năng chuyển đổi giữa kiểu dữ liệu nguyên thủy và đối tượng, mà còn định nghĩa sẵn nhiều hằng số tiện ích giúp lập trình viên làm việc hiệu quả hơn. Lớp `Boolean` cung cấp hai hằng số `TRUE` và `FALSE` biểu diễn giá trị logic. Lớp `Byte`, `Short`, `Integer`, `Long`, `Float` và `Double` đều có các hằng số `MIN_VALUE` và `MAX_VALUE` để biểu thị giá trị nhỏ nhất và lớn nhất mà kiểu dữ liệu có thể lưu trữ. Riêng `Float` và `Double` còn có thêm các hằng số đặc biệt như `NaN` (Not-a-Number), `NEGATIVE_INFINITY` và `POSITIVE_INFINITY` để xử lý các giá trị toán học đặc biệt. Lớp `Character` cung cấp các hằng số liên quan đến mã Unicode như `MAX_RADIX`, `MIN_RADIX`, giá trị ký tự lớn nhất/nhỏ nhất và các hằng số phân loại ký tự. Việc sử dụng các hằng số này giúp chương trình trở nên rõ ràng, an toàn và dễ bảo trì hơn.

• Boolean • Boolean FALSE • Boolean TRUE • Byte • byte MIN_VALUE • byte MAX_VALUE • Character • int MAX_RADIX • char MAX_VALUE • int MIN_RADIX • char MIN_VALUE • Unicode classification constants • Double • double MAX_VALUE • double MIN_VALUE • double NaN • double NEGATIVE_INFINITY • double POSITIVE_INFINITY	• Float • float MAX_VALUE • float MIN_VALUE • float NaN • float NEGATIVE_INFINITY • float POSITIVE_INFINITY • Integer • int MIN_VALUE • int MAX_VALUE • Long • long MIN_VALUE • long MAX_VALUE • Short • short MIN_VALUE • short MAX_VALUE
---	--

Hình 5.12 Hằng số trong Java

Xét ví dụ sau:

```
double d = (new Integer(Integer.MAX_VALUE)).
           doubleValue();
System.out.println(d); // 2.147483647E9

String input = "test 1-2-3";
int output = 0;
for (int index=0; index<input.length(); index++) {
    char c = input.charAt(index);
    if (Character.isDigit(c))
        output = output * 10 + Character.digit(c, 10);
}
System.out.println(output); //123
```

Trong ví dụ trên, phương thức `Character.isDigit()` trả về giá trị boolean cho biết đầu vào có phải là một số hay không

```
public static int digit(char ch, int radix)
//Returns the numeric value of the character ch in the specified radix.
```

Để ý rằng `Character.digit(c, radix)`: giá trị của ký tự `c` trong hệ cơ số `radix`. Giá trị này biểu diễn ở hệ 10.

Ví dụ:

```
Character.digit('1', 10) = 1;
Character.digit('5', 10) = 5;
Character.digit('A', 10) = -1;
Character.digit('A', 16) = 10;
Character.digit('E', 16) = 14;
```

5.4.4. Xâu

Trong Java, xâu (String) là một đối tượng đặc biệt được sử dụng để lưu trữ và xử lý chuỗi ký tự. Việc tạo xâu có thể được thực hiện trực tiếp bằng cách sử dụng dấu ngoặc kép, chẳng hạn như `String s = "Hello";`. Ngoài ra, Java còn cho phép tạo xâu bằng từ khóa `new`, ví dụ `String s = new String("Hello");`. Cách đầu tiên nhanh gọn và thường được sử dụng nhiều hơn, trong khi cách thứ hai hữu ích khi cần tạo đối tượng xâu mới hoàn toàn, ngay cả khi nội dung trùng lặp.

Một đặc điểm quan trọng của lớp String là tính bất biến (immutable). Khi một xâu đã được tạo, nội dung của nó không thể thay đổi; mọi thao tác chỉnh sửa sẽ tạo ra một đối tượng String mới thay thế. Điều này giúp tăng tính an toàn trong lập trình và tránh những lỗi không mong muốn khi nhiều đối tượng cùng tham chiếu tới một xâu. Ví dụ, khi nối thêm ký tự vào một xâu ban đầu, Java sẽ tạo một xâu mới chứa nội dung đã chỉnh sửa thay vì thay đổi trực tiếp xâu cũ.

Để thao tác và xử lý dữ liệu dạng xâu, lớp String cung cấp một loạt phương thức tiện ích. Các phương thức phổ biến bao gồm `length()` để lấy độ dài xâu, `charAt(int index)` để truy cập ký tự tại vị trí chỉ định, `substring(int start, int end)` để trích xuất một phần xâu, `toLowerCase()` và `toUpperCase()` để chuyển đổi chữ thường và chữ hoa, hay `equals()` và `compareTo()` để so sánh nội dung giữa các xâu. Nhờ hệ thống phương thức phong phú này, việc làm việc với chuỗi trong Java trở nên linh hoạt và hiệu quả.

Ghép xâu

Ghép xâu (string concatenation) là quá trình nối hai hoặc nhiều xâu lại với nhau để tạo thành một xâu mới. Trong Java, cách đơn giản nhất để ghép xâu là sử dụng toán tử cộng (+). Khi sử dụng toán tử này, Java sẽ tự động tạo ra một đối tượng String mới chứa nội dung của các xâu ban đầu được ghép liên tiếp. Ví dụ, `String fullName = firstName + " " + lastName;` sẽ tạo ra một xâu mới bằng cách nối họ

và tên với một khoảng trắng ở giữa. Đây là cách nhanh gọn, dễ đọc và thường được dùng trong các chương trình nhỏ hoặc khi số lượng phép ghép không nhiều.

```
String a = "This" + " is a " + "String";  
//a = "This is a String"
```

Hiểu về cách ghép xâu trong Java cũng bao gồm việc nắm rõ cơ chế tự động chuyển đổi kiểu dữ liệu khi sử dụng với toán tử cộng (+) trong lệnh `println()`. Khi một xâu xuất hiện ở bên trái toán tử +, mọi giá trị phía sau sẽ được tự động chuyển thành chuỗi và nối tiếp vào. Ví dụ, câu lệnh `System.out.println("answer = " + 1 + 2 + 3);` sẽ được thực hiện tuần tự từ trái sang phải: "answer = " ghép với 1 thành "answer = 1", sau đó ghép với 2 thành "answer = 12", rồi ghép tiếp với 3 thành "answer = 123". Ngược lại, nếu đặt phép cộng số học trong ngoặc như `System.out.println("answer = " + (1+2+3));`, Java sẽ tính tổng 1+2+3 trước thành 6, sau đó mới chuyển sang chuỗi và nối vào, tạo ra kết quả "answer = 6". Điều này cho thấy, dấu ngoặc và thứ tự thực hiện phép cộng ảnh hưởng trực tiếp đến kết quả khi ghép xâu.

```
System.out.println("answer = " + 1 + 2 + 3);  
System.out.println("answer = " + (1+2+3));
```

Các phương thức của xâu

Trong Java, lớp 'String' cung cấp nhiều phương thức hữu ích để xử lý và thao tác với chuỗi ký tự. Chẳng hạn, `toLowerCase()` và `toUpperCase()` lần lượt chuyển toàn bộ chuỗi sang dạng chữ thường và chữ in hoa. Phương thức `trim()` loại bỏ khoảng trắng ở đầu và cuối chuỗi, giúp dữ liệu gọn gàng hơn khi nhập từ bàn phím hoặc đọc từ file. Để tìm vị trí xuất hiện đầu tiên của một ký tự trong chuỗi, ta dùng `indexOf()`, chẳng hạn `"Joe Smith".indexOf('e')` trả về '2' vì ký tự 'e' nằm ở vị trí thứ 2 (đếm từ 0). Phương thức `length()` trả về độ dài chuỗi, còn `charAt()` lấy ký tự tại một vị trí cụ thể. Ngoài ra, `substring()` cho phép trích ra một phần chuỗi — nếu truyền vào một tham số, nó lấy từ vị trí đó đến hết chuỗi; nếu truyền vào hai tham số, nó lấy từ vị trí bắt đầu đến ngay trước vị trí kết thúc. Nhờ những phương thức này, việc xử lý chuỗi trong Java trở nên trực quan và linh hoạt hơn.

```
String name = "Joe Smith";  
name.toLowerCase();           // "joe smith"  
name.toUpperCase();           // "JOE SMITH"  
"Joe Smith ".trim();           // "Joe Smith"
```

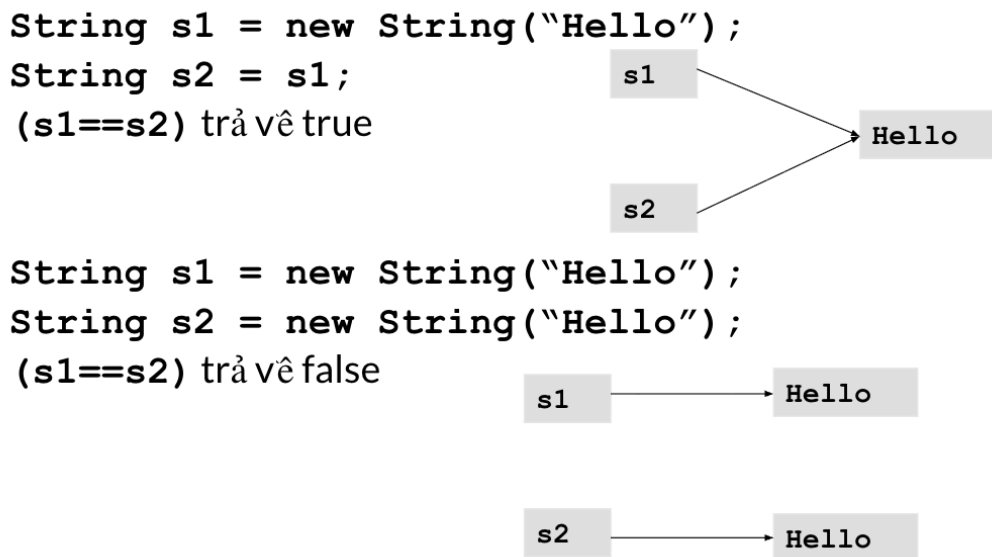
```
"Joe Smith".indexOf('e');           // 2
"Joe Smith".length();               // 9
"Joe Smith".charAt(5);              // 'm'
"Joe Smith".substring(5);           // "mith"
"Joe Smith".substring(2,5);         // "e S"
```

So sánh hai chuỗi

Để so sánh hai chuỗi trong Java, chúng ta không nên sử dụng toán tử ‘==’ vì toán tử này chỉ so sánh địa chỉ tham chiếu (reference) chứ không so sánh nội dung. Thay vào đó, phương thức ‘equals()’ của lớp ‘String’ được sử dụng để so sánh nội dung của hai chuỗi ký tự. Ví dụ, với đoạn lệnh ‘if ("Joe".equals(name)) name += " Smith";’, chương trình sẽ kiểm tra xem biến ‘name’ có cùng nội dung với chuỗi ‘"Joe"’ hay không. Nếu bằng nhau, nó sẽ nối thêm ‘" Smith"’ vào sau ‘name’. Việc đặt chuỗi hằng ‘"Joe"’ ở phía trước phương thức ‘equals()’ còn giúp tránh lỗi ‘NullPointerException’ nếu ‘name’ bị null.

Ngoài phương thức ‘equals()’, Java còn cung cấp ‘equalsIgnoreCase()’ để so sánh hai chuỗi mà không phân biệt chữ hoa hay chữ thường. Chẳng hạn, ‘boolean same = "Joe".equalsIgnoreCase("joe");’ sẽ trả về ‘true’ vì nội dung hai chuỗi giống nhau nếu bỏ qua khác biệt về kiểu chữ. Điều này rất hữu ích khi xử lý dữ liệu nhập từ người dùng, nơi mà cách viết hoa/thường không quan trọng đối với logic so sánh.

Cuối cùng, khi cần so sánh thứ tự từ điển của hai chuỗi (lexicographical order), ta có thể dùng phương thức ‘compareTo()’ hoặc ‘compareToIgnoreCase()’. Các phương thức này trả về giá trị nguyên: bằng 0 nếu hai chuỗi giống nhau, số dương nếu chuỗi gọi phương thức lớn hơn, và số âm nếu nhỏ hơn. Đây là cách phổ biến để sắp xếp danh sách chuỗi hoặc kiểm tra vị trí tương đối của chúng trong bảng chữ cái. So với ‘equals()’, ‘compareTo()’ không chỉ cho biết hai chuỗi có bằng nhau không, mà còn xác định thứ tự sắp xếp của chúng.



Hình 5.13 String equals

Hình ảnh 5.13 minh họa sự khác nhau giữa so sánh bằng toán tử `==` khi áp dụng cho các đối tượng `String` trong Java. Ở ví dụ đầu tiên, biến `s1` được khởi tạo với một đối tượng `String` chứa giá trị "Hello". Sau đó, `s2` được gán trực tiếp bằng `s1`, nghĩa là cả hai biến cùng tham chiếu đến cùng một đối tượng trong bộ nhớ. Do đó, khi so sánh `s1 == s2`, kết quả trả về là `true` vì địa chỉ tham chiếu giống nhau.

Ngược lại, ở ví dụ thứ hai, cả `s1` và `s2` đều được tạo ra bằng từ khóa `new String("Hello")`. Mặc dù nội dung giống nhau, nhưng mỗi lệnh `new` sẽ tạo ra một đối tượng mới trong bộ nhớ, dẫn đến hai biến trỏ đến hai vị trí khác nhau. Vì toán tử `==` chỉ so sánh địa chỉ tham chiếu, nên `s1 == s2` trả về `false`. Điều này nhấn mạnh rằng, khi muốn so sánh nội dung của chuỗi, ta cần dùng phương thức `equals()` thay vì `==`.

Điểm đặc biệt của String

Trong Java, có hai cách phổ biến để khởi tạo một biến kiểu `'String'`. Cách thứ nhất là gán trực tiếp một giá trị chuỗi (string literal) cho biến. Ví dụ: `'String str1 = "Java is Hot";'`. Đây là hình thức khởi tạo ngầm định (implicit construction), trong đó `'str1'` được khai báo là một tham chiếu tới `'String'` và giá trị `"Java is Hot"` được lấy từ bộ nhớ dùng chung dành cho các string literal. Cách này đơn giản, dễ viết và

được khuyến khích sử dụng vì hiệu quả về mặt bộ nhớ.

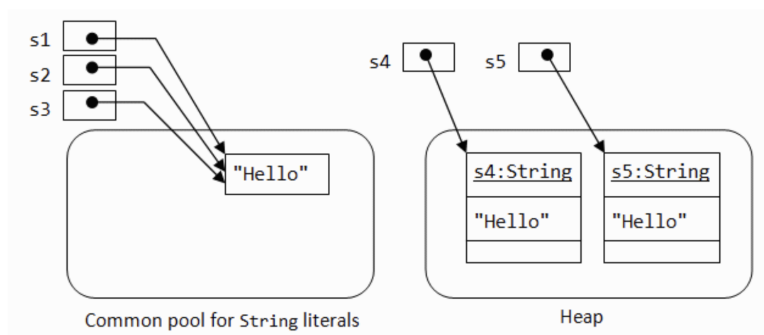
Cách thứ hai là sử dụng toán tử ‘new’ để tạo đối tượng ‘String’. Ví dụ: ‘String str2 = new String("I'm cool");’. Đây là hình thức khởi tạo tường minh (explicit construction), trong đó ‘str2’ được khai báo là một tham chiếu tới ‘String’ và được gán một đối tượng mới được tạo ra trên heap. Mặc dù nội dung chuỗi có thể giống với một literal đã tồn tại, toán tử ‘new’ vẫn buộc JVM tạo ra một đối tượng mới, không tái sử dụng dữ liệu từ bộ nhớ chung.

Cơ chế này liên quan trực tiếp đến khái niệm *String Pool* trong Java. Tất cả các string literal đều được lưu trữ trong một vùng bộ nhớ dùng chung gọi là common pool. Nhờ vậy, các biến có cùng nội dung literal sẽ cùng tham chiếu đến một đối tượng, giúp tiết kiệm bộ nhớ và tăng hiệu suất so sánh chuỗi. Ngược lại, các đối tượng ‘String’ được tạo bằng ‘new’ nằm ngoài pool, lưu trữ riêng lẻ trên heap, nên không tận dụng được lợi thế chia sẻ bộ nhớ này.

```

• String s1 = "Hello";           // String literal
• String s2 = "Hello";           // String literal
• String s3 = s1;                 // same reference
• String s4 = new String("Hello"); // String object
• String s5 = new String("Hello"); // String object

```



Hình 5.14 String Pool

```

String s1 = new String("test");
String s2 = "test";
String s3 = String.valueOf("test");
System.out.println(s1==s2);
System.out.println(s1==s3);

```



```
System.out.println(s2==s3);

String s4 = new String("test");
String s5 = "test";
String s6 = String.valueOf("test");
System.out.println(s1==s4);
System.out.println(s2==s5);
System.out.println(s3==s6);
```

- Nhóm 1:

```
String s1 = new String("test");
String s2 = "test";
String s3 = String.valueOf("test");

System.out.println(s1 == s2);
System.out.println(s1 == s3);
System.out.println(s2 == s3);
```

1. `s1 == s2` → **false**: `s1` được tạo bằng `new String("test")` nên luôn tạo ra một đối tượng mới trên heap. `s2` là một *string literal* nên được lấy từ *String Pool*. Hai biến tham chiếu đến hai đối tượng khác nhau nên so sánh `==` trả về **false**.
2. `s1 == s3` → **false**: `s3` được tạo bởi `String.valueOf("test")` với tham số là literal, nên sẽ trả về chính tham chiếu trong *String Pool*. Do `s1` là đối tượng mới trên heap, hai tham chiếu khác nhau nên kết quả là **false**.
3. `s2 == s3` → **true**: Cả hai đều trỏ đến cùng một literal `"test"` trong *String Pool* nên địa chỉ giống nhau.

- Nhóm 2:

```
String s4 = new String("test");
String s5 = "test";
String s6 = String.valueOf("test");

System.out.println(s1 == s4);
System.out.println(s2 == s5);
```

```
System.out.println(s3 == s6);
```

1. `s1 == s4` $\rightarrow$ **false**: Mỗi lần `new String("test")` tạo ra một đối tượng mới trên heap, nên `s1` và `s4` khác nhau về tham chiếu.
2. `s2 == s5` $\rightarrow$ **true**: Cả hai đều là literal `"test"`, dùng chung tham chiếu trong *String Pool*.
3. `s3 == s6` $\rightarrow$ **true**: Cả hai được tạo từ `String.valueOf("test")` với tham số literal, nên cùng trả về tham chiếu trong *String Pool*.

- **Kết luận:**

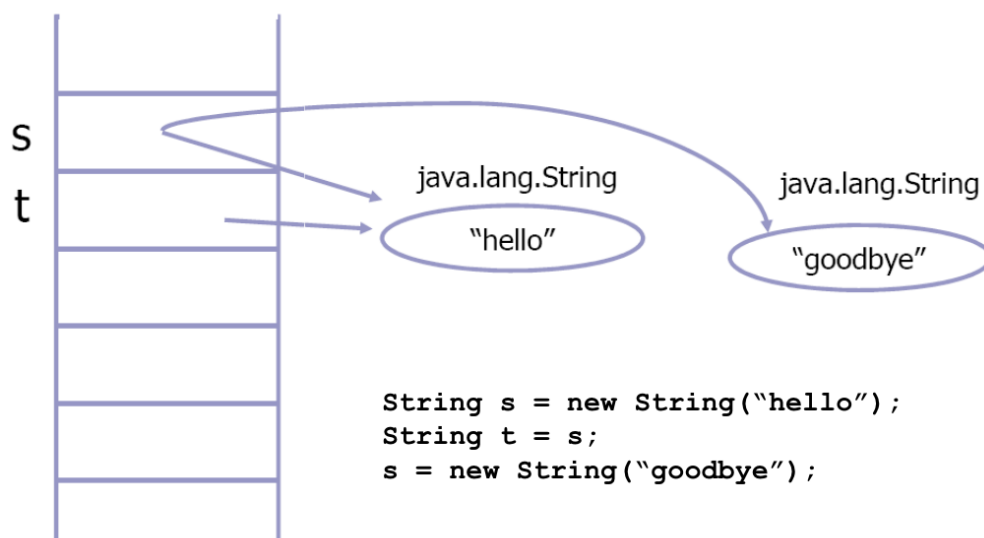
- `new String("...")`: Luôn tạo đối tượng mới trên heap, so sánh `==` với literal sẽ **false**.
- Literal hoặc `String.valueOf(literal)`: Dùng chung tham chiếu trong *String Pool*, so sánh `==` có thể **true** nếu cùng literal.
- `==` so sánh địa chỉ (tham chiếu), muốn so nội dung phải dùng `.equals()`.

5.4.5. StringBuffer

Trong Java, `String` là một kiểu dữ liệu bất biến (immutable). Điều này có nghĩa là một đối tượng `String`, sau khi được tạo ra, sẽ không thể thay đổi giá trị. Các chuỗi ký tự thuộc lớp `String` được thiết kế để giữ nguyên nội dung của chúng, đảm bảo tính toàn vẹn dữ liệu và an toàn khi nhiều luồng cùng truy cập.

Vì đặc tính bất biến, khi ta thực hiện các thao tác ghép nối (concatenation) giữa nhiều chuỗi `String`, chương trình sẽ không chỉnh sửa trực tiếp chuỗi ban đầu. Thay vào đó, một đối tượng `String` mới sẽ được tạo ra để chứa kết quả ghép nối. Việc tạo ra nhiều đối tượng mới như vậy có thể khiến chương trình tiêu tốn thêm bộ nhớ và giảm hiệu năng, đặc biệt khi thực hiện các phép nối chuỗi nhiều lần trong vòng lặp.

Ngược lại, `StringBuffer` là một kiểu dữ liệu biến đổi (mutable). Các đối tượng thuộc lớp `StringBuffer` cho phép thay đổi trực tiếp giá trị bên trong sau khi được tạo ra mà không cần sinh ra đối tượng mới. Điều này giúp tiết kiệm bộ nhớ và tăng tốc độ xử lý khi phải thao tác nhiều lần trên cùng một chuỗi, đặc biệt là trong các tình huống cần ghép nối hoặc chỉnh sửa nội dung liên tục.



Hình 5.15 StringBuffer

Hình vẽ 5.15 mô tả quá trình thay đổi tham chiếu của biến trong Java khi làm việc với đối tượng String.

Ban đầu, lệnh `String s = new String("hello");` tạo một đối tượng String mới với nội dung "hello" trong vùng nhớ heap. Biến tham chiếu `s` trỏ tới đối tượng này. Sau đó, lệnh `String t = s;` khiến biến `t` trỏ tới cùng một đối tượng "hello" mà `s` đang trỏ, tức là cả `s` và `t` cùng chia sẻ một vùng nhớ.

Tiếp theo, lệnh `s = new String("goodbye");` tạo ra một đối tượng String mới với nội dung "goodbye" và thay đổi tham chiếu của `s` để trỏ tới đối tượng mới này. Lúc này, `t` vẫn trỏ tới đối tượng "hello" ban đầu, còn `s` trỏ tới "goodbye". Điều này cho thấy, khi gán lại biến tham chiếu, ta chỉ thay đổi mối liên kết giữa biến và đối tượng chứ không làm thay đổi nội dung của đối tượng mà biến khác đang trỏ tới.

StringBuffer là một lớp trong Java được thiết kế để làm việc với các chuỗi ký tự có khả năng thay đổi nội dung sau khi được tạo ra. Khác với lớp String (bất biến), StringBuffer cho phép thêm, sửa, hoặc xóa ký tự mà không cần tạo đối tượng mới. Điều này đặc biệt hữu ích trong những tình huống mà chuỗi cần được chỉnh sửa nhiều lần.

Việc sử dụng StringBuffer trở nên cần thiết khi ta dự đoán rằng nội dung của chuỗi sẽ thay đổi, chẳng hạn như khi xử lý dữ liệu văn bản từ các nguồn động (ví dụ: đọc từng dòng từ một tệp văn bản và ghép chúng lại). Trong các tình huống

này, nếu dùng String thông thường, mỗi lần thay đổi sẽ tạo ra một đối tượng mới trong bộ nhớ, gây tốn kém tài nguyên.

Ngoài ra, StringBuffer được tối ưu hóa cho việc xây dựng và ghép nối chuỗi. Thực tế, khi lập trình viên dùng toán tử + để nối chuỗi trong nhiều trường hợp, trình biên dịch Java sẽ ngầm chuyển đổi quá trình này thành thao tác trên StringBuffer hoặc StringBuilder. Điều này giúp giảm số lượng đối tượng tạm thời được tạo ra và tăng hiệu suất xử lý chuỗi.

“Tính biến đổi” (mutability) nghĩa là bản thân đối tượng có thể thay đổi trạng thái sau khi đã được tạo. Khi nhiều biến/thuộc tính cùng tham chiếu đến cùng một đối tượng (cùng địa chỉ trên heap), thì bất kỳ thao tác nào làm thay đổi trạng thái bên trong đối tượng đó sẽ hiển thị ngay qua tất cả các tham chiếu. Nói cách khác, mọi “quan hệ” tới đối tượng (biến cục bộ, thuộc tính lớp, phần tử trong danh sách...) đều “nhìn” thấy giá trị mới vì chúng đang cùng trỏ đến một thực thể duy nhất.

Ví dụ với một đối tượng mutable:

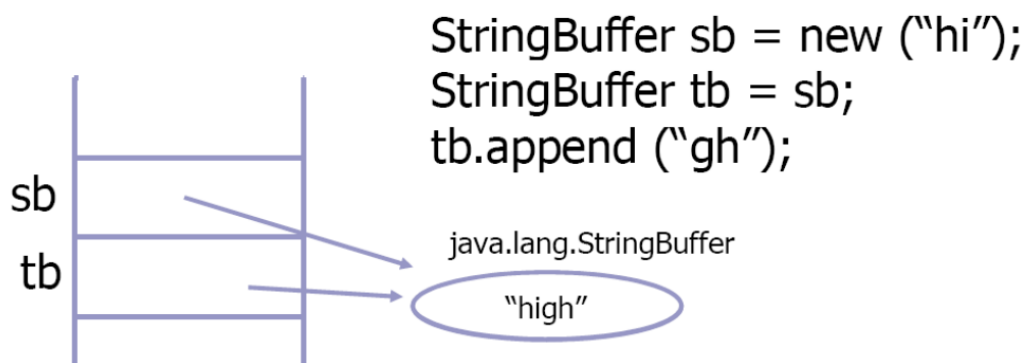
```
import java.util.ArrayList;
import java.util.List;

List<Integer> a = new ArrayList<>();
a.add(1);

List<Integer> b = a;           // b và a cùng tham chiếu đến MỘT danh sách
b.add(2);                     // sửa NỘI DUNG của danh sách

System.out.println(a);       // [1, 2]
System.out.println(b);       // [1, 2]
```

Cả a và b đều in [1, 2] vì đối tượng danh sách đã bị biến đổi (thêm phần tử 2). Tương tự, StringBuffer/StringBuilder là mutable: sb.append("x") sẽ làm sb (và mọi tham chiếu khác tới cùng sb) có nội dung mới.



Hình 5.16 Tính biến đổi

Phân biệt thật rõ biến đổi đối tượng với gán lại tham chiếu:

```
List<Integer> x = new ArrayList<>();
List<Integer> y = x;    // y trở cùng đối tượng với x
x.add(10);              // biến đổi -> y cũng thấy 10

x = new ArrayList<>();  // GÁN LẠI: x trở sang đối tượng KHÁC
x.add(99);

System.out.println(y); // [10] (không đổi, vẫn là đối tượng cũ)
System.out.println(x); // [99] (đối tượng mới)
```

Biến đổi (mutation): thay đổi nội dung nội tại của đối tượng mọi tham chiếu cùng đối tượng đều thấy cập nhật.

Gán lại (reassignment): chỉ thay đổi biến tham chiếu trở sang đối tượng khác các tham chiếu còn lại không bị ảnh hưởng.

Hệ quả thực tiễn: mutability giúp cập nhật dữ liệu hiệu quả nhưng dễ gây “tác dụng phụ” khi có nhiều nơi cùng dùng một đối tượng (aliasing). Khi cần an toàn, hãy cân nhắc dùng đối tượng bất biến (như String, BigDecimal) hoặc tạo bản sao phòng vệ (defensive copy) trước khi lưu/trao đổi đối tượng mutable; còn khi cần hiệu năng ghép/chỉnh sửa chuỗi lớn, dùng StringBuilder/StringBuffer và ý thức rằng mọi tham chiếu đến cùng thể hiện sẽ thấy thay đổi.

Các chuỗi của lớp `String` được thiết kế để không thay đổi, nó cho phép tối ưu hóa việc lưu trữ khi thực hiện chương trình. Khi các chuỗi được ghép nối với nhau một đối tượng mới được tạo ra để lưu trữ kết quả. Chính vì vậy ghép nối chuỗi thông thường rất tốn kém về bộ nhớ.

```
StringBuffer buffer = new StringBuffer(15);
buffer.append("This is ") ;
buffer.append("String") ;
buffer.insert(7," a") ;
buffer.append('.');
System.out.println(buffer.length());           // 17
System.out.println(buffer.capacity());         // 32
String output = buffer.toString() ;
System.out.println(output); // "This is a String."
```

Lớp `StringBuffer` cung cấp các đối tượng chuỗi có thể thay đổi, chúng ta sử dụng các chuỗi `buffer` khi dự đoán rằng các ký tự trong chuỗi có thể thay đổi. Và thường sử dụng trong trường hợp xây dựng các chuỗi một cách linh động, ví dụ như đọc dữ liệu text từ một tệp tin. `String buffer` an toàn khi sử dụng trong môi trường đa luồng.

5.4.6. Math

Lớp `Math` trong Java cung cấp nhiều hằng số và hàm toán học thường dùng. Tất cả các thành phần trong lớp `Math` đều là **static**, do đó có thể sử dụng trực tiếp thông qua cú pháp `Math.tênHàm(...)` hoặc `Math.tênHằng` mà không cần tạo đối tượng.

1. Các hằng toán học

- `Math.E`: Giá trị gần đúng của số e (cơ số logarit tự nhiên, ≈ 2.71828).
- `Math.PI`: Giá trị gần đúng của số π (≈ 3.14159).

2. Các hàm toán học phổ biến

- **Hàm so sánh**: `max(a, b)`, `min(a, b)` – tìm giá trị lớn nhất/nhỏ nhất.
- **Hàm giá trị tuyệt đối và làm tròn**:

- `abs(x)`: giá trị tuyệt đối.
- `floor(x)`: làm tròn xuống.
- `ceil(x)`: làm tròn lên.
- `round(x)`: làm tròn đến số nguyên gần nhất.

- **Hàm số học:**

- `sqrt(x)`: căn bậc hai.
- `pow(a, b)`: lũy thừa a^b .
- `log(x)`, `exp(x)`: logarit và hàm mũ.

- **Hàm lượng giác:** `sin`, `cos`, `tan`, `asin`, `acos`, `atan ...` (làm việc với radian).

- **Sinh số ngẫu nhiên:** `random()` trả về số thực ngẫu nhiên trong khoảng $[0.0, 1.0)$.

3. Ví dụ sử dụng

```
double r = Math.sqrt(25);           // Kết quả: 5.0
double area = Math.PI * r * r;      // Tính diện tích hình tròn
double x = Math.random();           // Số ngẫu nhiên trong [0.0, 1.0)
```

5.4.7. Lớp System

Lớp `System` trong Java là một lớp **final** cung cấp nhiều phương thức và trường **static** dùng để truy cập và thao tác với các tài nguyên và đặc tính của môi trường thực thi Java. Các thành phần quan trọng của lớp `System` gồm:

- **Dòng vào/ra chuẩn:**

- `System.in`: Dòng nhập chuẩn (thường là bàn phím).
- `System.out`: Dòng xuất chuẩn (thường là màn hình console).
- `System.err`: Dòng xuất lỗi chuẩn.

- **Thời gian hệ thống:**

- `currentTimeMillis()`: Trả về thời gian hiện tại tính bằng mili-giây kể từ mốc 1/1/1970 (Unix epoch).
- `nanoTime()`: Trả về thời gian chính xác cao hơn tính bằng nano-giây (thích hợp đo hiệu năng).

- **Quản lý bộ nhớ:**

- `gc()`: Yêu cầu trình dọn rác (Garbage Collector) thu hồi bộ nhớ (không đảm bảo thực thi ngay lập tức).

- **Biến môi trường và thuộc tính hệ thống:**

- `getProperty(key)`: Lấy giá trị thuộc tính hệ thống, ví dụ: `user.dir`, `java.version`.
- `getenv(name)`: Lấy giá trị biến môi trường.

- **Kết thúc chương trình:**

- `exit(status)`: Kết thúc chương trình với mã thoát `status` (0 = thành công, khác 0 = lỗi).

Ví dụ minh họa:

```
System.out.println("Java version: " + System.getProperty("java.version"));
long start = System.nanoTime();
// ... code cần đo thời gian ...
long end = System.nanoTime();
System.out.println("Thời gian chạy: " + (end - start) + " ns");
```

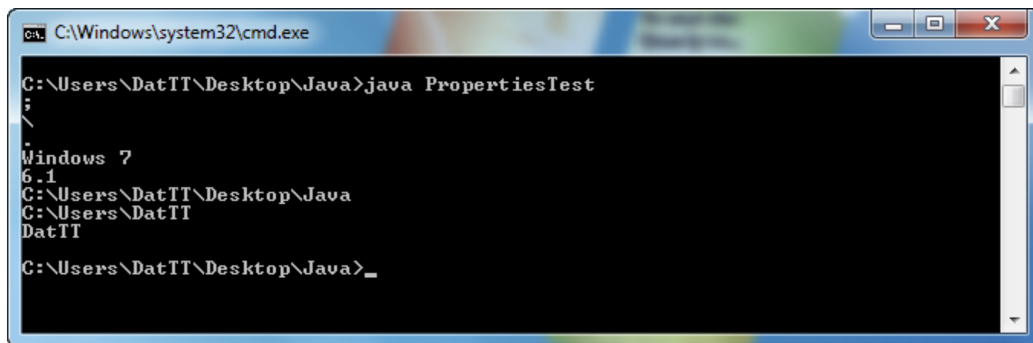
`currentTimeMillis()` Phương thức `currentTimeMillis()` của lớp `System` trả về thời gian hiện tại tính bằng mili-giây, được đo từ mốc thời gian chuẩn *Unix Epoch* (00:00:00 GMT ngày 1 tháng 1 năm 1970). Giá trị này thường được dùng để đo thời gian thực thi của một đoạn mã, tính toán khoảng thời gian giữa hai sự kiện hoặc ghi nhận thời điểm xảy ra một hành động trong chương trình. Tuy nhiên, độ chính xác và độ phân giải của giá trị trả về phụ thuộc vào hệ điều hành và phần cứng.

`exit()` & `gc()` Phương thức `exit(status)` cho phép kết thúc hoạt động của Java Virtual Machine (JVM) ngay lập tức, với mã thoát `status` xác định trạng thái kết thúc (0 thường chỉ ra chương trình kết thúc thành công, các giá trị khác biểu thị lỗi). Phương thức `gc()` yêu cầu bộ thu gom rác (*Garbage Collector*) hoạt động để giải phóng bộ nhớ của các đối tượng không còn được tham chiếu. Mặc dù lời gọi `gc()` chỉ là gợi ý cho JVM, nó không đảm bảo quá trình thu gom rác sẽ diễn ra ngay lập tức.

Các phương thức lấy thuộc tính hệ thống Lớp `System` cung cấp các phương thức như `getProperty(key)` và `getenv(name)` để truy cập các thông tin cấu hình của hệ thống. Ví dụ, `System.getProperty("java.version")` trả về phiên bản của Java Runtime Environment, `System.getProperty("java.home")` trả về thư mục cài đặt Java, và `System.getProperty("user.dir")` cho biết thư mục làm việc hiện tại. Những thông tin này hữu ích khi chương trình cần thích ứng với môi trường thực thi hoặc hiển thị cấu hình hệ thống cho người dùng.

```
import java.util.Properties;

public class PropertiesTest {
    public static void main(String[] args) {
        System.out.println(System.getProperty("path.separator"));
        System.out.println(System.getProperty("file.separator"));
        System.out.println(System.getProperty("java.class.path"));
        System.out.println(System.getProperty("os.name"));
        System.out.println(System.getProperty("os.version"));
        System.out.println(System.getProperty("user.dir"));
        System.out.println(System.getProperty("user.home"));
        System.out.println(System.getProperty("user.name"));
    }
}
```



Hình 5.17 Kết quả kiểm tra system

Hình 5.17 mô tả kết quả thực thi đoạn mã nguồn trên

5.5. Ví dụ và bài tập

Bài tập 1

Tiếp bài tập 2 của bài học trước, sử dụng thành viên lớp để cài đặt đếm số đối tượng của lớp NhanVien được tạo ra tại bất kỳ thời điểm nào, ở bất kỳ đâu. Cài đặt minh họa cách thức sử dụng.

Bài tập 2

Cài đặt phương thức, đầu vào là số lượng bất kỳ các đối tượng lớp NhanVien, đầu ra là tổng lương của các đối tượng này.

Bài tập 3

Sử dụng lớp System để đếm thời gian phép cộng xâu nhiều lần với String, và dùng phương thức append với StringBuffer để so sánh hiệu năng.

Chương 6

Kết tập và kế thừa

6.1. Tái sử dụng mã nguồn	128
6.2. Kết tập (Aggregation)	131
6.3. Kế thừa (Inheritance)	140
6.4. Ví dụ và bài tập	155

Mục tiêu: Nội dung của bài học giới thiệu về hai khái niệm cốt lõi trong lập trình hướng đối tượng: kết tập và kế thừa. Bài học bắt đầu với việc giải thích khái niệm tái sử dụng mã nguồn, một nguyên lý quan trọng giúp tối ưu hóa quá trình phát triển phần mềm. Phần tiếp theo phân tích bản chất và các khái niệm liên quan đến kết tập và kế thừa, đồng thời so sánh sự khác biệt giữa hai cơ chế này. Bài học cung cấp hướng dẫn cách biểu diễn kết tập và kế thừa trong UML, giúp thể hiện thiết kế và trực quan hóa các mối quan hệ giữa các lớp. Nội dung cuối tập trung vào việc áp dụng các kỹ thuật và nguyên lý về kết tập và kế thừa trên ngôn ngữ lập trình Java, giúp vận dụng kiến thức vào thực tế lập trình.

Nội dung:

- Tái sử dụng mã nguồn
- Kết tập (Aggregation)
- Kế thừa (Inheritance)
- Ví dụ và bài tập

6.1. Tái sử dụng mã nguồn

Tái sử dụng mã nguồn (Re-usability) là một khái niệm quan trọng trong phát triển phần mềm, tập trung vào việc sử dụng lại các đoạn mã đã được viết trước đó. Khi phát triển các ứng dụng phức tạp, việc tái sử dụng mã nguồn giúp tiết kiệm thời gian, giảm thiểu lỗi và tăng hiệu quả làm việc.

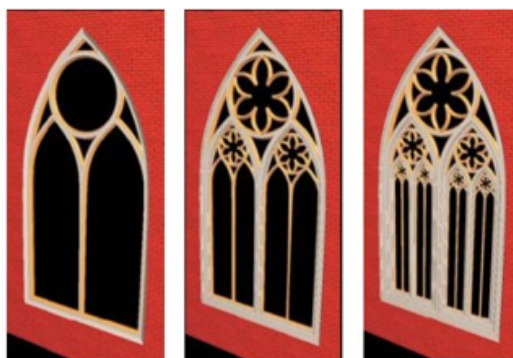
- Trong lập trình cấu trúc truyền thống, tái sử dụng mã được thực hiện thông qua việc xây dựng và gọi các hàm hoặc chương trình con. Đây là cách tiếp cận cơ bản nhất để tránh việc viết lại cùng một đoạn mã nhiều lần.
- Khi chuyển sang lập trình hướng đối tượng (OOP), khái niệm tái sử dụng mã nguồn được mở rộng và trở nên mạnh mẽ hơn. Trong thế giới thực, chúng ta thường gặp nhiều loại đối tượng có thuộc tính và hành vi tương tự hoặc liên quan đến nhau. OOP cho phép mô hình hóa các mối quan hệ này một cách tự nhiên, tạo điều kiện thuận lợi cho việc tái sử dụng mã.

Hình 6.1 minh họa ba mẫu cửa sổ phong cách Gothic với mức độ phức tạp tăng dần, thể hiện cách các thành phần cơ bản có thể được tái sử dụng và mở rộng để tạo ra các cấu trúc phức tạp hơn. Mẫu đầu tiên có thiết kế đơn giản với một vòm nhọn cơ bản, trong khi các mẫu tiếp theo bao gồm nhiều chi tiết trang trí và phân chia không gian. Sự tiến triển từ cửa sổ đơn giản đến phức tạp cho thấy khả năng tái sử dụng theo cách có hệ thống và có tổ chức. Thay vì phải thiết kế lại từ đầu mỗi kiểu cửa sổ, kiến trúc sư chỉ cần điều chỉnh và bổ sung vào mẫu cơ bản các yếu tố trang trí cụ thể. Quá trình này giống như cách lập trình viên tận dụng các đoạn mã đã có để xây dựng mã nguồn mới với chức năng mở rộng, giúp tiết kiệm thời gian và đảm bảo tính nhất quán trong toàn bộ hệ thống phần mềm.

Câu hỏi quan trọng đặt ra là: "Làm thế nào để tái sử dụng lớp (class) đã viết?" Trong lập trình hướng đối tượng, lớp là đơn vị cơ bản đóng gói cả dữ liệu (thuộc tính) và hành vi (phương thức) liên quan đến một khái niệm. Do đó lớp (class) tạo thành một mô hình hoàn chỉnh và độc lập có thể tái sử dụng. Khi quyết định sử dụng lại một lớp đã viết, chúng ta cần phân tích bản chất của mối quan hệ giữa lớp hiện có và lớp mới. Từ đó có thể cân nhắc lựa chọn cơ chế và kỹ thuật tái sử dụng mã phù hợp.

Ba cách chính để tái sử dụng lớp đã có, mỗi cách đều có những đặc điểm và ứng dụng riêng.

- *Sao chép* lớp cũ để tạo thành một lớp mới. Đây là cách đơn giản và dễ thực hiện nhất. Tuy nhiên, phương pháp này không được khuyến khích vì dẫn đến sự dư thừa mã nguồn và gây khó khăn trong quản lý khi có sự thay đổi. Khi cần sửa đổi, chúng ta phải cập nhật ở nhiều nơi, dễ dẫn đến sai sót và không đồng bộ giữa các phiên bản.



Hình 6.1 Các cửa sổ có cấu trúc tương đồng và độ phức tạp tăng dần

- Tạo ra lớp mới là sự *tập hợp* hoặc *sử dụng các đối tượng* của lớp cũ, còn gọi là **Kết tập (Aggregation)**. Lớp mới chứa tham chiếu đến đối tượng của lớp đã có như một thành phần. Kết tập cho phép tái sử dụng chức năng mà không tạo ra sự phụ thuộc chặt chẽ giữa các lớp, tăng tính linh hoạt và dễ bảo trì.
- Tạo ra lớp mới trên cơ sở *phát triển* từ lớp cũ, tức là **Kế thừa (Inheritance)**. Đây là cơ chế mạnh mẽ để tạo ra các lớp chuyên biệt từ các lớp tổng quát, hình thành tổ chức phân cấp các lớp một cách logic và chặt chẽ.

Ưu điểm của tái sử dụng mã nguồn

Tái sử dụng mã có thể được ví như chiếc mỏ lết trong hộp dụng cụ sửa chữa - đây là một công cụ đa năng có thể được sử dụng trong nhiều tình huống khác nhau.

Trước hết, việc tái sử dụng mã nguồn giúp giảm thiểu công sức và chi phí phát triển phần mềm. Thay vì phải viết lại từ đầu các chức năng đã tồn tại, lập trình viên có thể tận dụng các lớp và thành phần đã được xây dựng trước đó. Ví dụ như các thư viện chuẩn và các framework (Spring, Hibernate, v.v.) đã cung cấp sẵn nhiều lớp và giao diện có thể tái sử dụng, giúp tiết kiệm thời gian và nguồn lực đáng kể.

Tái sử dụng mã nguồn góp phần nâng cao chất lượng phần mềm. Các lớp và thành phần được tái sử dụng thường đã trải qua quá trình kiểm thử và tối ưu hóa, do đó có độ tin cậy cao hơn so với mã nguồn được viết mới. Khi sử dụng các thành phần phần đã được chứng minh là hoạt động tốt, chúng ta giảm thiểu khả năng xuất hiện lỗi và nâng cao tính ổn định của hệ thống.

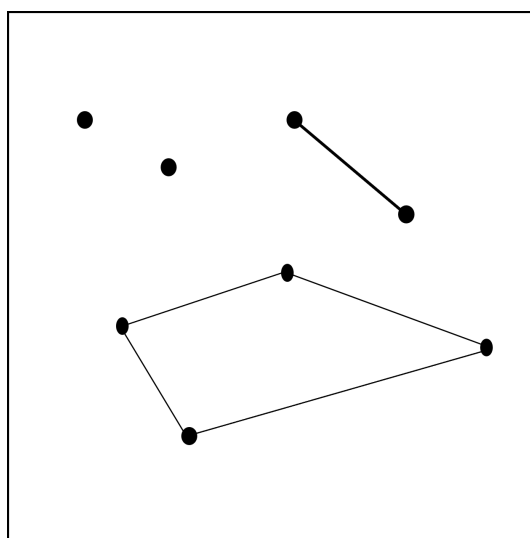
Tái sử dụng mã nguồn nâng cao khả năng mô hình hóa thế giới thực. Thông qua các cơ chế như kết tập và kế thừa, lập trình hướng đối tượng cho phép tạo ra các mô hình phần mềm phản ánh chính xác cấu trúc và mối quan hệ của các đối tượng trong thế giới thực.

Cuối cùng, tái sử dụng mã nguồn nâng cao khả năng bảo trì (maintainability) của phần mềm. Khi một lỗi được phát hiện hoặc một cải tiến được thực hiện trong một thành phần được tái sử dụng, tất cả các ứng dụng sử dụng thành phần đó đều được hưởng lợi từ sự cập nhật này. Điều này giúp giảm thiểu công sức bảo trì và đảm bảo tính nhất quán của phần mềm.

6.2. Kết tập (Aggregation)

6.2.1. Bản chất của kết tập

Chúng ta xem xét một ví dụ về các khái niệm trong hình học phẳng như được minh họa trong hình 6.2. Một Điểm (Point) là một đối tượng đơn giản chứa thông tin về tọa độ x, y. Khi xây dựng một Đoạn thẳng, chúng ta cần 2 điểm để xác định duy nhất cho đoạn thẳng. Khi xây dựng một Tứ giác, chúng ta kết nối 4 điểm thể hiện cho 4 đỉnh để xác định hình dạng của Tứ giác. Đối tượng Điểm có thể được sử dụng trong nhiều khái niệm hình học phẳng khác như Tam giác, Đường tròn (để xác định tâm), v.v. Đây là những ví dụ điển hình về cách các đối tượng phức tạp có thể được xây dựng từ các đối tượng đơn giản hơn thông qua mối quan hệ kết tập trong lập trình hướng đối tượng.



Hình 6.2 Hình Tứ giác được tạo thành từ các Điểm

Khái niệm **Kết tập (Aggregation)** thể hiện mối quan hệ "has-a" (chứa/có) hoặc "is-a-part-of" (là một phần của) giữa các đối tượng, cho phép một lớp chứa

đối tượng của lớp khác như một thành phần của nó. Trong ví dụ trên, Tứ giác có quan hệ kết tập với Điểm - một Tứ giác chứa (has-a) 4 đối tượng Điểm, hay nói cách khác, mỗi Điểm là một phần (is-a-part-of) của Tứ giác. Tứ giác có thể tái sử dụng thông tin của điểm (tọa độ x, y) trong các hoạt động của nó (tính toán kích thước, diện tích, v.v.). Ở chiều ngược lại, Điểm là một phần của Tứ giác, nhưng nó có thể tồn tại độc lập với Tứ giác và có thể được sử dụng trong nhiều ngữ cảnh khác nhau.

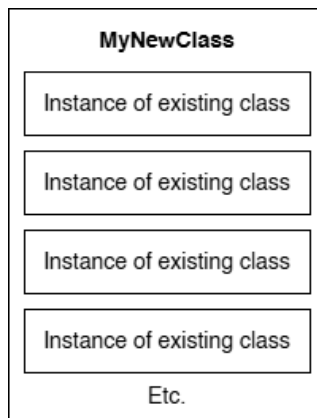
Như vậy bản chất của kết tập là việc tạo ra các đối tượng của các lớp có sẵn và sử dụng chúng như thành viên trong lớp mới. Điều này cho phép chúng ta tái sử dụng mã nguồn một cách hiệu quả thông qua việc sử dụng lại *các đối tượng* đã được định nghĩa trước đó. Trong mối quan hệ kết tập, chúng ta phân biệt hai vai trò chính: lớp mới đóng vai trò là **lớp toàn thể (Aggregate/Whole)**, trong khi lớp cũ đóng vai trò là **lớp thành phần (Part)**.

Lớp toàn thể chứa các đối tượng của lớp thành phần. Đồng thời, từ góc độ của lớp thành phần, nó được xem là "một phần" (is-a-part of) của lớp toàn thể. Khả năng tái sử dụng mã trong mối quan hệ kết tập được thực hiện thông qua việc tham chiếu đến các đối tượng thành phần. Lớp toàn thể có thể tái sử dụng cả dữ liệu (các thuộc tính) và các hành vi (các phương thức) của đối tượng thành phần mà không cần phải triển khai lại chúng.

Hình 6.3 mô tả một lớp mới (MyNewClass) chứa nhiều thể hiện (instances) của các lớp đã tồn tại. Mỗi thể hiện này đều là một đối tượng độc lập, được tạo ra từ các lớp có sẵn và được sử dụng trong lớp mới. Điều này minh họa trực quan cho cách kết tập hoạt động - lớp mới chỉ cần "tập hợp" các đối tượng của các lớp hiện có để tạo thành một đối tượng phức tạp hơn. Quan hệ kết tập phản ánh cấu trúc tự nhiên của nhiều thực thể trong thế giới thực. Ví dụ, một chiếc xe được tạo thành từ các thành phần là khung xe, động cơ, vỏ xe và bánh xe. Một điểm cần lưu ý là các đối tượng của lớp thành phần có thể tồn tại độc lập với lớp toàn thể. Đây là đặc điểm phân biệt quan hệ kết tập với quan hệ hợp thành (composition) - một mối quan hệ chặt chẽ hơn.

6.2.2. Biểu diễn kết tập bằng UML

Trong UML, mối quan hệ kết tập được biểu diễn bằng một đường thẳng nối hai lớp, với một "hình thoi rỗng" đặt ở đầu của lớp toàn thể. Hình thoi này là ký hiệu đặc trưng giúp phân biệt kết tập với các mối quan hệ khác như kế thừa. Tại các đầu của mối quan hệ trên đường kết nối, chúng ta có thể cung cấp thông tin về bội số quan hệ (multiplicity) và chỉ định tên vai trò (rolename).



Hình 6.3 Lớp MyNewClass chứa nhiều thể hiện (instances) của các lớp đã tồn tại

Bội số quan hệ (multiplicity) chỉ ra số lượng đối tượng của lớp tham gia vào mối quan hệ. Nó xác định có bao nhiêu thể hiện (instance) của một lớp có thể hoặc phải liên kết với một thể hiện của lớp khác. Giá trị của bội số là cơ sở để triển khai cài đặt thuộc tính, đặc biệt khi xác định một thuộc tính là đối tượng đơn lẻ hay tập hợp các đối tượng (mảng, danh sách, v.v.). Có nhiều cách biểu diễn bội số:

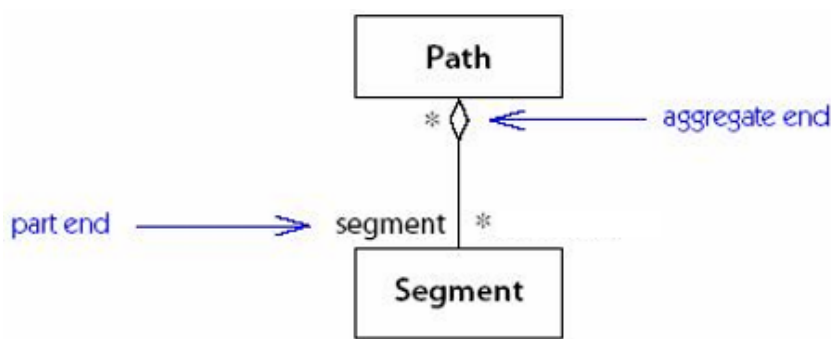
- Một số nguyên dương cụ thể như 1, 2, 3,... chỉ ra số lượng chính xác các đối tượng.
- Dải số như 0..1 (từ 0 đến 1), 2..4 (từ 2 đến 4) chỉ ra phạm vi số lượng đối tượng có thể tham gia.
- Ký hiệu * biểu thị "bất kỳ số nào", thường hiểu là từ 0 đến nhiều.
- Nếu không có bội số được chỉ định, mặc định là 1.

Tên vai trò (rolename) là nhãn (label) được gán cho mỗi đầu của mối quan hệ giữa hai lớp. Thông tin này mô tả chức năng hoặc ý nghĩa của một đối tượng khi nó được liên kết với đối tượng kia trong mối quan hệ. Tên vai trò thường được ánh xạ thành tên thuộc tính trong lớp khi triển khai cài đặt. Tên phải ngắn gọn, rõ ràng, thường dùng danh từ số ít hoặc số nhiều tùy theo bội số. Nếu không chỉ định tên vai trò, mặc định sẽ sử dụng tên của lớp với chữ cái đầu tiên viết thường.

Ví dụ, hình 6.4 minh họa mối quan hệ kết tập giữa class **Path** và class **Segment**. Trong sơ đồ, lớp Path đóng vai trò là lớp tổng thể (aggregate), trong khi lớp Segment là thành phần (part) cấu thành nên Path. Mối quan hệ này được đánh dấu bằng một hình thoi rỗng nối từ Path đến Segment, biểu thị rằng một đối tượng Path được cấu tạo từ nhiều đối tượng Segment, nhưng các đối tượng Segment vẫn có thể tồn

tại độc lập với Path. Quan hệ giữa hai lớp là quan hệ "nhiều-nhiều" (biểu thị bằng ký hiệu * ở hai đầu), nghĩa là một Path có thể bao gồm nhiều Segment, và một Segment có thể được sử dụng trong nhiều Path. Trong một thiết kế khác, nếu mỗi Segment chỉ thuộc về một Path, thì bội số phía Path có thể là 1 thay vì *. "segment" là tên vai trò phía class Segment, được dùng để khai báo thuộc tính trong class Path như đoạn mã dưới đây:

```
public class Path {
    private List<Segment> segment;
}
```



Hình 6.4 Biểu diễn quan hệ kết tập trong UML

Một ví dụ khác, hình 6.5 thể hiện mối quan hệ hợp thành (composition) trong biểu đồ lớp UML. Đây là một dạng đặc biệt của kết tập (aggregation), trong đó các đối tượng thành phần phụ thuộc hoàn toàn vào đối tượng tổng thể và không thể tồn tại độc lập. Trong ví dụ này, lớp Window là một lớp tổng hợp (composite class), chứa ba thành phần (part classes): Slider, Header và Panel. Mỗi quan hệ hợp thành được biểu diễn bằng đường thẳng có **hình thoi đặc (filled diamond)** tại đầu của lớp tổng thể. Đây là ký hiệu đặc trưng để phân biệt hợp thành với kết tập thông thường (sử dụng hình thoi rỗng).

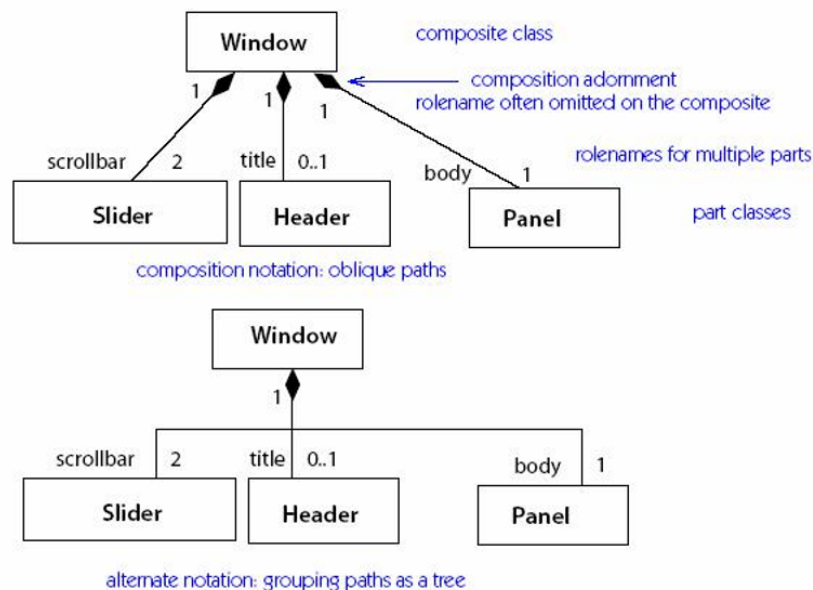
Biểu đồ còn thể hiện bội số (multiplicity) của mỗi thành phần: Window có 2 Slider, Window có 0 hoặc 1 Header và Window có đúng 1 Panel. Trong mỗi quan hệ này, các tên vai trò (rolenames) được chỉ định rõ ràng cho từng thành phần: "scrollbar" cho Slider, "title" cho Header, và "body" cho Panel. Vòng đời của các đối tượng thành phần gắn liền với vòng đời của đối tượng chứa. Khi đối tượng Window bị hủy, tất cả các đối tượng Slider, Header và Panel thuộc về nó cũng sẽ bị

hủy theo. Mỗi quan hệ hợp thành này có thể được triển khai như đoạn mã nguồn sau:

```
public class Window {
    private Slider[] scrollbars = new Slider[2]; // Multiplicity: 2
    private Header title;                       // Multiplicity: 0..1
    private Panel body;                         // Multiplicity: 1

    public Window() {
        // Khởi tạo các thành phần - thể hiện mối quan hệ hợp thành
        scrollbars[0] = new Slider();
        scrollbars[1] = new Slider();
        title = new Header(); // title có thể được khởi tạo hoặc không
        body = new Panel();
    }

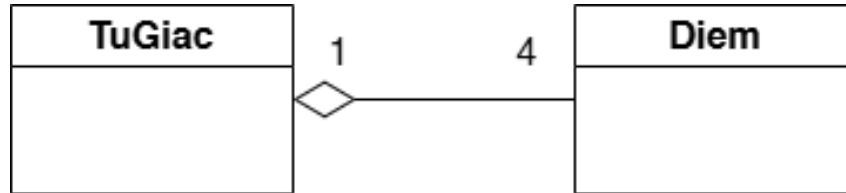
    // Khi đối tượng Window bị hủy, các thành phần cũng bị hủy theo
}
```



Hình 6.5 Ví dụ về quan hệ hợp thành (composition) trong UML

6.2.3. Minh họa kết tập trên Java

Trong phần này, chúng ta xây dựng mã nguồn Java minh họa cho mối quan hệ giữa hai lớp **TuGiac** và **Diem**, hình 6.6. Trong đó một **TuGiac** chứa chính xác 4 **Diem** (là các đỉnh xác định Tứ giác), và mỗi **Diem** có thể thuộc về 1 **TuGiac**.



Hình 6.6 **TuGiac** kết tập **Diem**

Mã nguồn của class **Diem**:

```

class Diem {
    private int x, y;
    public Diem(){ }
    public Diem(int x, int y) {
        this.x = x; this.y = y;
    }
    public void setX(int x){ this.x = x; }
    public int getX() { return x; }
    public void printDiem(){
        System.out.print("(" + x + ", " + y + ")");
    }
}

```

Mã nguồn của class **TuGiac**:

```

class TuGiac {
    private Diem d1, d2;
    private Diem d3, d4;
    public TuGiac(Diem p1, Diem p2, Diem p3, Diem p4){
        d1 = p1; d2 = p2; d3 = p3; d4 = p4;
    }
    public TuGiac(){

```

```

        d1 = new Diem();
        d2 = new Diem(0,1);
        d3 = new Diem (1,1);
        d4 = new Diem (1,0);
    }
    public void printTuGiac(){
        d1.printDiem();
        d2.printDiem();
        d3.printDiem();
        d4.printDiem();
        System.out.println();
    }
}

```

Mã nguồn của class **Test** chứa phương thức main:

```

public class Test {
    public static void main(String arg[]) {
        Diem d1 = new Diem(2,3);
        Diem d2 = new Diem(4,1);
        Diem d3 = new Diem (5,1);
        Diem d4 = new Diem (8,4);

        TuGiac tg1 = new TuGiac(d1, d2, d3, d4);
        TuGiac tg2 = new TuGiac();
        tg1.printTuGiac();
        tg2.printTuGiac();
    }
}

```

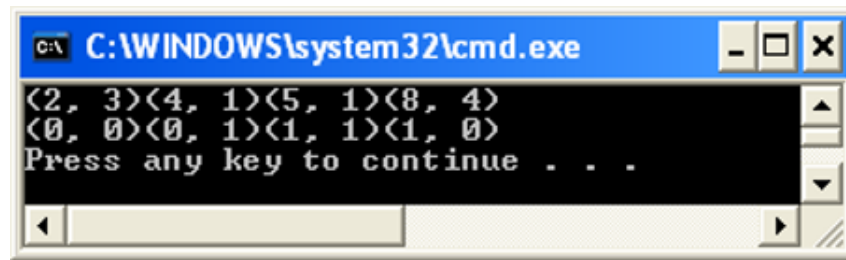
Mối quan hệ giữa class **TuGiac** và class **Diem** là một mối quan hệ kết tập điển hình. **TuGiac** "chứa" (has-a) bốn đối tượng **Diem**, thể hiện qua bốn thuộc tính **d1**, **d2**, **d3**, và **d4**. Lớp **TuGiac** có hai constructor. Một constructor với tham số *TuGiac(Diem p1, Diem p2, Diem p3, Diem p4)* nhận bốn đối tượng **Diem** từ bên ngoài. Một constructor không tham số *TuGiac()* tự tạo bốn đối tượng **Diem** mặc định từ các cặp tọa độ (0,0), (0,1), (1,1) và (1,0).

Trong ví dụ này, Lớp **TuGiac** tái sử dụng toàn bộ chức năng của lớp **Diem** mà không cần phải viết lại. Thay vì tự quản lý tọa độ của bốn đỉnh (cần 8 biến int),

TuGiac sử dụng bốn đối tượng Diem đã được định nghĩa trước. Lớp Diem đóng gói các thuộc tính x, y và cung cấp các phương thức để truy cập và thay đổi chúng. Lớp TuGiac tận dụng tính đóng gói này và không cần quan tâm đến cách Diem lưu trữ và xử lý tọa độ bên trong. Trong phương thức *printTuGiac()*, lớp TuGiac không tự triển khai logic để hiển thị tọa độ mà gọi phương thức *printDiem()* của từng đối tượng Diem. Constructor tham số của TuGiac cho phép truyền vào các đối tượng Diem đã tồn tại, cho phép tái sử dụng các đối tượng Diem trong nhiều ngữ cảnh khác nhau. Điều này được minh họa trong lớp Test:

```
TuGiac tg1 = new TuGiac(d1, d2, d3, d4);
```

Kết quả thực thi hiển thị trong cửa sổ cmd thông tin các đỉnh của 2 TuGiac như hình 6.7.



Hình 6.7 Kết quả thực thi ví dụ

Một cách tiếp cận khác để cài đặt mối quan hệ kết tập giữa lớp TuGiac và lớp Diem trong Java. Thay vì sử dụng bốn biến tham chiếu riêng biệt như cách cài đặt trước đó, phiên bản này sử dụng một mảng các đối tượng Diem để đại diện cho bốn đỉnh của tứ giác. Bản chất của mối quan hệ kết tập giữa các lớp trong thiết kế ban đầu vẫn được duy trì.

```
class TuGiac {
    private Diem[] diem = new Diem[4];
    public TuGiac(Diem p1, Diem p2, Diem p3, Diem p4){
        diem[0] = p1;
        diem[1] = p2;
        diem[2] = p3;
        diem[3] = p4;
    }
}
```

```
    }  
    public void printTuGiac(){  
        diem[0].printDiem();  
        diem[1].printDiem();  
        diem[2].printDiem();  
        diem[3].printDiem();  
        System.out.println();  
    }  
}
```

Cách tiếp cận sử dụng mảng mang lại một số ưu điểm về mặt tổ chức mã nguồn và khả năng mở rộng. Mã nguồn đã ngắn gọn hơn, chúng ta chỉ cần khai báo một thuộc tính mảng thay vì bốn thuộc tính riêng biệt. Phương thức *printTuGiac()* có thể cải tiến bằng cách sử dụng vòng lặp để gọi phương thức *printDiem()* cho từng phần tử trong mảng. Điều này rất thuận tiện khi chúng ta cần thực hiện các xử lý lặp lại với tất cả các đỉnh của TuGiac. Ngoài ra, nếu muốn mở rộng để làm việc với đa giác có số đỉnh khác, có thể thay đổi kích thước mảng phù hợp.

6.2.4. Thứ tự khởi tạo trong kết tập

Quá trình tạo mới một đối tượng yêu cầu tất cả các thuộc tính của đối tượng đều phải được khởi tạo và gán giá trị tương ứng. Yêu cầu này giúp cho đối tượng ở trạng thái hợp lệ sau khi được tạo ra. Hiểu rõ thứ tự khởi tạo rất quan trọng khi thiết kế các lớp có mối quan hệ phức tạp. Nếu không tuân thủ thứ tự khởi tạo, có thể dẫn đến các lỗi khó phát hiện khi chương trình cố gắng sử dụng các đối tượng chưa được khởi tạo đầy đủ.

Trong mối quan hệ kết tập, một nguyên tắc là **các đối tượng thành phần luôn được khởi tạo trước** đối tượng chứa nó. Điều này có nghĩa là các phương thức khởi tạo (constructor) của các lớp thành phần sẽ được thực hiện trước khi constructor của lớp chứa hoàn tất. Nguyên tắc này đảm bảo rằng khi đối tượng chứa được tạo ra, tất cả các thành phần của nó đã ở trạng thái sẵn sàng để sử dụng.

Trong ví dụ lớp TuGiac ở phần trước, khi tạo một đối tượng TuGiac mới, các đối tượng Diem (là thành phần của TuGiac) sẽ được khởi tạo trước.

```
TuGiac tg = new TuGiac(new Diem(1,2), new Diem(3,4),  
                       new Diem(5,6), new Diem(7,8));
```

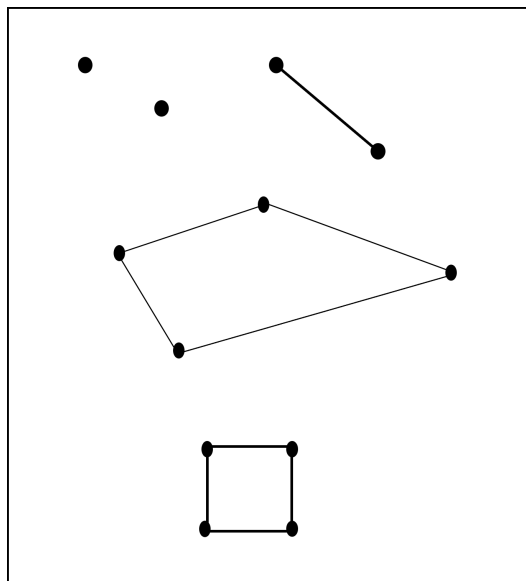
Trong câu lệnh trên, thứ tự khởi tạo sẽ là:

- 1. Constructor của class Diem được gọi để tạo ra bốn đối tượng Diem.
- 2. Sau đó, constructor của TuGiac được gọi, sử dụng bốn đối tượng Diem đã được tạo.

6.3. Kế thừa (Inheritance)

6.3.1. Tổng quan về kế thừa

Chúng ta nhắc lại ví dụ về các khái niệm trong hình học phẳng như được minh họa trong hình 6.8. Đầu tiên, chúng ta có lớp Điểm (Point), đại diện cho một điểm trong mặt phẳng với tọa độ xác định. Lớp Tứ giác được xây dựng dựa trên kết tập bốn đối tượng Điểm, tức là tứ giác được xác định từ 4 đỉnh là 4 điểm tạo thành. Đây là ví dụ điển hình cho mối quan hệ kết tập, trong đó một đối tượng phức hợp chứa các đối tượng thành phần. Tiếp theo, lớp **Hình vuông** được xem là *một trường hợp đặc biệt của Tứ giác*, với các tính chất bổ sung như các cạnh bằng nhau và các góc vuông. Lớp Hình vuông không cần phải xây dựng lại hoàn toàn từ đầu bằng cách định nghĩa lại các đối tượng Điểm cơ bản, vì nó có thể phát triển từ lớp Tứ giác đã định nghĩa trước đó. Lớp Tứ giác đã bao gồm đầy đủ các thuộc tính và phương thức để quản lý bốn điểm cấu thành hình tứ giác. Khi lớp Hình vuông được phát triển từ Tứ giác, nó tự động sở hữu tất cả các thuộc tính và phương thức này mà không cần phải định nghĩa lại.



Hình 6.8 Hình Vuông kế thừa các đặc điểm của hình Tứ Giác

Chúng ta sử dụng **kế thừa** để mô tả mối quan hệ này: Hình vuông kế thừa tất cả các thuộc tính và phương thức của tứ giác, đồng thời có thể mở rộng hoặc ghi đè để phù hợp với đặc điểm riêng.

Bản chất kế thừa

Kế thừa (Inherit, Derive) là quá trình tạo ra một lớp mới dựa trên việc phát triển từ một lớp đã có sẵn. Lớp mới này không chỉ thừa hưởng tất cả các thuộc tính và phương thức của lớp cũ mà còn có thể được mở rộng thêm để phát triển các tính năng mới, phù hợp với yêu cầu cụ thể hơn.

Trong mối quan hệ này, lớp cũ được gọi là **lớp cha (parent, superclass)** hoặc **lớp cơ sở (base class)**. Đây là lớp gốc cung cấp các đặc điểm chung mà các lớp con có thể kế thừa. Ngược lại, lớp mới được gọi là **lớp con (child, subclass)** hoặc **lớp dẫn xuất (derived class)**. Lớp con kế thừa từ lớp cha và có thể bổ sung hoặc thay đổi hành vi để phù hợp với mục đích sử dụng riêng. Kế thừa không chỉ đơn giản là việc sao chép các thuộc tính và phương thức từ lớp cha sang lớp con. Thực chất, kế thừa thể hiện mối quan hệ "là một" (is-a) hoặc "là một loại" (is-a-kind-of) giữa các lớp, nghĩa là lớp con là một dạng đặc biệt của lớp cha. Mọi đối tượng của lớp con cũng có thể được xem là đối tượng của lớp cha, thừa hưởng các đặc điểm và hành vi chung từ lớp cha. Ví dụ, nếu lớp cha là Hình tứ giác, thì lớp con có thể là Hình vuông hoặc Hình chữ nhật. Những hình này vẫn giữ nguyên các đặc điểm chung của hình tứ giác nhưng đồng thời có những tính chất riêng biệt.

Trong kế thừa, lớp con có thể **chi tiết hóa** để phù hợp với mục đích sử dụng mới. Việc chi tiết hóa này diễn ra theo hai hướng chính:

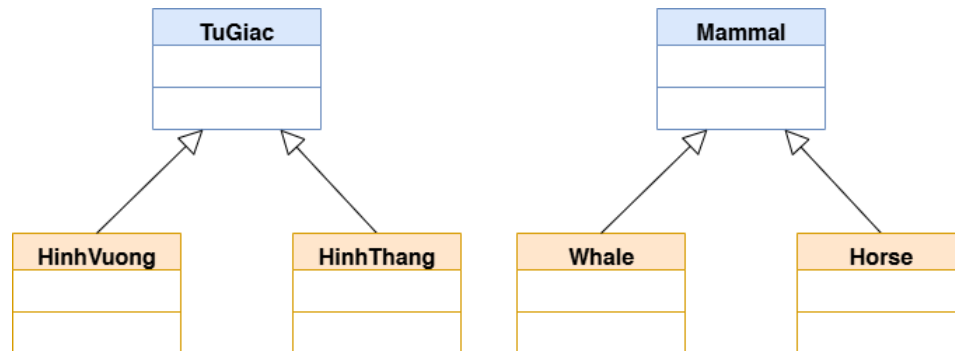
- *Extension (Mở rộng)*: Lớp con có thể thêm các thuộc tính hoặc hành vi mới mà lớp cha không có, từ đó mở rộng chức năng để đáp ứng yêu cầu cụ thể hơn.
- *Redefinition (Ghi đè phương thức - Method Overriding)*: Lớp con có thể chỉnh sửa lại các hành vi kế thừa từ lớp cha, tức là định nghĩa lại phương thức sao cho phù hợp hơn với đặc điểm riêng của lớp con. Đây là một kỹ thuật quan trọng giúp lớp con có thể tùy biến hành vi mà vẫn giữ được cấu trúc kế thừa.

Mối quan hệ kế thừa trong lập trình giống như mối quan hệ giữa cha mẹ và con cái trong cuộc sống, vừa có sự tương đồng, vừa có sự khác biệt. Câu tục ngữ "Con nhà tông, không giống lông thì cũng giống cánh" là một cách hình tượng, sinh động để giúp hiểu sâu sắc hơn về bản chất của kế thừa. Kế thừa không chỉ là một kỹ thuật

trong viết mã, mà còn là một cách tư duy giúp thiết kế phần mềm thông minh, linh hoạt và gần gũi với cách thế giới vận hành. Kế thừa giúp chúng ta mô hình hóa thế giới thực một cách logic và hiệu quả. Nó tạo ra một hệ thống phân cấp rõ ràng giữa các lớp, giúp tổ chức mã nguồn khoa học, dễ hiểu và dễ mở rộng. Khi phát triển phần mềm lớn, việc tận dụng kế thừa sẽ giúp giảm thiểu sự trùng lặp, tăng tính tái sử dụng và dễ dàng mở rộng hệ thống mà không làm ảnh hưởng đến các phần đã có.

Biểu diễn kế thừa trong UML

Ký pháp UML biểu diễn mối quan hệ kế thừa bằng một tam giác rỗng đặt ở đầu mũi tên, điểm hướng vào lớp cha. Mũi tên xuất phát từ lớp con và chỉ về lớp cha, thể hiện rằng lớp con kế thừa các đặc điểm và hành vi từ lớp cha. Trong hình 6.9, lớp Tứ Giác (TuGiac) là lớp cha, với hai lớp con là Hình Vuông (HinhVuong) và Hình Thang (HinhThang). Mũi tên có đầu tam giác rỗng chỉ từ hai lớp con này về lớp cha, cho thấy Hình Vuông và Hình Thang kế thừa từ Tứ Giác. Tương tự, ở hình bên phải, lớp Mammal (Động vật có vú) là lớp cha, với hai lớp con là Whale (Cá voi) và Horse (Ngựa). Mối quan hệ kế thừa cũng được biểu diễn bằng mũi tên có đầu tam giác rỗng từ Whale và Horse về Mammal.



Hình 6.9 Biểu diễn kế thừa trong UML

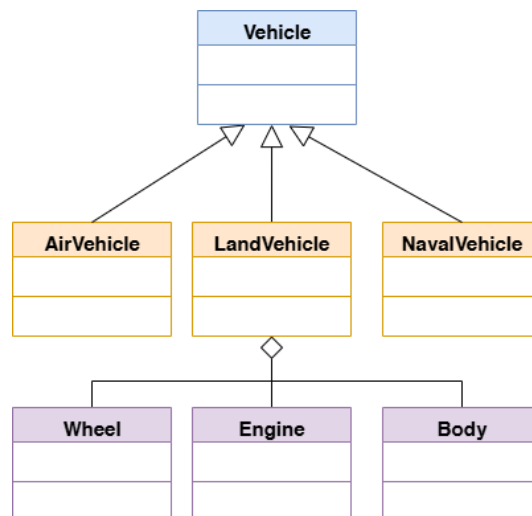
Phân biệt kết tập và kế thừa

- *Giống nhau*: cả kế thừa và kết tập đều là kỹ thuật để tái sử dụng mã nguồn trong OOP.
- *Khác nhau*: kế thừa là cơ chế tái sử dụng thông qua lớp còn kết tập là cơ chế tái sử dụng thông qua đối tượng.

Kế thừa	Kết tập
Kế thừa tái sử dụng thông qua lớp. - Tạo lớp mới bằng cách phát triển lớp đã có - Lớp con kế thừa dữ liệu và hành vi của lớp cha	Kết tập tái sử dụng thông qua đối tượng. - Tạo ra lớp mới là tập hợp các đối tượng của các lớp - Lớp toàn thể có thể sử dụng dữ liệu và hành vi thôn
Quan hệ "là một loại" ("is a kind of")	Quan hệ "là một phần" ("is a part of")
Ví dụ: Ô tô là một loại phương tiện vận tải	Ví dụ: Bánh xe là một phần của Ô tô

Bảng 6.1 So sánh kết tập và kế thừa

Hình 6.10 minh họa cả hai quan hệ kết tập và kế thừa trong một sơ đồ lớp. Lớp Vehicle (phương tiện) là lớp cha, với các lớp con như AirVehicle (phương tiện trên không), LandVehicle (phương tiện trên đất liền), và NavalVehicle (phương tiện trên biển) kế thừa từ lớp cha. Trong khi đó, LandVehicle kết tập các thành phần như Wheel (bánh xe), Engine (động cơ), và Body (thân xe), thể hiện mối quan hệ bộ phận-tổng thể.

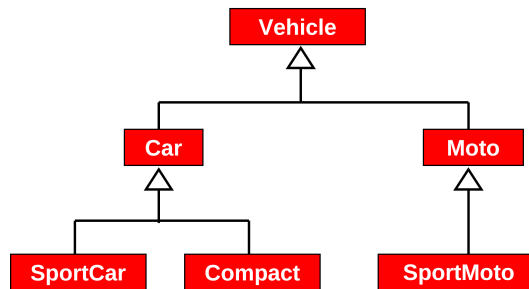


Hình 6.10 Sơ đồ lớp minh họa kết tập và kế thừa

Cây phân cấp kế thừa (Inheritance hierarchy)

Kế thừa tạo ra một **cấu trúc phân cấp** giữa các lớp, giúp tổ chức và quản lý mã nguồn theo cách logic và có hệ thống. Điều này rất hữu ích trong các hệ thống phức tạp, khi cần phân loại các đối tượng theo các đặc điểm chung và riêng. Cấu trúc phân cấp này có dạng cấu trúc hình cây, trong đó mỗi nút đại diện cho một

lớp, và các mối quan hệ kế thừa được thể hiện bằng các đường nối từ lớp con đến lớp cha. Trong cây kế thừa có hai loại dẫn xuất: *dẫn xuất trực tiếp* và *dẫn xuất gián tiếp*. Dẫn xuất trực tiếp nghĩa là một lớp con kế thừa trực tiếp từ lớp cha, ví dụ lớp B dẫn xuất trực tiếp từ lớp A. Trong khi đó, dẫn xuất gián tiếp là trường hợp một lớp con kế thừa từ lớp cha thông qua một hoặc nhiều lớp trung gian, ví dụ lớp C dẫn xuất gián tiếp từ lớp A thông qua lớp B.



Hình 6.11 Cây phân cấp kế thừa

Hình 6.11 cho thấy một ví dụ cụ thể về cây phân cấp kế thừa. Lớp gốc là Vehicle (phương tiện), từ đó có hai lớp con là Car (ô tô) và Moto (xe máy). Tiếp theo, Car lại có hai lớp con là SportCar (xe thể thao) và Compact (xe nhỏ gọn), còn Moto có lớp con là SportMoto (xe máy thể thao). Mối quan hệ này thể hiện rõ ràng sự phân cấp và mở rộng các lớp theo các đặc điểm cụ thể hơn. Trong ví dụ này, lớp Car dẫn xuất trực tiếp từ lớp Vehicle, tức là Car kế thừa trực tiếp các thuộc tính và phương thức của Vehicle. Tương tự, lớp SportCar dẫn xuất trực tiếp từ lớp Car, và lớp SportMoto dẫn xuất trực tiếp từ lớp Moto. Lớp SportCar dẫn xuất gián tiếp từ lớp Vehicle thông qua lớp Car. Điều này có nghĩa là SportCar kế thừa các đặc điểm của Vehicle nhưng thông qua việc kế thừa từ Car. Tương tự, lớp Compact cũng dẫn xuất gián tiếp từ Vehicle qua Car.

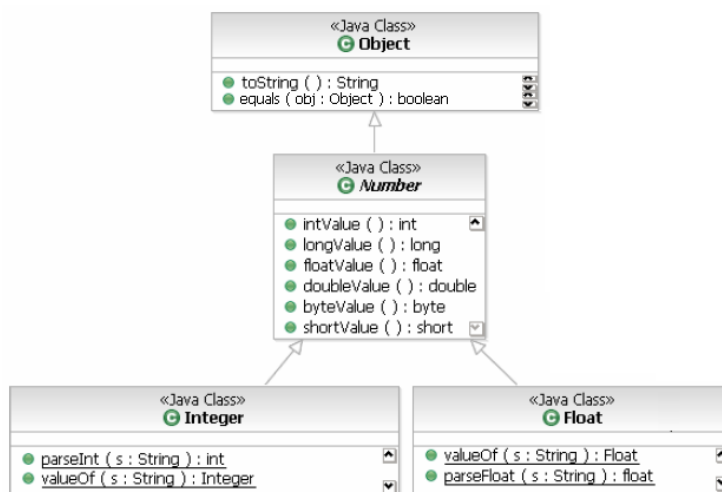
Trong cây phân cấp kế thừa, các lớp con có cùng lớp cha được gọi là anh chị em (siblings). Những lớp này nằm cùng một cấp độ trong cây và kế thừa trực tiếp từ một lớp cha chung. Thành viên (thuộc tính và phương thức) của lớp cha sẽ được kế thừa xuống tất cả các lớp con trong cây phân cấp. Điều quan trọng là một lớp con không chỉ kế thừa trực tiếp từ lớp cha của nó mà còn kế thừa tất cả các lớp tổ tiên ở phía trên trong cây. Ví dụ trong cây Vehicle, các lớp Car và Moto đều kế thừa trực tiếp từ lớp Vehicle, do đó Car và Moto là hai lớp anh chị em với nhau. Lớp SportCar kế thừa trực tiếp từ Car, nhưng đồng thời cũng kế thừa các thuộc tính và

phương thức của Vehicle - lớp tổ tiên cao hơn trong cây. Điều này giúp SportCar có đầy đủ các đặc điểm của một Vehicle nói chung, đồng thời có những đặc điểm riêng biệt của Car và SportCar.

Lớp Object

Trong ngôn ngữ lập trình Java, lớp Object nằm trong gói java.lang và đóng vai trò đặc biệt quan trọng trong hệ thống phân cấp lớp. Nếu một lớp nào đó không được định nghĩa rõ ràng là lớp con của một lớp khác, thì theo mặc định, lớp đó sẽ là con trực tiếp của lớp Object. Điều này có nghĩa rằng mọi lớp trong Java đều kế thừa trực tiếp hoặc gián tiếp từ lớp Object.

Lớp Object được xem là lớp gốc trên cùng của tất cả các cây phân cấp kế thừa trong Java, hình 6.12. Nó cung cấp những phương thức cơ bản mà tất cả các đối tượng trong Java đều có, như toString(), equals(), hashCode(), và clone(). Hai phương thức điển hình và quan trọng nhất là toString() và equals(). Phương thức toString() cho phép chuyển đổi đối tượng thành chuỗi, rất tiện lợi khi cần hiển thị thông tin đối tượng. Phương thức equals() dùng để so sánh tính bằng nhau của hai đối tượng, thay thế cho phép so sánh tham chiếu mặc định.



Hình 6.12 Lớp Object trong hệ thống phân cấp lớp của Java

Nhờ có lớp Object, Java đảm bảo rằng mọi đối tượng đều có những đặc điểm và hành vi tối thiểu chung, từ đó giúp việc quản lý và thao tác với các đối tượng trở nên đồng nhất và linh hoạt.

6.3.2. Nguyên lý kế thừa

Nội dung của nguyên lý kế thừa giúp lập trình viên xác định quyền truy cập các thành viên (thuộc tính và phương thức) của lớp cha và trả lời cho câu hỏi "Lớp con có thể kế thừa được gì?". Các ngôn ngữ lập trình hướng đối tượng cung cấp các chỉ định truy cập như *private*, *default* (phạm vi package trong Java), *protected* và *public* để kiểm soát phạm vi truy cập này.

Chỉ định truy cập *protected* đóng vai trò quan trọng trong cơ chế kế thừa. Cụ thể, các thành viên được khai báo với truy cập *protected* trong lớp cha có thể được truy cập từ chính lớp cha, từ các lớp con kế thừa, cũng như từ các lớp khác nằm trong cùng một package với lớp cha. Điều này tạo ra một phạm vi truy cập rộng hơn so với *private* nhưng vẫn giới hạn hơn so với *public*.

Lớp con có thể kế thừa được gì?

- Kế thừa được các thành viên được khai báo là *public* và *protected* của lớp cha
- Không kế thừa được các thành viên *private*
- Các thành viên có chỉ định truy cập mặc định nếu lớp cha cùng gói với lớp con

Bảng sau tổng hợp các chỉ định truy cập trong ngôn ngữ lập trình Java:

	<i>public</i>	Không có (default)	<i>protected</i>	<i>private</i>
Cùng lớp cha	Yes	Yes	Yes	Yes
Lớp con cùng gói	Yes	Yes	Yes	No
Lớp con khác gói	Yes	No	Yes	No
Khác gói, non-inheritance	Yes	No	No	No

Bảng 6.2 Các chỉ định truy cập và nguyên lý kế thừa

- *private*, đây là mức truy cập hạn chế nhất, chỉ cho phép truy cập trong chính lớp khai báo. Do đó, các thành viên *private* không thể được kế thừa hay truy cập trực tiếp từ lớp con.
- *Không có (default)*, nếu không khai báo rõ ràng, thành viên thuộc phạm vi package, tức chỉ được truy cập trong cùng một package. Lớp con kế thừa có thể truy cập các thành viên này nếu nó nằm trong cùng package với lớp cha. Nếu lớp con ở package khác, các thành viên này bị ẩn, không kế thừa.

- *protected*, thành viên *protected* có thể được truy cập trong lớp cha, trong các lớp con dù có nằm ở package khác hay không, và trong các lớp cùng package với lớp cha.
- *public*, có thể truy cập từ bất kỳ đâu, do đó lớp con có thể kế thừa và sử dụng.

Về nguyên lý truy cập: *protected* giống như *default* (*package-private*) khi truy cập thành viên lớp cha từ một tham chiếu khác tham chiếu hiện tại. Nguyên lý kế thừa: *protected* rộng hơn *default* vì luôn được kế thừa, không cần cùng package. Lưu ý về phạm vi truy cập *protected*: Lớp con có thể truy cập trực tiếp các thành viên *protected* của lớp cha trên chính đối tượng hiện tại (ví dụ: `this.memberA`). Tuy nhiên, nếu lớp con chứa một đối tượng riêng biệt của lớp cha (ví dụ: `A a = new A();`), thì việc truy cập các thành viên *protected* của đối tượng `a` này bị hạn chế theo quy tắc truy cập package.

- Nếu lớp con và lớp cha **khác package**, từ đối tượng `a` sẽ **không thể truy cập** thành viên *protected*.
- Nếu lớp con và lớp cha **cùng package**, từ đối tượng `a` sẽ **được phép truy cập** thành viên *protected*.

Minh họa, mã nguồn lớp cha:

```
package packageA;

public class A {
    protected int memberA = 10;
}
```

Mã nguồn class B là lớp con kế thừa từ A:

```
package packageB;

import packageA.A;

public class B extends A {

    public void testAccess() {
        // Truy cập memberA của lớp cha trên đối tượng hiện tại (this)
        System.out.println(this.memberA); // Được phép, vì B kế thừa A
    }
}
```

```

// Tạo đối tượng lớp cha
A a = new A();

// TH1: Nếu B khác package với A (như ví dụ này)
// Không được phép truy cập memberA qua đối tượng a
// System.out.println(a.memberA); // Lỗi biên dịch

// Nếu B cùng package với A, dòng trên sẽ hợp lệ
}
}

```

Một số trường hợp đặc biệt mà chúng ta không được phép kế thừa, đó là các phương thức khởi tạo và hủy đối tượng. Các phương thức này đảm nhiệm vai trò rất quan trọng trong việc khởi đầu và kết thúc vòng đời của đối tượng, bao gồm việc cấp phát tài nguyên khi tạo đối tượng và giải phóng tài nguyên khi đối tượng không còn sử dụng nữa. Tuy nhiên, vì chúng chỉ biết cách làm việc với từng lớp cụ thể mà không thể áp dụng chung cho các lớp con, nên chúng không thể được kế thừa.

Bên cạnh đó, toán tử gán = cũng là một trường hợp đặc biệt không được kế thừa. Toán tử này có nhiệm vụ tương tự như phương thức khởi tạo, tức là sao chép trạng thái hoặc giá trị từ một đối tượng sang đối tượng khác. Do tính chất đặc thù và cách hoạt động phụ thuộc vào từng lớp cụ thể, toán tử gán không thể được kế thừa như các phương thức thông thường.

6.3.3. Cú pháp kế thừa trong Java

Một lớp con muốn kế thừa từ một lớp cha sẽ sử dụng từ khóa extends theo cú pháp:

```
<Lớp con> extends <Lớp cha>
```

Ví dụ:

```

class HìnhVuong extends TuGiac {
    // Các thành phần của lớp HìnhVuong
}

```

Lưu ý: nếu lớp cha được định nghĩa với từ khóa final, thì lớp đó không thể bị kế thừa. Từ khóa final trong Java được dùng để ngăn chặn việc mở rộng lớp, nhằm bảo vệ tính toàn vẹn và tránh sự thay đổi không mong muốn từ các lớp con.

Một số ví dụ minh họa về kế thừa trong ngôn ngữ Java. Ví dụ 1:

```
public class TuGiac {
    protected Diem d1, d2, d3, d4;
    public void setD1(Diem _d1) {d1=_d1;}
    public Diem getD1(){return d1;}
    public void printTuGiac(){...}
    ...
}

public class HìnhVuong extends TuGiac {
    public HìnhVuong(){
        d1 = new Diem(0,0); d2 = new Diem(0,1);
        d3 = new Diem(1,0); d4 = new Diem(1,1);
    }
}

public class Test{
    public static void main(String args[]){
        HìnhVuong hv = new HìnhVuong();
        hv.printTuGiac();
    }
}
```

Trong ví dụ này, chúng ta có lớp cha TuGiac định nghĩa các thuộc tính d1, d2, d3, d4 với phạm vi truy cập là protected. Điều này cho phép các lớp con kế thừa có thể truy cập trực tiếp các thuộc tính này. Khi tạo đối tượng hv thuộc lớp HìnhVuong, có thể gọi phương thức printTuGiac(), một phương thức public của lớp cha thông qua đối tượng lớp con.

Ví dụ 2:

```
class Person {
    protected String name;
    protected Date birthday;
    public String getName() {return name;}
    ...
}

class Employee extends Person {
    private double salary;
```



```

        public boolean setSalary(double sal){
            salary = sal;
            return true;
        }
        public String getDetail(){
            String s = name + ", " + birthday + ", " + salary;
        }
    }
    public class Test {
        public static void main(String args[]){
            Employee e = new Employee();
            e.setName("John");
            e.setSalary(3.0);
        }
    }

```

Trong ví dụ 2, nếu lớp Person khai báo các thuộc tính name và birthday với phạm vi private thì các thuộc tính này chỉ có thể được truy cập trực tiếp bên trong lớp Person và không thể truy cập trực tiếp từ lớp con Employee. Khi đó việc cố gắng truy cập trực tiếp name và birthday trong phương thức getDetail() của Employee sẽ gây lỗi biên dịch.

Ví dụ 3.1, quan hệ kế thừa giữa lớp con và lớp cha trong cùng package:

```

public class Person {
    Date birthday;
    String name;
    ...
}
public class Employee extends Person {
    ...
    public String getDetail() {
        String s;
        String s = name + ", " + birthday;
        s += ", " + salary;
        return s;
    }
}

```

Ví dụ 3.2, quan hệ kế thừa giữa lớp con và lớp cha thuộc các package khác nhau:

```
package abc;
public class Person {
    protected Date birthday;
    protected String name;
    ...
}

import abc.Person;
public class Employee extends Person {
    ...
    public String getDetail() {
        String s;
        s = name + "," + birthday + "," + salary;
        return s;
    }
}
```

6.3.4. Khởi tạo và hủy bỏ đối tượng trong kế thừa

Khởi tạo đối tượng trong kế thừa tuân theo nguyên tắc: lớp cha luôn được khởi tạo trước lớp con. Khi một đối tượng của lớp con được tạo, các phương thức khởi tạo của lớp con luôn phải gọi phương thức khởi tạo của lớp cha ngay ở câu lệnh đầu tiên. Điều này đảm bảo rằng phần thuộc tính và hành vi kế thừa từ lớp cha được thiết lập đầy đủ trước khi lớp con thực hiện các thao tác khởi tạo riêng.

Có hai cách gọi phương thức khởi tạo của lớp cha trong lớp con.

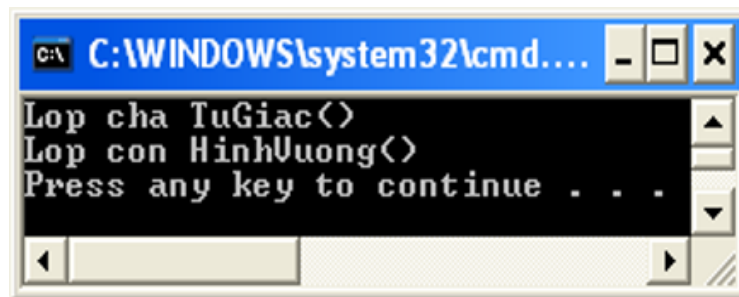
- Thứ nhất là gọi tự động, hay gọi ngầm định (implicit), xảy ra khi lớp cha **chỉ có** phương thức khởi tạo mặc định không tham số. Trong trường hợp này, trình biên dịch sẽ tự động chèn lệnh gọi phương thức khởi tạo mặc định của lớp cha mà không cần khai báo rõ ràng trong mã nguồn lớp con.
- Thứ hai là gọi trực tiếp, hay gọi tường minh (explicit), khi lớp cha có các phương thức khởi tạo có tham số hoặc khi lập trình viên muốn gọi rõ ràng một phương thức khởi tạo cụ thể của lớp cha bằng từ khóa `super`.

Về việc hủy bỏ đối tượng, quá trình này diễn ra ngược lại với khởi tạo. Có nghĩa là khi một đối tượng bị hủy, lớp con sẽ được hủy trước, sau đó đến lớp cha. Điều này đảm bảo rằng các tài nguyên và trạng thái do lớp con quản lý được giải phóng đúng cách trước khi lớp cha kết thúc vòng đời của đối tượng.

Ví dụ minh họa tình huống lớp con tự động gọi constructor của lớp cha:

```
public class TuGiac {  
    protected Diem d1, d2;  
    protected Diem d3, d4;  
    public TuGiac(){  
        System.out.println ("Lop cha TuGiac()");  
    }  
    //...  
}  
  
public class HinhVuong extends TuGiac {  
    public HinhVuong(){  
        //Tu dong gọi TuGiac()  
        System.out.println("Lop con HinhVuong()");  
    }  
}  
  
public class Test {  
    public static void main(String arg[]){  
        HinhVuong hv = new HinhVuong();  
    }  
}
```

Hình 6.13 minh họa kết quả khi thực thi ví dụ trên.



Hình 6.13 Ví dụ về lớp con tự động gọi constructor của lớp cha

Khi ta tạo một đối tượng HinhVuong, constructor của lớp TuGiac được gọi trước tiên, in ra thông báo "Lop cha TuGiac()". Sau đó, constructor của lớp HinhVuong

tiếp tục được thực thi và in ra "Lop con HìnhVuong()". Mặc dù trong constructor của lớp con không có lệnh gọi tường minh đến constructor của lớp cha nhưng kết quả cho thấy lời gọi được thực hiện tự động.

Tình huống gọi tường minh constructor của lớp cha trong phương thức khởi tạo của lớp con, lời gọi này sẽ là câu lệnh đầu tiên và có cú pháp như sau:

```
super(Danh_sach_tham_so);
```

Việc gọi này là bắt buộc trong trường hợp lớp cha không có constructor mặc định (không tham số). Điều này xảy ra khi lớp cha đã được định nghĩa constructor với một hoặc nhiều tham số mà không có constructor không tham số nào. Nếu lớp con không gọi tường minh constructor của lớp cha trong trường hợp này, chương trình sẽ báo lỗi biên dịch.

Ngược lại, constructor của lớp con không nhất thiết phải có tham số, miễn sao nó gọi được constructor phù hợp của lớp cha bằng super. Điều này cho phép lớp con linh hoạt trong việc khởi tạo mà vẫn đảm bảo lớp cha được khởi tạo đúng cách.

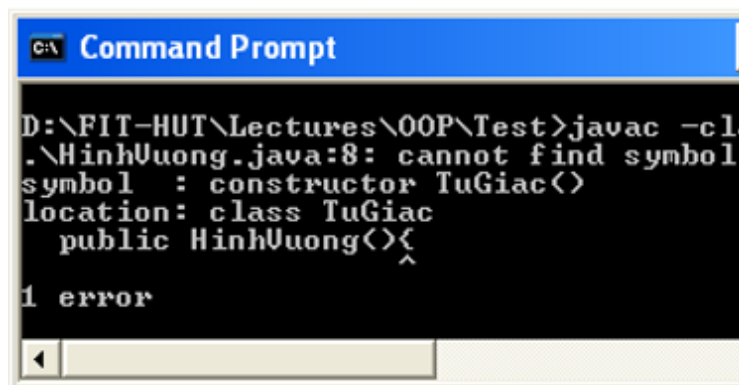
```
public class TuGiac {
    protected Diem d1, d2;
    protected Diem d3, d4;
    public TuGiac(Diem d1, Diem d2, Diem d3, Diem d4){
        System.out.println("Lop cha TuGiac(d1, d2, d3, d4)");
        this.d1 = d1; this.d2 = d2;
        this.d3 = d3; this.d4 = d4;
    }
}

public class HìnhVuong extends TuGiac {
    public HìnhVuong(){
        System.out.println("Lop con HìnhVuong()");
    }
}

public class Test {
    public static void main(String arg[]){
        HìnhVuong hv = new HìnhVuong();
    }
}
```

Lớp HìnhVuong kế thừa từ TuGiac và định nghĩa một constructor không tham số. Tuy nhiên, lớp TuGiac không có constructor mặc định (không tham số). Trong

constructor của `HinhVuong`, không có câu lệnh gọi tường minh `super(...)` để gọi constructor của lớp cha. Khi biên dịch, chương trình cố gắng tự động chèn lời gọi `super()` không tham số vào đầu constructor của lớp con, nhưng lớp cha `TuGiac` không có constructor mặc định phù hợp. Điều này dẫn đến lỗi biên dịch: "cannot find symbol: constructor `TuGiac()`". Lỗi này phản ánh việc lớp con không thể gọi constructor mặc định của lớp cha vì nó không tồn tại, minh họa trong hình 6.14.



Hình 6.14 Lỗi biên dịch do thiếu lời gọi tường minh constructor của lớp cha

Để khắc phục, trong constructor của `HinhVuong` cần phải gọi trực tiếp constructor của lớp cha với đầy đủ tham số phù hợp. Ví dụ:

```
public class TuGiac {
    protected Diem d1,d2,d3,d4;
    public TuGiac(Diem d1, Diem d2, Diem d3, Diem d4){
        System.out.println("Lop cha TuGiac(d1, d2, d3, d4)");
        this.d1 = d1; this.d2 = d2;
        this.d3 = d3; this.d4 = d4;
    }
}

public class HinhVuong extends TuGiac {
    public HinhVuong(){
        super(new Diem(0,0), new Diem(0,1), new Diem(1,1),new Diem(1,0));
        System.out.println("Lop con HinhVuong()");
    }
}
```

Tình huống constructor của lớp con HìnhVuong được xây dựng là constructor có tham số.

```
public class TuGiac {
    protected Diem d1,d2,d3,d4;
    public TuGiac(Diem d1, Diem d2, Diem d3, Diem d4){
        System.out.println("Lop cha TuGiac(d1,d2,d3,d4)");
        this.d1 = d1; this.d2 = d2;
        this.d3 = d3; this.d4 = d4;
    }
}

public class HìnhVuong extends TuGiac {
    public HìnhVuong(Diem d1, Diem d2, Diem d3, Diem d4){
        super(d1, d2, d3, d4);
        System.out.println("Lop con HìnhVuong(d1,d2,d3,d4)");
    }
}
```

6.4. Ví dụ và bài tập

Bài tập 1 Cho đoạn mã nguồn chương trình Java sau. Hãy cho biết kết quả nhận được khi biên dịch chương trình và giải thích?

```
public class TG1 {
    private String name;

    public TG1(String name) {
    }
}

public class HV1 extends TG1 {
    public HV1() { }
    public void test() {
    }
}

public class TG2 {
    private String name;
```

```

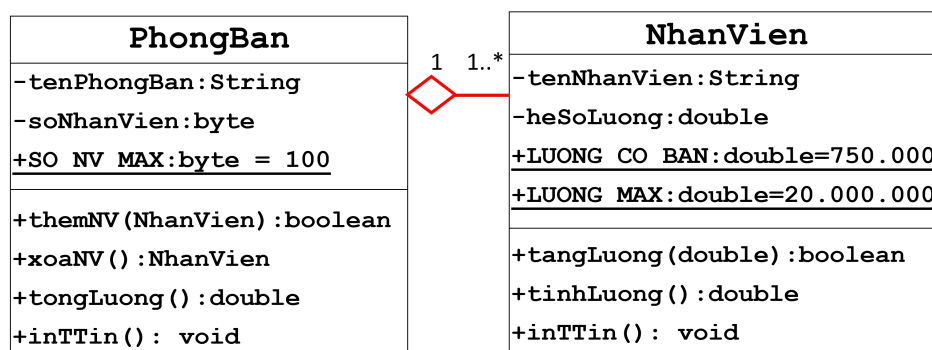
    public TG2(String name) {
    }
}

public class HV2 extends TG2 {
    public void test() {
    }
}

```

Bài tập 2 Viết mã nguồn cho lớp PhongBan với các thuộc tính và phương thức như biểu đồ lớp trong hình 6.15 cùng phương thức khởi tạo với số lượng tham số cần thiết, biết rằng:

- Việc thêm/xóa nhân viên được thực hiện theo cơ chế của stack
- tongLuong() trả về tổng lương của các nhân viên trong phòng
- inTTin() hiển thị thông tin của phòng và thông tin của các nhân viên trong phòng



Hình 6.15 Sơ đồ lớp PhongBan

Thảo luận các câu hỏi sau với bài tập 2:

- Lớp cũ? Lớp mới?
- Lớp mới tái sử dụng lớp cũ thông qua?
- Lớp mới tái sử dụng được những gì của lớp cũ?

Chương 7

Một số kỹ thuật trong kế thừa

7.1. Ghi đè (Overriding)	157
7.2. Lớp trừu tượng (abstract class)	163
7.3. Giao diện (interface)	166

Nội dung

- Ghi đè (Overloading)
- Lớp trừu tượng (abstract class) và phương thức trừu tượng (abstract method)
- Giao diện (interface) và thực thi giao diện

7.1. Ghi đè (Overriding)

Ghi đè (tiếng Anh là overriding) là một kỹ thuật ứng dụng trong kế thừa. Ở bài trước, chúng ta đã học về khái niệm về kế thừa: khi mà chúng ta xây dựng lớp con, thì lớp con mà có mối quan hệ với lớp cha là "lớp con *là một loại* của lớp cha", trong tiếng Việt chúng ta dùng từ "*là*" (mối quan hệ "is-a"). Ví dụ: "Sinh viên *là một* người", "A student is-a person".

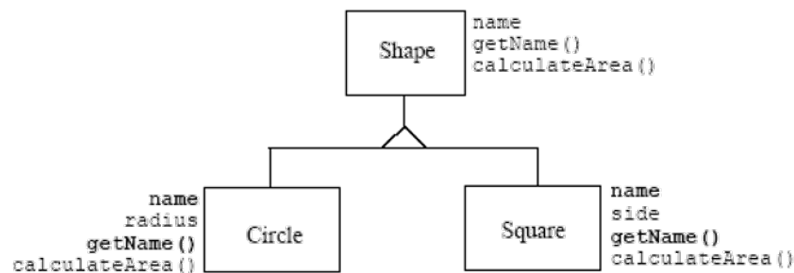
Khi lớp con kế thừa lớp cha, lớp con kế thừa tất cả những cái thành phần dữ liệu và tất cả những cái hành vi của lớp cha. Và sau đó thì chúng ta sẽ **chi tiết hóa** để làm sao cho nó phù hợp với cả mục đích sử dụng mới.

Ở trong bài kỳ trước chúng ta đã học một cái là **extension** (mở rộng), tức là chúng ta bổ sung thêm các cái phương thức và thuộc tính mới. Cái này nó được thể hiện ở chính cái gọi là từ khóa extends, "mở rộng", được sử dụng trong Java. Ngoài ra, việc chi tiết hóa còn được thể hiện ở một khái niệm khác là **redefinition** (định nghĩa lại), tức là **chúng ta sẽ chỉnh sửa lại các cái hành vi của lớp cha**.

Lấy một ví dụ: bây giờ các bạn có thể xây dựng một cái lớp tên là Người, sau đó xây dựng một cái lớp là Sinh viên kế thừa từ lớp Người. Khi mà các bạn xây dựng cái lớp gọi là lớp Người ấy, thì chúng ta có thể xây dựng những cái phương

thức cơ bản ví dụ như là ăn, ngủ, chơi. Lớp Sinh viên thì sẽ bổ sung thêm những cái phương thức và thuộc tính mới. Ví dụ như bổ sung thêm mã số sinh viên, CPA, GPA..., thêm những phương thức mới ví dụ như đi học, đi thi. Đây là những phương thức mới mà lớp Người không có. Tuy nhiên, nếu mà so sánh với cả một người bình thường thì việc ăn và ngủ của sinh viên nó cũng sẽ rất khác. Chẳng hạn, nếu như người bình thường thì có thể là họ sẽ đến tối thì bắt đầu gọi là sắp xếp giường chiếu, lên giường, tắt đèn đi ngủ. Thế còn sinh viên thì ngủ như thế nào? Sinh viên có rất nhiều cách ngủ, ví dụ như là có thể ngủ gật trong lớp học, hoặc có thể là nằm trên xe máy, che quyển sách vào mặt ngủ ngon lành. Hay ví dụ như ăn thì sao? Ăn thì người bình thường có thể bày cái mâm, bát đĩa rồi ngồi ăn. Sinh viên thì có thể lén lút cầm cái bánh mì vào lớp, khi nào giảng viên không chú ý là cúi xuống ngán bàn cắn một miếng. Rõ ràng những cách thức ăn, ngủ này khác với cả cái phương thức mà chúng ta cài đặt ở lớp cha. Đây chính là ý tưởng của overriding - ghi đè.

Kỹ thuật đã được sử dụng là **ghi đè phương thức (method overriding)**. Phương thức ghi đè ở lớp con sẽ thay thế hoặc làm rõ hơn phương thức ở lớp cha, làm cho nó phù hợp hơn với đặc trưng của lớp con. Khi hành vi của lớp con cần khác biệt so với cách cài đặt trong lớp cha, **việc ghi đè cho phép định nghĩa lại cách lớp cha thực hiện ý tưởng của nó**.



Hình 7.1 Ví dụ về ghi đè

Ví dụ về lớp Hình (Shape): Giả sử có một lớp cha là Shape với phương thức `calculateArea()` để tính diện tích. Đối với một hình bất kỳ, việc tính diện tích có thể chưa xác định rõ ràng.

Tuy nhiên, khi có các lớp cụ thể kế thừa từ Shape, ví dụ như lớp **Hình tròn (Circle)** có thuộc tính `radius` (bán kính) hoặc lớp **Hình vuông (Square)** có thuộc tính `side` (cạnh), thì việc tính diện tích có thể được xác định rõ ràng ($\pi * r^2$ cho Hình tròn, $side * side$ cho Hình vuông).

Phương thức tính diện tích ở lớp Circle hoặc Square có thể khác biệt hoàn toàn so với nhau và so với lớp cha. Kỹ thuật được áp dụng ở đây chính là ghi đè phương

thức `calculateArea()`.

Về mặt lập trình trong Java, việc ghi đè phương thức rất đơn giản. Cần viết một phương thức ở lớp con **hoàn toàn trùng tên và trùng danh sách tham số** với phương thức ở lớp cha. Phương thức này sẽ ghi đè nội dung của lớp cha, và nội dung mới trong lớp con có thể được viết theo yêu cầu cụ thể.

Việc trùng tên và danh sách tham số được gọi là **trùng về chữ ký phương thức (method signature)**. Khái niệm này tương tự với **nạp chồng (overloading)** ở chỗ cả hai đều sử dụng phương thức trùng tên. Tương tự như nạp chồng (overloading), mục đích của việc các phương thức trùng tên là vì **bản chất công việc giống nhau**. Tên phương thức trùng nhau thể hiện việc thực hiện cùng một công việc, ví dụ `calculateArea()` dù được cài đặt khác nhau ở `Circle` hay `Square` thì mục đích vẫn là tính diện tích.

Ví dụ mã nguồn:

```
class Shape {
    protected String name;
    Shape(String n) { name = n; }
    public String getName() { return name; }
    public float calculateArea() { return 0.0f; }
}

class Circle extends Shape {
    private int radius;
    Circle(String n, int r){
        super(n);
        radius = r;
    }

    public float calculateArea() {
        float area = (float)(3.14f * radius * radius);
        return area;
    }
}

class Square extends Shape {
    private int side;
    Square(String n, int s) {
```

```

        super(n);
        side = s;
    }
    public float calculateArea() {
        float area = (float) side * side;
        return area;
    }
}

```

- Lớp Shape: public float calculateArea(). Rõ ràng chúng ta chưa biết tính diện tích hình bất kỳ như thế nào, nên tạm thời chúng ta sẽ return 0;
- Lớp Circle kế thừa Shape, có thuộc tính radius. Hình tròn thì chúng ta đã có công thức tính diện tích rồi, do đó chúng ta sẽ ghi đè calculateArea(): $area = \pi * radius * radius$.
- Lớp Square kế thừa Shape, có thuộc tính side. Tương tự, ta ghi đè phương thức calculateArea(): $area = side * side$.

Khi tạo một đối tượng của lớp Circle và gọi calculateArea(), phương thức của lớp Circle sẽ được thực thi. Tương tự, khi tạo đối tượng Square và gọi calculateArea(), phương thức của lớp Square sẽ được thực thi.

Về nguyên tắc, phương thức ghi đè ở lớp con **có thể thay thế hoàn toàn phương thức ở lớp cha**. Tuy nhiên, trong thực tế, đôi khi cần tái sử dụng mã nguồn của lớp cha.

Ví dụ, lớp Nhân viên có phương thức inThongTin() để in thông tin. Khi xây dựng lớp Trưởng phòng kế thừa từ Nhân viên và ghi đè inThongTin(), chúng ta có thể vẫn muốn tái sử dụng phương thức của lớp cha, in lại các thông tin cơ bản của một nhân viên, sau đó in thêm phụ cấp và số năm đương chức.

Để tái sử dụng mã nguồn của lớp cha, có thể sử dụng **từ khóa super** để gọi đến phương thức của lớp cha. Từ khóa super dùng để gọi phương thức hoặc phương thức khởi tạo của lớp cha. Nếu không ghi đè, việc sử dụng super không khác biệt so với gọi trực tiếp, nhưng khi ghi đè, super trở nên hữu ích. (Lưu ý là từ khóa super vẫn không giúp chúng ta truy cập được các phương thức hay thuộc tính private của lớp cha). Khi một phương thức được ghi đè trong lớp con, về bản chất, lớp con đó có cả hai phương thức: phương thức của lớp con và phương thức của lớp cha. Từ khóa super cho phép gọi tường minh phương thức của lớp cha.

```
package abc;
public class Person {
    protected String name;
    protected int age;
    public String getDetail() {
        String s = name + "," + age;
        return s;
    }
}

import abc.Person;
public class Employee extends Person {
    double salary;
    public String getDetail() {
        String s = super.getDetail() + "," + salary;
        return s;
    }
}
```

Ví dụ: Lớp cha Person có phương thức getDetail(). Lớp con Employee ghi đè getDetail(). Trong getDetail() của Employee, có thể sử dụng super.getDetail() để dùng lại phương thức từ lớp cha, sau đó cộng thêm thông tin riêng của Employee (ví dụ: salary).

Lưu ý: Việc sử dụng super.getDetail() để gọi phương thức của lớp cha có thể thực hiện ở bất kỳ đâu trong phương thức của lớp con, khác với super() gọi phương thức khởi tạo lớp cha phải ở dòng đầu tiên của phương thức khởi tạo lớp con. Khi đó, Java sẽ hiểu rằng đây là lời gọi đến phương thức getDetail() của lớp cha, không phải của lớp con.

Một số quy định trong ghi đè:

1. **Phương thức ghi đè ở lớp con phải có danh sách tham số giống hệt phương thức ở lớp cha.** Nếu danh sách tham số khác, đó không phải là ghi đè và cũng không hẳn là nạp chồng theo định nghĩa (nạp chồng dùng cho các phương thức trong cùng một lớp), mặc dù về mặt lập trình có thể coi như có hai phương thức trùng tên với chữ ký khác nhau tồn tại trong lớp con (một kế thừa từ cha, một mới định nghĩa). Để ẩn hoàn toàn phương thức ở lớp cha, tên phương thức và danh sách tham số phải giống hệt. Ngoài ra, **kiểu trả về cũng phải giống hệt phương thức ở lớp cha.**

2. **Chỉ định truy cập (access modifiers) của phương thức ghi đè ở lớp con không được chặt hơn phương thức ở lớp cha.** Thứ tự chặt là: private > default > protected > public. Ví dụ, nếu phương thức lớp cha là protected, phương thức lớp con ghi đè có thể là protected hoặc public, nhưng không được phép là private.

Ví dụ về quy định chỉ định truy cập và kiểu trả về:

```
class Parent {
    public void doSomething() {}
    protected int doSomething2() {
        return 0;
    }
}

class Child extends Parent {
    protected void doSomething() {}
    protected void doSomething2() {}
}
```

- Lớp Parent: public void doSomething(), protected int doSomething2().
- Lớp Child: protected void doSomething(), protected void doSomething2().
- Trường hợp 1 (doSomething): protected ở lớp con chặt hơn public ở lớp cha (vì protected cho phép truy cập trong cùng gói và lớp con, còn public cho phép truy cập từ mọi nơi), do đó không ghi đè được.
- Trường hợp 2 (doSomething2): Kiểu trả về khác nhau (int vs void), do đó không ghi đè được.

Việc ghi đè có thể diễn ra qua nhiều cấp kế thừa (ghi đè của ghi đè).

Một số phương thức không được ghi đè:

1. **Các phương thức static.** Phương thức static thuộc về lớp, không phải của từng đối tượng.
2. **Các phương thức private.** Phương thức private chỉ hiển thị trong nội bộ lớp khai báo, không thể truy cập từ lớp con để ghi đè.

3. **Các phương thức final ở lớp cha.** Đối với thành phần dữ liệu, từ khóa final được sử dụng cho biến để khai báo hằng số. Tương tự, khi sử dụng final cho phương thức, phương thức đó **không thể bị ghi đè**.

Lý do sử dụng final cho phương thức có thể bao gồm:

- **Chỉ định rằng phương thức này là cuối cùng, không được thay đổi.** Điều này hữu ích khi phát triển API hoặc thư viện, thông báo cho người kế thừa rằng việc thay đổi phương thức này có thể phá vỡ các chức năng khác.
- **Tối ưu hóa hiệu quả (efficiency).** Khi một phương thức được khai báo là final, trình biên dịch có thể xác định trực tiếp phương thức nào sẽ được gọi tại thời điểm biên dịch (liên kết tĩnh), giúp tăng tốc độ thực thi so với việc phải dò tìm phương thức phù hợp tại thời điểm chạy (liên kết động). Tuy nhiên, với các ngôn ngữ hiện đại như Java, sự khác biệt về hiệu năng này hiện không còn quá quan trọng như trong C++.

Bài tập thực hành: Xây dựng lớp Trưởng phòng kế thừa từ Nhân viên. Ghi đè phương thức tính lương (lương Trưởng phòng = lương cơ bản * hệ số lương + phụ cấp). Ghi đè phương thức in thông tin (in thông tin Nhân viên và bổ sung phụ cấp, chức vụ, số năm đương chức).

7.2. Lớp trừu tượng (abstract class)

Quay lại ví dụ về các lớp hình (Shape, Circle, Square, Triangle), thể hiện trong Hình 7.1. Lớp Shape là một khái niệm rất chung chung. Khi hình dung về một "hình", khó có thể chỉ ra một đối tượng cụ thể nào xuất hiện trong suy nghĩ mỗi người. Còn trong thực tế lập trình, chúng ta thường **không cần tạo ra một đối tượng cụ thể từ lớp Shape**. Lớp Shape tồn tại chủ yếu để làm lớp cơ sở cho các lớp con cụ thể hơn như Circle, Square kế thừa, nhằm **chia sẻ những đặc tính chung** của tất cả các loại hình. Những lớp như vậy được gọi là **lớp trừu tượng**.

Một đặc điểm nữa của những lớp như Shape là có những phương thức mà **không biết cài đặt cụ thể như thế nào**. Ví dụ phương thức calculateArea() của Shape. Mặc dù một hình bất kỳ đều có diện tích, nhưng không có cách cài đặt chung cho mọi loại hình tại cấp lớp Shape. Trong ví dụ trước đó, chúng ta đã cài đặt trả về 0, thực ra là không chính xác. Một hình thực ra có diện tích đầy, nhưng không phải là 0.

Những phương thức mà chúng ta không biết cách cài đặt cụ thể như vậy được gọi là **phương thức trừu tượng (abstract method)**. Những phương thức này

chỉ có ý nghĩa trong ngữ cảnh chung, không có cài đặt cụ thể

Phương thức trừu tượng được sinh ra **để bị ghi đè**. Trong hầu hết các ngôn ngữ lập trình, phương thức trừu tượng chỉ bao gồm **chữ ký mà không có cài đặt**. Các lớp con cụ thể sẽ định nghĩa lại (ghi đè) phương thức trừu tượng đó.

Đặc trưng của lớp trừu tượng là:

1. Là những lớp không rõ ràng, không thể hình dung đối tượng cụ thể.
2. Thường chứa những phương thức không rõ ràng (phương thức trừu tượng).

Khi lập trình:

- Để biến một lớp thành lớp trừu tượng, thêm từ khóa **abstract** vào trước khai báo lớp. Ví dụ: `public abstract class Shape`.
- Lớp đã khai báo là abstract không thể được khởi tạo đối tượng trực tiếp. Lệnh `Shape s = new Shape();` sẽ báo lỗi.
- Để khai báo một phương thức trừu tượng, thêm từ khóa **abstract** vào trước khai báo phương thức. Sau chữ ký phương thức là dấu chấm phẩy (;), không có phần thân phương thức (không có cặp ngoặc nhọn {}). Ví dụ: `public abstract float calculateArea();`
- Các phương thức trừu tượng được khai báo trong lớp trừu tượng và sẽ được ghi đè (định nghĩa lại) bởi các lớp con kế thừa.

Ví dụ mã nguồn được sửa lại:

```
abstract class Shape {
    protected String name;
    Shape(String n) { name = n; }
    public String getName() { return name; }
    public abstract float calculateArea();
}
```

```
class Circle extends Shape {
    private int radius;
    Circle(String n, int r){
        super(n);
        radius = r;
    }
}
```

```
}

    public float calculateArea() {
        float area = (float) (3.14 * radius * radius);
        return area;
    }
}
```

Lớp Shape là abstract class, phương thức calculateArea() là abstract và kết thúc bằng dấu chấm phẩy. Điều này thể hiện đúng ý tưởng rằng "một hình có diện tích, nhưng tôi không biết tính như thế nào ở cấp độ chung này". Các lớp con như Square, Circle sẽ kế thừa từ lớp abstract Shape và ghi đè (định nghĩa cụ thể) phương thức calculateArea().

Lớp con kế thừa lớp trừu tượng bắt buộc phải ghi đè lại tất cả những phương thức trừu tượng của lớp cha. Nếu lớp con không ghi đè hết các phương thức trừu tượng của lớp cha, đồng nghĩa với việc nó kế thừa các phương thức trừu tượng đó. Vì lớp con này chứa phương thức trừu tượng, nên nó bắt buộc phải là một lớp trừu tượng.

Khi kiểm tra xem một lớp con có cần là lớp trừu tượng hay không, cần xem xét không chỉ các phương thức trừu tượng trực tiếp trong lớp cha mà cả các phương thức trừu tượng kế thừa từ các lớp cha cấp cao hơn (lớp cha của lớp cha...) mà chưa được ghi đè hoàn toàn trên cây kế thừa. Nếu vẫn còn phương thức trừu tượng chưa được cài đặt, lớp con đó vẫn phải là lớp trừu tượng.

Vì phương thức trừu tượng được sinh ra để ghi đè, nên không được khai báo là **final** hoặc **static**. Các chỉ định truy cập hợp lệ cho phương thức trừu tượng là public hoặc protected. abstract private và abstract static, abstract final là không hợp lệ.

Lưu ý về lớp trừu tượng:

- Nếu một lớp có một hoặc nhiều phương thức trừu tượng, nó bắt buộc phải là lớp trừu tượng.
- Ngược lại, một lớp trừu tượng không bắt buộc phải có phương thức trừu tượng. Một lớp bình thường không có phương thức trừu tượng nào vẫn có thể được khai báo là lớp trừu tượng bằng cách thêm từ khóa abstract.

Lớp trừu tượng có thể kế thừa một lớp trừu tượng khác như một lớp bình thường. Nếu một lớp con kế thừa từ một lớp trừu tượng có nhiều phương thức trừu tượng,

lớp con đó phải ghi đè tất cả các phương thức trừu tượng đó hoặc tự trở thành lớp trừu tượng. Bản chất của lớp trừu tượng không khác gì một lớp bình thường, chỉ khác là chúng ta chỉ định cho trình biên dịch không cho phép tạo đối tượng trực tiếp từ lớp này. Tuy nhiên, nếu một lớp chứa phương thức trừu tượng, nó bắt buộc phải là lớp trừu tượng, vì nếu có thể tạo đối tượng từ một lớp có phương thức chưa cài đặt, việc gọi phương thức đó sẽ không thực hiện được.

Trong biểu đồ lớp (UML), các phương thức trừu tượng hoặc lớp trừu tượng thường được thể hiện bằng **chữ nghiêng (italic)**.

7.3. Giao diện (interface)

Quan trọng: Khái niệm "giao diện" ở đây hoàn toàn không liên quan đến giao diện người sử dụng (User Interface - UI), đồ họa, form, nút bấm. Giao diện trong ngữ cảnh này là một kỹ thuật trong lập trình hướng đối tượng liên quan đến kế thừa, cùng với ghi đè và lớp trừu tượng.

Trước khi tìm hiểu sâu hơn về interface, cần hiểu về đơn kế thừa (single inheritance) và đa kế thừa (multiple inheritance).

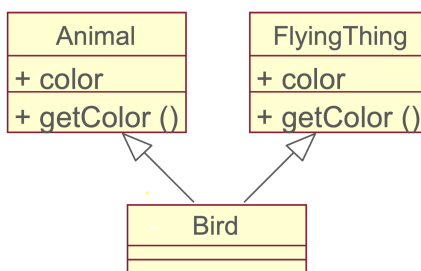
Đa kế thừa là khi một lớp kế thừa từ nhiều lớp. Cần phân biệt với trường hợp nhiều lớp kế thừa từ một lớp, đó vẫn là đơn kế thừa. Mối quan hệ kế thừa là "**is-a**". Ví dụ, lớp Student kế thừa Person vì "Student is a Person" (Sinh viên là một người).

Trong thực tế, đa kế thừa là một khái niệm rất tự nhiên. Ví dụ, FemaleStudent (Sinh viên nữ). FemaleStudent là một Student và cũng có thể coi là một Woman (Phụ nữ). Như vậy, FemaleStudent có thể kế thừa cả Student và Woman, đây là ví dụ về đa kế thừa. Trong các ngôn ngữ lập trình, ngôn ngữ C++ hỗ trợ đa kế thừa.

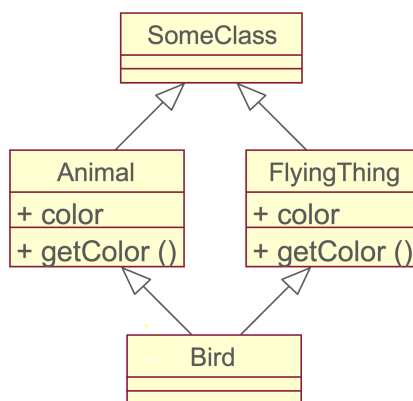
Về mặt lập trình, đa kế thừa gây ra một số vấn đề:

1. **Xung đột về tên (name collision)**. Nếu hai lớp cha có cùng tên thuộc tính hoặc phương thức, lớp con kế thừa từ cả hai sẽ gặp khó khăn trong việc xác định nên sử dụng thuộc tính hoặc phương thức nào. Ví dụ, nếu cả lớp Student và lớp Woman đều có thuộc tính name, age hoặc phương thức eat(), sleep(), lớp FemaleStudent sẽ không biết chọn phương thức sleep() nào khi được gọi (sleep() của Student hay sleep() của Woman). Vấn đề này thường được trình biên dịch phát hiện và báo lỗi.
2. **Vấn đề "Diamond shape" (vấn đề hình kim cương)**. Vấn đề này xảy ra khi một lớp kế thừa từ hai lớp cha, và hai lớp cha này lại cùng kế thừa từ một lớp cha chung cấp cao hơn. Ví dụ: Lớp FemaleStudent kế thừa Student và

Woman. Cả Student và Woman đều kế thừa từ Person. Lớp Person có các thuộc tính name, age và phương thức eat(), sleep(). Khi đó, FemaleStudent dường như có hai bản sao của name, age, eat(), sleep() thông qua hai đường dẫn kế thừa khác nhau (FemaleStudent -> Student -> Person và FemaleStudent -> Woman -> Person). Vấn đề này khó giải quyết hơn xung đột tên trực tiếp và có thể khiến việc thiết kế trở nên phức tạp, khó kiểm soát.



Hình 7.2 Xung đột tên trong đa kế thừa



Hình 7.3 Vấn đề "hình kim cương" trong đa kế thừa

Các ngôn ngữ lập trình hiện đại như Java và C# giải quyết vấn đề này bằng cách không cho phép đa kế thừa lớp. Một lớp trong Java chỉ được phép kế thừa từ một lớp cha duy nhất (đơn kế thừa).

Để hỗ trợ các bài toán thiết kế tương tự đa kế thừa, các ngôn ngữ này đưa ra khái niệm giao diện (interface).

Giao diện giống như một lớp trừu tượng đặc biệt. Tất cả các phương thức trong giao diện đều là trừu tượng. Thay vì một lớp con kế thừa nhiều lớp cha, nó sẽ kế thừa một lớp cha duy nhất và thực thi (implement) một hoặc nhiều giao diện.

Ví dụ FemaleStudent: Có thể kế thừa từ lớp Student và thực thi giao diện Woman. Về mặt tư tưởng, có thể hiểu là "Sinh viên nữ là một Sinh viên và có những đặc điểm của Phụ nữ".

Đặc điểm của giao diện:

- Giống như một lớp trừu tượng.
- Tất cả các phương thức đều là trừu tượng (public abstract).
- Không có cài đặt cụ thể cho các phương thức.
- Không có thuộc tính (biến thành viên) thông thường. Chỉ có thể chứa hằng số.

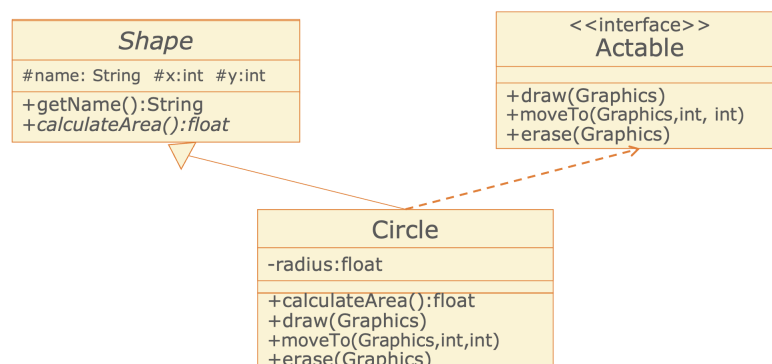
Việc sử dụng giao diện giải quyết vấn đề nhập nhằng (name conflict) vì giao diện không có thuộc tính thông thường và các phương thức của giao diện chỉ là chữ ký (abstract), không có cài đặt cụ thể. Lớp thực thi giao diện sẽ cung cấp cài đặt cụ thể, do đó không có nhiều cài đặt khác nhau để lựa chọn.

Về mặt lập trình:

- Khai báo giao diện bằng từ khóa **interface**. Ví dụ: interface Woman.
- Liệt kê các phương thức bên trong giao diện. Các phương thức này chỉ bao gồm chữ ký và kết thúc bằng dấu chấm phẩy (;), **không có phần thân**. Ví dụ: eat();, sleep();, makeup();.
- Mặc định, tất cả các phương thức khai báo trong interface đều là **public abstract**. Không cần viết rõ từ khóa public và abstract, mặc dù việc viết rõ cũng không gây lỗi.
- Thuộc tính trong interface bắt buộc phải là hằng số, ngầm định là **public static final**.
- Một giao diện có thể **extends (kế thừa)** một giao diện khác.
- Một giao diện không thể kế thừa từ một lớp. Giao diện và lớp là hai khái niệm khác nhau.
- Một lớp thực thi giao diện sử dụng từ khóa **implements**. Một lớp có thể thực thi nhiều giao diện, liệt kê các giao diện cách nhau bằng dấu phẩy sau từ khóa implements. Ví dụ: class FemaleStudent extends Student implements Woman.

- Khi một lớp implements một giao diện, nó giống như kế thừa các phương thức trừu tượng. Do đó, lớp đó bắt buộc phải ghi đè (override) lại **tất cả các phương thức** được định nghĩa trong giao diện và cung cấp cài đặt cụ thể.
- Nếu lớp thực thi không ghi đè hết tất cả các phương thức trừu tượng của giao diện, nó phải tự trở thành một lớp trừu tượng (**abstract class**).
- Tương tự lớp trừu tượng, giao diện không thể được khởi tạo đối tượng trực tiếp.

Về mặt tư tưởng, giao diện giống như định nghĩa một tập hợp các khả năng hoặc hành vi mà các lớp thực thi nó sẽ có. Nó thể hiện mối quan hệ "**có những đặc điểm của**". Ví dụ, FemaleStudent implements Woman có thể hiểu là "Sinh viên nữ là Sinh viên và có những đặc điểm của Phụ nữ".



Hình 7.4 Ví dụ về interface

Về quy ước đặt tên (coding convention) trong Java, interface thường được đặt tên bằng *tính từ* thể hiện khả năng (ví dụ: `Serializable` - có thể tồn tại được, `Drawable` - có thể vẽ được). Tuy nhiên, đây chỉ là quy ước phổ biến, không bắt buộc. Một quy ước khác là thêm chữ **I** vào trước tên interface (ví dụ: `IWoman`). Việc tuân theo quy ước nào tùy thuộc vào dự án hoặc công ty.

Ví dụ: Lớp `Circle`, `Rectangle` kế thừa lớp `Shape2D`. Lớp `Cube`, `Sphere` kế thừa lớp `Shape3D`. Tất cả các lớp này đều cần có khả năng vẽ lên màn hình, nên chúng có thể cùng **implements** giao diện `Drawable`. Giao diện `Drawable` có thể định nghĩa phương thức `draw()`. Mỗi lớp thực thi (`Circle`, `Rectangle`, `Cube`, `Sphere`) sẽ ghi đè phương thức `draw()` theo cách riêng của nó (ví dụ: vẽ hình tròn, hình vuông, chiếu hình khối 3D lên 2D và vẽ).

Trong thực tế doanh nghiệp, giao diện được sử dụng như một bản hợp đồng (contract). Trong quy trình phát triển phần mềm, giai đoạn thiết kế định nghĩa các

chức năng và cách các thành phần khác nhau tương tác. Interface đóng vai trò định nghĩa "hợp đồng" về những gì một thành phần (lớp) phải cung cấp. Nếu một lớp thực thi một interface, nó cam kết sẽ có những phương thức được định nghĩa trong interface đó.

Interface đảm bảo rằng các lớp khác nhau, dù có cây kế thừa khác nhau, vẫn có thể tương tác thông qua một tập hợp phương thức chung được định nghĩa trong interface.

Tuy nhiên, việc lạm dụng interface (ví dụ: tạo interface cho mọi lớp, như class Student implements IStudentInterface) có thể làm mất ý nghĩa của interface và làm phức tạp cấu trúc không cần thiết.

Trong các phiên bản Java sau này (từ **Java 8 trở lên**), để giảm bớt một số hạn chế của interface (ví dụ: không thể tái sử dụng mã cài đặt), các khái niệm mới đã được giới thiệu:

- **Default method:** Cho phép thêm cài đặt mặc định cho một phương thức trong interface. Phương thức này được khai báo với từ khóa default. Lớp thực thi interface có thể sử dụng cài đặt mặc định này hoặc ghi đè nó. Default method giúp hỗ trợ các trường hợp giống đa kế thừa và giải quyết vấn đề name conflict giữa các default method từ các interface khác nhau bằng cách yêu cầu lớp thực thi ghi đè phương thức đó và chỉ định rõ default method nào sẽ được gọi bằng cách sử dụng cú pháp `InterfaceName.super.methodName()`.
- **Static method:** Cho phép định nghĩa các phương thức static trong interface. Phương thức static trong interface hoạt động tương tự như trong lớp và có thể gọi trực tiếp từ interface.

Khi lập trình, đặc biệt là lập trình cho các thiết bị IoT, cần phải lưu ý phiên bản Java để bảo đảm tính tương thích của mã nguồn.

Chương 8

Đa hình

8.1. Upcasting và Downcasting	171
8.2. Liên kết tĩnh và Liên kết động	180
8.3. Đa hình (Polymorphism)	184
8.4. Ví dụ và bài tập	187

Mục tiêu: Trong lập trình hướng đối tượng, đa hình (polymorphism) là một khái niệm cốt lõi giúp tăng tính linh hoạt và mở rộng của chương trình. Bài học tập trung vào việc hiểu sâu về đa hình thông qua các kỹ thuật upcasting và downcasting trong Java. Đây là những phương pháp chuyển đổi kiểu dữ liệu giữa lớp cha và lớp con, giúp khai thác tối đa khả năng tái sử dụng và mở rộng mã nguồn. Chúng ta cũng sẽ phân biệt rõ ràng giữa liên kết tĩnh và liên kết động. Liên kết tĩnh xảy ra khi phương thức được xác định tại thời điểm biên dịch, trong khi liên kết động cho phép chương trình quyết định phương thức cụ thể được gọi tại thời điểm chạy, từ đó hỗ trợ đa hình động. Cuối cùng, bài học sẽ cung cấp các ví dụ minh họa và bài tập thực hành với ngôn ngữ lập trình Java, giúp củng cố kiến thức và áp dụng thành thạo đa hình trong các tình huống thực tế.

Nội dung:

- Upcasting và Downcasting
- Liên kết tĩnh và Liên kết động
- Đa hình (Polymorphism)
- Ví dụ và bài tập

8.1. Upcasting và Downcasting

8.1.1. Chuyển đổi kiểu dữ liệu

Trong các ngôn ngữ lập trình hướng đối tượng nói chung, việc chuyển đổi kiểu dữ liệu là một khái niệm cơ bản và rất quan trọng để đảm bảo rằng các phép toán

và biểu thức được thực hiện chính xác cũng như tránh lỗi kiểu dữ liệu. Đối với ngôn ngữ lập trình Java, các kiểu dữ liệu được chia thành hai nhóm: kiểu dữ liệu nguyên thủy và kiểu dữ liệu tham chiếu.

Chuyển đổi kiểu dữ liệu nguyên thủy Chuyển đổi kiểu dữ liệu nguyên thủy có thể được chia thành hai trường hợp chính: chuyển đổi tự động và chuyển đổi ép kiểu (casting).

Trước tiên, Java sẽ tự động thực hiện chuyển đổi kiểu dữ liệu khi các kiểu dữ liệu tương thích với nhau, đặc biệt là khi chuyển đổi từ kiểu dữ liệu có phạm vi hẹp hơn sang kiểu có phạm vi rộng hơn. Ví dụ, khi biến nguyên kiểu `int` được gán cho biến kiểu `double`, Java sẽ tự động chuyển đổi giá trị `int` sang `double` mà không cần sự can thiệp của lập trình viên. Điều này xảy ra bởi vì kiểu `double` có thể chứa tất cả giá trị của kiểu `int` mà không bị mất dữ liệu, nên quá trình chuyển đổi này là an toàn. Cụ thể, đoạn mã minh họa như sau:

```
int i;  
double d = i;
```

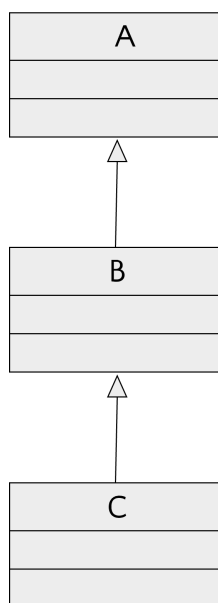
Trong trường hợp ngược lại, khi chuyển đổi từ kiểu dữ liệu có phạm vi rộng hơn sang kiểu hẹp hơn, Java không tự động thực hiện chuyển đổi vì có nguy cơ mất dữ liệu hoặc sai lệch giá trị. Do đó, lập trình viên phải thực hiện ép kiểu (casting) một cách rõ ràng. Ví dụ, khi muốn gán một biến `int` sang biến `byte`, ta cần sử dụng cú pháp ép kiểu bằng cách đặt kiểu đích trong dấu ngoặc đơn trước biến hoặc giá trị. Nếu không ép kiểu, chương trình sẽ báo lỗi biên dịch. Ví dụ:

```
int i;  
byte b = (byte) i;
```

Việc ép kiểu này báo hiệu cho trình biên dịch rằng lập trình viên đã nhận thức được rủi ro tiềm ẩn khi chuyển đổi và chịu trách nhiệm đảm bảo tính hợp lệ của dữ liệu. Nếu giá trị của `i` vượt quá phạm vi của kiểu `byte`, dữ liệu có thể bị mất hoặc thay đổi không mong muốn.

Chuyển đổi kiểu dữ liệu tham chiếu Kiểu dữ liệu tham chiếu (reference type) là các biến chứa địa chỉ của đối tượng trong bộ nhớ. Việc chuyển đổi kiểu dữ liệu tham chiếu thường gặp khi làm việc với các lớp có quan hệ kế thừa. Chuyển đổi kiểu dữ liệu tham chiếu chỉ có thể thực hiện khi các kiểu dữ liệu đó tương thích, tức là thuộc cùng một cây phân cấp kế thừa.

Hai lớp được xem là tương thích nếu chúng nằm trên cùng một cây kế thừa — nghĩa là một lớp có thể là cha hoặc con của lớp kia, hoặc cùng kế thừa từ một lớp cha chung. Ví dụ, nếu lớp A là lớp cha của lớp B, và lớp B lại là lớp cha của lớp C như trong hình 8.1, thì các kiểu tham chiếu của các lớp này có thể được chuyển đổi qua lại trong phạm vi quan hệ kế thừa đó.



Hình 8.1 Một cây kế thừa đơn giản

Khi khai báo một biến tham chiếu kiểu A và khởi tạo nó bằng một đối tượng thuộc lớp B (lớp con của A), ta có:

```
A var1 = new B();
```

Ở đây, biến var1 có kiểu A, nhưng thực tế nó tham chiếu đến một đối tượng kiểu B. Đây là một ví dụ của **upcasting**, tức chuyển từ kiểu con lên kiểu cha, được Java cho phép tự động vì nó an toàn.

Ngược lại, khi muốn chuyển một biến tham chiếu kiểu cha về kiểu con, ví dụ:

```
A var1 = new A();  
C var2 = (C) var1;
```


thì cần thực hiện **downcasting**, tức ép kiểu rõ ràng bằng cách đặt kiểu đích trong dấu ngoặc đơn. Tuy nhiên, downcasting chỉ hợp lệ nếu đối tượng thực tế mà biến `var1` tham chiếu là thể hiện của lớp `C` hoặc lớp con của `C`. Nếu không, chương trình sẽ gặp lỗi thời gian chạy (`ClassCastException`).

Như vậy, có hai loại chuyển đổi kiểu dữ liệu tham chiếu chính:

- *Upcasting*: chuyển từ kiểu con sang kiểu cha, thường được thực hiện tự động và an toàn.
- *Downcasting*: chuyển từ kiểu cha xuống kiểu con, cần ép kiểu rõ ràng và phải đảm bảo tính hợp lệ để tránh lỗi runtime.

Đối với upcasting và downcasting, điều quan trọng cần hiểu là các thao tác này chỉ liên quan đến kiểu dữ liệu tham chiếu của biến, chứ không trực tiếp thay đổi hay xét đến bản chất thực sự của đối tượng mà biến đó tham chiếu. Ví dụ, một biến có thể được khai báo là kiểu `A`, nhưng thực tế nó có thể tham chiếu đến một đối tượng thuộc lớp con `B` hoặc `C`. Khi thực hiện upcasting hay downcasting, ta chỉ thay đổi kiểu dữ liệu tham chiếu của biến, tức là "gán nhãn" cho biến đó là kiểu nào trong hệ thống phân cấp lớp.

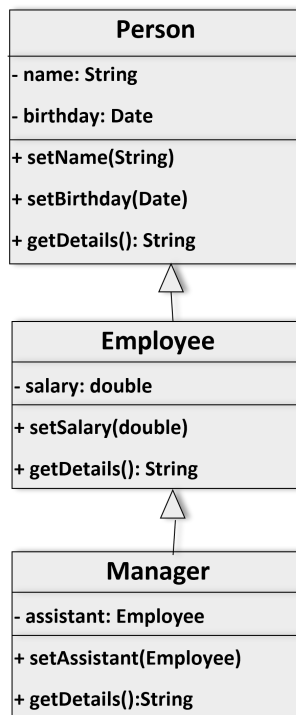
Bản chất thực tế của đối tượng trong bộ nhớ không bị thay đổi bởi việc chuyển đổi kiểu tham chiếu này. Đối tượng vẫn giữ nguyên kiểu lớp mà nó được tạo ra bằng câu lệnh `new`. Ví dụ, nếu một đối tượng được tạo ra từ lớp `C`, thì dù biến tham chiếu đến nó có kiểu `A` hay `B`, đối tượng đó vẫn là một thể hiện của lớp `C`. Điều này có ý nghĩa rất quan trọng trong lập trình, bởi vì các phương thức gọi trên biến tham chiếu sẽ được thực thi dựa trên **bản chất thực sự của đối tượng** (nguyên tắc đa hình - polymorphism), chứ không phải dựa trên kiểu tham chiếu của biến.

8.1.2. Upcasting

Upcasting là khái niệm mô tả việc chuyển đổi kiểu dữ liệu tham chiếu của một đối tượng từ lớp dẫn xuất (subclass) lên lớp cơ sở (superclass) trong cây phân cấp kế thừa. Nói cách khác, upcasting là quá trình “đi lên” trên cây kế thừa, từ lớp con lên lớp cha.

Việc upcasting cho phép ta **nhìn nhận một đối tượng thuộc lớp dẫn xuất như thể nó là một đối tượng thuộc lớp cơ sở**. Điều này giúp tăng tính trừu tượng và hỗ trợ tính đa hình (polymorphism). Khi một biến tham chiếu kiểu lớp cơ sở trỏ tới một đối tượng lớp con, ta có thể sử dụng biến đó để gọi các phương thức được định nghĩa trong lớp cơ sở, trong khi phương thức thực thi sẽ là phương thức của đối tượng lớp con (nếu có ghi đè).

Một điểm đáng chú ý là **upcasting** được thực hiện một cách tự động, không cần phải ép kiểu rõ ràng. Điều này bởi vì việc chuyển từ lớp con lên lớp cha luôn an toàn, không làm mất dữ liệu hay gây lỗi kiểu.



Hình 8.2 Ví dụ về upcasting

Hình 8.2 minh họa một ví dụ về Upcasting. Trong ví dụ này, Employee là lớp con kế thừa từ Person. Lớp Person định nghĩa các thuộc tính và phương thức cơ bản như name, birthday, và các phương thức setName(), setBirthday(), getDetails(). Lớp Employee mở rộng thêm thuộc tính salary và các phương thức liên quan như setSalary() và ghi đè phương thức getDetails().

```
public class Test1 {
    public static void main(String arg[]) {
        Employee e = new Employee();
        Person p;
        p = e; // Upcasting: biến p (Person) tham chiếu đến đối tượng e (Employee)
        p.setName("Hoa");
    }
}
```

```

        p.setSalary(350000);           // Lỗi biên dịch (compile error)
    }
}

```

Ở đây, biến `e` được khai báo và khởi tạo là một đối tượng thuộc lớp `Employee`. Sau đó, biến `p` có kiểu tham chiếu là `Person` được gán bằng biến `e`. Đây chính là upcasting, tức là biến kiểu cha (`Person`) tham chiếu đến một đối tượng thuộc lớp con (`Employee`).

Điều quan trọng cần lưu ý là khi biến `p` được khai báo là kiểu `Person`, nó chỉ có thể truy cập các thành phần (thuộc tính và phương thức) được định nghĩa trong lớp `Person` hoặc các lớp cha của nó. Mặc dù thực tế `p` đang tham chiếu đến một đối tượng `Employee`, nhưng trình biên dịch chỉ cho phép gọi các phương thức hiện hữu trong lớp `Person`.

Do đó, khi gọi `p.setSalary(350000);`, trình biên dịch sẽ báo lỗi vì phương thức `setSalary()` không thuộc lớp `Person` mà chỉ có trong lớp `Employee`. Đây là một minh chứng rõ ràng cho việc upcasting chỉ thay đổi kiểu tham chiếu của biến, không làm thay đổi bản chất đối tượng thực tế. Ngược lại, việc gọi `p.setName("Hoa");` là hợp lệ vì phương thức `setName()` được định nghĩa trong lớp `Person`. Như vậy việc gọi phương thức không tồn tại trong kiểu tham chiếu sẽ gây lỗi biên dịch, giúp tránh sai sót khi lập trình.

Ví dụ về upcasting trong quá trình truyền tham số:

```

class Manager extends Employee {
    Employee assistant;
    // ...
    public void setAssistant(Employee e) {
        assistant = e;
    }
    // ...
}

public class Test2 {
    public static void main(String arg[]) {
        Manager junior, senior;
        // ...
        senior.setAssistant(junior);
    }
}

```

```
}
```

Trong lớp `Manager`, phương thức `setAssistant(Employee e)` nhận vào một tham số kiểu `Employee`. Khi gọi `senior.setAssistant(junior);`, biến `junior` có kiểu là `Manager` (là lớp con của `Employee`), được truyền vào một tham số kiểu `Employee`. Đây là một ví dụ điển hình của upcasting ngầm định: biến `junior` kiểu `Manager` được tự động chuyển sang kiểu `Employee` khi truyền vào phương thức.

Việc nhận tham số kiểu `Employee` trong phương thức `setAssistant()` giúp tăng tính linh hoạt và mở rộng cho chương trình. Bất kỳ đối tượng nào thuộc `Employee` hoặc các lớp con của nó (như `Manager`) đều có thể được sử dụng làm trợ lý. Điều này cho phép thiết kế phần mềm theo nguyên tắc mở rộng mà không làm thay đổi giao diện phương thức.

```
public class Test3 {  
    String static teamInfo(Person p1, Person p2) {  
        return "Leader: " + p1.getName() + ", member: " + p2.getName();  
    }  
    public static void main(String arg[]) {  
        Employee e1, e2;  
        Manager m1, m2;  
        // ...  
        System.out.println(teamInfo(e1, e2));  
        System.out.println(teamInfo(m1, m2));  
        System.out.println(teamInfo(m1, e2));  
    }  
}
```

Phương thức tĩnh `teamInfo` được định nghĩa trong lớp `Test3` có tham số đầu vào là hai biến kiểu `Person`. Trong phương thức `main`, các biến `e1`, `e2` kiểu `Employee` và `m1`, `m2` kiểu `Manager` được khai báo. Sau đó các biến `e1`, `e2`, `m1`, `m2` đều được truyền làm đối số cho tham số kiểu `Person` và được tự động chuyển sang kiểu lớp cha (`Person`) khi truyền vào phương thức. Điều này cho phép phương thức `teamInfo` hoạt động linh hoạt với bất kỳ đối tượng nào thuộc lớp `Person` hoặc các lớp con của nó. Chúng ta không cần phải viết lại nhiều phiên bản phương thức cho từng lớp con cụ thể.

Trong phương thức `teamInfo`, việc gọi `p1.getName()` và `p2.getName()` là hợp lệ vì `getName()` được định nghĩa trong lớp `Person`. Nhờ tính kế thừa, các đối tượng `Employee` và `Manager` cũng có thể sử dụng phương thức này.

8.1.3. Downcasting

Downcasting được hiểu là quá trình chuyển đổi một đối tượng từ kiểu lớp cơ sở (superclass) về kiểu lớp dẫn xuất (subclass), tức là "đi xuống" trong cây phân cấp thừa kế. Đây là bước ngược lại với upcasting, vốn chuyển từ lớp con lên lớp cha.

Downcasting cho phép lập trình viên nhìn nhận một đối tượng được tham chiếu bởi biến kiểu lớp cha như thể nó thuộc về lớp con cụ thể hơn. Điều này rất hữu ích khi ta muốn truy cập các thuộc tính hoặc phương thức đặc thù chỉ có trong lớp con mà biến tham chiếu kiểu lớp cha không thể trực tiếp truy cập được. Ví dụ, một biến kiểu Person thực chất có thể trỏ tới một đối tượng Employee hoặc Manager. Để sử dụng các phương thức riêng của Employee hay Manager, ta cần thực hiện downcasting về đúng kiểu lớp con tương ứng.

Tuy nhiên, khác với upcasting, downcasting không được thực hiện tự động bởi trình biên dịch. Lập trình viên bắt buộc phải chỉ định rõ ràng việc ép kiểu (casting) khi muốn chuyển đổi kiểu từ lớp cha về lớp con. Việc ép kiểu này được thực hiện bằng toán tử ép kiểu, ví dụ: (Manager) p. Sự cần thiết của ép kiểu rõ ràng nhằm đảm bảo rằng lập trình viên có ý thức về việc chuyển đổi kiểu này, vì nếu ép kiểu sai (ví dụ biến kiểu cha thực chất không tham chiếu tới đối tượng kiểu con tương ứng), chương trình sẽ phát sinh lỗi runtime `ClassCastException`.

Ví dụ về downcasting theo sơ đồ lớp trong hình 8.2:

```
public class Test2 {  
    public static void main(String arg[]) {  
        Employee e = new Employee();  
        Person p = e; // upcasting  
        Employee ee = (Employee) p; // downcasting  
        Manager m = (Manager) ee; // run-time error  
  
        Person p2 = new Manager();  
        Employee e2 = (Employee) p2;  
  
        Person p3 = new Employee();  
        Manager e3 = (Manager) p3;  
    }  
}
```

Trong ví dụ này, biến e được upcast thành kiểu Person và gán cho biến p. Đây là một ví dụ điển hình của upcasting, được Java thực hiện tự động vì Employee là một

lớp con của `Person`. Sau đó, để truy cập lại các đặc tính riêng của lớp `Employee`, biến `p` được downcast trở lại thành kiểu `Employee` và gán cho biến `ee`. Việc này yêu cầu phải ép kiểu rõ ràng bằng cách sử dụng toán tử (`Employee`).

Ví dụ cũng cho thấy một trường hợp gây lỗi tại thời điểm chạy. Cụ thể, khi biến `ee` (thực chất là một `Employee`) bị ép kiểu thành `Manager`, chương trình sẽ phát sinh lỗi runtime ném ra ngoại lệ `ClassCastException`. Lý do là `ee` không phải là một đối tượng thuộc lớp `Manager` hay lớp con của `Manager`, nên việc ép kiểu này không hợp lệ.

Biến `p2` kiểu `Person` tham chiếu đến một đối tượng `Manager`. Khi đó, việc ép kiểu từ `Person` về `Employee` (lớp cha của `Manager`) được phép và hợp lệ, vì `Manager` là lớp con của `Employee`. Điều này cho thấy downcasting có thể diễn ra qua các cấp kế thừa, miễn là kiểu thực của đối tượng phù hợp. Biến `p3` có kiểu `Person` tham chiếu đến một đối tượng `Employee` và sau đó ép kiểu `p3` thành `Manager`. Chương trình có thể lại gặp lỗi thời gian chạy khi sử dụng đối tượng `e3` bởi vì đối tượng thực sự không phải là `Manager`.

Nguyên tắc: một đối tượng `A` chỉ được tham chiếu tới bởi các biến tham chiếu kiểu `A`, hoặc biến tham chiếu có kiểu là các lớp cha của `A`. Ngược lại sẽ bị lỗi (lớp con của `A`, lớp không có quan hệ tới `A`).

Toán tử `instanceof` để đảm bảo an toàn trước khi thực hiện downcasting, lập trình viên thường kết hợp với câu lệnh kiểm tra kiểu `instanceof` để xác định chắc chắn đối tượng có phải thuộc kiểu lớp con mong muốn hay không. Điều này giúp tránh lỗi và tăng tính ổn định cho chương trình.

Toán tử này cung cấp phép kiểm tra kiểu động, giúp lập trình viên xác định chính xác kiểu thực tế của đối tượng tại thời điểm chạy chương trình, từ đó đưa ra các xử lý phù hợp. Toán tử `instanceof` trả về giá trị boolean: `true` nếu đối tượng là thể hiện của lớp được chỉ định hoặc lớp con của nó, và `false` nếu không phải hoặc nếu đối tượng đó là `null`. Việc trả về `false` khi đối tượng là `null` giúp tránh lỗi `NullPointerException` trong quá trình kiểm tra kiểu.

Ví dụ:

```
public class Employee extends Person {}
public class Student extends Person {}

public class Test{
    public doSomething(Person e) {
        if (e instanceof Employee) {...
```

```

        } else if (e instanceof Student) {...
        } else {...
        }
    }
}

```

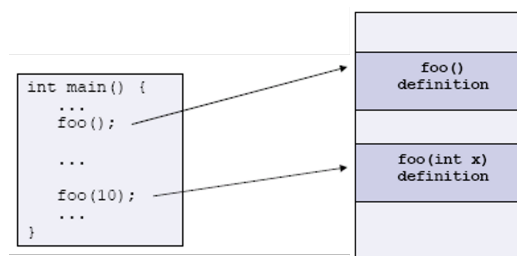
Trong ví dụ minh họa, phương thức `doSomething` nhận một tham số kiểu `Person`, nhưng thực tế tham số này có thể là một thể hiện của `Employee`, `Student` hoặc các lớp con khác. Để xử lý đúng kiểu đối tượng, ta sử dụng toán tử `instanceof` để kiểm tra xem biến `e` có phải là `Employee` hay `Student` trước khi thực hiện các thao tác đặc thù.

Như vậy, trong downcasting thay vì ép kiểu trực tiếp có thể dẫn đến nguy cơ lỗi `ClassCastException`. Chúng ta sử dụng `instanceof` để xác nhận kiểu trước, chỉ khi nào đúng kiểu mới tiến hành ép kiểu.

8.2. Liên kết tĩnh và Liên kết động

8.2.1. Liên kết lời gọi hàm

Liên kết lời gọi hàm (function call binding) là quy trình xác định chính xác khối mã của hàm nào sẽ được thực thi khi một lời gọi hàm được thực hiện. Quá trình này quyết định cách thức chương trình liên kết giữa lời gọi hàm và định nghĩa hàm tương ứng.



Hình 8.3 Liên kết lời gọi hàm

Trong ngôn ngữ lập trình C, việc liên kết lời gọi hàm khá đơn giản vì mỗi hàm có một tên duy nhất. Khi chương trình gọi một hàm, trình biên dịch sẽ xác định ngay khối mã hàm cần thực thi dựa trên tên hàm đó. Trong hình 8.3, hàm `foo()` được định nghĩa với hai phiên bản khác nhau: một phiên bản không có tham số và

một phiên bản nhận một tham số kiểu int. Khi chương trình gọi `foo()` hoặc `foo(10)`, liên kết lời gọi hàm sẽ xác định chính xác phiên bản hàm nào cần được thực thi dựa trên cách gọi và tham số truyền vào. Quá trình này được thực hiện tại thời điểm biên dịch (compile-time binding) trong các ngôn ngữ như C, nơi mà kiểu và số lượng tham số của hàm là duy nhất và rõ ràng. Do đó, trình biên dịch có thể dễ dàng xác định địa chỉ của hàm cần gọi và tạo ra mã máy phù hợp.

Trong lập trình hướng đối tượng, đặc biệt là với các ngôn ngữ hỗ trợ đa hình như Java, liên kết lời gọi phương thức có thể phức tạp hơn. Liên kết lời gọi phương thức (method call binding) là quá trình xác định đoạn mã thực thi tương ứng với lời gọi phương thức trong chương trình. Khi một phương thức được gọi, hệ thống cần biết chính xác khối mã nào sẽ được thực hiện để đảm bảo chương trình hoạt động đúng như mong đợi.

Liên kết lời gọi phương thức có thể được chia thành hai loại chính: liên kết tĩnh (static binding) và liên kết động (dynamic binding).

- *Liên kết tĩnh (static binding)* xảy ra khi việc xác định hàm hoặc phương thức cần gọi được thực hiện tại thời điểm biên dịch. Điều này thường áp dụng cho các phương thức không bị ghi đè, các phương thức tĩnh hoặc các lời gọi phương thức mà kiểu đối tượng đã được biết chắc chắn từ trước.
- *Liên kết động (dynamic binding)* xảy ra khi việc xác định phương thức cần gọi được hoãn lại đến thời điểm chạy chương trình. Khi đó, chương trình sẽ xác định chính xác phương thức tương ứng với kiểu thực tế của đối tượng tại thời điểm gọi.

8.2.2. Liên kết tĩnh

Liên kết tĩnh còn được gọi là liên kết tại thời điểm biên dịch (early binding hoặc compile-time binding), nghĩa là quá trình xác định phương thức nào sẽ được gọi được thực hiện hoàn toàn trong giai đoạn biên dịch mã nguồn. Khi trình biên dịch xử lý chương trình, nó đã biết chính xác phương thức nào sẽ được gọi dựa trên kiểu dữ liệu tĩnh của đối tượng hoặc các thông tin cố định khác. Do đó, chỉ có một phiên bản duy nhất của phương thức được thực thi khi chương trình chạy, và mọi lỗi liên quan đến phương thức như không tồn tại hoặc sai kiểu tham số sẽ được phát hiện ngay trong quá trình biên dịch, giúp giảm thiểu lỗi khi chạy chương trình.

Một ưu điểm nổi bật của liên kết tĩnh là tốc độ thực thi nhanh. Bởi vì trình biên dịch đã xác định sẵn địa chỉ của phương thức cần gọi, nên chương trình không mất thời gian để tìm kiếm hay quyết định phương thức tại thời điểm chạy. Điều này làm cho các lời gọi phương thức tĩnh trở nên hiệu quả hơn về mặt hiệu suất.

Trong Java, ví dụ điển hình của liên kết tĩnh là các phương thức được khai báo với từ khóa **static**. Các phương thức static không thuộc về bất kỳ đối tượng cụ thể nào mà thuộc về lớp, do đó việc gọi các phương thức này được xác định ngay từ thời điểm biên dịch. Điều này cũng giải thích tại sao các phương thức static không bị ghi đè (override) trong các lớp kế thừa, vì bản chất của chúng là liên kết tĩnh, không hỗ trợ đa hình.

Ví dụ về liên kết tĩnh trong Java:

```
class Human {
    public static void walk() {
        System.out.println("Human walks");
    }
}

public class Boy extends Human {
    public static void walk() {
        System.out.println("Boy walks");
    }

    public static void main(String args[]) {
        // Reference is of Human type and object is Boy type
        Human obj1 = new Boy();
        // Reference is of Human type and object is Human type.
        Human obj2 = new Human();
        // Reference is of Human type and object is Human type.
        Boy obj3 = new Boy();
        obj1.walk();
        obj2.walk();
        obj3.walk();

        obj1 = obj3;
        obj1.walk();
    }
}
```

Cả hai phương thức walk() trong lớp Human và Boy đều là phương thức static, nên chúng được liên kết tĩnh. Khi chạy chương trình, chúng ta khai báo ba đối tượng: obj1 là tham chiếu kiểu Human trỏ đến một đối tượng Boy, obj2 là tham chiếu kiểu Human trỏ đến đối tượng Human, và obj3 là tham chiếu kiểu Boy trỏ đến đối tượng Boy.

Khi gọi `walk()` thông qua các biến này, kết quả in ra cho thấy rằng phương thức được gọi phụ thuộc vào kiểu tham chiếu (reference type), không phải kiểu thực tế của đối tượng (object type). Cụ thể, `obj1.walk()` và `obj2.walk()` đều gọi phương thức `walk()` của lớp `Human`, in ra "Human walks" mặc dù `obj1` thực sự trỏ tới một đối tượng `Boy`. Ngược lại, `obj3.walk()` gọi phương thức `walk()` của lớp `Boy`, in ra "Boy walks".

Khi gán `obj1 = obj3`; và gọi lại `obj1.walk()`, kết quả vẫn là "Human walks" vì kiểu tham chiếu của `obj1` vẫn là `Human`.

Ví dụ này minh họa rõ ràng bản chất của liên kết tĩnh trong Java: các phương thức static được quyết định gọi dựa trên kiểu tham chiếu tại thời điểm biên dịch, chứ không phải dựa trên kiểu đối tượng thực tế tại thời điểm chạy chương trình. Ngoài phương thức static, trong Java các phương thức khai báo là `private` hoặc `final` cũng được liên kết tĩnh.

8.2.3. Liên kết động

Trong liên kết động (dynamic binding) lời gọi phương thức không được xác định ngay tại thời điểm biên dịch mà được trì hoãn và quyết định trong quá trình chạy chương trình (run-time). Điều này có nghĩa là phiên bản cụ thể của phương thức được chọn để thực thi sẽ dựa trên kiểu thực tế của đối tượng tại thời điểm gọi, chứ không phải dựa trên kiểu tham chiếu tĩnh của biến. Quá trình này còn được gọi là late binding hoặc run-time binding.

Java mặc định sử dụng liên kết động cho tất cả các phương thức instance (không phải static). Khi một phương thức được gọi trên một biến tham chiếu, JVM sẽ xác định đúng phiên bản phương thức tương ứng với đối tượng thực tế mà biến đó tham chiếu tới để thực thi.

Nếu một phương thức được khai báo với từ khóa **final**, điều này đồng nghĩa với việc tắt cơ chế liên kết động cho phương thức đó. Phương thức `final` sẽ được liên kết tĩnh (static binding), tức là lời gọi phương thức sẽ được xác định ngay tại thời điểm biên dịch dựa trên kiểu tham chiếu, không bị ảnh hưởng bởi kiểu đối tượng thực tế. Từ khóa `final` ngăn cản ghi đè (override) phương thức. Khi phương thức không thể bị ghi đè, trình biên dịch không cần chờ tới lúc chạy mới xác định phiên bản phương thức được gọi. Vì nó biết chắc chắn phương thức nào sẽ được thực thi, nên có thể liên kết tĩnh.

Ngoài Java, các ngôn ngữ lập trình hướng đối tượng khác cũng hỗ trợ cơ chế liên kết động nhưng có cách thể hiện khác nhau. Trong C, liên kết động được hỗ trợ thông qua các từ khóa như `virtual`, `abstract` và `override`. Các phương thức được

đánh dấu virtual có thể bị ghi đè trong các lớp dẫn xuất, và lời gọi phương thức sẽ được xác định tại thời điểm chạy dựa trên kiểu đối tượng thực tế.

Trong C++, cơ chế liên kết động được thực hiện thông qua từ khóa virtual. Khi một phương thức được khai báo là virtual trong lớp cơ sở, lời gọi phương thức sẽ được xác định tại thời điểm chạy dựa trên kiểu đối tượng thực tế, cho phép đa hình động. Nếu không khai báo virtual, phương thức sẽ được liên kết tĩnh, tương tự như trong Java với các phương thức static hoặc final.

Ví dụ về liên kết động trong Java:

```
public class Test {
    public static void main(String arg[]){
        Person p = new Person();
        // ...
        Employee e = new Employee();
        // ...
        Manager m = new Manager();
        // ...
        Person pArr[] = {p, e, m};
        for (int i=0; i< pArr.length; i++){
            System.out.println(pArr[i].getDetail());
        }
    }
}
```

Nhắc lại về sơ đồ lớp trong hình 8.2. Cả ba lớp Person, Employee và Manager đều có chung một phương thức quan trọng là getDetails(), được định nghĩa lại (override) ở các lớp con để trả về thông tin chi tiết phù hợp với từng loại đối tượng.

Trong ví dụ này, mảng pArr được khai báo là mảng các đối tượng kiểu Person, nhưng lại chứa các thể hiện của cả Person, Employee và Manager. Khi duyệt qua mảng và gọi getDetails() trên từng phần tử, Java sẽ sử dụng cơ chế liên kết động để xác định phiên bản phương thức getDetails() chính xác tương ứng với kiểu thực tế của đối tượng đang được tham chiếu. Nhờ đó, dù biến mảng có kiểu tham chiếu là Person, nhưng phương thức được gọi sẽ là phiên bản của lớp thực thể đối tượng.

8.3. Đa hình (Polymorphism)

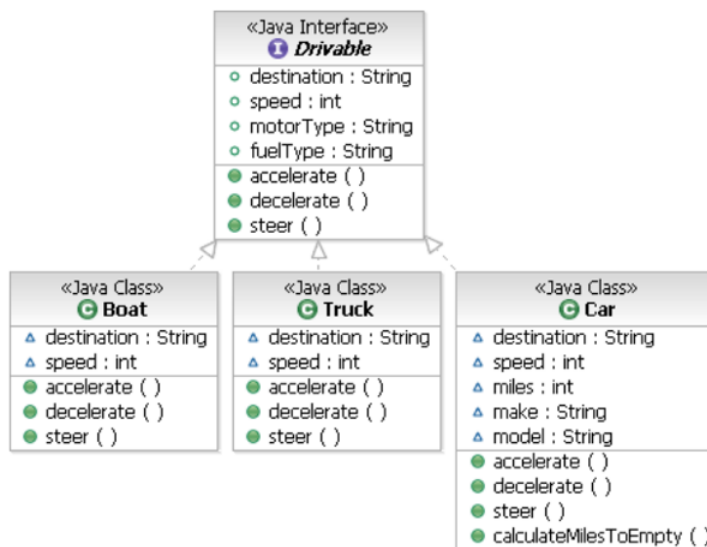
Đa hình (Polymorphism) thể hiện khả năng một đối tượng có thể được xử lý theo nhiều hình thức khác nhau. Để minh họa cho khái niệm này, chúng ta có thể

tưởng tượng về một tình huống rất gần gũi trong cuộc sống hàng ngày: khi đi du lịch, bạn có thể lựa chọn nhiều phương tiện khác nhau như ô tô, thuyền hoặc máy bay để đến nơi cần đến.



Hình 8.4 Minh họa cho đa hình

Mặc dù các phương tiện này khác nhau về bản chất, cách thức hoạt động và trải nghiệm khi sử dụng, nhưng mục đích cuối cùng của chúng là giống nhau, đó là giúp bạn đến được điểm đến mong muốn. Điều này tương tự như trong lập trình hướng đối tượng, các lớp khác nhau có thể cùng đáp ứng một tập hợp các thông điệp giống nhau, từ đó cung cấp các dịch vụ tương tự nhưng với cách thức thực hiện khác nhau. Điều này có nghĩa là các lớp con có thể triển khai các phương thức của một giao diện hoặc lớp cha theo cách riêng biệt, phù hợp với đặc điểm và yêu cầu cụ thể của từng lớp.



Hình 8.5 Nguyên lý đa hình thông qua giao diện

Trong hình 8.5, giao diện Drivable, định nghĩa các thuộc tính và phương thức chung. Từ giao diện này, có ba lớp cụ thể triển khai: Boat (thuyền), Truck (xe tải) và Car (ô tô). Mỗi lớp này đều có các thuộc tính riêng biệt, ví dụ như Car có thêm miles, make, model và một phương thức đặc biệt calculateMilesToEmpty() bên cạnh các phương thức chung.

Điều quan trọng là mặc dù cả ba lớp đều nhận và phản hồi các thông điệp như accelerate(), decelerate() hay steer(), cách thức thực hiện các phương thức này có thể khác nhau tùy thuộc vào từng loại phương tiện. Ví dụ, cách tăng tốc của một chiếc thuyền sẽ khác với xe tải hay ô tô. Nhờ đó, các đối tượng thuộc các lớp này có thể được trao đổi cho nhau mà không làm ảnh hưởng đến đối tượng gửi thông điệp, tức là phần mã gọi các phương thức đó không cần biết chi tiết bên trong cách thực hiện, chỉ cần tin tưởng rằng đối tượng sẽ thực hiện đúng chức năng.

Khả năng này chính là biểu hiện của đa hình trong lập trình hướng đối tượng, giúp tăng tính linh hoạt và khả năng mở rộng cho phần mềm. Khi cần thêm một loại phương tiện mới, chỉ cần tạo một lớp mới triển khai giao diện Drivable và định nghĩa lại các phương thức theo cách riêng mà không cần thay đổi mã nguồn hiện có. Điều này giảm thiểu sự phụ thuộc giữa các thành phần và giúp chương trình dễ bảo trì hơn.

Đa hình còn được giải thích theo nghĩa là "nhiều hình thức thực hiện" hoặc "nhiều kiểu tồn tại". Đây là một đặc tính quan trọng cho phép một biến tham chiếu có thể thay đổi hành vi tùy theo đối tượng mà nó đang tham chiếu đến. Điều này có nghĩa là cùng một lời gọi phương thức có thể dẫn đến các hành động khác nhau, tùy thuộc vào kiểu thực tế của đối tượng mà biến đó trỏ tới trong thời gian chạy chương trình.

Đa hình trong lập trình được chia thành hai loại chính: đa hình phương thức và đa hình đối tượng.

- *Đa hình phương thức* đề cập đến việc cùng một tên phương thức có thể xuất hiện nhiều lần trong cùng một lớp hoặc trong các lớp kế thừa, nhưng được phân biệt dựa vào danh sách tham số khác nhau. Đây là cơ chế gọi là nạp chồng phương thức (method overloading), giúp tăng tính linh hoạt và khả năng tái sử dụng mã nguồn trong quá trình thiết kế phần mềm.
- *Đa hình đối tượng* liên quan đến cách nhìn nhận một đối tượng theo nhiều kiểu khác nhau. Các đối tượng thuộc các lớp khác nhau có thể cùng đáp ứng chung một danh sách các thông điệp (phương thức) nhưng lại có cách giải nghĩa và thực thi các thông điệp đó theo phương thức riêng biệt. Đây chính là biểu hiện của việc ghi đè phương thức (method overriding) trong các lớp kế

thừa, cho phép các lớp con có thể tùy biến hành vi của mình khi nhận cùng một lời gọi phương thức từ lớp cha hoặc giao diện.

Nhìn nhận đối tượng theo nhiều kiểu khác nhau, chính là Upcasting và Down-casting:

```
public class Test3 {  
    public static void main(String args[]){  
        Person p1 = new Employee();  
        Person p2 = new Manager();  
  
        Employee e = (Employee) p1;  
        Manager m = (Manager) p2;  
    }  
}
```

Các đối tượng khác nhau giải nghĩa các thông điệp theo cách thức khác nhau, chính là liên kết động (dynamic binding):

```
Person p1 = new Person();  
Person p2 = new Employee();  
Person p3 = new Manager();  
// ...  
System.out.println(p1.getDetail());  
System.out.println(p2.getDetail());  
System.out.println(p3.getDetail());
```

8.4. Ví dụ và bài tập

Bài tập 1 Giả sử lớp Sub kế thừa từ lớp cha Sandwich. Tạo hai đối tượng từ các lớp này:

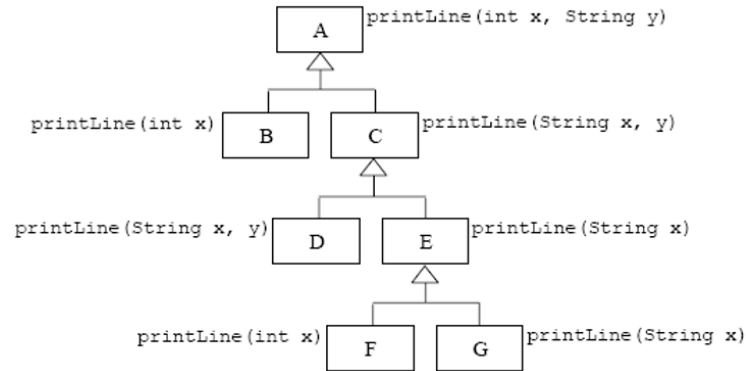
```
Sandwich x = new Sandwich();  
Sub y = new Sub();
```

Phép gán nào sau đây là hợp lệ? Giải thích?

- $x = y$;
- $y = x$;

- `y = new Sandwich();`
- `x = new Sub();`

Bài tập 2 Cho biểu đồ lớp như sau:



Hình 8.6 Biểu đồ lớp cho bài tập 2

Phương thức `printLine()` của lớp nào sẽ được sử dụng trong mỗi trường hợp dưới đây, biết rằng `z` là một đối tượng của lớp `F`? Giải thích ngắn gọn?

- `z.printLine(1)`
- `z.printLine(2, "Object-Oriented Programming")`
- `z.printLine("Java")`
- `z.printLine("Object-Oriented Programming", "Java")`
- `z.printLine("Object-Oriented Programming", 3)`

Bài tập 3 Những điều kiện nào trả về true? (Có thể xem Java documentation để biết các quan hệ thừa kế giữa các lớp) Biết rằng `System.out` là một đối tượng của lớp `PrintStream`.

- `System.out instanceof PrintStream`
- `System.out instanceof OutputStream`
- `System.out instanceof LogStream`

- System.out instanceof Object
- System.out instanceof String
- System.out instanceof Writer

Bài tập 4 Kiểm tra các đoạn mã sau đây và vẽ sơ đồ lớp tương ứng?

```
abstract public class Animal {
    abstract public void greeting();
}
public class Cat extends Animal {
    public void greeting() {
        System.out.println("Meow!");
    }
}
public class Dog extends Animal {
    public void greeting() {
        System.out.println("Woof!");
    }
}
```

```
public void greeting(Dog another) {
    System.out.println("Wooooooooooof!");
}
}
public class BigDog extends Dog {
    public void greeting() {
        System.out.println("Woow!");
    }
    public void greeting(Dog another) {
        System.out.println("Woowooooowwww!");
    }
}
```

Hình 8.7 Mã nguồn cho bài tập 4

Bài tập 5 Giải thích các đầu ra (hoặc các lỗi nếu có) cho chương trình thử nghiệm sau:

```
public class TestAnimal {
    public static void main(String[] args) {
        // Using the subclasses
        Cat cat1 = new Cat();
        cat1.greeting();
        Dog dog1 = new Dog();
        dog1.greeting();
        BigDog bigDog1 = new BigDog();
        bigDog1.greeting();

        // Using Polymorphism
        Animal animal1 = new Cat();
        animal1.greeting();
        Animal animal2 = new Dog();
        animal2.greeting();
        Animal animal3 = new BigDog();
```

```
        animal3.greeting();
        Animal animal4 = new Animal();

        // Downcast
        Dog dog2 = (Dog)animal2;
        BigDog bigDog2 = (BigDog)animal3;
        Dog dog3 = (Dog)animal3;
        Cat cat2 = (Cat)animal2;
        dog2.greeting(dog3);
        dog3.greeting(dog2);
        dog2.greeting(bigDog2);
        bigDog2.greeting(dog2);
        bigDog2.greeting(bigDog1);
    }
}
```

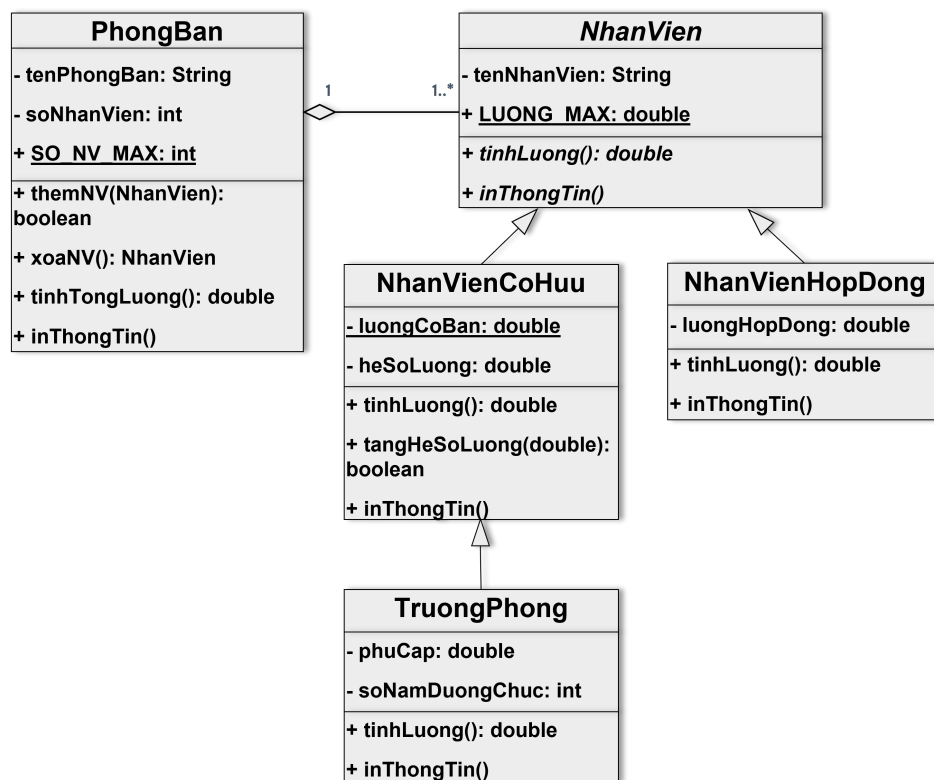
Hình 8.8 Mã nguồn cho bài tập 5

Bài tập 6 Phân tích xây dựng các lớp như mô tả sau:

- Hàng điện máy <mã hàng, tên hàng, nhà sản xuất, giá, thời gian bảo hành, điện áp, công suất>
- Hàng sành sứ < mã hàng, tên hàng, nhà sản xuất, giá, loại nguyên liệu>
- Hàng thực phẩm <mã hàng, tên hàng, nhà sản xuất, giá, ngày sản xuất, ngày hết hạn dùng>

Viết chương trình tạo mỗi loại một mặt hàng cụ thể. Xuất thông tin về các mặt hàng này.

Bài tập 7 Xây dựng các lớp trong biểu đồ lớp sau:



Hình 8.9 Mã nguồn cho bài tập 7

Chương 9

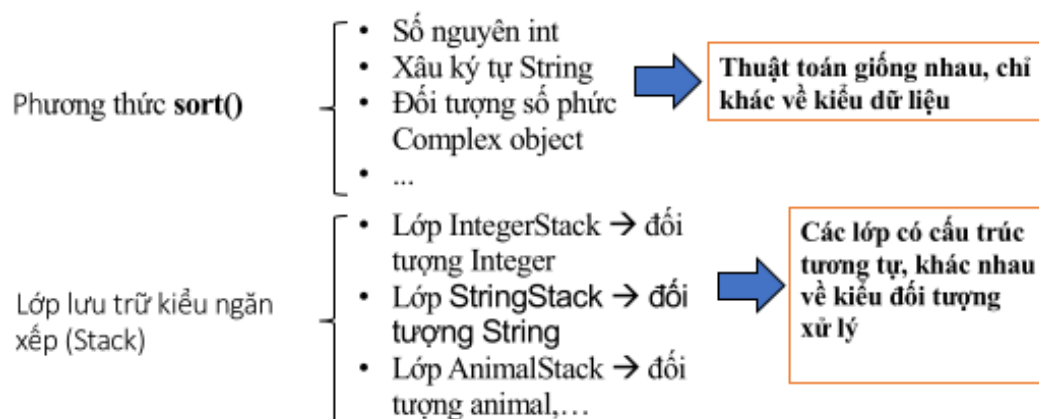
Lập trình tổng quát

Mục tiêu: Chương này sẽ giới thiệu về lập trình tổng quát và cách thực hiện trong ngôn ngữ lập trình Java cũng như giới thiệu về Java collection framework với các cấu trúc tổng quát: List, HashMap, Tree, Set, Vector,... Định nghĩa và sử dụng Java Template với ký tự đại diện (wildcard) cũng được trình bày. Cuối cùng kết thúc chương sẽ gồm các ví dụ và bài tập về các vấn đề trên với ngôn ngữ lập trình Java.

9.1. Giới thiệu về lập trình tổng quát

Lập trình tổng quát (Generic programming) có nghĩa là tổng quát hóa chương trình để có thể hoạt động với các kiểu dữ liệu khác nhau, kể cả kiểu dữ liệu trong tương lai.

Ví dụ:



Hình 9.1 Ví dụ lập trình tổng quát

Lập trình tổng quát với ngôn ngữ C, ta sử dụng con trỏ dạng tổng quát có kiểu void. Lập trình tổng quát với ngôn ngữ C++, ta sử dụng template. Với ngôn ngữ Java thì trước bản 1.5, người ta lợi dụng các kỹ thuật upcasting, downcasting và lớp Object để thực hiện lập trình tổng quát. Từ sau phiên bản 1.5 trở đi thì Java đề xuất sử dụng template cho lập trình tổng quát. Ví dụ lập trình tổng quát trong ngôn ngữ C với hàm memcpy() của thư viện string.h:

```
void* memcpy(void* region1, const void* region2, size_t n);
```

Hàm memcpy() bên trên được khai báo tổng quát bằng cách sử dụng các con trỏ kiểu void. Điều này giúp cho hàm có thể sử dụng với nhiều kiểu dữ liệu khác nhau: dữ liệu được truyền vào một cách tổng quát thông qua địa chỉ và kích thước kiểu dữ liệu. Hay nói cách khác, để sao chép dữ liệu, ta chỉ cần địa chỉ và kích cỡ của chúng.

Lập trình tổng quát với ngôn ngữ Java trước phiên bản 1.5, tất cả các đối tượng chưa rõ kiểu sẽ đều được coi là đối tượng lớp Object do lớp Object là lớp tổ tiên tổng quát nhất nên có thể chấp nhận các đối tượng thuộc lớp hậu duệ của nó. Ví dụ:

```
public class ArrayList {
    public Object get(int i) { . . . }
    public void add(Object o) { . . . }
    . . .
    private Object[] elementData;
}
```

Lớp ArrayList nêu ở trên đã khai báo phương thức add nên nó có thể thêm các phần tử có kiểu đối tượng khác nhau vào trong danh sách của nó. Ví dụ:

```
List myList = new ArrayList(); // tạo đối tượng lớp ArrayList tên myList
myList.add("Fred"); // thêm 1 đối tượng lớp String vào myList
myList.add(new Dog()); // thêm 1 đối tượng lớp Dog vào myList
myList.add(new Integer(42)); // thêm 1 đối tượng lớp Integer vào myList
```

Hạn chế là ta phải thực hiện ép kiểu và có thể ép sai kiểu dẫn tới lỗi chạy chương trình runtime (run-time error), ví dụ:

```
// lệnh dưới là lấy ra đối tượng lớp Dog chứ không phải lớp String!!!!
String name = (String) myList.get(1); // lỗi
```

Từ Java phiên bản 1.5 trở đi, ta sử dụng template để thực hiện lập trình tổng quát. Ví dụ:

```
// danh sách chỉ chấp nhận các đối tượng có kiểu là Integer
List<Integer> myList = new LinkedList<Integer>();
myList.add(new Integer(0));
Integer x = myList.iterator().next(); //Không cần ép kiểu
myList.add(new String("Hello")); //Lỗi biên dịch (Compile Error)
```

9.2. Định nghĩa và sử dụng Template

9.2.1. Lớp tổng quát

Lớp tổng quát (generic class) là lớp có thể nhận kiểu dữ liệu là một lớp bất kỳ. Cú pháp:

Tên Lớp <kiểu 1, kiểu 2, kiểu 3...>{....}

Các phương thức hay thuộc tính của lớp tổng quát có thể sử dụng các kiểu được khai báo như mọi lớp bình thường khác. Ví dụ:

```
public class Information<T> {
    private T value;
    public Information(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}
```

Trong lớp Information khai báo ở trên thì T là tên kiểu, ta sẽ phải thay thế T bằng một kiểu cụ thể, ví dụ:

```
Information<String> mystring = new Information<String>("hello");
Information<Circle> circle = new Information<Circle>(new Circle());
Information<2DShape> shape = new Information<>(new 2DShape());
```

Chú ý: Không sử dụng các kiểu dữ liệu nguyên thủy cho các lớp tổng quát. Ví dụ:

```
Information<int> integer = new Information<int>(2012); // Error
Information<Integer> integer = new Information<Integer>(2012); // OK
```

Quy ước đặt tên kiểu được trình bày trong hình 9.2

Tên kiểu	Mục đích
E	Các thành phần trong một collection
K	Kiểu khóa trong Map
V	Kiểu giá trị trong Map
T	Các kiểu thông thường
S, U	Các kiểu thông thường khác

Hình 9.2 Quy ước đặt tên kiểu

9.2.2. Phương thức tổng quát

Phương thức tổng quát (generic method) là các phương thức tự định nghĩa kiểu tham số của nó. Phương thức tổng quát có thể được viết trong lớp bất kỳ (tổng quát hoặc không). Cú pháp:

(chỉ định truy cập) <kiểu1, kiểu 2... > (kiểu trả về) tên phương thức (danh sách tham số) { ... }

Ví dụ:

```
public static <E> void print(E[ ] a) { ... }
```

Ví dụ minh họa:

```
public class ArrayTool {
    //phương thức in các phần tử trong mảng String
    public static void print(String[ ] a) {
        for (String e : a) System.out.print(e + " ");
        System.out.println();
    }
    //phương thức in các phần tử của mảng có kiểu dữ liệu bất kỳ
    public static <E> void print(E[ ] a) {
        for (E e : a) System.out.print(e + " ");
        System.out.println();
    }
}

...
```

```
String[ ] str = new String[5];
Point[ ] p = new Point[3];
int[ ] intnum = new int[2];
```

```
ArrayTool.print(str); // OK
ArrayTool.print(p); // OK
```

Chú ý không dùng được với kiểu dữ liệu nguyên thủy, nếu sử dụng sẽ báo lỗi, ví dụ lệnh `ArrayTool.print(intnum);` sẽ gây lỗi.

9.2.3. Giới hạn kiểu dữ liệu tổng quát

Có thể giới hạn các kiểu dữ liệu tổng quát sử dụng phải là dẫn xuất của một hoặc nhiều lớp như sau:

- Giới hạn 1 lớp với cú pháp `<type-param extends bound>`
- Giới hạn nhiều lớp với cú pháp `<type-param extends bound1 & bound2 & ..>`

Ví dụ:

```
// chấp nhận các kiểu là lớp con của lớp 2DShape
public class Information<T extends 2DShape> {
    private T value;
    public Information(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}

// khai báo OK
Information<Point> pointInfo = new Information<Point>(new Point());

// khai báo lỗi
Information<String> stringInfo = new Information<String>(); // error
```

9.2.4. Lập trình tổng quát trong Java collections framework

Collection là đối tượng có khả năng chứa các đối tượng khác. Các thao tác thông thường trên collection là:

- Thêm/Xoá đối tượng vào/khỏi collection
- Kiểm tra một đối tượng có ở trong collection không
- Lấy một đối tượng từ collection
- Duyệt các đối tượng trong collection
- Xoá toàn bộ collection

Các collection đầu tiên của Java gồm:

- Mảng
- Vector: Mảng động
- Hashtable: Bảng băm

Java Collections Framework (gọi tắt là JCF) kể từ Java 1.2 đã được nâng lên thành một kiến trúc hợp nhất để biểu diễn và thao tác trên các collection. JCF giúp cho việc xử lý các collection độc lập với biểu diễn chi tiết bên trong của chúng.

Một số lợi ích của Collections Framework:

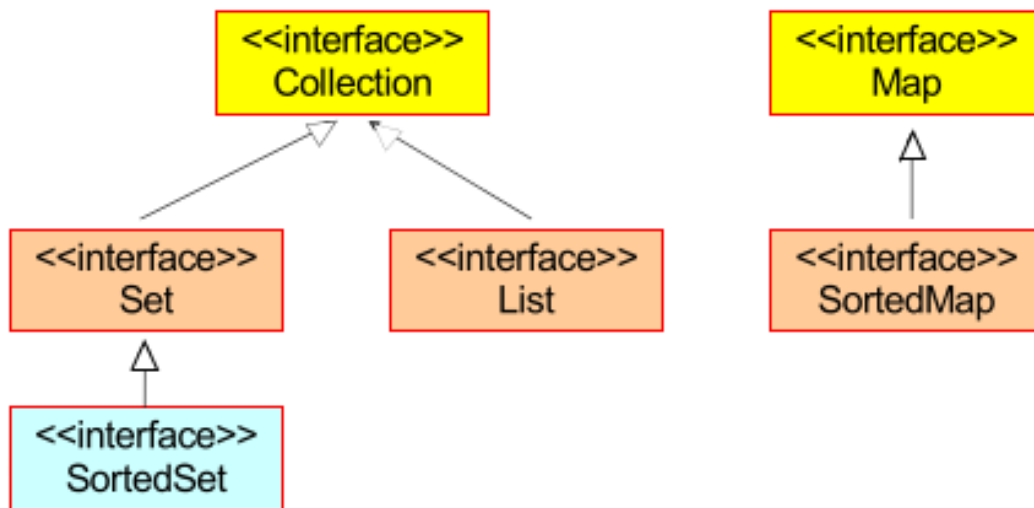
- Giảm thời gian lập trình
- Tăng cường hiệu năng chương trình
- Dễ mở rộng các collection mới
- Khuyến khích việc sử dụng lại mã chương trình

Collections Framework bao gồm:

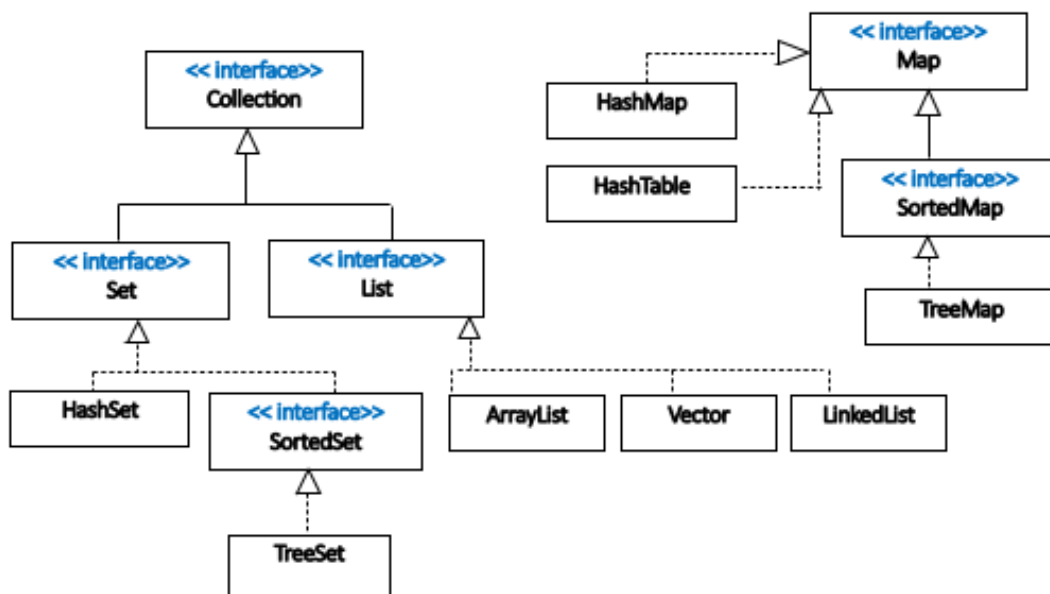
- Interfaces: Là các giao diện thể hiện tính chất của các kiểu collection khác nhau như List, Set, Map.
- Implementations: Là các lớp collection có sẵn được cài đặt các collection interfaces.
- Algorithms: Là các phương thức tính để xử lý trên collection, ví dụ: sắp xếp danh sách, tìm phần tử lớn nhất...

Java collections framework có các giao diện sau:

- List: Tập các đối tượng tuần tự, kế tiếp nhau, có thể lặp lại.
- Set: Tập các đối tượng không lặp lại.
- Map: Tập các cặp khóa-giá trị (key-value) và không cho phép khóa lặp lại.



Hình 9.3 Giao diện Java collection framework



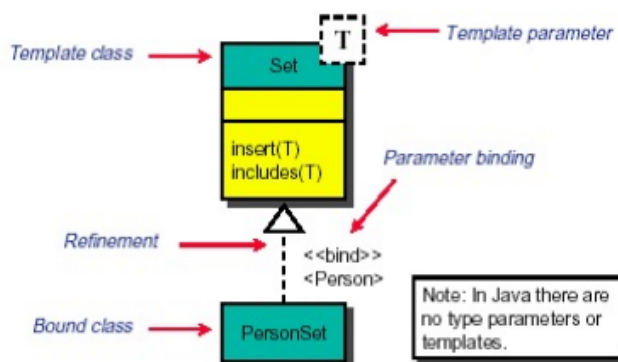
Hình 9.4 Cây phân cấp kế thừa của Java collections framework

Hình 9.5 là sự so sánh giữa Collection và Array trong Java.

Collection	Array
Collection (có thể) truy xuất theo dạng ngẫu nhiên	Mảng truy xuất 1 cách tuần tự
Collection có thể chứa nhiều loại đối tượng/dữ liệu khác nhau	Mảng chứa 1 loại đối tượng/dữ liệu nhất định
Dùng Java Collection, chỉ cần khai báo và gọi những phương thức đã được định nghĩa sẵn	Dùng tổ chức dữ liệu theo mảng phải lập trình hoàn toàn
Duyệt các phần tử tập hợp thông qua Iterator	Duyệt các phần tử mảng tuần tự thông qua chỉ số mảng

Hình 9.5 Collection và Array trong Java

Các giao diện và lớp thực thi trong Collection framework của Java đều được xây dựng theo template. Điều này cho phép ta xác định tập các phần tử cùng kiểu bất kỳ nào đó và cho phép ta chỉ định kiểu dữ liệu của các Collection nhằm hạn chế việc thao tác sai kiểu dữ liệu.



Hình 9.6 Template trong Java

Ví dụ:

```

public interface List<E> {
    void add(E x);
}

```

```

        Iterator<E> iterator();
    }

    List<String> myList = new ArrayList<String>();
    myList.add("Fred"); // OK
    myList.add(new Dog()); // Compile error!
    String s = myList.get(0);

```

Giao diện Collection

Giao diện Collection xác định giao diện cơ bản cho các thao tác với một tập các đối tượng:

- Thêm vào collection
- Xóa khỏi collection
- Kiểm tra có là thành viên

Giao diện Collection chứa các phương thức thao tác trên các phần tử riêng lẻ hoặc theo khối. Giao diện Collection cung cấp các phương thức cho phép thực hiện duyệt qua các phần tử trên collection (lặp) và chuyển tập hợp sang mảng.

Giao diện Collection khai báo các phương thức như hình dưới đây, trong đó phương thức retainAll(Collection c) giữ lại các phần tử của Collection c trong Collection đang xét (nghĩa là loại bỏ những phần tử không có trong Collection c trong Collection đang xét):

```

public interface Collection {
    // Basic Operations
    int size();
    boolean isEmpty();
    boolean contains(Object element);
    boolean add(Object element);
    boolean remove(Object element);
    Iterator iterator();

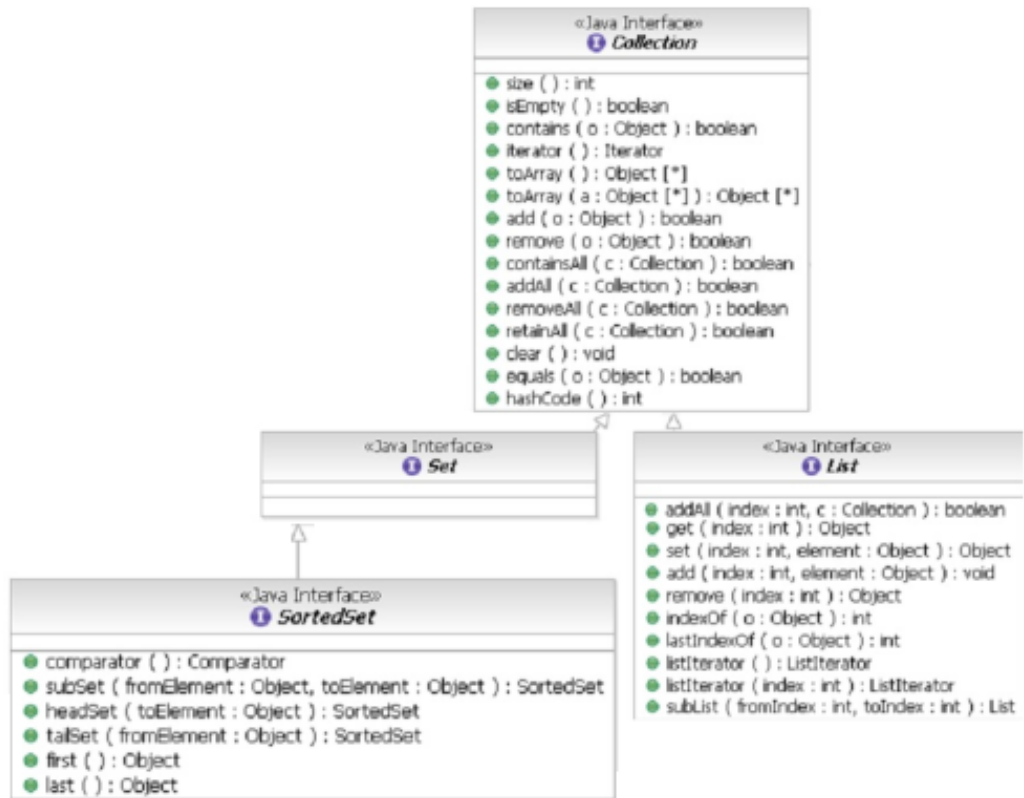
    // Bulk Operations
    boolean addAll(Collection c);
    boolean removeAll(Collection c);
    boolean retainAll(Collection c);

    /**
     * // Array Operations
     */
    Object[] toArray();
    Object[] toArray(Object a[]);
}

```

Hình 9.7 Các phương thức chính của Giao diện Collection

Các giao diện con kế thừa giao diện Collection:



Hình 9.8 Giao diện con của Collection

Giao diện Set

Giao diện Set kế thừa từ Collection nên cũng hỗ trợ toàn bộ các thao tác xử lý trên Collection. Set là một tập hợp các phần tử không được trùng lặp. Set không có thêm phương thức riêng ngoài các phương thức kế thừa từ Collection.

Ví dụ:

- Set of cars: BMW, Ford, Jeep, Chevrolet, Nissan, Toyota, VW
- Nationalities in the class: Chinese, American, Canadian, Indian

Giao diện SortedSet

Giao diện SortedSet kế thừa giao diện Set và có đặc điểm sau:

- Các phần tử được sắp xếp theo một thứ tự
- Không có các phần tử trùng nhau
- Cho phép một phần tử là null
- Các đối tượng đưa vào trong một SortedSet phải cài đặt giao diện Comparable hoặc lớp cài đặt SortedSet phải nhận một Comparator trên kiểu của đối tượng đó (khi đó ta mới có thể so sánh được các đối tượng trong SortedSet với nhau do Comparable/Comparator yêu cầu phải định nghĩa các phương thức so sánh). Một số phương thức của giao diện SortedSet:

- first(): lấy phần tử đầu tiên (nhỏ nhất)
- last(): lấy phần tử cuối cùng (lớn nhất)
- SortedSet subSet(Object e1, Object e2): lấy một tập các phần tử nằm trong khoảng từ e1 tới e2

Ví dụ ta có một đối tượng t là đối tượng của một lớp cài đặt giao diện SortedSet. Khi đó nếu một lớp A cài đặt giao diện Comparable thì ta có thể thực hiện thao tác thêm đối tượng lớp A vào t, ví dụ t.add(new A());

Ta có một ví dụ khác. Lớp TreeSet là một lớp thực thi giao diện SortedSet, do đó ta có thể khởi tạo đối tượng lớp TreeSet sử dụng một đối tượng Comparator<T> như sau:

```
import java.util.*;
public class Main {
    public static void main(String a[]){
        TreeSet<String> ts = new TreeSet<String>(new MyComp());
        ts.add("RED");
        ts.add("ORANGE");
        ts.add("BLUE");
        ts.add("GREEN");
        System.out.println(ts); // in ra BLUE, GREEN, ORANGE, RED
    }
}
class MyComp implements Comparator<String>{
    @Override
    public int compare(String str1, String str2) {
        return str1.compareTo(str2);
    }
}
```

Giao diện List

Giao diện List kế thừa từ Collection. List cung cấp thêm các phương thức để xử lý Collection kiểu danh sách (danh sách là một collection với các phần tử được xếp theo chỉ số).

Một số phương thức của List:

```
Object get(int index);
Object set(int index, Object o);
void add(int index, Object o);
Object remove(int index);
int indexOf(Object o);
int lastIndexOf(Object o);
```

Giao diện Map

Giao diện Map là giao diện cơ bản để thao tác với một tập hợp bao gồm cặp khóa-giá trị như thêm một cặp khóa-giá trị, xóa một cặp khóa-giá trị, lấy về giá trị với khóa đã có, kiểm tra có phải là thành viên (khóa hoặc giá trị)... Giao diện Map cung cấp 3 cách nhìn cho nội dung của tập hợp: tập các khóa, tập các giá trị, tập các ánh xạ khóa-giá trị.

Giao diện Map cung cấp các thao tác xử lý trên các bảng ánh xạ:

- Xác định giao diện cơ bản để thao tác với một tập hợp bao gồm cặp khóa-giá trị
- Bảng ánh xạ lưu các phần tử theo khoá và không được có 2 khoá trùng nhau

Một số phương thức của Map:

```
Object put(Object key, Object value); //chèn vào cặp khóa-giá trị key-value
Object get(Object key); // lấy giá trị từ khóa key
Object remove(Object key); // xóa cặp khóa-giá trị key-value
boolean containsKey(Object key); // kiểm tra khóa key có không
boolean containsValue(Object value); // kiểm tra giá trị value có không
Set keySet(); // lấy ra tập khóa
Collection values(); // lấy ra tập giá trị
Set entrySet(); // lấy ra tập ánh xạ khóa-giá trị
```

Giao diện SortedMap

Giao diện SortedMap thừa kế giao diện Map:

- các phần tử được sắp xếp theo thứ tự

- tương tự SortedSet, tuy nhiên việc sắp xếp được thực hiện với các khóa

Phương thức của giao diện SortedMap cũng tương tự Map, bổ sung thêm:

- firstKey(): returns the first (lowest) value currently in the map
- lastKey(): returns the last (highest) value currently in the map

Các lớp thực thi giao diện Collection

Java đã xây dựng sẵn một số lớp thực thi các giao diện Set, List và Map và cài đặt các phương thức HashMap là bảng băm (cài đặt của Map), tương ứng như ArrayList, LinkedList, HashSet, LinkedHashMap, TreeSet, HashMap...

		IMPLEMENTATIONS				
		Hash Table	Resizable Array	Balanced Tree	Linked List	Legacy
I N T E R F A C E S	Set	HashSet		TreeSet		
	List		ArrayList		LinkedList	Vector, Stack
	Map	HashMap		TreeMap		HashTable, Properties

Hình 9.9 Các lớp thực thi giao diện Collection

ArrayList là mảng động, nếu các phần tử thêm vào vượt quá kích cỡ mảng, mảng sẽ tự động tăng kích cỡ.

LinkedList là danh sách liên kết nhằm hỗ trợ thao tác trên đầu và cuối danh sách cũng như được sử dụng để tạo ngăn xếp, hàng đợi, cây ...

HashSet được sử dụng để lưu các phần tử vào trong một bảng băm (xem lại trong môn Cấu trúc dữ liệu và thuật toán). HashSet không cho phép lưu trùng lặp nhưng cho phép phần tử null. HashSet chỉ yêu cầu một đối tượng khi gọi phương thức add(Object o): đối tượng này sẽ đóng vai trò là các khóa còn giá trị ứng với nó trong bảng băm của HashSet được coi là không xác định (dummy). Độ phức tạp tính toán của các thao tác cơ bản của HashSet không phải lúc nào cũng đảm bảo bằng $O(1)$. Ví dụ về HashSet:

```
HashSet<String> hs = new HashSet<String>();
hs.add("geeks");
hs.add("prac");
hs.add("con");
hs.add("geeks");
```

```

hs.add("prac");
hs.add(null);
hs.add(null);
System.out.println(hs); // in ra [null, prac, geeks, con]

```

LinkedHashSet là bảng băm kết hợp với linked list nhằm đảm bảo thứ tự các phần tử:

- Thừa kế HashSet và thực thi giao diện Set
- Khác HashSet ở chỗ nó lưu trữ trong một danh sách móc nối đôi
- Thứ tự các phần tử được sắp xếp theo thứ tự được insert vào tập hợp

Lớp TreeSet là một lớp thực thi giao diện SortedSet. Các đối tượng đưa vào trong TreeSet phải cài đặt giao diện Comparable hoặc TreeSet phải nhận một Comparator trên kiểu của đối tượng đó. Phần tử có giá trị null không được phép trong TreeSet. TreeSet cho phép lấy các phần tử trong tập hợp theo thứ tự đã sắp xếp (theo tiêu chí xác định trong Comparable/Comparator):

- Các phần tử được thêm vào TreeSet tự động được sắp xếp
- Thông thường, ta có thể thêm các phần tử vào HashSet, sau đó convert về

TreeSet để duyệt theo thứ tự nhanh hơn

HashMap là bảng băm cài đặt của Map. Giống HashSet, HashMap được sử dụng để lưu các phần tử vào trong một bảng băm. Mỗi phần tử trong HashMap gồm hai đối tượng ứng với cặp khóa-giá trị (key-value) và ta cần đưa vào cặp khóa-giá trị khi gọi phương thức chèn vào là put(Object k, Object v). HashMap không cho phép trùng lặp khóa nhưng cho phép chèn khóa null. Về bản chất thì một đối tượng HashSet sử dụng một đối tượng HashMap bên trong nó để lưu các phần tử nên tốc độ của các thao tác của HashSet thực hiện chậm hơn HashMap. Độ phức tạp tính toán của các thao tác cơ bản của HashMap luôn đảm bảo bằng O(1). Ví dụ về HashMap:

```

HashMap<String, Integer> people = new HashMap<String, Integer>();
people.put("John", 32); // thêm tên-tuổi

```

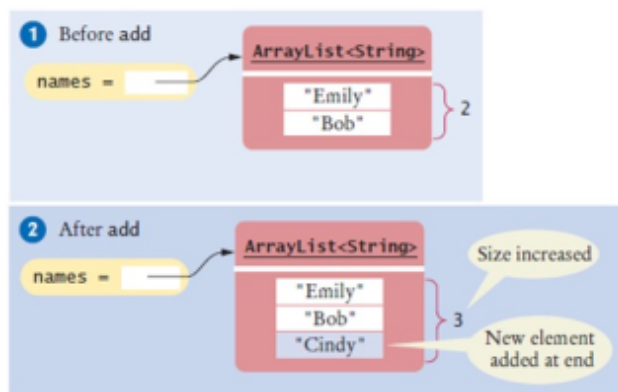
LinkedHashMap là bảng băm kết hợp với linked list nhằm đảm bảo thứ tự các phần tử (cài đặt của Map) còn TreeMap là cấu trúc Cây tương tự TreeSet và cài đặt Map.

Các lớp cũ được xây dựng trong các phiên bản Java đời đầu như Vector hoặc Hashtable được cài đặt bổ sung thêm các collection interface. Ngoài ra ta có thể dùng ArrayList thay cho Vector hoặc dùng HashMap thay cho Hashtable.

Ví dụ dùng ArrayList:

```
ArrayList<String> names = new ArrayList<String>();  
names.add("Emily");  
names.add("Bob");  
names.add("Cindy");
```

Kết quả:

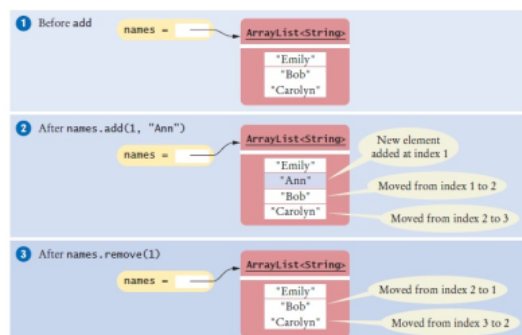


Hình 9.10 Kết quả của ví dụ

Ví dụ tiếp:

```
String name = names.get(0);  
names.add(1, "Ann");  
names.remove(1);
```

Kết quả:



Hình 9.11 Kết quả ví dụ tiếp

Câu hỏi: sau khi thực hiện đoạn chương trình sau, danh sách `names` có chứa các phần tử nào?


```

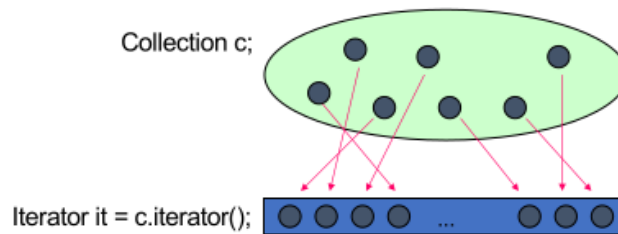
ArrayList<String> names = new ArrayList<String>;
names.add("Bob");
names.add(0, "Ann");
names.remove(1);
names.add("Cal");

```

Kết quả: danh sách names có chứa 2 phần tử "Ann", "Cal"

Giao diện Iterator và Comparator/Comparable

Giao diện Iterator và Comparator sử dụng để duyệt và so sánh trên các Collection. Các phần tử trong collection có thể được duyệt thông qua Iterator.



Hình 9.12 Iterator

Iterator cung cấp cơ chế thuận tiện để duyệt (lặp) qua toàn bộ nội dung của tập hợp, mỗi lần là một đối tượng trong tập hợp. Iterator của các tập hợp đã sắp xếp duyệt theo thứ tự đã sắp xếp của tập hợp.

Iterator có các phương thức:

- iterator(): yêu cầu container trả về một iterator
- next(): trả về phần tử tiếp theo
- hasNext(): kiểm tra có tồn tại phần tử tiếp theo hay không
- remove(): xóa phần tử gần nhất của iterator

Định nghĩa iterator:

```

public interface Iterator {
    boolean hasNext();
    Object next();
    void remove();
}

```

Sử dụng iterator:

```
Collection c;  
Iterator i = c.iterator();  
while (i.hasNext()) {  
    Object o = i.next();  
    // Process this object  
}
```

Cách sử dụng iterator nêu ở trên tương tự vòng lặp for:

```
for (String name : names) {  
    System.out.println(name);  
}
```

Giao diện Comparator được sử dụng để cho phép so sánh hai đối tượng trong tập hợp:

- Giao diện Comparator khai báo một phương thức compare(Object obj1, Object obj2) dùng để so sánh hai đối tượng. Phương thức này lấy 2 tham số đầu vào và trả về -1, 0 hoặc 1.
- Không cần thiết nếu tập hợp đã có khả năng so sánh tự nhiên (vd. String, Integer...)

Ví dụ lớp Person:

```
class Person {  
    private int age;  
    private String name;  
    public void setAge(int age){  
        this.age=age;  
    }  
    public int getAge(){  
        return this.age;  
    }  
    public void setName(String name){  
        this.name=name;  
    }  
    public String getName(){  
        return this.name;  
    }  
}
```

Ví dụ Cài đặt AgeComparator:

```
class AgeComparator implements Comparator {
    public int compare(Object ob1, Object ob2) {
        int ob1Age = ((Person)ob1).getAge();
        int ob2Age = ((Person)ob2).getAge();
        if(ob1Age > ob2Age)
            return 1;
        else if(ob1Age < ob2Age)
            return -1;
        else
            return 0;
    }
}
```

Ví dụ Sử dụng AgeComparator:

```
public class ComparatorExample {
    public static void main(String args[ ]) {
        ArrayList<Person> lst = new ArrayList<Person>();
        Person p = new Person();
        p.setAge(35);
        p.setName("A");
        lst.add(p);
        p = new Person();
        p.setAge(30);
        p.setName("B");
        lst.add(p);
        p = new Person();
        p.setAge(32);
        p.setName("C");
        lst.add(p);
        System.out.println("Order before sorting");
        for (Person person : lst) {
            System.out.println(person.getName() + "\t"
                + person.getAge());
        }
        Collections.sort(lst, new AgeComparator());
        System.out.println("\n\nOrder of person")
    }
}
```

```

        + "after sorting by age");
    for (Iterator<Person> i = lst.iterator(); i.hasNext();) {
        Person person = i.next();
        System.out.println(person.getName() + "\t"
            + person.getAge());
    } // end for
} // end main
} // end class

```

Giao diện Comparable hơi khác một chút so với Comparator. Comparator so sánh hai đối tượng với nhau (qua phương thức `compare(Object obj1, Object obj2)`) trong khi Comparable cho phép so sánh chính bản thân đối tượng với đối tượng khác qua phương thức `compareTo(Object otherObject)`.

9.2.5. Ký tự đại diện (Wildcard)

Quan hệ thừa kế giữa hai lớp không có ảnh hưởng gì đến quan hệ giữa các cấu trúc tổng quát dùng cho hai lớp đó. Ví dụ:

- Dog và Cat là các lớp con của Animal
- Có thể đưa các đối tượng Dog và Cat vào một `ArrayList<Animal>` (sử dụng phương thức `add`)
- Tuy nhiên, `ArrayList<Dog>`, `ArrayList<Cat>` lại không có quan hệ gì với `ArrayList<Animal>`

Do đó, ta không thể ép kiểu `ArrayList<ChildClass>` về kiểu `ArrayList<ParentClass>`. Ví dụ nếu lớp Child là lớp con của lớp Parent thì câu lệnh sau sẽ bị lỗi: `ArrayList<Parent> myList = new ArrayList<Child>();`

Ví dụ:

```

public class Test {
    public static void main(String args[]) {
        List<String> lst0 = new LinkedList<String>();
        List<Object> lst1 = lst0; // Error
        printList(lst0); // Error
    }
    void static printList(List<Object> lst) {
        Iterator it = lst.iterator();
        while (it.hasNext())
            System.out.println(it.next());
    }
}

```

```
    }
}
```

Để giải quyết vấn đề nêu trên, ta sử dụng kí tự đại diện (wildcard). Kí tự đại diện ? dùng để hiển thị cho một kiểu dữ liệu bất kỳ. Khi biên dịch, dấu ? có thể được thay thế bởi bất kì kiểu dữ liệu nào.

Ví dụ sử dụng Wildcards:

```
public class Test {
    void printList(List<?> lst) {
        Iterator it = lst.iterator();
        while (it.hasNext())
            System.out.println(it.next());
    }
    public static void main(String args[]) {
        List<String> lst0 = new LinkedList<String>();
        List<Employee> lst1 = new LinkedList<Employee>();
        printList(lst0); // String
        printList(lst1); // Employee
    }
}
```

Lưu ý cách làm sau là không hợp lệ:

```
ArrayList<?> list = new ArrayList<String>();
list.add("a1"); //compile error
list.add(new Object()); //compile error
```

Nguyên nhân là vì không biết list là danh sách liên kết cho kiểu dữ liệu nào, nên không thể thêm phần tử vào list, kể cả đối tượng của lớp Object.

Cách sử dụng kí tự đại diện:

- "? extends Type": Xác định một tập các kiểu con của Type. Đây là wildcard hữu ích

- "? super Type": Xác định một tập các kiểu cha của Type

- "?": Xác định tập tất cả các kiểu hoặc bất kỳ kiểu nào

Ví dụ:

"? extends Animal" có nghĩa là kiểu gì đó thuộc loại Animal (là Animal hoặc con của Animal)

Lưu ý: Hai cú pháp sau là tương đương:

```
public void foo( ArrayList<? extends Animal> a)
public <T extends Animal> void foo( ArrayList<T> a)
```

Câu hỏi: cho biết sự khác nhau giữa hai khối lệnh dưới đây?

```
public void print1(List<Employee> list) {
    for (Employee e : list) {
        System.out.println(e);
    }
}
```

và

```
public void print2(List<? extends Employee> list) {
    for (Employee e : list) {
        System.out.println(e);
    }
}
```

Trả lời: hàm print1 làm việc với lớp Employee còn hàm print2 làm việc với lớp Employee và lớp con của lớp Employee!

9.2.6. Bài tập và Lời giải

Bài tập 9.1. Trừu tượng hoá mô tả sau: một quyển sách là tập hợp các chương, chương là tập hợp các trang.

- Phác hoạ các lớp Book, Chapter, và Page
- Tạo các thuộc tính cần thiết cho các lớp, sử dụng Collection
- Tạo các phương thức cho lớp Chapter cho việc thêm trang và xác định một chương có bao nhiêu trang
- Tạo các phương thức cho lớp Book cho việc thêm chương và xác định quyển sách có bao nhiêu chương, và số trang cho quyển sách

Đáp án:

// file Page.java

```
public class Page implements Comparable<Page> {
    private int num;
    public Page(int num) {
        this.num = num;
    }
}
```

```
    }
    public void setNum(int n) {
        num = n;
    }
    public int getNum() {
        return num;
    }
    public int compareTo(Page o) {
        int n2 = o.getNum();
        if(num > n2) return 1;
        else if (num < n2) return -1;
        return 0;
    }
}

// file Chapter.java

import java.util.Collection;
import java.util.Iterator;
import java.util.TreeSet;

public class Chapter implements Comparable<Chapter> {
    Collection<Page> ss;
    private int num;

    public Chapter(int num) {
        ss = new TreeSet<Page>();
        this.num = num;
    }

    public int getNum() {
        return num;
    }

    public int addPage(Page p) {
        if(p.getNum() <= 0) return 0;
        ss.add(p);
        return 1;
    }
}
```

```
    }

    public int getChapterPageNumber() {
        return ss.size();
    }

    public int getFirstPageNumber() {
        try {
            Page pp = ((TreeSet<Page>)ss).first();
            return pp.getNum();
        } catch(Exception ex) {}

        return -1;
    }

    public int getLastPageNumber() {
        try {
            Page pp = ((TreeSet<Page>)ss).last();
            return pp.getNum();
        } catch(Exception ex) {}

        return -1;
    }

    public Iterator<Page> getIterator() {
        return ss.iterator();
    }

    public int compareTo(Chapter o) {
        int n2 = o.getNum();
        if(num > n2) return 1;
        else if (num < n2) return -1;
        return 0;
    }
}
```

// file Book.java


```
import java.util.Collection;
import java.util.Iterator;
import java.util.TreeSet;

public class Book {
    Collection<Chapter> ss;

    public Book() {
        ss = new TreeSet<Chapter>();
    }

    public int addChapter(Chapter c) {
        if(c.getNum() <= 0) return 0;
        ss.add(c);
        return 1;
    }

    public int getTotalChapterNumber() {
        return ss.size();
    }

    public int getTotalPageNumber() {
        Iterator<Chapter> i = getIterator();
        int page = 0;
        while(i.hasNext()) {
            Chapter c = i.next();
            page += c.getChapterPageNumber();
        }
        return page;
    }

    public Iterator<Chapter> getIterator() {
        return ss.iterator();
    }
}
```

```
// main function
```

```

public static void main(String[] args) {
    Book book = new Book();
    Chapter c1 = new Chapter(1);
    for(int i=0;i<23;i++)
        c1.addPage(new Page(i+1));
    Chapter c2 = new Chapter(2);
    for(int i=23;i<43;i++)
        c2.addPage(new Page(i+1));
    book.addChapter(c2);
    book.addChapter(c1);
    Iterator<Chapter> i = book.getIterator();
    while(i.hasNext()) {
        Chapter c = i.next();
        Iterator<Page> iinc = c.getIterator();
        while(iinc.hasNext()) {
            System.out.println(iinc.next().getNum());
        }
    }
}

```

Bài tập 9.2. Xây dựng lớp Stack tổng quát với các kiểu dữ liệu.

StackOfChars	StackOfIntegers	
- elements: char[]	- elements: int[]	
- size: int	- size: int	
+ StackOfChars()	+ StackOfIntegers()	
+ StackOfChars (capacity: int)	+ StackOfIntegers (capacity: int)	
+ isEmpty(): boolean	+ isEmpty(): boolean	
+ isFull(): boolean	+ isFull(): boolean	
+ peak(): char	+ peak(): int	
+ push(value:char): void	+ push(value:int): void	
+ pop(): char	+ pop(): int	
+ getSize(): int	+ getSize(): int	...

Đáp án:

// file Stack.java

```

import java.util.LinkedList;
public class Stack<T> {

```

```
LinkedList<T> list;
private int capacity;
public Stack(int size) {
    list = new LinkedList<T>();
    capacity = size;
}
public int getCapacity() {
    return capacity;
}
public boolean isEmpty() {
    return list.isEmpty();
}
public boolean isFull() {
    return list.size()==capacity;
}
public int getSize() {
    return list.size();
}
public void push(T element) {
    if(!isFull())
        list.addFirst(element);
}
public T pop() {
    if(!isEmpty())
        return list.removeFirst();
    return null;
}
public T peak() {
    if(!isEmpty())
        return list.getFirst();
    return null;
}
}

//in main function

public static void main(String[] args) {
    Stack<Character> st = new Stack<Character>(10);
```

```
    st.push('a');
    st.push('b');
    System.out.println("peak:"+st.peak());
    while (!st.isEmpty()) {
        System.out.println(st.pop());
    }
    Stack<Integer> st2 = new Stack<Integer>(10);
    st2.push(12);
    st2.push(234);
    System.out.println("peak:"+st2.peak());
    while (!st2.isEmpty()) {
        System.out.println(st2.pop());
    }
}
```

Chương 10

Ngoại lệ và xử lý ngoại lệ

Mục tiêu: Chương này sẽ giải thích ngoại lệ là gì và mô tả các lợi ích của việc xử lý ngoại lệ hướng đối tượng cũng như giải thích được mô hình xử lý ngoại lệ. Chương sẽ trình bày cách sử dụng khối try/catch/finally để bắt và xử lý ngoại lệ trong Java, từ đó giúp sinh viên hiểu và biết cách sử dụng ủy nhiệm ngoại lệ cũng như biết cách tạo ra và sử dụng ngoại lệ tự định nghĩa.

10.1. Ngoại lệ

Ngoại lệ là gì?

Ngoại lệ (Exception) là một sự kiện xảy ra trong quá trình thực thi chương trình, nó phá vỡ luồng bình thường của chương trình. Nói cách khác chương trình xuất hiện ngoại lệ được hiểu là có lỗi xảy ra khi chạy chương trình.

Ví dụ thực hiện phép chia cho 0 khi chạy chương trình thì sẽ xuất hiện ngoại lệ.

Khi xảy ra một ngoại lệ, nếu không xử lý thì chương trình kết thúc ngay và trả lại quyền điều khiển cho hệ điều hành.

Cách xử lý lỗi truyền thống

Viết mã xử lý tại nơi phát sinh ra lỗi làm cho chương trình trở nên rối do không phải lúc nào cũng đầy đủ thông tin để xử lý. Vì vậy ta không nhất thiết phải xử lý ngay tại nơi phát sinh lỗi.

Ta có thể truyền trạng thái lên mức trên thông qua tham số, giá trị trả lại hoặc biến tổng thể (flag).

Tuy nhiên thì việc truyền trạng thái lên mức trên có thể gây khó hiểu và nhầm lẫn.

Ví dụ chương trình sau sẽ xuất hiện ngoại lệ chia cho 0 nếu tham số denom có giá trị bằng 0:

```
int devide(int num, int denom, int *error){  
    if (denom != 0) {  
        error = 0;  
        return num/denom;  
    } else {  
        error = 1;  
        return 0;  
    }  
}
```

Với cách xử lý lỗi thông thường sẽ khó kiểm soát được hết các trường hợp như lỗi số học, lỗi bộ nhớ,...

Lập trình viên thường quên không xử lý lỗi do bản chất con người nhiều khi lơ đãng hoặc thiếu kinh nghiệm, cố tình bỏ qua.

10.2. Bắt và xử lý ngoại lệ

10.2.1. Mục đích của xử lý ngoại lệ

Xử lý ngoại lệ giúp chương trình đáng tin cậy hơn, tránh kết thúc bất thường. Để xử lý ngoại lệ ta có thể tách biệt khối lệnh có thể gây ngoại lệ và khối lệnh xử lý ngoại lệ.

Khi xảy ra ngoại lệ, nếu không có cơ chế xử lý thích hợp thì chương trình bị ngắt khi ngoại lệ xảy ra, dẫn tới các tài nguyên không được giải phóng và lãng phí. Ví dụ khi thực hiện vào/ra tệp tin: nếu ngoại lệ xảy ra (ví dụ như chuyển đổi kiểu không đúng) làm chương trình kết thúc mà không đóng tệp tin lại => tệp tin không thể truy cập/hồng và tài nguyên cấp phát không được giải phóng.

10.2.2. Mô hình xử lý ngoại lệ

Mô hình xử lý ngoại lệ trong lập trình Hướng Đối Tượng sẽ:

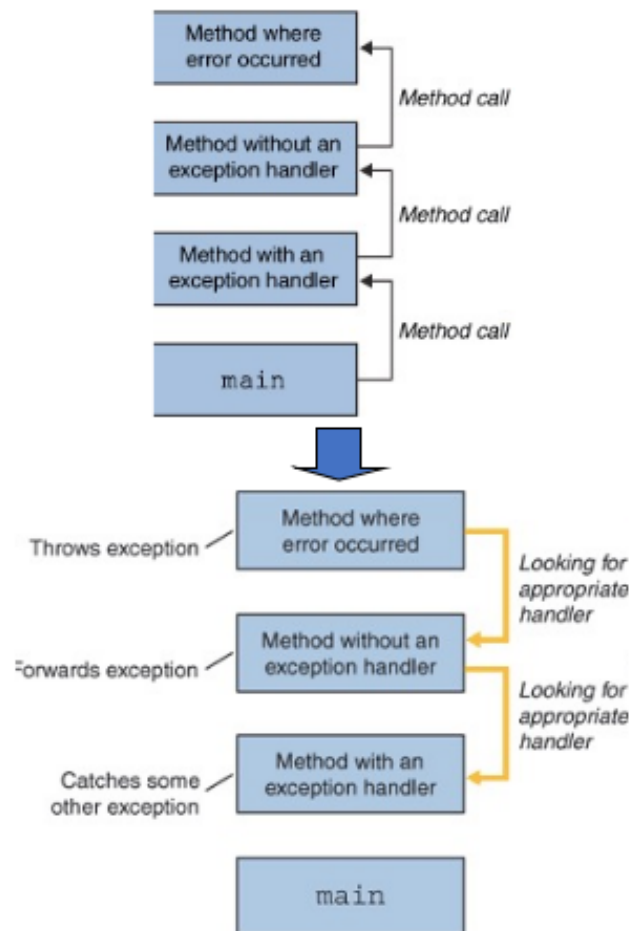
- Đóng gói các điều kiện không mong đợi trong một đối tượng. Khi xảy ra ngoại lệ, đối tượng tương ứng với ngoại lệ được tạo ra chứa thông tin chi tiết về ngoại lệ.
- Cung cấp cơ chế hiệu quả trong việc xử lý lỗi.
- Tách biệt luồng điều khiển bất thường với luồng bình thường.

Ngoại lệ cần phải được xử lý ở tại phương thức sinh ra ngoại lệ hoặc ủy nhiệm cho phương thức gọi đến.

10.2.3. Xử lý ngoại lệ trong Java

Java có cơ chế xử lý ngoại lệ rất mạnh. Xử lý ngoại lệ trong Java được thực hiện theo mô hình hướng đối tượng:

- Tất cả các ngoại lệ đều là thể hiện của một lớp kế thừa từ lớp Throwable hoặc các lớp con của nó.
- Các đối tượng này có nhiệm vụ chuyển thông tin về ngoại lệ (loại và trạng thái của chương trình) từ vị trí xảy ra ngoại lệ đến nơi quản lý/xử lý nó.



Hình 10.1 Xử lý ngoại lệ trong Java

Khối try/catch

Khối try ... catch: phân tách đoạn chương trình thông thường và phần xử lý

ngoại lệ. Lệnh try {...} sẽ bọc khối lệnh có khả năng gây ra ngoại lệ bên trong nó còn lệnh catch() {...} sẽ bắt và xử lý ngoại lệ. Cách dùng:

```
try {  
    // Doan ma co the gay ngoai le  
} catch (ExceptionType e) {  
    // Xu ly ngoai le  
}
```

với ExceptionType là một lớp con của Throwable.

Ví dụ chương trình xuất hiện ngoại lệ nhưng không được xử lý làm chương trình dừng đột ngột: chương trình dưới đây cố gắng truy cập vào phần tử đầu tiên của tham số args (là tham số ở đầu vào của chương trình) nhưng khi ta thực thi chương trình này mà không có tham số đầu vào sẽ dẫn tới ngoại lệ `ArrayIndexOutOfBoundsException`:

```
class NoException {  
    public static void main(String args[]){  
        String t = args[0]; //Xuất hiện ngoại lệ khi thực thi  
        // chương trình dừng đột ngột!  
        System.out.println(t);  
    }  
}
```

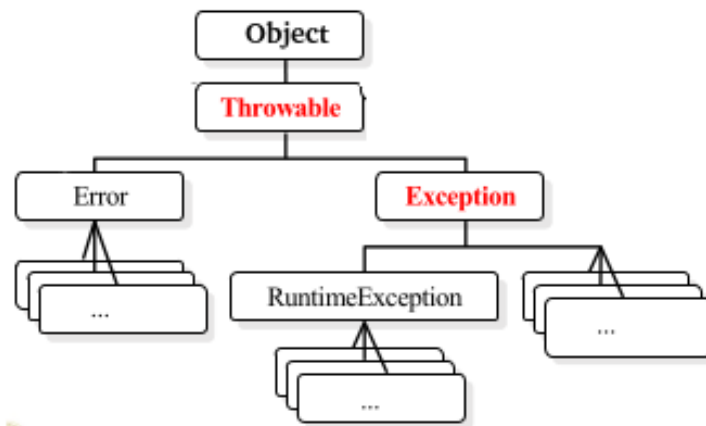
Ta xử lý ngoại lệ phát sinh ở chương trình trên bằng khối try/catch, khi đó mỗi lần chương trình chạy mà không có tham số đầu vào thì chương trình sẽ hiển thị nội dung ra màn hình là 'Hay nhap tham so khi chay!'.

```
class ArgExceptionDemo {  
    public static void main(String args[]) {  
        try {  
            String t = args[0];  
            System.out.println(t);  
        }  
        catch(Exception e) {  
            System.out.println("Hay nhap tham so khi chay!");  
        }  
    }  
}
```


Ví dụ ngoại lệ chia cho 0, khi có lỗi xảy ra ta in ra màn hình thông tin về lỗi:

```
public class ChiaCho0Demo {
    public static void main(String args[]) {
        try {
            int num = calculate(9,0);
            System.out.println(num);
        }
        catch(Exception e) {
            System.err.println("Co loi xay ra: " + e.toString());
        }
    }
    static int calculate(int no, int no1) {
        int num = no / no1;
        return num;
    }
}
```

Lớp Throwable



Hình 10.2 Cây phân cấp ngoại lệ trong Java

Lớp này chứa một biến kiểu String để lưu thông tin chi tiết về ngoại lệ đã xảy ra. Lớp này có một số phương thức cơ bản như:

- new Throwable(String s): Tạo một ngoại lệ với thông tin về ngoại lệ là s
- String getMessage(): Lấy thông tin về ngoại lệ

- void printStackTrace(): In ra tất cả các thông tin liên quan đến ngoại lệ (tên, loại, vị trí...)

Ví dụ in ra màn hình thông tin ngoại lệ khi ngoại lệ xảy ra:

```
public class StckExceptionDemo {
    public static void main(String args[]){
        try {
            int num = calculate(9,0);
            System.out.println(num);
        }
        catch(Exception e) {
            System.err.println("Co loi xay ra : " + e.getMessage());
            e.printStackTrace();
        }
    }
    static int calculate(int no, int no1)
    {
        int num = no / no1;
        return num;
    }
}
```

Lớp Error

Lớp Error gồm các ngoại lệ nghiêm trọng không thể kiểm tra (unchecked exception) vì có thể xảy ra ở nhiều phần của chương trình hay còn gọi là ngoại lệ không thể phục hồi (un-recoverable exception). Những loại ngoại lệ của lớp Error không phát sinh từ mã nguồn của chương trình nên bạn không cần kiểm tra trong mã nguồn Java của bạn. Các lớp con của lớp Error:

- VirtualMachineError: InternalError, OutOfMemoryError, StackOverflowError, UnknownError
- ThreadDeath
- LinkageError:
 - IncompatibleClassChangeError
 - AbstractMethodError, InstantiationException, NoSuchFieldError, NoSuchMethodError...

Lớp Exception

Lớp Exception chứa các loại ngoại lệ nên/phải bắt và xử lý hoặc ủy nhiệm. Người

dùng có thể tạo ra các ngoại lệ của riêng mình bằng cách kế thừa từ lớp `Exception`. Lớp `RuntimeException` là lớp con của `Exception`. Đối tượng `RuntimeException` có thể được “tung” ra trong quá trình JVM thực hiện (trong quá trình chạy chương trình). Ta không bắt buộc phải bắt ngoại lệ `RuntimeException` dù có thể xảy ra lỗi và không nên viết ngoại lệ của riêng mình kế thừa từ lớp `RuntimeException`.

Một số lớp con của `Exception`

- `ClassNotFoundException`, `SQLException`
- `java.io.IOException`: `FileNotFoundException`, `EOFException`...
- và lớp `RuntimeException`, lớp này chứa các lớp con khác như `NullPointerException`, `BufferOverflowException`, `ClassCastException`, `ArithmeticException`, `IndexOutOfBoundsException` (`ArrayIndexOutOfBoundsException`, `StringIndexOutOfBoundsException`...), `IllegalArgumentException` (`NumberFormatException`, `InvalidParameterException`...)

Ví dụ `IOException`:

```
import java.io.InputStreamReader;
import java.io.IOException;
public class HelloWorld {
    public static void main(String[] args) {
        InputStreamReader isr = new InputStreamReader(System.in);
        try {
            System.out.print("Nhập vào 1 ký tự: ");
            char c = (char) isr.read();
            System.out.println("Ký tự vừa nhập: " + c);
        } catch (IOException ioe) {
            ioe.printStackTrace();
        }
    }
}
```

Khối `try` – `catch` lồng nhau

Những phần nhỏ trong khối mã sinh ra một lỗi, nhưng toàn bộ cả khối thì lại sinh ra một lỗi khác nên ta cần có các xử lý ngoại lệ lồng nhau: sử dụng các khối `try/catch` lồng nhau. Khi các khối `try/catch` lồng nhau, khối `try` bên trong sẽ được thực hiện trước. Ví dụ:

```
try {
```

```
// Doan ma co the gay ra IOException
try {
    // Doan ma co the gay ra NumberFormatException
}
catch (NumberFormatException e1) {
    // Xu ly loi sai dinh dang so
}
} catch (IOException e2) {
    // Xu ly loi vao ra
}
```

Nhiều khối catch

Một đoạn mã có thể gây ra nhiều hơn một ngoại lệ nên ta có thể sử dụng nhiều khối catch:

```
try {
    // Doan ma co the gay ra nhieu ngoai le
} catch (ExceptionType1 e1) {
    // Xu ly ngoai le 1
} catch (ExceptionType2 e2) {
    // Xu ly ngoai le 2
} ...
```

ExceptionType1 phải là lớp con hoặc ngang hàng với ExceptionType2 (trong cây phân cấp kế thừa).

Ví dụ gây lỗi khi ExceptionType1 không phải lớp con hoặc không phải lớp ngang hàng ExceptionType2:

```
class MultipleCatch {
    public static void main(String args[ ]) {
        try {
            String num = args[0];
            int numValue = Integer.parseInt(num);
            System.out.println("Dien tich: " + numValue * numValue);
        } catch (Exception e1) {
            System.out.println("Hay nhap canh cua hv!");
        } catch (NumberFormatException e2) {
            System.out.println("Not a number!");
        }
    }
}
```

```

    }
}
}

```

Đoạn mã trên xuất hiện lỗi biên dịch do lớp `Exception` là lớp tổ tiên của lớp `NumberFormatException`, màn hình sau biên dịch sẽ hiện dòng chữ: `"java.lang.NumberFormatException has already been caught"`

Ví dụ nhiều khối catch hợp lệ:

```

class MultipleCatch1 {
    public static void main(String args[]) {
        try {
            String num = args[0];
            int numValue = Integer.parseInt(num);
            System.out.println("Dien tich:" + numValue * numValue);
        } catch(ArrayIndexOutOfBoundsException e1) {
            System.out.println("Hay nhap canh cua hv!");
        } catch(NumberFormatException e2) {
            System.out.println("Hay nhap 1 so!");
        }
    }
}

```

Ví dụ nhiều khối catch hợp lệ (ví dụ khác):

```

class MultiCatch2 {
    public static void main( String args[]) {
        try {
            // format a number
            // read a file
            // something else...
        } catch(IOException e) {
            System.out.println("I/O error "+e.getMessage());
        } catch(NumberFormatException e) {
            System.out.println("Bad data "+e.getMessage());
        } catch(Throwable e) {
            // catch all
            System.out.println("error: " + e.getMessage());
        }
    }
}

```

```

    }
}

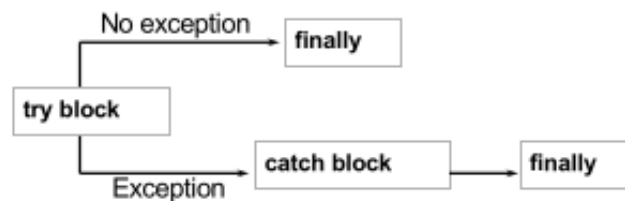
```

Khối finally

Ta sử dụng khối finally để đảm bảo thực hiện tất cả các công việc cần thiết khi có ngoại lệ xảy ra như:

- Đóng file, đóng socket, connection
- Giải phóng tài nguyên (nếu cần)...

Khối lệnh finally luôn luôn được thực hiện dù ngoại lệ có xảy ra hay không.



Hình 10.3 Khối finally

Cú pháp try ... catch ... finally sẽ là:

```

try {
    // Khởi lệnh có thể sinh ngoại lệ
} catch(ExceptionType e) {
    // Bắt và xử lý ngoại lệ
} finally {
    /* Thực hiện các công việc cần thiết
    dù ngoại lệ có xảy ra hay không */
}

```

Nếu đã có khối try thì bắt buộc phải có khối catch hoặc khối finally hoặc cả hai.

Ví dụ:

```

class StrExceptionDemo {
    static String str;
    public static void main(String s[]) {
        try {
            System.out.println("Trước ngoại lệ");
            staticLengthMethod();
            System.out.println("Sau ngoại lệ");
        }
    }
}

```

```

    }
    catch(NullPointerException ne) {
        System.out.println("Da xay ra loi");
    }
    finally {
        System.out.println("Trong finally");
    }
}
static void staticLengthMethod() {
    System.out.println(str.length());
}
}

```

Khi chạy chương trình trên thì màn hình sẽ in ra 3 dòng:

```

Truoc ngoai le
Da xay ra loi
Trong finally

```

Ta có một ví dụ khác trong thực tế: khi mở file để đọc ta cần đóng file kể cả khi có ngoại lệ xảy ra hay không. Đoạn mã như sau:

```

public void openFile()
{

    try {
        // constructor may throw FileNotFoundException
        FileReader reader = new FileReader("someFile");
        int i=0;
        while(i != -1)
        {
            //reader.read() may throw IOException
            i = reader.read();
            System.out.println((char) i );
        }
    } catch (FileNotFoundException e) {
        //do something clever with the exception
    } catch (IOException e) {
        //do something clever with the exception
    } finally {

```

```

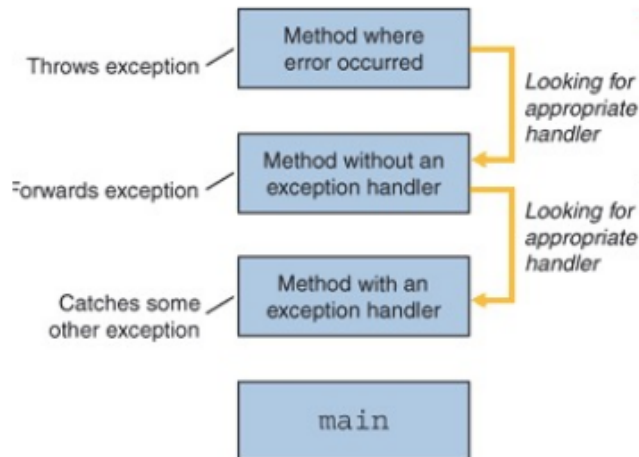
        reader.close(); // Close reader
        System.out.println("--- File End ---");
    }

}

```

10.3. Ủy nhiệm ngoại lệ

Ta có hai cách làm việc với ngoại lệ: Sử dụng khối try ... catch (finally nếu cần) để xử lý ngoại lệ ngay tại nơi nó phát sinh hoặc ủy nhiệm cho vị trí khác nếu ta không muốn xử lý ngay (sử dụng các từ khóa throw và throws).



Hình 10.4 Ủy nhiệm ngoại lệ trong Java

Phương thức có thể ủy nhiệm ngoại lệ cho vị trí gọi nó bằng cách sử dụng throws ExceptionType ở phần khai báo phương thức để báo hiệu cho vị trí gọi nó biết là nó có thể phát sinh ngoại lệ ExceptionType và sử dụng throw để tung ra ngoại lệ kiểu ExceptionType trong thân phương thức khi cần.

Ví dụ:

```

public void myMethod(int param) throws Exception {
    if (param < 10) {
        throw new Exception("Too low!");
    }
    //Blah, Blah, Blah...
}

```


Nếu phương thức có chứa câu lệnh tung ngoại lệ (throw) thì phải khai báo phương thức phải khai báo là có tung ngoại lệ đó hoặc lớp cha của ngoại lệ đó, ví dụ sau gây lỗi biên dịch:

```
public void myMethod(int param) {
    if (param < 10) {
        throw new Exception("Too low!");
    }
    //Blah, Blah, Blah...
}
```

Biên dịch đoạn lệnh trên sẽ gặp lỗi "unreported exception java.lang.Exception must be caught or declared to be thrown".

Phương thức tung ra RuntimeException thì không cần phải khai báo là nó sẽ tung ra RuntimeException vì ngoại lệ kiểu này mặc định được ủy nhiệm cho JVM. Ví dụ sau hợp lệ:

```
class Test {
    public void myMethod(int param) {
        if (param < 10) {
            throw new RuntimeException("Too low!");
        }
        //Blah, Blah, Blah...
    }
}
```

Tại vị trí gọi phương thức có ủy nhiệm ngoại lệ (trừ RuntimeException) ta phải hoặc bắt ngoại lệ ủy nhiệm và xử lý ngay bằng try...catch (finally nếu cần) hoặc ủy nhiệm tiếp cho vị trí gọi mình. Ví dụ không hợp lệ:

```
public class DelegateExceptionDemo {
    public static void main(String args[]){
        int num = calculate(9,3);
        System.out.println("Lan 1: " + num);
        num = calculate(9,0); //Ngoại lệ không được xử lý!
        System.out.println("Lan 2: " + num);
    }
    static int calculate(int no, int no1) throws ArithmeticException {
        if (no1==0) throw new ArithmeticException("Loi chia 0");
    }
}
```

```

        int num = no / no1;
        return num;
    }
}

```

Ví dụ khác cũng không hợp lệ:

```

public class DelegateExceptionDemo {
    public static void main(String args[]){
        int num = calculate(9,3);
        System.out.println("Lan 1: " + num);
        num = calculate(9,0); //Ngoại lệ không được xử lý!
        System.out.println("Lan 2: " + num);
    }
    static int calculate(int no, int no1) throws Exception {
        if (no1==0) throw new ArithmeticException("Loi chia 0");
        int num = no / no1;
        return num;
    }
}

```

Ta sửa lại hai ví dụ trên để biên dịch thành công như sau:

```

public class DelegateExceptionDemo {
    public static void main(String args[]){
        try {
            int num = calculate(9,3);
            System.out.println("Lan 1: " + num);
            num = calculate(9,0);
            System.out.println("Lan 2: " + num);
        } catch(Exception e) { // OK, xử lý ngoại lệ
            System.out.println(e.getMessage());
        }
    }
    static int calculate(int no, int no1) throws ArithmeticException {
        if (no1 == 0) throw new ArithmeticException("Loi chia 0!");
        int num = no / no1;
        return num;
    }
}

```

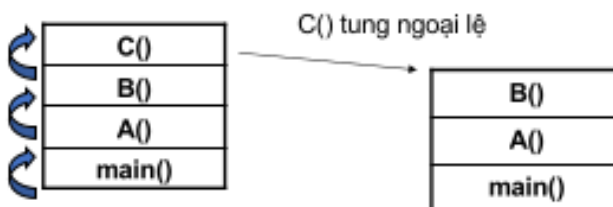
```
    }
}
```

Một phương thức có thể ủy nhiệm nhiều hơn 1 ngoại lệ, ví dụ:

```
public void myMethod(int tuoi, String ten)
throws ArithmeticException, NullPointerException {
    if (tuoi < 18) {
        throw new ArithmeticException("Chua du tuoi!");
    }
    if (ten == null) {
        throw new NullPointerException("Thieu ten!");
    }
    //Blah, Blah, Blah...
}
```

Lan truyền ngoại lệ

Ta có tình huống như sau. Giả sử trong `main()` gọi phương thức `A()`, trong `A()` gọi `B()`, trong `B()` gọi `C()`. Khi đó một ngăn xếp các phương thức được tạo ra. Giả sử trong `C()` xảy ra ngoại lệ.



Hình 10.5 Lan truyền ngoại lệ

Nếu `C()` gặp lỗi và tung ra ngoại lệ nhưng trong `C()` lại không xử lý ngoại lệ này, thì chỉ còn một nơi có thể xử lý chính là nơi mà `C()` được gọi, đó là trong phương thức `B()`. Nếu trong `B()` cũng không xử lý thì phải xử lý ngoại lệ này trong `A()`... Quá trình này gọi là lan truyền ngoại lệ. Nếu đến `main()` cũng không xử lý ngoại lệ được tung từ `C()` thì chương trình sẽ phải dừng lại.

Kế thừa và ủy nhiệm ngoại lệ

Khi override một phương thức của lớp cha, phương thức ở lớp con không được phép tung ra các ngoại lệ mới. Ngoài ra phương thức ghi đè trong lớp con chỉ được phép tung ra các ngoại lệ giống hoặc là lớp con hoặc là tập con của các ngoại lệ được tung ra ở lớp cha.

Ví dụ không hợp lệ:

```
class Disk {  
    void readFile() throws EOFException {}  
}  
class FloppyDisk extends Disk {  
    void readFile() throws IOException {} // ERROR!  
}
```

Ví dụ hợp lệ:

```
class Disk {  
    void readFile() throws IOException {}  
}  
class FloppyDisk extends Disk {  
    void readFile() throws EOFException {} //OK  
}
```

Ưu điểm của ủy nhiệm ngoại lệ

Ưu điểm của ủy nhiệm ngoại lệ là dễ thực hiện, làm chương trình dễ đọc và an toàn hơn. Ta dễ dàng chuyển điều khiển đến nơi có khả năng xử lý ngoại lệ và ta có thể ném nhiều loại ngoại lệ.

Ngoài ra ủy nhiệm ngoại lệ còn một số ưu điểm khác:

- Tách xử lý ngoại lệ khỏi đoạn mã thông thường
- Không bỏ sót ngoại lệ (ném tự động)
- Gom nhóm và phân loại các ngoại lệ
- Làm chương trình dễ đọc và an toàn hơn

10.4. Tạo ngoại lệ tự định nghĩa

Người dùng có thể tự định nghĩa lớp ngoại lệ riêng do thư viện sẵn có của Java không đủ để kiểm soát tất cả các lỗi. Để định nghĩa ngoại lệ mới, ta cần:

- Kế thừa từ một lớp Exception hoặc lớp con của nó
- Lớp ngoại lệ vừa được xây dựng có tất cả các phương thức của lớp Throwable

Ví dụ:

```
public class MyException extends Exception {  
    public MyException(String msg) {  
        super(msg);  
    }  
}
```

```

    }
    public MyException(String msg, Throwable cause){
        super(msg, cause);
    }
}

```

Sử dụng ngoại lệ người dùng định nghĩa

Ta sử dụng lớp MyException vừa được định nghĩa ở trên:

```

public class FileExample {
    // tung ngoại lệ ta tự định nghĩa phía trên
    public void copyFile(String fName1, String fName2) throws MyException {
        if (fName1.equals(fName2))
            throw new MyException("Same FileName");//sinh ngoại lệ
        System.out.println("Copy completed");
    }
}

```

Ta bắt và xử lý ngoại lệ MyException như ngoại lệ thông thường:

```

public class Test {
    public static void main(String[] args) {
        FileExample obj = new FileExample();
        try {
            String a = args[0];
            String b = args[1];
            obj.copyFile(a,b);
        } catch (MyException e1) {
            System.out.println(e1.getMessage());
        }
        catch(Exception e2) {
            System.out.println(e2.toString());
        }
    }
}

```

10.5. Tổng kết

- Bất cứ khi nào có một lỗi xảy ra khi thực hiện chương trình thì một ngoại lệ đã xuất hiện.
- Mọi ngoại lệ đều phải được xử lý nếu không muốn chương trình kết thúc một cách bất thường.
- Xử lý ngoại lệ cho phép kết hợp và xử lý lỗi tại một nơi.
- Java sử dụng khối try/catch để quản lý ngoại lệ.
- Các câu lệnh trong khối try tung ra ngoại lệ, và việc xử lý các ngoại lệ đó diễn ra trong khối catch.
- Nhiều khối catch có thể được sử dụng để xử lý tách biệt các loại ngoại lệ khác nhau.
- Từ khóa throws được sử dụng để liệt kê mỗi danh sách các ngoại lệ mà một phương thức có thể tung ra.
- Từ khóa throw được sử dụng để tung ra một ngoại lệ.
- Khối finally để thực hiện các công việc cần thiết dù có ngoại lệ xảy ra hay không.

10.6. Bài tập và Lời giải

Bài tập 10.1. Hệ thống liên tục nhận các giá trị đầu vào là xâu đại diện cho một số nguyên, yêu cầu mỗi lần nhận được một số thì tính trung bình cộng của các giá trị đã nhận. Xây dựng phương thức public double getAverage(string x) với thuộc tính average, N có sẵn trong lớp. Hãy cài đặt phương thức trong đó có xử lý các ngoại lệ xảy ra:

- Xâu đầu vào là xâu rỗng
- Xâu đầu vào không phải là số
- Xâu đầu vào không phải là số nguyên

Đáp án:

```
import java.util.Scanner;
public class Main {
    static double average = 0.0;
    static int N=1;
    public static void main(String[] args) {
        Scanner console = new Scanner(System.in);
        while(true) {
```

```

        getAverage(console.nextLine());
    }
}

public static double getAverage(String s) {
    int i;
    try {
        i = Integer.parseInt(s);
        System.out.println("Input is " + i);
        double m = average*(N-1) + i;
        average = m/N;
        System.out.println("average is " + average);
        N++;
    } catch (java.lang.NumberFormatException ex1) {
        System.out.println("Can nhap vao so nguyen!");
    }
    return average;
}
}

```

Bài tập 10.2. Hệ thống cần đọc file để lấy ra một dãy các con số (mỗi dòng một số nguyên) trong đó dòng 1 của file là số lượng các số có trong file và mỗi dòng tiếp theo là một số nguyên. Xây dựng phương thức đọc dãy số public void readListIntegers(String fileName). Hãy xử lý các ngoại lệ:

- Xâu tên file là xâu rỗng
- Không tìm thấy file
- Không mở được file
- Các xâu trong từng dòng của file không phải là đại diện cho con số

Đáp án:

```

import java.io.*;

public class Main {
    public static void readListIntegers(String file) {
        try {
            checkEmptyString(file);
            BufferedReader input =
                new BufferedReader(new FileReader(file));
            try {

```

```
        int n = Integer.parseInt(input.readLine());
        String line = null;
        while (n > 0) {
            n = n - 1;
            line = input.readLine();
            int i = Integer.parseInt(line);
            System.out.println("OK int:" + i);
        }
    }
    finally {
        input.close();
    }
}
catch (EmptyStringException ex){
    System.out.println("FileName Empty Exception");
    ex.printStackTrace();
}
catch (NumberFormatException ex) {
    System.out.println("Need integer in file!");
    ex.printStackTrace();
}
catch (FileNotFoundException ex){
    System.out.println("File Not Found Exception");
    ex.printStackTrace();
}
catch (IOException ex){
    System.out.println("IO Exception");
    ex.printStackTrace();
}
}

public static void main(String[] args) {
    readListIntegers("filenamets");
}

public static void checkEmptyString(String s)
throws EmptyStringException {
    if(s.trim().equals("")) throw new EmptyStringException();
}
```



```
}
```

```
class EmptyStringException extends Exception {}
```

Bài tập 10.3. Hệ thống cần đọc file đầu vào để lấy ra một dãy các con số (mỗi dòng một số nguyên), sau đó tách ra thành 4 phần bằng nhau và ghi ra các file khác nhau. Trong file đầu vào, dòng 1 của file đầu vào là số lượng các số có trong file và mỗi dòng sau đó là một số nguyên. Xây dựng phương thức đọc dãy số public void splitListIntegers(String fileName). Hãy xử lý các ngoại lệ:

- Xâu tên file là xâu rỗng
- Không tìm thấy file
- Không mở được file
- Các xâu trong từng dòng của file không phải là đại diện cho con số
- Không ghi được file mới

Đáp án:

```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
public class Main {
    public static void splitListIntegers(String file) {
        OutputData out = readListIntegers(file);
        if(out.data==null) return;
        for(int i=0;i<out.data.length;i++) {
            try {
                File myObj = new File("filename"+i+".txt");
                System.out.println();
                if (myObj.createNewFile()) {
                    System.out.println("File created: "
                        + myObj.getName());
                } else {
                    System.out.println("File already exists.");
                }
            } catch (IOException e) {
```

```

        System.out.println("An error occurred.");
        e.printStackTrace();
    }
    try {
        FileWriter myWriter =
            new FileWriter("filename"+i+".txt");
        try {
            myWriter.write("" + out.size[i]+"\\n");
            if(out.size[i]!=0) {
                for(int j=0;j<out.data[i].length;j++)
                    myWriter.write(""+out.data[i][j]+"\\n");
            }
        }
        finally {
            myWriter.close();
        }
        System.out.println("Successfully wrote to the file.");
    } catch (IOException e) {
        System.out.println("An error occurred.");
        e.printStackTrace();
    }
}

}

public static OutputData readListIntegers(String file) {
    OutputData output = new OutputData();
    try {
        checkEmptyString(file);
        BufferedReader input =
            new BufferedReader(new FileReader(file));
        try {
            String line = null;
            int num = 0, n = -1;
            int j = 0, k = -1;
            while ((n<num)&&((line = input.readLine()) != null)){
                int i = Integer.parseInt(line);
                System.out.println("OK int:" + i);
                if(num==0) {

```

```

        if(i<=0) break;
        else {
            num = i;
            n = 0;
            int[] [] oo;
            if(num%4==0) {
                oo = new int[4] [num/4];
            } else {
                oo = new int[4] [num/4 + 1];
            }
            output.data = oo;
        }
    } else {
        n++;
        k++;
        int dim2 = output.data[0].length;
        if(k == dim2) {k=0; j++;}
        output.data[j] [k] = i;
        output.size[j] = k+1;
        output.n = n;
    }
}

}

}
finally {
    input.close();
}
}

catch (EmptyStringException ex){
    System.out.println("FileName Empty Exception");
    ex.printStackTrace();
}

catch (NumberFormatException ex) {
    System.out.println("Need integer in file!");
    output.data = null;
    output.n = 0;
    ex.printStackTrace();
}
}

```

```
        catch (FileNotFoundException ex){
            System.out.println("File Not Found Exception");
            ex.printStackTrace();
        }
        catch (IOException ex){
            System.out.println("IO Exception");
            output.data = null;
            output.n = 0;
            ex.printStackTrace();
        }
        return output;
    }

    public static void main(String[] args) {
        splitListIntegers("filenamets");
    }

    public static void checkEmptyString(String s)
    throws EmptyStringException {
        if(s.trim().equals("")) throw new EmptyStringException();
    }
}

class EmptyStringException extends Exception {}
class OutputData {
    int[] [] data;
    int[] size = new int[4];
    int n;
}
```

Chương 11

Lập trình giao diện GUI với JavaFX

11.1. Giới thiệu về JavaFX	242
11.2. Cài đặt JavaFX	245
11.3. Các thành phần giao diện JavaFX	248
11.4. UI Controls trong JavaFX	256
11.5. Layout Panes trong JavaFX	260
11.6. Mô hình xử lý sự kiện trong JavaFX	268
11.7. Kéo thả giao diện với SceneBuilder	282
11.8. Bài tập	286
11.9. Lời giải bài tập chương 11	286

11.1. Giới thiệu về JavaFX

11.1.1. Tổng quan về JavaFX và lập trình GUI

JavaFX là một thư viện Java mạnh mẽ, được sử dụng để xây dựng các ứng dụng giao diện đồ họa người dùng (Graphical User Interface - GUI). Trong thời đại ngày nay, GUI đã trở thành một phần quan trọng của bất kỳ phần mềm nào, bởi giao diện đồ họa giúp người dùng dễ dàng tương tác với ứng dụng thông qua các yếu tố trực quan như nút bấm, hộp chọn, thanh cuộn, và nhiều thành phần khác, thay vì phải nhập lệnh thông qua dòng lệnh truyền thống. JavaFX không chỉ cung cấp các công cụ để tạo giao diện đồ họa mà còn hỗ trợ phát triển các ứng dụng đa nền tảng, chạy được trên nhiều thiết bị từ máy tính để bàn, máy tính bảng đến điện thoại thông minh.

JavaFX được thiết kế để thay thế cho các thư viện đồ họa cũ như AWT và Swing, đồng thời mang lại những tính năng hiện đại và linh hoạt hơn. Bên cạnh đó, JavaFX còn giúp lập trình viên dễ dàng tạo các hiệu ứng động và tích hợp các yếu tố đa phương tiện vào ứng dụng của mình, tạo ra trải nghiệm người dùng mượt mà và hấp dẫn. Không chỉ dừng lại ở việc xây dựng giao diện đơn giản, JavaFX còn hỗ trợ các yếu tố đồ họa phức tạp như 2D, 3D, đồ họa động, và các tính năng phát âm

thanh và video, giúp lập trình viên phát triển những ứng dụng phong phú và hiện đại hơn.

11.1.2. Lịch sử phát triển JavaFX

JavaFX bắt đầu là một dự án cá nhân của Chris Oliver khi ông làm việc tại SeeBeyond Technology Corporation. Vào năm 2005, SeeBeyond được mua lại bởi Sun Microsystems, và JavaFX chính thức ra mắt vào năm 2007 tại hội nghị JavaOne. Được phát triển để trở thành một giải pháp giao diện đồ họa toàn diện cho Java, JavaFX nhận được sự chú ý từ cộng đồng lập trình viên ngay từ khi ra mắt.

JavaFX 1.0, phiên bản đầu tiên, được phát hành vào năm 2008 và tích hợp sẵn vào môi trường phát triển NetBeans. Phiên bản này giúp lập trình viên dễ dàng sử dụng và xây dựng các ứng dụng GUI mà không cần phải thiết lập thêm thư viện phức tạp. Đến năm 2014, JavaFX đã được tích hợp vào bộ công cụ Java SDK 8, khiến nó trở thành một phần không thể thiếu của Java và nâng cao sự phổ biến. Tuy nhiên, đến năm 2018, JavaFX được tách ra khỏi Java SDK bắt đầu từ phiên bản Java 11, trở thành một thư viện độc lập. Điều này giúp JavaFX linh hoạt hơn trong việc cập nhật, phát triển mà không phụ thuộc vào Java SDK, đáp ứng kịp thời nhu cầu phát triển đa dạng của các ứng dụng hiện đại.

11.1.3. Các tính năng nổi bật của JavaFX

JavaFX mang đến nhiều tính năng ưu việt giúp lập trình viên xây dựng ứng dụng GUI một cách hiệu quả và chuyên nghiệp hơn. Một số tính năng nổi bật của JavaFX bao gồm:

1. Hỗ trợ FXML: FXML là một ngôn ngữ khai báo dựa trên XML cho phép lập trình viên định nghĩa giao diện người dùng một cách trực quan mà không cần viết mã Java trực tiếp. Điều này giúp tách biệt phần giao diện và phần logic của ứng dụng, giúp cho việc phát triển, bảo trì và mở rộng ứng dụng trở nên dễ dàng hơn. Với FXML, lập trình viên có thể mô tả các thành phần giao diện như nút bấm, thanh công cụ, khung văn bản, và các tương tác sự kiện.

2. Scene Builder: JavaFX cung cấp công cụ Scene Builder cho phép thiết kế giao diện bằng thao tác kéo thả. Đây là một công cụ mạnh mẽ, giúp các lập trình viên và nhà thiết kế giao diện làm việc một cách nhanh chóng, dễ dàng tạo ra giao diện đồ họa mà không cần phải viết mã thủ công. Công cụ này xuất tệp đầu ra là FXML, giúp lập trình viên tích hợp nhanh chóng vào ứng dụng JavaFX.

3. Tương thích với Swing: Một trong những ưu điểm lớn của JavaFX là tính tương thích cao với Swing, thư viện GUI phổ biến của Java trước đó. Điều này có

nghĩa là các dự án đang được phát triển trên nền tảng Swing có thể dễ dàng chuyển đổi và mở rộng với JavaFX, giúp lập trình viên tận dụng lại mã nguồn hiện có và tăng cường chức năng của ứng dụng mà không cần phải xây dựng lại từ đầu.

4. CSS Styling: JavaFX hỗ trợ tùy chỉnh giao diện thông qua các kiểu CSS, cho phép lập trình viên sử dụng các thuộc tính CSS để thay đổi màu sắc, kích thước, và định dạng của các thành phần giao diện một cách dễ dàng. Tính năng này giúp cho việc quản lý giao diện trở nên trực quan hơn, cho phép lập trình viên tạo ra giao diện có phong cách độc đáo và chuyên nghiệp.

5. Bộ thư viện UI Controls phong phú: JavaFX cung cấp một thư viện phong phú các thành phần giao diện như Button, TextField, ComboBox, TableView, và các điều khiển khác. Các thành phần này giúp lập trình viên xây dựng nhanh chóng các giao diện thân thiện và tiện ích cho người dùng.

6. Đồ họa và Đa phương tiện: JavaFX hỗ trợ đồ họa 2D, 3D, cũng như khả năng phát âm thanh, video và các yếu tố đa phương tiện khác. Điều này đặc biệt hữu ích cho các ứng dụng yêu cầu hình ảnh động, hiệu ứng 3D hoặc phát nội dung âm thanh và video. Với khả năng mở rộng đồ họa, JavaFX là nền tảng lý tưởng để phát triển các ứng dụng như trò chơi, các công cụ học tập, hoặc các ứng dụng truyền thông đa phương tiện.

11.1.4. So sánh JavaFX với các thư viện AWT và Swing

Trước JavaFX, Java đã có hai thư viện chính hỗ trợ phát triển GUI là AWT và Swing. Cả hai đều đóng góp quan trọng trong việc xây dựng các ứng dụng GUI truyền thống, tuy nhiên chúng đều có những hạn chế nhất định:

AWT (Abstract Windowing Toolkit): Là thư viện đồ họa đầu tiên của Java, AWT ra đời từ JDK 1.0 và cung cấp các thành phần giao diện cơ bản. Tuy nhiên, AWT phụ thuộc vào nền tảng hệ điều hành, dẫn đến việc giao diện và hành vi của ứng dụng không nhất quán trên các hệ điều hành khác nhau. Giao diện của AWT cũng khá thô sơ và thiếu linh hoạt trong việc tùy chỉnh.

Swing: Được phát triển trên nền tảng AWT, Swing ra đời như một phần mở rộng giúp tăng cường tính năng và khả năng tùy chỉnh giao diện. Swing không phụ thuộc vào nền tảng hệ điều hành, cung cấp giao diện nhất quán trên mọi hệ điều hành và mang lại trải nghiệm người dùng tốt hơn so với AWT. Tuy nhiên, Swing vẫn có những hạn chế trong việc hỗ trợ đa phương tiện và không phù hợp với các giao diện hiện đại.

JavaFX được thiết kế để khắc phục những hạn chế của AWT và Swing, với giao diện trực quan, hỗ trợ đa phương tiện và tương thích với các nền tảng hiện đại. Nó

cung cấp một bộ công cụ toàn diện cho việc xây dựng giao diện đồ họa người dùng, đồng thời đáp ứng nhu cầu của các ứng dụng hiện đại như trò chơi, các công cụ học tập, và các ứng dụng doanh nghiệp.

11.1.5. Lợi ích khi sử dụng JavaFX trong phát triển GUI

Sử dụng JavaFX mang lại nhiều lợi ích cho lập trình viên, bao gồm tính linh hoạt trong thiết kế, khả năng mở rộng, và hỗ trợ đa nền tảng. JavaFX cho phép lập trình viên xây dựng giao diện đồ họa dễ dàng, với các công cụ như Scene Builder và CSS Styling. Tính năng tương thích với Swing giúp các dự án cũ có thể chuyển đổi và tiếp tục phát triển mà không cần phải thay đổi nhiều mã nguồn. Bên cạnh đó, JavaFX hỗ trợ đồ họa 2D, 3D và phát đa phương tiện, giúp các ứng dụng trở nên sinh động và hấp dẫn hơn. Tất cả những yếu tố này giúp JavaFX trở thành một nền tảng lý tưởng cho phát triển GUI.

11.2. Cài đặt JavaFX

11.2.1. Giới thiệu về cài đặt JavaFX

Trước khi bắt đầu phát triển ứng dụng với JavaFX, bạn cần đảm bảo rằng môi trường phát triển đã được cấu hình để hỗ trợ thư viện này. Từ phiên bản JDK 11 trở đi, JavaFX không còn là một phần mặc định của Java Development Kit (JDK), điều này đòi hỏi lập trình viên phải thực hiện một số bước cài đặt bổ sung để tích hợp JavaFX vào các dự án của mình. Để bắt đầu, bạn có thể tải xuống JavaFX từ trang chủ OpenJFX tại <https://openjfx.io/>. Đây là phiên bản mã nguồn mở của JavaFX, được cập nhật và phát triển bởi cộng đồng.

Sau khi truy cập vào trang web, bạn cần chọn mục *Download* và tải về phiên bản JavaFX phù hợp với hệ điều hành của mình như Windows, macOS hoặc Linux. Sau khi tải xong, bạn hãy giải nén gói cài đặt và ghi nhớ đường dẫn đến thư mục `lib`, nơi chứa các tệp JAR cần thiết cho dự án.

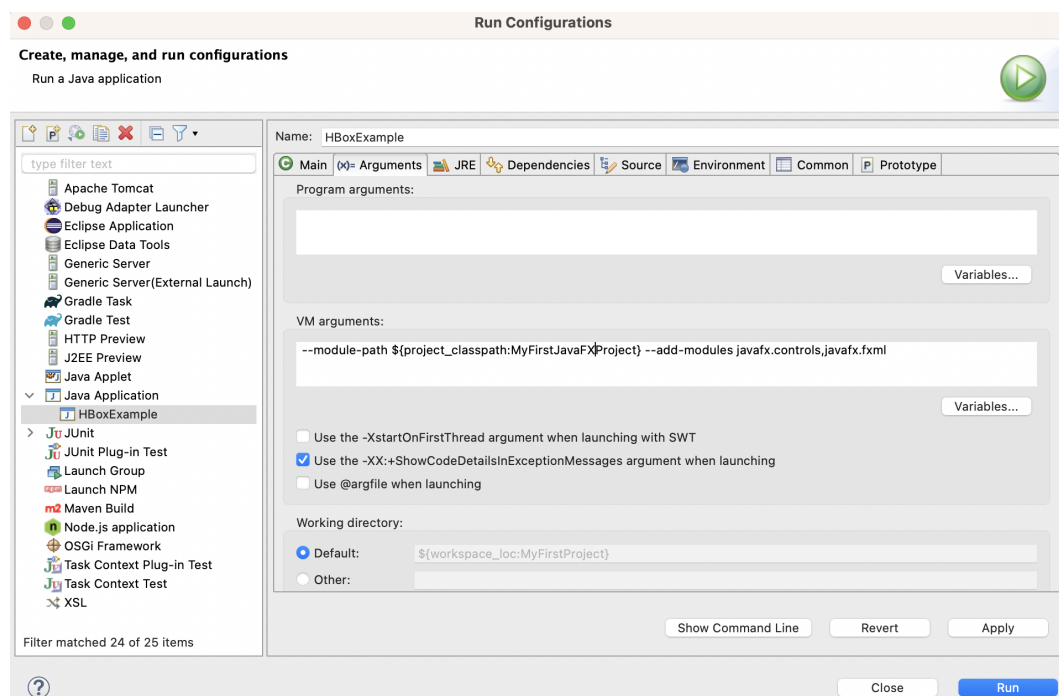
11.2.2. Cấu hình JavaFX trong Eclipse

Đối với những ai sử dụng Eclipse, việc tích hợp JavaFX yêu cầu một số bước cấu hình cơ bản. Đầu tiên, bạn hãy mở Eclipse và tạo một dự án Java mới. Sau đó, nhấp chuột phải vào dự án và chọn *Properties*. Trong mục *Java Build Path*, chuyển đến tab *Libraries* và nhấn nút *Add External JARs*. Tại đây, bạn cần chọn tất cả các tệp JAR trong thư mục `lib` của JavaFX mà bạn đã tải về trước đó. Sau khi hoàn tất, nhấn *OK* để thêm thư viện vào dự án.

Tiếp theo, để chạy ứng dụng JavaFX, bạn cần cấu hình tham số VM. Trong Eclipse, nhấp vào mũi tên bên cạnh biểu tượng *Run* và chọn *Run Configurations...*. Trong phần cấu hình, chọn dự án của bạn và thêm các tham số sau vào phần *VM arguments*:

```
--module-path ${project_classpath:REPLACE_ME_WITH_YOUR_PROJECT_NAME}
--add-modules javafx.controls,javafx.fxml
```

Lưu ý, bạn cần thay `REPLACE_ME_WITH_YOUR_PROJECT_NAME` bằng đường dẫn thực tế tới project của bạn. Đồng thời, bạn cần bỏ chọn *Bỏ chọn: Use the -XstartOnFirstThread argument when launching with SWT*. Chi tiết xem Hình 11.1



Hình 11.1 Cấu hình JavaFx trong Eclipse

11.2.3. Cài đặt JavaFX Scene Builder

Một công cụ quan trọng giúp việc thiết kế giao diện trong JavaFX trở nên dễ dàng hơn là Scene Builder. Đây là công cụ cho phép bạn tạo giao diện bằng cách kéo thả các thành phần một cách trực quan. Để cài đặt Scene Builder, bạn có thể truy cập trang web chính thức tại <https://gluonhq.com/products/scene-builder/> và tải về phiên bản mới nhất. Sau khi hoàn tất cài đặt, bạn có thể tích hợp Scene

Builder vào Eclipse. Hãy mở Eclipse và vào *Window* → *Preferences* → *JavaFX*, sau đó nhấn nút *Browse...* để chọn đường dẫn đến file thực thi của Scene Builder (ví dụ: *SceneBuilder.exe* trên Windows). Nhờ tích hợp này, bạn có thể chỉnh sửa các tệp FXML trực tiếp bằng Scene Builder thông qua Eclipse bằng cách nhấp chuột phải vào tệp FXML và chọn *Open with Scene Builder*.

11.2.4. Cấu hình JavaFX trong các IDE khác

Ngoài Eclipse, bạn cũng có thể sử dụng JavaFX trong các môi trường phát triển tích hợp khác như IntelliJ IDEA và NetBeans. Với IntelliJ IDEA, bạn có thể tạo một dự án Java mới hoặc mở một dự án hiện tại. Sau đó, truy cập vào phần *Project Structure* và thêm thư mục `lib` của JavaFX vào phần phụ thuộc của dự án. Bạn cũng cần cấu hình tham số VM tương tự như trong Eclipse để đảm bảo ứng dụng JavaFX có thể chạy được. Trong NetBeans, việc cấu hình JavaFX tương đối đơn giản vì IDE này hỗ trợ tích hợp tốt từ các phiên bản cũ. Bạn chỉ cần tạo một dự án JavaFX mới và đảm bảo rằng thư viện JavaFX đã được thêm vào đường dẫn của dự án.

11.2.5. Kiểm tra cài đặt với ứng dụng Hello World

Sau khi hoàn tất cài đặt và cấu hình, bạn có thể kiểm tra xem JavaFX đã hoạt động chính xác hay chưa bằng cách tạo một ứng dụng "Hello World" đơn giản. Ví dụ mã nguồn dưới đây tạo một giao diện với một nút bấm hiển thị thông điệp "Hello World" khi nhấp vào:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class HelloWorld extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button btn = new Button("Say 'Hello World'");
        btn.setOnAction(new EventHandler<ActionEvent>() {
            @Override
            public void handle(ActionEvent event) {
                System.out.println("Hello World!");
            }
        });
        StackPane root = new StackPane();
        root.getChildren().add(btn);
        primaryStage.setScene(new Scene(root, 300, 350));
        primaryStage.show();
    }
}
```

```

        }
    });

    StackPane root = new StackPane();
    root.getChildren().add(btn);

    Scene scene = new Scene(root, 300, 250);
    primaryStage.setTitle("Hello World!");
    primaryStage.setScene(scene);
    primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
}

```

Nếu ứng dụng chạy thành công và hiển thị giao diện với một nút bấm, điều đó có nghĩa là JavaFX đã được cài đặt đúng cách và sẵn sàng cho các ứng dụng phức tạp hơn.

11.3. Các thành phần giao diện JavaFX

Một ứng dụng JavaFX cơ bản bao gồm các thành phần chính là *Stage*, *Scene* và *Node*. Các thành phần này kết hợp với nhau để tạo ra giao diện người dùng và quản lý các tương tác.

11.3.1. Stage

Stage là cửa sổ chính trong một ứng dụng JavaFX, đại diện cho một khung hiển thị mà qua đó mọi thành phần đồ họa khác được hiển thị. Khi bạn tạo một ứng dụng JavaFX, một *Stage* chính, gọi là *primaryStage*, được cung cấp bởi JavaFX và được quản lý thông qua phương thức `start()` trong lớp `Application`.

Stage có thể được tùy chỉnh bằng cách sử dụng các phương thức như `setTitle()`, `setWidth()`, `setHeight()`, và `show()`. Dưới đây là một ví dụ minh họa:

```

import javafx.application.Application;
import javafx.stage.Stage;
import javafx.stage.StageStyle;

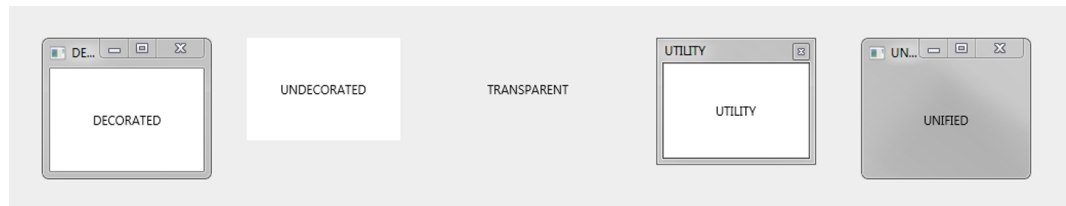
```

```
public class MyStageExample extends Application {  
    @Override  
    public void start(Stage primaryStage) {  
        primaryStage.setTitle("My JavaFX Stage Example");  
        primaryStage.setWidth(400);  
        primaryStage.setHeight(300);  
  
        // Thiết lập kiểu trang trí cho Stage  
        primaryStage.initStyle(StageStyle.DECORATED);  
        primaryStage.show();  
    }  
  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

Trong đoạn mã trên, `primaryStage` là đối tượng *Stage* chính. Chúng ta sử dụng phương thức `setTitle()` để đặt tiêu đề cho cửa sổ, `setWidth()` và `setHeight()` để thiết lập kích thước. Lệnh `initStyle(StageStyle.DECORATED)` thiết lập kiểu hiển thị cho *Stage*. JavaFX hỗ trợ nhiều kiểu hiển thị như minh họa trên Hình 11.2

- `StageStyle.DECORATED`: Kiểu mặc định, hiển thị cửa sổ với thanh tiêu đề và các nút điều khiển.
- `StageStyle.UNDECORATED`: Cửa sổ không có thanh tiêu đề và các nút điều khiển.
- `StageStyle.TRANSPARENT`: Cửa sổ trong suốt, cho phép nội dung bên dưới hiển thị qua cửa sổ.
- `StageStyle.UTILITY`: Cửa sổ dạng nhỏ gọn dành cho các tiện ích.

Hình minh họa: Kết quả khi chạy chương trình trên sẽ hiển thị một cửa sổ với kích thước và tiêu đề đã được thiết lập. Khi thay đổi kiểu `StageStyle`, giao diện của cửa sổ sẽ thay đổi theo, như bỏ đi thanh tiêu đề khi sử dụng `StageStyle.UNDECORATED`.



Hình 11.2 Các kiểu trang trí Stage

11.3.2. Scene

Scene đại diện cho nội dung hiển thị trong một *Stage*. Mỗi *Stage* chỉ có một *Scene* duy nhất, và *Scene* này chứa các thành phần giao diện được tổ chức theo cấu trúc cây gọi là *Scene Graph*. Ví dụ sau đây minh họa cách tạo một *Scene* với nút bấm:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class MySceneExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button btn = new Button("Click Me");

        btn.setOnAction(new EventHandler<ActionEvent>() {
            @Override
            public void handle(ActionEvent event) {
                System.out.println("Button clicked!");
            }
        });

        StackPane root = new StackPane();
        root.getChildren().add(btn);

        Scene scene = new Scene(root, 300, 200);
        primaryStage.setScene(scene);
```

```
        primaryStage.setTitle("JavaFX Scene Example");
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Trong ví dụ trên, chúng ta tạo một *Button* và thêm nó vào *StackPane*, sau đó đặt *StackPane* làm gốc cho *Scene*. Cuối cùng, *Scene* được thêm vào *Stage* để hiển thị. Kết quả chạy chương trình hiển thị một cửa sổ với nút bấm. Khi người dùng nhấp vào nút, thông báo "Button clicked!" xuất hiện trong console.

11.3.3. Node

Node là thành phần cơ bản trong JavaFX, đại diện cho các đối tượng đồ họa và giao diện người dùng như hình tròn, hình chữ nhật, nút bấm, hộp văn bản và nhiều thành phần khác. Mỗi *Node* có thể có các thuộc tính như vị trí, kích thước, màu sắc và sự kiện tương tác. Các *Node* được tổ chức trong một *Scene Graph* để tạo thành giao diện ứng dụng.

Ví dụ sau minh họa cách tạo một *Node* đơn giản là hình tròn:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class MyNodeExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Hình tròn có bán kính 50 và màu xanh
        Circle circle = new Circle(50, Color.BLUE);
        circle.setStroke(Color.BLACK); // Viền đen cho hình tròn
        circle.setStrokeWidth(2);
    }
}
```

```
StackPane root = new StackPane();
root.getChildren().add(circle);

Scene scene = new Scene(root, 300, 250);
primaryStage.setScene(scene);
primaryStage.setTitle("JavaFX Node Example");
primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
}
```

Trong đoạn mã trên, *Circle* là một *Node* được tạo với bán kính 50 pixel và màu nền là xanh dương. Lệnh `circle.setStroke(Color.BLACK)` thiết lập màu viền cho hình tròn, và `circle.setStrokeWidth(2)` thiết lập độ dày viền. Hình tròn này được thêm vào *StackPane* và hiển thị trên *Scene*.

11.3.4. Scene Graph

Trong JavaFX, *Scene Graph* là cấu trúc cây tổ chức các *Node* trong một *Scene*. Mỗi *Scene* có một *Node* gốc (root node) và các *Node* con được sắp xếp theo các nhánh. Điều này giúp quản lý hiển thị, sự kiện và các thuộc tính của giao diện một cách dễ dàng.

Ví dụ, khi thêm hoặc xóa một *Node* trong *Scene Graph*, giao diện sẽ tự động được cập nhật mà không cần làm mới thủ công. Đây là một tính năng mạnh mẽ giúp JavaFX dễ dàng phát triển các giao diện linh hoạt và giàu tương tác.

11.3.5. Cách tạo ứng dụng JavaFX

Để tạo một ứng dụng JavaFX cơ bản, bạn cần kế thừa lớp `Application` và ghi đè phương thức `start()`, nơi bạn sẽ tạo *Stage*, thiết lập *Scene* và các thành phần giao diện. Dưới đây là một ví dụ minh họa cách tạo ứng dụng JavaFX cơ bản:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
```

```
import javafx.stage.Stage;

public class BasicJavaFXApp extends Application {
    @Override
    public void start(Stage primaryStage) {
        Label label = new Label("Welcome to JavaFX!");
        StackPane root = new StackPane(label);
        Scene scene = new Scene(root, 400, 300);

        primaryStage.setTitle("Basic JavaFX Application");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Trong đoạn mã trên, lớp `BasicJavaFXApp` kế thừa từ `Application` và ghi đè phương thức `start()` để thiết lập *Stage* và *Scene*. Chúng ta tạo một *Label* và thêm nó vào *StackPane*, sau đó đặt *Scene* này làm nội dung cho *Stage*.

11.3.6. Vòng đời ứng dụng JavaFX

Mỗi ứng dụng JavaFX có một vòng đời xác định, bao gồm các giai đoạn khởi tạo, hiển thị và kết thúc. JavaFX cung cấp ba phương thức chính mà bạn có thể ghi đè để kiểm soát các giai đoạn này:

- `init()`: Phương thức này được gọi đầu tiên khi ứng dụng khởi động, trước khi `start()`. Nó thường được dùng để khởi tạo các tài nguyên không liên quan đến giao diện đồ họa, chẳng hạn như đọc dữ liệu hoặc thiết lập các thông số cấu hình.
- `start(Stage primaryStage)`: Đây là phương thức chính được gọi để khởi động giao diện của ứng dụng. Tại đây, bạn sẽ tạo *Stage*, *Scene* và thêm các thành phần giao diện.
- `stop()`: Phương thức này được gọi khi ứng dụng kết thúc, chẳng hạn khi cửa

số ứng dụng bị đóng. Nó thường được dùng để giải phóng tài nguyên hoặc lưu trạng thái ứng dụng.

Ví dụ minh họa một ứng dụng sử dụng cả ba phương thức:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class JavaFXLifecycleExample extends Application {
    @Override
    public void init() {
        System.out.println("Application is initializing...");
    }

    @Override
    public void start(Stage primaryStage) {
        Button btn = new Button("Close");

        StackPane root = new StackPane(btn);
        Scene scene = new Scene(root, 300, 200);
        primaryStage.setScene(scene);
        primaryStage.setTitle("JavaFX Lifecycle Example");
        primaryStage.show();
    }

    @Override
    public void stop() {
        System.out.println("Application is stopping...");
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Khi chạy ứng dụng, bạn sẽ thấy thông báo "Application is initializing..." xuất hiện trên console khi ứng dụng khởi tạo và "Application is stopping..." khi ứng dụng đóng lại.

11.3.7. Cách xây dựng Scene Graph

Trong JavaFX, *Scene Graph* là cấu trúc cây tổ chức các *Node* trong *Scene*. Cây này có một *Node* gốc (*root node*) và các *Node* con được sắp xếp theo các nhánh. Các *Node* có thể là *Parent Node* (có chứa các *Node* con) hoặc *Leaf Node* (không chứa *Node* con). Khi một *Node* trong *Scene Graph* thay đổi, JavaFX sẽ tự động cập nhật giao diện để phản ánh sự thay đổi đó.

Ví dụ dưới đây minh họa cách xây dựng một *Scene Graph* đơn giản:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class SceneGraphExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button btn1 = new Button("Button 1");
        Button btn2 = new Button("Button 2");
        Button btn3 = new Button("Button 3");

        VBox root = new VBox();
        root.getChildren().addAll(btn1, btn2, btn3);

        Scene scene = new Scene(root, 300, 200);
        primaryStage.setScene(scene);
        primaryStage.setTitle("Scene Graph Example");
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

```
}

```

Trong ví dụ trên, *VBox* là *Parent Node* chứa ba *Button* làm con. Khi các *Node* con được thêm vào hoặc gỡ bỏ, JavaFX sẽ tự động quản lý và cập nhật giao diện để hiển thị thay đổi. *Scene Graph* giúp tạo ra các giao diện phức tạp một cách dễ dàng và trực quan.

11.4. UI Controls trong JavaFX

UI Controls là các thành phần giao diện người dùng cho phép người dùng tương tác với ứng dụng JavaFX. JavaFX cung cấp một bộ sưu tập phong phú các *UI Controls*, bao gồm *Button*, *Label*, *TextField*, *CheckBox*, *RadioButton*, *ComboBox*, và nhiều thành phần khác (xem Hình 11.3). Các *UI Controls* này được tích hợp sẵn trong JavaFX để tạo các giao diện người dùng phức tạp, thân thiện và tương tác.



Hình 11.3 Các UI controls trong JavaFX

11.4.1. Button

Button là một trong những *UI Controls* phổ biến nhất, cho phép người dùng nhấn để thực hiện một hành động. Dưới đây là ví dụ về cách tạo và xử lý sự kiện cho Button:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class ButtonExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button button = new Button("Click Me");
        btn.setOnAction(new EventHandler<ActionEvent>() {
            @Override
            public void handle(ActionEvent event) {
                System.out.println("Button clicked!");
            }
        });

        StackPane root = new StackPane(button);
        Scene scene = new Scene(root, 300, 200);

        primaryStage.setTitle("Button Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Trong ví dụ trên, chúng ta tạo một Button với nhãn "Click Me". Lệnh `button.setOnAction(...)` thiết lập một trình xử lý sự kiện để in ra thông báo

"Button was clicked!" khi người dùng nhấn vào nút. Phần giao diện được tổ chức trong `StackPane` và hiển thị trên *Stage* chính.

11.4.2. Label

`Label` là một thành phần giao diện dùng để hiển thị văn bản không thể chỉnh sửa. `Label` thường được sử dụng để mô tả thông tin hoặc hướng dẫn người dùng.

Ví dụ dưới đây minh họa cách sử dụng `Label`:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class LabelExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        Label label = new Label("Hello, JavaFX!");

        StackPane root = new StackPane(label);
        Scene scene = new Scene(root, 300, 200);

        primaryStage.setTitle("Label Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Trong ví dụ này, `Label` hiển thị văn bản "Hello, JavaFX!". `Label` được thêm vào `StackPane` để hiển thị trên *Scene*.

11.4.3. TextField

`TextField` là một thành phần giao diện cho phép người dùng nhập và chỉnh sửa văn bản. Nó thường được sử dụng để thu thập dữ liệu từ người dùng, chẳng hạn

như tên, địa chỉ email, v.v.

Dưới đây là ví dụ về TextField:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class TextFieldExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        TextField textField = new TextField();
        textField.setPromptText("Enter your name");

        button.setOnAction(new EventHandler<ActionEvent>() {
            @Override
            public void handle(ActionEvent event) {
                String input = textField.getText();
                System.out.println("Hello, " + input + "!");
            }
        });

        VBox root = new VBox(10, textField, button);
        Scene scene = new Scene(root, 300, 200);

        primaryStage.setTitle("TextField Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Trong đoạn mã này, TextField cho phép người dùng nhập tên. Nút Button có

sự kiện `onAction` để lấy văn bản từ `TextField` và hiển thị thông báo chào mừng. Lệnh `textField.setPromptText(...)` hiển thị văn bản hướng dẫn khi `TextField` trống.

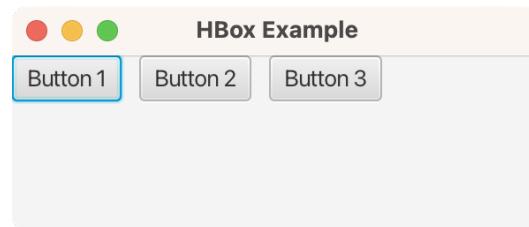
11.5. Layout Panes trong JavaFX

Layout Panes là các lớp dùng để sắp xếp các thành phần giao diện (UI Controls) trong một *Scene*. JavaFX cung cấp nhiều loại layout khác nhau, mỗi loại có các đặc điểm riêng biệt giúp định vị và sắp xếp các thành phần con theo những cách khác nhau. Các loại layout chính bao gồm `HBox`, `VBox`, `BorderPane`, `GridPane`, `FlowPane`, và `StackPane`.

11.5.1. HBox

`HBox` là một layout sắp xếp các thành phần con theo hàng ngang, từ trái sang phải. Các thành phần trong `HBox` có thể được căn chỉnh và có khoảng cách giữa chúng.

Hình 11.4 là ví dụ minh họa cách sử dụng `HBox`.



Hình 11.4 Ví dụ layout Hbox

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.HBox;
import javafx.stage.Stage;

public class HBoxExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button button1 = new Button("Button 1");
        Button button2 = new Button("Button 2");
```

```
Button button3 = new Button("Button 3");

// Tạo một HBox với khoảng cách 10px giữa các thành phần
HBox hbox = new HBox(10);
hbox.getChildren().addAll(button1, button2, button3);

Scene scene = new Scene(hbox, 300, 100);
primaryStage.setTitle("HBox Example");
primaryStage.setScene(scene);
primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
}
```

Trong ví dụ trên, HBox được tạo với khoảng cách 10 pixel giữa các nút bấm (Button) như Hình 11.4. Các thành phần được sắp xếp theo hàng ngang từ trái sang phải. Lệnh `hbox.getChildren().addAll(...)` thêm tất cả các *Node* vào HBox.

11.5.2. VBox

VBox là một layout sắp xếp các thành phần con theo cột dọc, từ trên xuống dưới. Các thành phần có thể được căn chỉnh theo chiều ngang hoặc có khoảng cách giữa chúng. Hình 11.5 là ví dụ minh họa cách sử dụng VBox.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class VBoxExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        Label label1 = new Label("Label 1");
        Label label2 = new Label("Label 2");
        Label label3 = new Label("Label 3");
```




Hình 11.5 Ví dụ layout VBox

```
// Tạo một VBox với khoảng cách 15px giữa các thành phần
VBox vbox = new VBox(15);
vbox.getChildren().addAll(label1, label2, label3);

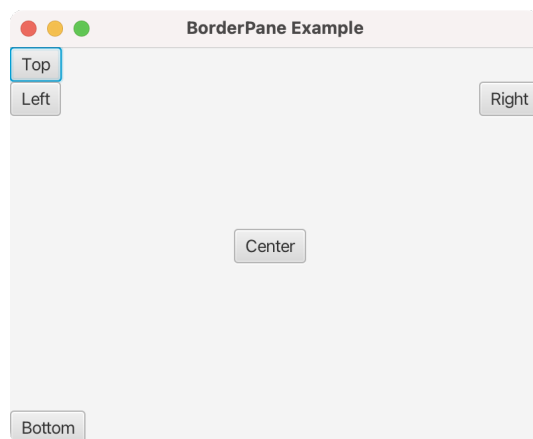
Scene scene = new Scene(vbox, 200, 150);
primaryStage.setTitle("VBox Example");
primaryStage.setScene(scene);
primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
}
```

Trong ví dụ này, các `Label` được sắp xếp theo cột dọc với khoảng cách 15 pixel giữa chúng. `VBox` là layout lý tưởng để tổ chức các thành phần dọc theo chiều dọc, chẳng hạn như danh sách các nhãn hoặc các nút.

11.5.3. `BorderPane`

`BorderPane` chia giao diện thành năm khu vực: `Top`, `Bottom`, `Left`, `Right`, và `Center`. Bạn có thể đặt các thành phần vào các khu vực này để tạo nên giao diện bố cục phức tạp hơn. Hình 11.6 là ví dụ sử dụng `BorderPane`.



Hình 11.6 Ví dụ layout BorderPane

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.BorderPane;
import javafx.stage.Stage;

public class BorderPaneExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button topButton = new Button("Top");
        Button bottomButton = new Button("Bottom");
        Button leftButton = new Button("Left");
        Button rightButton = new Button("Right");
        Button centerButton = new Button("Center");

        BorderPane borderPane = new BorderPane();
        borderPane.setTop(topButton);
        borderPane.setBottom(bottomButton);
        borderPane.setLeft(leftButton);
        borderPane.setRight(rightButton);
        borderPane.setCenter(centerButton);

        Scene scene = new Scene(borderPane, 400, 300);
        primaryStage.setTitle("BorderPane Example");
    }
}
```

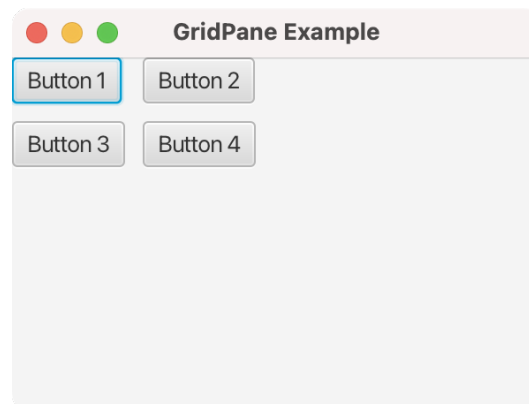
```
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Trong ví dụ này, các nút được sắp xếp vào các khu vực `Top`, `Bottom`, `Left`, `Right`, và `Center` của `BorderPane`. `BorderPane` rất hữu ích khi bạn muốn tổ chức giao diện theo các vùng chính.

11.5.4. GridPane

`GridPane` cho phép sắp xếp các thành phần theo dạng lưới với các hàng và cột. Đây là layout lý tưởng để tạo các biểu mẫu hoặc giao diện có cấu trúc bảng. Hình 11.7 minh họa cách sử dụng `GridPane`.



Hình 11.7 Ví dụ layout `GridPane`

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class GridPaneExample extends Application {
```

```
@Override
public void start(Stage primaryStage) {
    Button button1 = new Button("Button 1");
    Button button2 = new Button("Button 2");
    Button button3 = new Button("Button 3");
    Button button4 = new Button("Button 4");

    GridPane gridPane = new GridPane();
    gridPane.setHgap(10); // Khoảng cách ngang giữa các cột
    gridPane.setVgap(10); // Khoảng cách dọc giữa các hàng

    gridPane.add(button1, 0, 0); // Vị trí cột 0, hàng 0
    gridPane.add(button2, 1, 0); // Vị trí cột 1, hàng 0
    gridPane.add(button3, 0, 1); // Vị trí cột 0, hàng 1
    gridPane.add(button4, 1, 1); // Vị trí cột 1, hàng 1

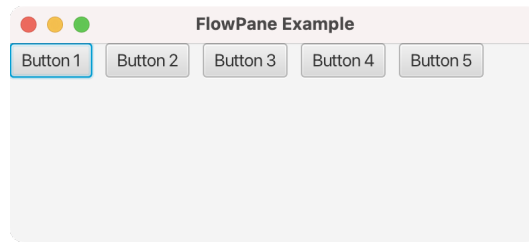
    Scene scene = new Scene(gridPane, 300, 200);
    primaryStage.setTitle("GridPane Example");
    primaryStage.setScene(scene);
    primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
}
```

Trong ví dụ này, các nút được sắp xếp vào các ô của **GridPane** với khoảng cách 10 pixel giữa các hàng và cột. Lệnh `gridPane.add(...)` thêm các thành phần vào các vị trí cụ thể trong lưới.

11.5.5. FlowPane

FlowPane sắp xếp các thành phần theo dòng. Khi không còn chỗ trong dòng hiện tại, các thành phần sẽ được tự động đẩy xuống dòng tiếp theo. Điều này rất hữu ích khi cần tạo các giao diện có thể tự động thích ứng với kích thước cửa sổ. Hình 11.8 minh họa cách sử dụng **FlowPane**.



Hình 11.8 Ví dụ layout FlowPane

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.FlowPane;
import javafx.stage.Stage;

public class FlowPaneExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button button1 = new Button("Button 1");
        Button button2 = new Button("Button 2");
        Button button3 = new Button("Button 3");
        Button button4 = new Button("Button 4");
        Button button5 = new Button("Button 5");

        FlowPane flowPane = new FlowPane();
        flowPane.setHgap(10); // Khoảng cách ngang giữa các thành phần
        flowPane.setVgap(10); // Khoảng cách dọc giữa các thành phần
        flowPane.getChildren().addAll(
            button1, button2, button3, button4, button5
        );

        Scene scene = new Scene(flowPane, 400, 150);
        primaryStage.setTitle("FlowPane Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

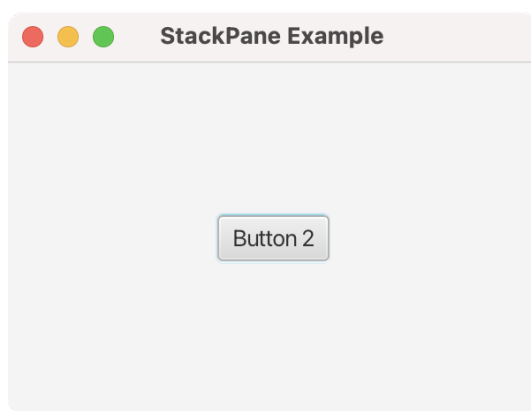
    public static void main(String[] args) {
```

```
        launch(args);  
    }  
}
```

Trong ví dụ trên, các nút được sắp xếp trong `FlowPane` với khoảng cách ngang và dọc được đặt là 10 pixel. Khi cửa sổ nhỏ lại, các thành phần sẽ tự động đẩy xuống dòng tiếp theo.

11.5.6. StackPane

`StackPane` xếp chồng các thành phần con lên nhau theo thứ tự thêm vào. Thành phần cuối cùng được thêm vào sẽ nằm trên cùng. Hình 11.9 minh họa cách sử dụng `StackPane`.



Hình 11.9 Ví dụ layout StackPane

```
import javafx.application.Application;  
import javafx.scene.Scene;  
import javafx.scene.control.Button;  
import javafx.scene.layout.StackPane;  
import javafx.stage.Stage;  
  
public class StackPaneExample extends Application {  
    @Override  
    public void start(Stage primaryStage) {  
        Button button1 = new Button("Button 1");  
        Button button2 = new Button("Button 2");  
    }  
}
```

```
StackPane stackPane = new StackPane();
stackPane.getChildren().addAll(button1, button2);

Scene scene = new Scene(stackPane, 300, 200);
primaryStage.setTitle("StackPane Example");
primaryStage.setScene(scene);
primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
}
```

Trong ví dụ trên, `Button 2` được thêm vào sau và nằm trên `Button 1`. `StackPane` là layout hữu ích khi bạn cần hiển thị các thành phần chồng lên nhau.

11.6. Mô hình xử lý sự kiện trong JavaFX

Trong JavaFX, chúng ta có thể phát triển các ứng dụng GUI, ứng dụng web và các ứng dụng đồ họa. Trong những ứng dụng này, khi người dùng tương tác với các thành phần giao diện (node), một sự kiện sẽ được coi là đã xảy ra. Ví dụ, nhấn vào một nút, di chuyển chuột, nhập ký tự qua bàn phím, chọn một mục từ danh sách, cuộn trang đều là những hoạt động dẫn đến sự kiện.

11.6.1. Các loại sự kiện trong JavaFX

Các sự kiện có thể được chia thành hai loại chính:

Sự kiện tiền cảnh (Foreground Events): Đây là những sự kiện yêu cầu sự tương tác trực tiếp của người dùng. Chúng được sinh ra do người dùng tương tác với các thành phần trong giao diện đồ họa, chẳng hạn như nhấn nút, di chuyển chuột, nhập ký tự qua bàn phím, chọn một mục từ danh sách hoặc cuộn trang.

Sự kiện hậu cảnh (Background Events): Đây là những sự kiện không yêu cầu sự tương tác của người dùng. Chúng có thể bao gồm các gián đoạn từ hệ điều hành, lỗi phần cứng hoặc phần mềm, hết thời gian chờ, hoàn thành thao tác, v.v.

JavaFX cung cấp hỗ trợ cho việc xử lý nhiều loại sự kiện khác nhau. Lớp `Event` trong gói `javafx.event` là lớp cơ sở cho mọi sự kiện. Một số loại sự kiện phổ biến trong JavaFX bao gồm:

Sự kiện chuột (Mouse Event): Đây là sự kiện nhập xảy ra khi chuột được nhấn. Nó được đại diện bởi lớp `MouseEvent` và bao gồm các hành động như nhấn chuột (*mouse clicked*), chuột được nhấn giữ (*mouse pressed*), chuột được nhả ra (*mouse released*), di chuyển chuột (*mouse moved*), chuột đi vào vùng mục tiêu (*mouse entered target*), chuột ra khỏi vùng mục tiêu (*mouse exited target*), v.v.

Sự kiện bàn phím (Key Event): Đây là sự kiện nhập xảy ra khi người dùng gõ phím trên một *Node*. Nó được đại diện bởi lớp `KeyEvent` và bao gồm các hành động như phím được nhấn (*key pressed*), phím được nhả ra (*key released*) và gõ phím (*key typed*).

Sự kiện kéo thả (Drag Event): Đây là sự kiện nhập xảy ra khi chuột được kéo. Nó được đại diện bởi lớp `DragEvent` và bao gồm các hành động như kéo vào mục tiêu (*drag entered*), thả kéo (*drag dropped*), kéo qua mục tiêu (*drag over*), v.v.

Sự kiện cửa sổ (Window Event): Đây là các sự kiện liên quan đến hành động hiển thị/ẩn cửa sổ. Nó được đại diện bởi lớp `WindowEvent` và bao gồm các hành động như ẩn cửa sổ (*window hiding*), hiển thị cửa sổ (*window shown*), ẩn cửa sổ (*window hidden*), đang hiển thị cửa sổ (*window showing*), v.v.

11.6.2. Xử lý sự kiện

Xử lý sự kiện là cơ chế kiểm soát và xác định các hành động xảy ra khi một sự kiện được kích hoạt. Cơ chế này có mã được gọi là *event handler* sẽ được thực thi khi sự kiện xảy ra. Trong JavaFX, các sự kiện được xử lý thông qua các trình xử lý sự kiện (`EventHandler`) và bộ lọc sự kiện (`Event Filters`).

Mỗi sự kiện trong JavaFX có các thành phần sau:

Mục tiêu (Target): Đây là *Node* nơi sự kiện xảy ra. Mục tiêu có thể là cửa sổ, *Scene* hoặc một *Node* cụ thể.

Nguồn (Source): Nguồn là nơi sinh ra sự kiện. Trong ví dụ nhấn nút, chuột là nguồn của sự kiện.

Loại (Type): Đây là loại sự kiện xảy ra, chẳng hạn trong trường hợp sự kiện chuột - nhấn chuột (*mouse pressed*), nhả chuột (*mouse released*) là các loại sự kiện.

Ví dụ, nếu chúng ta có một ứng dụng với một *Circle* và hai nút dừng (*Stop*) và phát (*Play*) được chèn vào qua một đối tượng *Group*. Khi bạn nhấn vào nút phát, nguồn của sự kiện sẽ là chuột, *target node* sẽ là nút phát và loại sự kiện được sinh ra là sự kiện nhấn chuột.

11.6.3. Các giai đoạn xử lý sự kiện trong JavaFX

Khi một sự kiện được sinh ra, JavaFX trải qua các giai đoạn sau đây:

Xây dựng tuyến đường (Route Construction)

Khi một sự kiện được sinh ra, tuyến đường mặc định của sự kiện được xác định bằng cách xây dựng một chuỗi phân phối sự kiện (*Event Dispatch Chain*). Đây là con đường từ *Stage* đến *Node* nguồn của sự kiện.

Ví dụ, khi nhấn vào nút phát trong ví dụ trên, chuỗi phân phối sự kiện sẽ đi từ *Stage* đến *Node Play Button*.

Giai đoạn bắt sự kiện (Event Capturing Phase)

Sau khi chuỗi phân phối sự kiện được xây dựng, *Node* gốc của ứng dụng sẽ phân phối sự kiện. Sự kiện sẽ di chuyển qua tất cả các *Node* trong chuỗi phân phối (từ gốc đến *Node* mục tiêu). Nếu bất kỳ *Node* nào trong chuỗi có bộ lọc sự kiện được đăng ký cho sự kiện đã sinh ra, bộ lọc sẽ được thực thi. Nếu không có *Node* nào có bộ lọc cho sự kiện, sự kiện sẽ được chuyển đến *Node* mục tiêu và được xử lý bởi nó.

Giai đoạn lan truyền sự kiện (Event Bubbling Phase)

Trong giai đoạn này, sự kiện sẽ di chuyển ngược lại từ *Node* mục tiêu lên đến *Stage* (từ dưới lên trên). Nếu bất kỳ *Node* nào trong chuỗi phân phối có trình xử lý sự kiện được đăng ký cho sự kiện đã sinh ra, trình xử lý sẽ được thực thi. Nếu không có *Node* nào có trình xử lý, sự kiện sẽ tiếp tục lan truyền lên và hoàn tất khi đến *Node* gốc.

11.6.4. Trình xử lý và bộ lọc sự kiện

Trình xử lý và bộ lọc sự kiện (Event Handlers and Filters) là nơi chứa logic ứng dụng để xử lý sự kiện. Một *Node* có thể đăng ký nhiều trình xử lý hoặc bộ lọc sự kiện. Trong trường hợp *Node* cha - con, bạn có thể cung cấp một bộ lọc hoặc trình xử lý chung cho *Node* cha, điều này sẽ được xử lý mặc định cho tất cả các *Node* con.

Như đã đề cập ở trên, trong quá trình xử lý sự kiện, một bộ lọc sẽ được thực thi trong giai đoạn bắt sự kiện và một trình xử lý sẽ được thực thi trong giai đoạn lan truyền sự kiện. Tất cả các trình xử lý và bộ lọc đều triển khai giao diện *EventHandler* của gói *javafx.event*.

11.6.5. Tạo và Đăng ký Trình Xử Lý Sự Kiện

Trong JavaFX, trình xử lý sự kiện (*Event Handlers*) được định nghĩa là các hành động sẽ được thực thi sau khi một sự kiện đã xảy ra. Các trình xử lý sự kiện có thể được tạo và đăng ký để phản hồi các sự kiện phức tạp như sự kiện chuột, bàn phím,

cửa sổ, v.v. JavaFX cung cấp các phương thức tiện lợi để đăng ký trình xử lý sự kiện vào các thành phần giao diện.

Một số lớp trong JavaFX định nghĩa các thuộc tính trình xử lý sự kiện, cho phép lập trình viên đăng ký trình xử lý sự kiện một cách dễ dàng. Việc đặt giá trị cho thuộc tính trình xử lý sự kiện bằng một trình xử lý sự kiện do người dùng định nghĩa sẽ tự động đăng ký trình xử lý đó để nhận các loại sự kiện tương ứng. Các phương thức setter của các thuộc tính trình xử lý sự kiện này là phương thức tiện lợi để đăng ký trình xử lý sự kiện.

Ví dụ sau minh họa cách tạo và đăng ký một trình xử lý sự kiện cho sự kiện chuột:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.input.MouseEvent;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;
import javafx.event.EventHandler;

public class MouseEventExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button button = new Button("Hover Over Me");

        // Đăng ký trình xử lý sự kiện chuột
        button.setOnMouseEntered(new EventHandler<MouseEvent>() {
            @Override
            public void handle(MouseEvent event) {
                System.out.println("Mouse entered the button area.");
            }
        });

        StackPane root = new StackPane(button);
        Scene scene = new Scene(root, 300, 200);
        primaryStage.setTitle("Mouse Event Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
}
```

```

    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

Trong ví dụ này, trình xử lý sự kiện được đăng ký cho sự kiện `onMouseEntered` của `Button`. Khi con trỏ chuột di chuyển vào vùng của nút, trình xử lý sự kiện được thực thi và hiển thị thông báo "Mouse entered the button area." trên console. Phương thức `setOnMouseEntered()` là một phương thức tiện lợi để đăng ký trình xử lý sự kiện chuột.

11.6.6. Đăng ký Trình Xử Lý Sự Kiện Thông Qua Thuộc Tính

Như đã đề cập, một số lớp trong JavaFX cung cấp các thuộc tính trình xử lý sự kiện, cho phép đăng ký trình xử lý sự kiện thông qua các phương thức setter tương ứng. Việc sử dụng các phương thức setter giúp đơn giản hóa việc đăng ký trình xử lý và đảm bảo rằng chúng sẽ được kích hoạt khi sự kiện tương ứng xảy ra.

Dưới đây là ví dụ sử dụng thuộc tính trình xử lý sự kiện cho sự kiện nhấn chuột (`MouseEvent`):

```

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class ButtonClickExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button button = new Button("Click Me");

        // Đăng ký trình xử lý sự kiện cho sự kiện nhấn chuột
        button.setOnMouseClicked(new EventHandler<MouseEvent>() {
            @Override
            public void handle(MouseEvent event) {
                System.out.println("Button clicked!");
            }
        });
    }
}

```

```

        }
    });

    StackPane root = new StackPane(button);
    Scene scene = new Scene(root, 300, 200);
    primaryStage.setTitle("Button Click Example");
    primaryStage.setScene(scene);
    primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
}

```

Trong ví dụ này, trình xử lý sự kiện được đăng ký bằng cách sử dụng phương thức `setOnMouseClicked()`. Khi người dùng nhấn chuột vào nút, trình xử lý sẽ được kích hoạt và hiển thị thông báo "Button clicked!" trên console.

Lưu ý: Trong JavaFX, bạn có thể đăng ký nhiều trình xử lý sự kiện và bộ lọc cho cùng một *Node*. Điều này cho phép ứng dụng linh hoạt trong việc xử lý nhiều sự kiện hoặc các hành vi khác nhau của cùng một thành phần giao diện.

11.6.7. Event Filters trong JavaFX

Event Filters cho phép bạn xử lý sự kiện trong giai đoạn bắt sự kiện (*Event Capturing Phase*) của quá trình xử lý sự kiện. Giai đoạn bắt sự kiện là giai đoạn khi sự kiện di chuyển qua tất cả các *Node* trong chuỗi phân phối sự kiện (*Dispatch Chain*). Một *Node* trong chuỗi này có thể có một hoặc nhiều bộ lọc, hoặc không có bộ lọc nào, để xử lý sự kiện.

Event Filters xử lý các sự kiện như sự kiện chuột, sự kiện cuộn, sự kiện bàn phím, v.v., trong giai đoạn bắt sự kiện. Tuy nhiên, để cung cấp logic xử lý sự kiện, các *Event Filters* cần phải được đăng ký với *Node*. Nếu một *Node* không chứa bộ lọc sự kiện, sự kiện sẽ được chuyển trực tiếp tới *Node* mục tiêu. Ngược lại, một bộ lọc duy nhất có thể được sử dụng cho nhiều *Node* và nhiều loại sự kiện. Nói cách đơn giản, *Event Filters* cho phép *Node* cha cung cấp xử lý chung cho các *Node* con và cũng có thể chặn sự kiện để ngăn *Node* con xử lý sự kiện đó.

Thêm và Xóa *Event Filters*

Để thêm một bộ lọc sự kiện vào một *Node*, bạn cần đăng ký bộ lọc này bằng phương thức `addEventFilter()` của lớp *Node*. Ví dụ:

```
// Tạo trình xử lý sự kiện chuột
EventHandler<MouseEvent> eventHandler = new EventHandler<MouseEvent>() {
    @Override
    public void handle(MouseEvent e) {
        System.out.println("Hello World");
        circle.setFill(Color.DARKSLATEBLUE);
    }
};

// Thêm bộ lọc sự kiện
circle.addEventFilter(MouseEvent.MOUSE_CLICKED, eventHandler);
```

Tương tự, bạn có thể xóa một bộ lọc bằng phương thức `removeEventFilter()` như sau:

```
circle.removeEventFilter(MouseEvent.MOUSE_CLICKED, eventHandler);
```

Ví dụ về *Event Filters*

Dưới đây là ví dụ minh họa cách sử dụng *Event Filters* để xử lý sự kiện chuột trong JavaFX. Lưu mã này vào một tệp có tên `EventFiltersExample.java`.

```
import javafx.application.Application;
import javafx.event.EventHandler;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.input.MouseEvent;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.Text;
import javafx.stage.Stage;

public class EventFiltersExample extends Application {
```

```
@Override
public void start(Stage stage) {
    // Vẽ một hình tròn
    Circle circle = new Circle();

    // Thiết lập vị trí của hình tròn
    circle.setCenterX(300.0f);
    circle.setCenterY(135.0f);

    // Thiết lập bán kính của hình tròn
    circle.setRadius(25.0f);

    // Thiết lập màu sắc cho hình tròn
    circle.setFill(Color.BROWN);

    // Thiết lập độ rộng của viền hình tròn
    circle.setStrokeWidth(20);

    // Thiết lập văn bản
    Text text = new Text("Click on the circle to change its color");

    // Thiết lập font chữ cho văn bản
    text.setFont(Font.font(null, FontWeight.BOLD, 15));

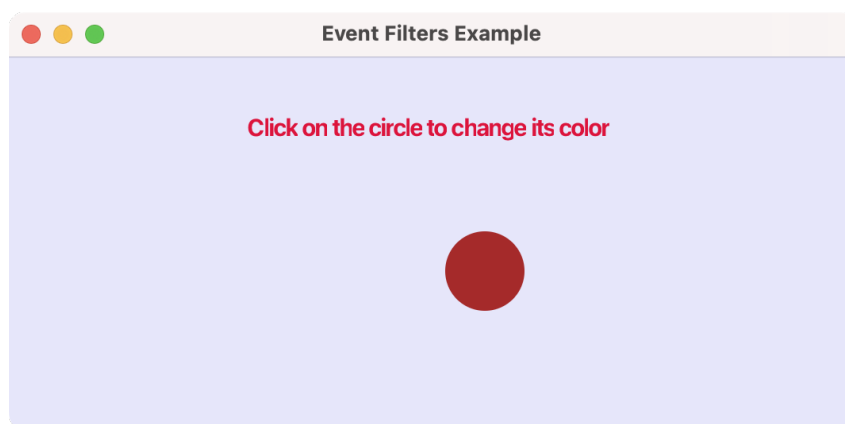
    // Thiết lập màu sắc cho văn bản
    text.setFill(Color.CRIMSON);

    // Thiết lập vị trí của văn bản
    text.setX(150);
    text.setY(50);

    // Tạo trình xử lý sự kiện chuột
    EventHandler<MouseEvent> eventHandler = new EventHandler<MouseEvent>() {
        @Override
        public void handle(MouseEvent e) {
            System.out.println("Hello World");
            circle.setFill(Color.DARKSLATEBLUE);
        }
    };
}
```

```
    }  
};  
  
// Đăng ký bộ lọc sự kiện  
circle.addEventFilter(MouseEvent.MOUSE_CLICKED, eventHandler);  
  
// Tạo đối tượng Group  
Group root = new Group(circle, text);  
  
// Tạo đối tượng Scene  
Scene scene = new Scene(root, 600, 300);  
  
// Thiết lập màu nền cho Scene  
scene.setFill(Color.LAVENDER);  
  
// Thiết lập tiêu đề cho Stage  
stage.setTitle("Event Filters Example");  
  
// Thêm Scene vào Stage  
stage.setScene(scene);  
  
// Hiển thị nội dung của Stage  
stage.show();  
}  
  
public static void main(String args[]){  
    launch(args);  
}  
}
```

Trong ví dụ này, chúng ta tạo một hình tròn và một văn bản mô tả (xem Hình 11.10). Khi người dùng nhấp chuột vào hình tròn, *Event Filter* sẽ xử lý sự kiện và in ra thông báo "Hello World" trên console, đồng thời thay đổi màu sắc của hình tròn thành `Color.DARKSLATEBLUE`. Bộ lọc sự kiện được đăng ký với hình tròn bằng cách sử dụng `addEventFilter()`.



Hình 11.10 Ví dụ Event Filter

11.6.8. Event Handlers trong JavaFX

Event Handlers cho phép bạn xử lý sự kiện trong giai đoạn lan truyền (*bubbling phase*) của quá trình xử lý sự kiện. Trong giai đoạn lan truyền, sự kiện di chuyển từ *Node* mục tiêu lên đến *Node* gốc (*Stage*). Tương tự như *Event Filters*, một *Node* có thể có một hoặc nhiều trình xử lý sự kiện, hoặc không có trình xử lý nào để xử lý sự kiện. Nếu *Node* không chứa trình xử lý sự kiện, sự kiện sẽ di chuyển tiếp đến *Node* gốc và quá trình hoàn tất. Ngược lại, nếu *Node* trong chuỗi phân phối sự kiện có trình xử lý, trình xử lý sẽ được thực thi.

Một trình xử lý có thể được sử dụng cho nhiều *Node* và nhiều loại sự kiện khác nhau. Nếu trình xử lý sự kiện của *Node* con không kết thúc (**consume**) sự kiện, trình xử lý sự kiện của *Node* cha cho phép *Node* cha hành động trên sự kiện sau khi *Node* con xử lý nó, và cung cấp xử lý sự kiện chung cho nhiều *Node* con.

Thêm và Xóa Trình Xử Lý Sự Kiện

Để thêm một trình xử lý sự kiện vào *Node*, bạn cần đăng ký trình xử lý này bằng phương thức `addEventHandler()` của lớp *Node*, như sau:

```
// Tạo trình xử lý sự kiện chuột
EventHandler<javafx.scene.input.MouseEvent> eventHandler =
    new EventHandler<javafx.scene.input.MouseEvent>() {
        @Override
        public void handle(javafx.scene.input.MouseEvent e) {
            System.out.println("Hello World");
            circle.setFill(Color.DARKSLATEBLUE);
        }
    }
```



```

    }
};
// Thêm trình xử lý sự kiện
circle.addEventHandler(
    javafx.scene.input.MouseEvent.MOUSE_CLICKED, eventHandler
);

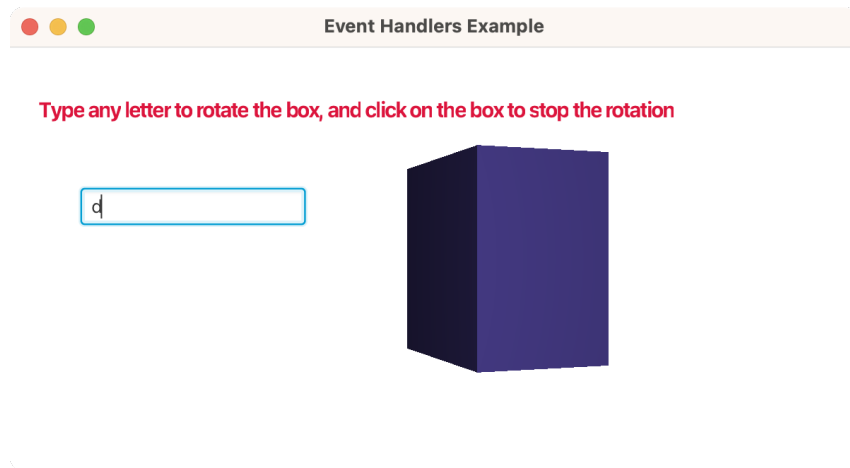
```

Tương tự, bạn có thể xóa một trình xử lý sự kiện bằng phương thức `removeEventHandler()` như sau:

```
circle.removeEventHandler(MouseEvent.MOUSE_CLICKED, eventHandler);
```

Ví dụ về *Event Handlers*

Dưới đây là một ví dụ minh họa cách sử dụng *Event Handlers* để xử lý sự kiện bàn phím và sự kiện chuột trong JavaFX (xem Hình 11.11). Lưu mã này vào một tệp có tên `EventHandlersExample.java`.



Hình 11.11 Ví dụ Event Handler

```

import javafx.animation.RotateTransition;
import javafx.application.Application;
import javafx.event.EventHandler;
import javafx.scene.Group;
import javafx.scene.PerspectiveCamera;
import javafx.scene.Scene;
import javafx.scene.control.TextField;

```

```
import javafx.scene.input.KeyEvent;
import javafx.scene.paint.Color;
import javafx.scene.paint.PhongMaterial;
import javafx.scene.shape.Box;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.Text;
import javafx.scene.transform.Rotate;
import javafx.stage.Stage;
import javafx.util.Duration;

public class EventHandlersExample extends Application {

    @Override
    public void start(Stage stage) {
        // Tạo một hình hộp
        Box box = new Box();

        // Thiết lập thuộc tính cho hình hộp
        box.setWidth(150.0);
        box.setHeight(150.0);
        box.setDepth(100.0);

        // Thiết lập vị trí của hình hộp
        box.setTranslateX(350);
        box.setTranslateY(150);
        box.setTranslateZ(50);

        // Thiết lập văn bản
        Text text = new Text(
            "Type any letter to rotate the box, " +
            "and click on the box to stop the rotation"
        );

        // Thiết lập font chữ cho văn bản
        text.setFont(Font.font(null, FontWeight.BOLD, 15));
```

```
// Thiết lập màu sắc cho văn bản
text.setFill(Color.CRIMSON);

// Thiết lập vị trí của văn bản
text.setX(20);
text.setY(50);

// Thiết lập vật liệu cho hình hộp
PhongMaterial material = new PhongMaterial();
material.setDiffuseColor(Color.DARKSLATEBLUE);
box.setMaterial(material);

// Thiết lập hoạt ảnh xoay cho hình hộp
RotateTransition rotateTransition = new RotateTransition();
rotateTransition.setDuration(Duration.millis(1000));
rotateTransition.setNode(box);
rotateTransition.setAxis(Rotate.Y_AXIS);
rotateTransition.setByAngle(360);
rotateTransition.setCycleCount(50);
rotateTransition.setAutoReverse(false);

// Tạo một TextField
TextField textField = new TextField();
textField.setLayoutX(50);
textField.setLayoutY(100);

// Xử lý sự kiện nhập phím
EventHandler<KeyEvent> eventHandlerTextField =
new EventHandler<KeyEvent>() {
    @Override
    public void handle(KeyEvent event) {
        // Phát hoạt ảnh
        rotateTransition.play();
    }
};

// Đăng ký trình xử lý sự kiện cho TextField
textField.addEventHandler(KeyEvent.KEY_TYPED, eventHandlerTextField);
```

```
// Xử lý sự kiện nhấn chuột trên hình hộp
EventHandler<javafx.scene.input.MouseEvent> eventHandlerBox =
    new EventHandler<javafx.scene.input.MouseEvent>() {
        @Override
        public void handle(javafx.scene.input.MouseEvent e) {
            rotateTransition.stop();
        }
    };
// Đăng ký trình xử lý sự kiện cho hình hộp
box.addEventHandler(
    javafx.scene.input.MouseEvent.MOUSE_CLICKED, eventHandlerBox
);

// Tạo một đối tượng Group
Group root = new Group(box, textField, text);

// Tạo đối tượng Scene
Scene scene = new Scene(root, 600, 300);

// Thiết lập camera
PerspectiveCamera camera = new PerspectiveCamera(false);
camera.setTranslateX(0);
camera.setTranslateY(0);
camera.setTranslateZ(0);
scene.setCamera(camera);

// Thiết lập tiêu đề cho Stage
stage.setTitle("Event Handlers Example");

// Thêm Scene vào Stage
stage.setScene(scene);

// Hiển thị nội dung của Stage
stage.show();
}
```

```
public static void main(String args[]){  
    launch(args);  
}  
}
```

Trong ví dụ này, chúng ta sử dụng *Event Handlers* để xử lý sự kiện nhập phím và sự kiện nhấn chuột. Khi người dùng nhập bất kỳ ký tự nào vào `TextField`, hình hộp sẽ xoay. Khi người dùng nhấn chuột vào hình hộp, hoạt ảnh xoay sẽ dừng lại.

11.7. Kéo thả giao diện với SceneBuilder

SceneBuilder là một công cụ thiết kế giao diện đồ họa trực quan cho các ứng dụng JavaFX. Công cụ này cho phép bạn tạo và quản lý các giao diện bằng cách sử dụng thao tác kéo-thả để thêm các thành phần vào giao diện và sắp xếp chúng theo mong muốn. SceneBuilder tạo ra các tệp định dạng **FXML**, là các tệp XML mô tả cấu trúc giao diện, giúp dễ dàng tích hợp với mã Java để xử lý logic ứng dụng.

11.7.1. Giao diện chính của SceneBuilder

Khi mở SceneBuilder, bạn sẽ thấy giao diện chính bao gồm các phần chính như sau:

- **Library Panel:** Bảng này hiển thị các thành phần giao diện có thể được thêm vào FXML. Các thành phần bao gồm các nút, nhãn, trường văn bản, hộp điều khiển, v.v.
- **Content Panel:** Khu vực trung tâm nơi bạn thiết kế giao diện bằng cách kéo-thả các thành phần từ **Library Panel**.
- **Inspector Panel:** Bảng này hiển thị các thuộc tính của thành phần được chọn, cho phép bạn tùy chỉnh kích thước, màu sắc, văn bản, sự kiện và nhiều thuộc tính khác.
- **Hierarchy Panel:** Hiển thị cấu trúc cây của các thành phần trong giao diện, cho phép bạn sắp xếp và quản lý các thành phần một cách dễ dàng.

11.7.2. Thiết kế giao diện bằng cách kéo-thả

Để tạo giao diện bằng cách sử dụng SceneBuilder, bạn có thể thực hiện các bước sau:

1. Tạo một tệp FXML mới hoặc mở một tệp FXML hiện có trong SceneBuilder.

2. Kéo-thả các thành phần từ **Library Panel** vào **Content Panel** để thêm chúng vào giao diện.
3. Chỉnh sửa thuộc tính của từng thành phần bằng cách sử dụng **Inspector Panel**. Ví dụ, bạn có thể thay đổi nhãn của một nút, thiết lập sự kiện nhấn nút hoặc điều chỉnh kích thước và vị trí của một trường văn bản.
4. Sắp xếp các thành phần trong **Hierarchy Panel** để tổ chức cấu trúc giao diện.
5. Lưu tệp FXML sau khi hoàn tất việc thiết kế.

11.7.3. Sử dụng tệp FXML trong JavaFX

Trong JavaFX, tệp FXML được sử dụng để tách biệt phần giao diện người dùng (UI) và logic ứng dụng. Tệp FXML là một tệp XML mô tả cấu trúc và các thành phần của giao diện đồ họa. Cách tiếp cận này giúp giảm thiểu sự phức tạp khi xây dựng giao diện, đồng thời cải thiện khả năng bảo trì và mở rộng ứng dụng. Để sử dụng tệp FXML trong ứng dụng JavaFX, bạn cần tải tệp này và liên kết nó với một lớp điều khiển.

Tải tệp FXML

Để tải tệp FXML vào ứng dụng JavaFX, bạn sử dụng lớp **FXMLLoader**. Phương thức `load()` của lớp này đọc tệp FXML và tạo ra cây *Scene Graph* từ tệp.

Ví dụ minh họa cách tải tệp FXML:

```
import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class FXMLExample extends Application {
    @Override
    public void start(Stage primaryStage) throws Exception {
        // Tải tệp FXML
        Parent root = FXMLLoader.load(getClass().getResource("sample.fxml"));

        // Tạo Scene từ cây giao diện
        Scene scene = new Scene(root);
```

```

        // Thiết lập tiêu đề và thêm Scene vào Stage
        primaryStage.setTitle("FXML Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

Trong ví dụ này:

- Lớp `FXMLLoader` được sử dụng để tải tệp `sample.fxml` bằng phương thức `load()`. Phương thức này tạo ra một đối tượng `Parent`, là gốc của cây *Scene Graph*.
- Tệp `FXML` cần phải nằm trong cùng thư mục với tệp mã nguồn hoặc được đặt trong một thư mục mà `getResource()` có thể tìm thấy.
- Sau khi tải tệp `FXML`, `Scene` được tạo từ đối tượng `Parent` và thêm vào `Stage`.

Liên kết FXML với lớp điều khiển

Khi sử dụng `FXML`, bạn có thể liên kết giao diện người dùng với một lớp điều khiển (*controller*) để xử lý logic ứng dụng. Lớp điều khiển là nơi chứa các phương thức xử lý sự kiện, và các thành phần giao diện được liên kết thông qua các thuộc tính `fx:id` trong tệp `FXML`.

Ví dụ:

```

<!-- File: sample.fxml -->
<VBox xmlns="http://javafx.com/javafx/8"
xmlns:fx="http://javafx.com/fxml/1" fx:controller="sample.Controller">
    <children>
        <Button fx:id="myButton" text="Click Me!"
            onAction="#handleButtonAction" />
    </children>
</VBox>

```

Trong ví dụ này:

- Thuộc tính `fx:controller` xác định lớp điều khiển liên kết với tệp FXML.
- Thuộc tính `fx:id` cho phép bạn liên kết các thành phần giao diện (ví dụ: `Button`) với các thuộc tính trong lớp điều khiển. Trong *SceneBuilder*, chọn một thành phần (ví dụ: `Button`) và thiết lập `fx:id` cho thành phần này (ví dụ: `myButton`). Sau đó, thiết lập các sự kiện cho thành phần, chẳng hạn như `onAction` để xử lý sự kiện nhấn nút.
- `onAction` xác định phương thức sẽ được gọi khi xảy ra sự kiện (ví dụ: nhấn nút).

Lớp điều khiển tương ứng:

```
// File: Controller.java
package sample;

import javafx.fxml.FXML;
import javafx.scene.control.Button;

public class Controller {
    @FXML
    private Button myButton;

    @FXML
    private void handleButtonAction() {
        System.out.println("Button clicked!");
    }
}
```

Giải thích:

- Chú thích `@FXML` được sử dụng để liên kết thuộc tính `myButton` trong lớp `Controller` với `Button` trong tệp FXML.
- Phương thức `handleButtonAction()` được gọi khi sự kiện `onAction` của `Button` được kích hoạt.

Sử dụng FXML trong JavaFX giúp tách biệt giao diện và logic, giúp cho việc phát triển ứng dụng trở nên rõ ràng và dễ bảo trì hơn. *SceneBuilder* là một công cụ mạnh

mẽ để tạo tệp FXML một cách trực quan, nhưng bạn vẫn có thể chỉnh sửa tệp này trực tiếp bằng tay khi cần thiết.

11.7.4. Ưu điểm của việc sử dụng SceneBuilder

Sử dụng SceneBuilder mang lại nhiều lợi ích, bao gồm:

- **Thiết kế giao diện nhanh chóng và dễ dàng:** Không cần phải viết mã bằng tay cho các thành phần giao diện cơ bản.
- **Tăng năng suất:** SceneBuilder cho phép bạn tập trung vào logic ứng dụng thay vì việc tạo giao diện bằng mã.
- **Trực quan và linh hoạt:** Giao diện trực quan giúp bạn dễ dàng tạo và chỉnh sửa giao diện người dùng.

11.8. Bài tập

Bài tập 11.1. Chạy lại các ví dụ cung cấp trong chương 5.

Bài tập 11.2. Viết ứng dụng JavaFX, sử dụng VBox. Bên trên là nút sayHello. Bên dưới là TextArea. Khi bấm vào nút sayHello, in dòng chữ Hello World trên TextArea.

Bài tập 11.3. Làm lại Bài tập 11.2, nhưng sử dụng Scene Builder.

11.9. Lời giải bài tập chương 11

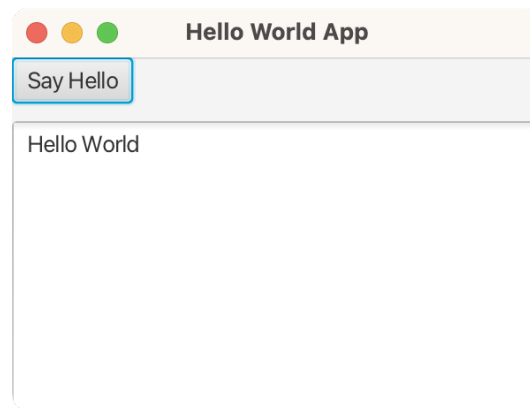
11.1 Lưu ý: đặt tên file cho chuẩn và cấu hình như mục 11.2.2.

11.2 Kết quả chương trình như Hình 11.12

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.TextArea;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;

public class HelloWorldApp extends Application {

    @Override
    public void start(Stage primaryStage) {
```



Hình 11.12 Kết quả chạy chương trình

```
// Tạo nút bấm
Button sayHelloButton = new Button("Say Hello");

// Tạo TextArea
TextArea textArea = new TextArea();
textArea.setEditable(false); // Chỉ cho phép đọc

// Xử lý sự kiện khi bấm nút
sayHelloButton.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        textArea.appendText("Hello World\n");
    }
});

// Sắp xếp các thành phần trong VBox
VBox vbox = new VBox(10); // Khoảng cách giữa các phần tử là 10 pixel
vbox.getChildren().addAll(sayHelloButton, textArea);

// Thiết lập scene và hiển thị stage
Scene scene = new Scene(vbox, 300, 200);
primaryStage.setTitle("Hello World App");
primaryStage.setScene(scene);
primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
}
```

Chương 12

Phân tích thiết kế hướng đối tượng và biểu đồ lớp

12.1. Phân tích thiết kế hướng đối tượng	288
12.2. Biểu đồ lớp (Class diagram)	296
12.3. Quan hệ giữa các lớp	299
12.4. Bài tập	306

12.1. Phân tích thiết kế hướng đối tượng

Nhiều người phát triển phần mềm vẫn cho rằng việc xây dựng một dự án phần mềm chủ yếu là quá trình viết mã trên bàn phím. Chính vì suy nghĩ này, họ thường không dành đủ thời gian cho các giai đoạn quan trọng như phân tích yêu cầu và thiết kế hệ thống.

Hậu quả là trong quá trình phát triển, nhóm dễ rơi vào tình trạng chạy theo việc hoàn thành chương trình mà không thực sự hiểu rõ yêu cầu của khách hàng, hoặc hiểu sai do giao tiếp nội bộ kém hiệu quả. Việc thiếu trao đổi rõ ràng giữa các thành viên cũng dẫn đến tình trạng không thể tích hợp các module do các lập trình viên khác thực hiện. Đến khi phát hiện ra các vấn đề này, họ mới nhận thấy tầm quan trọng của việc phân tích và thiết kế — nhưng lúc đó thì đã quá muộn.

Chính vì vậy, cần phải xây dựng một quy trình hiệu quả để thu thập và phân tích yêu cầu, cũng như thiết kế phần mềm một cách bài bản. Quy trình này nên được thể hiện dưới dạng một “ngôn ngữ thống nhất” – đóng vai trò như cầu nối giúp các thành viên trong nhóm phát triển phần mềm phối hợp với nhau dễ dàng và hiệu quả hơn trong suốt vòng đời dự án.

Mục đích của phân tích và thiết kế hướng đối tượng là chuyển đổi các yêu cầu của bài toán thực tế thành một bản thiết kế cụ thể cho hệ thống phần mềm sẽ được xây dựng. Giai đoạn này tập trung chủ yếu vào việc phân tích các yêu cầu của hệ thống và xây dựng các mô hình đại diện cho cấu trúc và hành vi của hệ thống, tất

cả đều được thực hiện trước khi bắt đầu lập trình.

Việc phân tích và thiết kế được tiến hành nhằm đảm bảo rằng các mục tiêu và yêu cầu của hệ thống đã được xác định và ghi nhận một cách chính xác, logic và đầy đủ. Điều này giúp giảm thiểu rủi ro phát sinh lỗi trong các giai đoạn sau và đảm bảo rằng sản phẩm cuối cùng sẽ đáp ứng đúng nhu cầu thực tế.

Ngoài ra, phân tích và thiết kế còn cung cấp nhiều góc nhìn khác nhau về hệ thống – phục vụ cho nhiều đối tượng liên quan như người dùng, khách hàng, kỹ sư phân tích và kỹ sư thiết kế. Nhờ đó, các bên có thể hiểu rõ hơn về hệ thống, hỗ trợ việc giao tiếp, đánh giá và điều chỉnh kịp thời trước khi đi vào giai đoạn phát triển chi tiết.

Phân tích và thiết kế hướng đối tượng (OOAD) bao gồm hai giai đoạn chính:

- **Phân tích hướng đối tượng (Object-Oriented Analysis – OOA):** Đây là giai đoạn tập trung vào việc hiểu rõ và mô hình hóa các yêu cầu của hệ thống dựa trên những gì khách hàng mong muốn. Mục tiêu là xây dựng các mô hình khái niệm, phản ánh các thực thể, mối quan hệ và hành vi trong thế giới thực mà hệ thống cần hỗ trợ. Giai đoạn này giúp xác định các lớp, đối tượng và các tương tác giữa chúng ở mức trừu tượng, không gắn liền với công nghệ hay chi tiết cài đặt.
- **Thiết kế hướng đối tượng (Object-Oriented Design – OOD):** Sau khi có được các mô hình khái niệm từ giai đoạn phân tích, giai đoạn thiết kế sẽ tiếp tục bổ sung các chi tiết kỹ thuật cần thiết để hiện thực hóa hệ thống. Điều này bao gồm việc xác định rõ cấu trúc lớp, giao diện, cơ chế tương tác, các thuật toán xử lý và kiến trúc tổng thể của hệ thống. Thiết kế hướng đối tượng đóng vai trò là cầu nối giữa mô hình khái niệm và quá trình lập trình, đảm bảo hệ thống được xây dựng đúng với yêu cầu ban đầu.

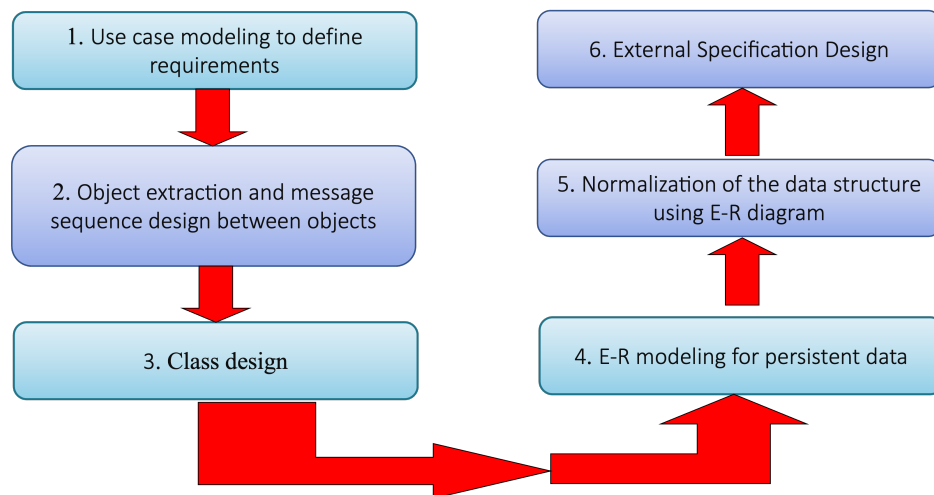
12.1.1. Phân tích hướng đối tượng (Object-Oriented Analysis – OOA):

Phân tích hướng đối tượng là giai đoạn quan trọng trong quy trình phát triển phần mềm, nhằm chuyển đổi các yêu cầu của bài toán thực tế thành bản thiết kế khái niệm cho hệ thống sẽ được xây dựng. Cụ thể, giai đoạn này tập trung vào:

- Mục tiêu của giai đoạn này là chuyển hóa các yêu cầu từ thế giới thực (bài toán) thành một bản thiết kế có cấu trúc rõ ràng, phản ánh cách thức mà hệ thống phần mềm sẽ được triển khai trong thực tế.
- Phân tích các yêu cầu của hệ thống một cách có hệ thống và logic, từ đó xác định rõ các thực thể, hành vi và mối quan hệ cần có trong hệ thống. Thiết kế

các mô hình khái niệm, đại diện cho cấu trúc và hành vi của hệ thống, trước khi bước vào giai đoạn hiện thực hóa bằng lập trình.

- Đảm bảo rằng các mục tiêu và yêu cầu của hệ thống được ghi nhận một cách đầy đủ, rõ ràng và hợp lý, từ đó giúp định hướng chính xác cho các bước tiếp theo trong quá trình phát triển phần mềm.
- Cung cấp nhiều góc nhìn khác nhau về cùng một hệ thống cho các đối tượng liên quan như người dùng, khách hàng, kỹ sư phân tích và kỹ sư thiết kế. Điều này giúp tăng cường sự hiểu biết chung và hỗ trợ hiệu quả cho việc trao đổi, đánh giá và điều chỉnh hệ thống trong suốt vòng đời phát triển.



Hình 12.1 Phương pháp OOAD.

Phân tích và đặc tả yêu cầu phần mềm: Quá trình phân tích yêu cầu phần mềm bắt đầu bằng việc xác định rõ các chức năng, tính năng và ràng buộc mà hệ thống cần phải đáp ứng. Đây là bước nền tảng để đảm bảo rằng phần mềm sẽ được phát triển đúng với mong đợi của người dùng và các bên liên quan.

Sau khi thu thập yêu cầu, các yêu cầu này sẽ được đặc tả thông qua mô hình hướng đối tượng, tức là sử dụng các đối tượng và mô tả sự tương tác giữa chúng để biểu diễn hành vi và cấu trúc của hệ thống.

Các mô hình được xây dựng cần có các thành phần phản ánh các đối tượng và khái niệm trong đời thực, sao cho chúng dễ hiểu đối với người dùng cuối hoặc khách hàng không có nền tảng kỹ thuật. Điều này giúp tăng khả năng giao tiếp và xác nhận yêu cầu giữa kỹ sư phần mềm và người dùng.

Đồng thời, quá trình mô hình hóa cần giữ nguyên cấu trúc, mối quan hệ và hành vi vốn có giữa các thực thể, nhằm đảm bảo tính chính xác và nhất quán trong việc chuyển đổi từ thế giới thực sang hệ thống phần mềm.

Ví dụ: Mô hình hóa yêu cầu phần mềm cho một phòng bán ô tô

1. Các thực thể (đối tượng trong hệ thống):

- Khách hàng: người đến mua ô tô, có thể tạo phiếu đặt hàng và thanh toán.
- Người bán hàng: nhân viên tư vấn và hỗ trợ khách hàng chọn mua xe.
- Xe ô tô: sản phẩm được trưng bày và bán.
- Phiếu đặt hàng: tài liệu ghi nhận thông tin về xe khách hàng chọn mua.
- Hóa đơn thanh toán: ghi nhận thông tin thanh toán khi khách hàng mua xe.

2. Các tương tác và quan hệ giữa các thực thể:

- Người bán hàng: dẫn khách hàng tham quan phòng trưng bày xe để giới thiệu sản phẩm.
- Khách hàng: chọn một chiếc xe ô tô mà họ quan tâm.
- Khách hàng: lập phiếu đặt hàng, ghi rõ thông tin xe được chọn và thông tin cá nhân.
- Khách hàng: thanh toán, hệ thống tạo ra hóa đơn thanh toán tương ứng.
- Xe ô tô: được giao đến tay khách hàng sau khi hoàn tất các thủ tục.

3. Mô hình hóa hướng đối tượng:

- Các đối tượng: Customer, Salesperson, Car, OrderForm, Invoice.
- Các quan hệ: Customer tương tác với Salesperson để chọn Car. Customer tạo OrderForm và sau đó nhận Invoice. Car được liên kết với OrderForm và Invoice. Sau khi thanh toán, Car được giao cho Customer.

4. Mục tiêu của mô hình:

- Phản ánh đúng luồng nghiệp vụ thực tế trong quy trình bán xe ô tô.
- Mô hình sử dụng các khái niệm đời thực, dễ hiểu với người dùng.
- Hỗ trợ việc trao đổi giữa kỹ sư phần mềm và khách hàng khi phân tích và thiết kế hệ thống.

12.1.2. Thiết kế hướng đối tượng (Object-Oriented Design – OOD)

Thiết kế hướng đối tượng là giai đoạn tiếp theo sau phân tích hướng đối tượng (OOA), với mục đích chính là chuyển hóa các mô hình khái niệm đã xây dựng trong OOA thành thiết kế kỹ thuật chi tiết, sẵn sàng cho việc lập trình và triển khai hệ thống. Cụ thể, mục tiêu của thiết kế hướng đối tượng bao gồm:

Hiện thực hóa các mô hình khái niệm từ bước phân tích hướng đối tượng (OOA), bằng cách xác định chi tiết các thành phần sẽ được triển khai trong phần mềm. Ánh xạ các khái niệm từ OOA thành các lớp thực thi cụ thể, trong đó bao gồm việc thiết kế:

- Các lớp (classes) với các thuộc tính (attributes) và phương thức (methods) tương ứng.
- Các ràng buộc và giao diện (interfaces) phục vụ tương tác giữa các thành phần của hệ thống.
- Đặc tả chi tiết hệ thống theo công nghệ cụ thể sẽ được sử dụng trong giai đoạn lập trình (ví dụ: Java, C++, .NET...).

Tổ chức chương trình thành các tập hợp đối tượng cộng tác với nhau, trong đó mỗi đối tượng là một thực thể (instance) của một lớp đã thiết kế. Dựa trên kết quả phân tích ban đầu, thiết kế hướng đối tượng sẽ:

- Cải thiện và tối ưu hóa mô hình.
- Xác định rõ mối quan hệ giữa các lớp, cách các đối tượng tương tác và trao đổi dữ liệu.
- Thiết kế các biểu đồ: Biểu đồ tĩnh (static diagrams): như biểu đồ lớp (Class Diagram) và biểu đồ đối tượng (Object Diagram), dùng để mô tả cấu trúc hệ thống. Biểu đồ động (dynamic diagrams): như biểu đồ trình tự (Sequence Diagram) và biểu đồ trạng thái (State Diagram), dùng để mô tả hành vi và tương tác giữa các đối tượng theo thời gian.

Thiết kế biểu đồ lớp (Class Diagram Design): Biểu đồ lớp là một trong những thành phần cốt lõi của thiết kế hướng đối tượng, dùng để mô tả cấu trúc tĩnh của hệ thống thông qua các lớp (class) và mối quan hệ giữa các lớp.

Mục tiêu của thiết kế biểu đồ lớp:

- Xác định các thành viên (thuộc tính và phương thức) của từng lớp trong hệ thống.

- Xác định các mối quan hệ giữa các lớp như: kế thừa, kết hợp, liên kết, phụ thuộc, v.v.

Một trong những kỹ thuật phổ biến và trực quan được sử dụng trong giai đoạn thiết kế là thẻ CRC (Class-Responsibility-Collaboration). Đây là phương pháp đơn giản nhưng hiệu quả để suy nghĩ và mô hình hóa các lớp trong hệ thống. Thẻ CRC – Class Responsibility Collaboration Card. Mỗi thẻ CRC đại diện cho một lớp trong hệ thống và bao gồm ba phần chính:

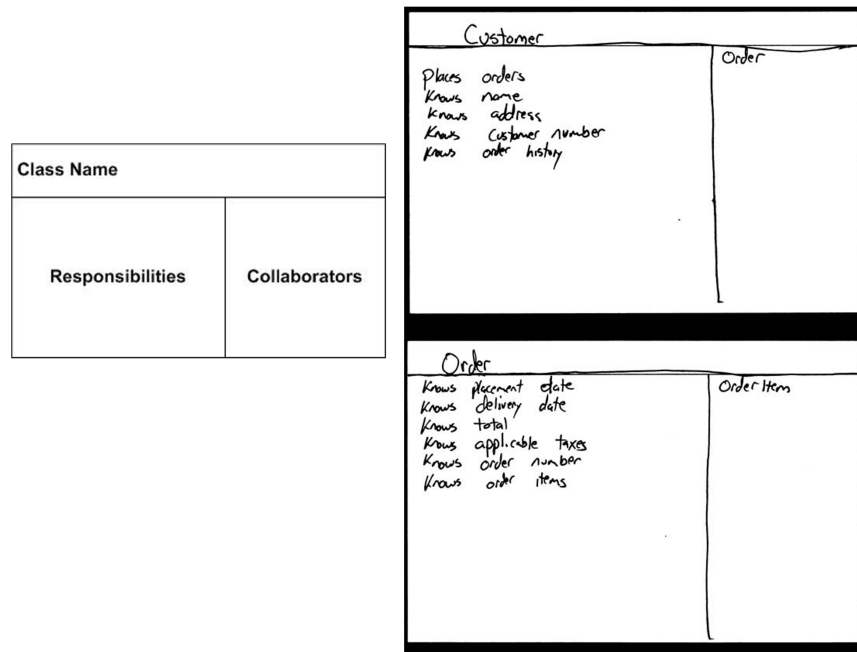
- Tên lớp (Class Name): Đặt tên theo vai trò hoặc chức năng chính mà lớp đảm nhiệm trong hệ thống. Tên lớp nên rõ ràng, dễ hiểu và phản ánh mục đích sử dụng.
- Trách nhiệm của lớp (Responsibilities): Liệt kê những hành vi hoặc chức năng chính mà lớp phải thực hiện. Thường được mô tả dưới dạng tên các phương thức (hàm thành phần) mà lớp cung cấp.
- Cộng tác (Collaboration): Xác định các lớp khác mà lớp này cần tương tác hoặc phụ thuộc để thực hiện trách nhiệm của mình. Điều này giúp làm rõ quan hệ và sự phối hợp giữa các lớp trong toàn hệ thống.

Ý nghĩa của CRC:

- Giúp nhà thiết kế hình dung và phân tích vai trò, trách nhiệm và mối liên hệ giữa các lớp một cách rõ ràng trước khi chuyển sang vẽ biểu đồ UML chính thức.
- Hỗ trợ làm việc nhóm hiệu quả, dễ dàng thảo luận và phân tách hệ thống thành các phần độc lập để phát triển.

Trong quá trình phân tích và thiết kế hệ thống theo phương pháp hướng đối tượng, các nhà phát triển phần mềm đã tổng kết ra 5 bước cơ bản để thiết kế đối tượng một cách hiệu quả. Các bước này phản ánh tiến trình từ việc nhận diện đến tổ chức, mở rộng và tái sử dụng các đối tượng trong hệ thống.

- Bước 1: Phát hiện đối tượng (Object Discovery) - Đây là bước đầu tiên, thường diễn ra trong giai đoạn phân tích hệ thống. Mục tiêu là xác định các đối tượng chính trong thế giới thực có liên quan đến bài toán, chẳng hạn như khách hàng, sản phẩm, đơn hàng, v.v. Việc phát hiện đối tượng giúp xây dựng các mô hình khái niệm ban đầu và làm nền tảng cho thiết kế hệ thống sau này.



Hình 12.2 CRC Card.

- Bước 2: Lắp ráp đối tượng (Object Assembly) - Sau khi xác định được các đối tượng, bước tiếp theo là xác định đặc điểm chi tiết của từng đối tượng bao gồm việc thêm các thuộc tính (attributes) và hàm thành phần (methods) vào mỗi đối tượng, để phản ánh đúng hành vi và trạng thái của đối tượng trong thực tế. Đây là giai đoạn tinh chỉnh mô hình đối tượng.
- Bước 3: Xây dựng hệ thống (System Construction) - Ở bước này, các đối tượng được phát triển và tích hợp lại với nhau để hình thành nên một hệ thống hoàn chỉnh. Cần xem xét các tương tác giữa các đối tượng, chẳng hạn như truyền dữ liệu, gọi phương thức, kế thừa hoặc phụ thuộc, để đảm bảo hệ thống hoạt động đúng chức năng.
- Bước 4: Mở rộng hệ thống (System Extension) - Trong quá trình phát triển phần mềm, thường xuyên có yêu cầu thêm tính năng mới. Bước này đề cập đến việc mở rộng hệ thống bằng cách bổ sung thêm lớp mới, đối tượng mới và thiết lập các mối quan hệ mới giữa các thành phần hiện có và mới thêm vào. Thiết kế hướng đối tượng hỗ trợ tốt việc mở rộng mà không làm ảnh hưởng đến phần cốt lõi của hệ thống.
- Bước 5: Tái sử dụng đối tượng (Object Reuse) - Đây là bước rất quan trọng trong thiết kế phần mềm hiện đại. Mục tiêu là tái sử dụng các lớp và đối tượng

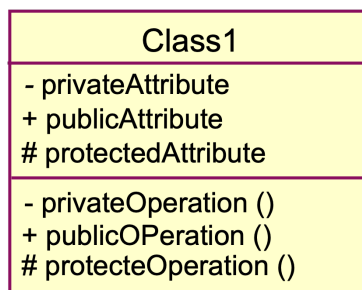
đã có, thông qua các cơ chế như kế thừa, đa hình, giao diện và mẫu thiết kế (design patterns). Tái sử dụng không chỉ giúp tiết kiệm thời gian và công sức phát triển, mà còn nâng cao tính nhất quán và độ tin cậy của hệ thống.

Một số lưu ý quan trọng khi phát triển các lớp trong thiết kế hướng đối tượng: Trong quá trình thiết kế và phát triển phần mềm hướng đối tượng, việc xây dựng và tổ chức các lớp (classes) là một nhiệm vụ cốt lõi. Dưới đây là một số nguyên tắc và kinh nghiệm thực tiễn cần lưu ý:

1. Tạo lớp trước, hoàn thiện sau: Trong giai đoạn đầu, nên bắt đầu bằng việc xác định các lớp chính cần có trong hệ thống. Việc phát triển và hoàn thiện chi tiết bên trong lớp (như thuộc tính, phương thức) có thể được thực hiện dần dần trong quá trình xây dựng hệ thống. Đây là cách tiếp cận linh hoạt và thực tế, giúp tránh việc thiết kế thừa hoặc sai hướng ngay từ đầu.
2. Không cần xác định toàn bộ thành viên của lớp ngay lập tức. Khi phân tích hoặc thiết kế lớp, không nên cố gắng xác định tất cả các thành phần (thuộc tính, phương thức) ngay từ đầu. Cấu trúc lớp sẽ được làm rõ dần trong quá trình phát triển phần mềm. Đây là phương pháp “learn as you go” – học và hoàn thiện dần trong quá trình thực hiện.
3. Phát hiện và sử dụng lại lớp là nhiệm vụ quan trọng
4. Việc phát hiện ra các lớp cần thiết là một trong những nhiệm vụ quan trọng nhất trong thiết kế hệ thống. Nếu hệ thống có thể tận dụng lại các lớp đã có sẵn (từ thư viện hoặc module đã phát triển trước đó), thì việc phát triển sẽ nhanh chóng, nhất quán và hiệu quả hơn.
5. Tuân thủ thiết kế khi lập trình. Khi bước vào giai đoạn lập trình, cần bám sát theo thiết kế đã được xác lập ở giai đoạn trước. Không nên hoang mang khi thấy khác biệt so với phương pháp lập trình truyền thống (như lập trình tuyến tính hoặc thủ tục). Mặc dù có thể phải làm việc với nhiều đối tượng và lớp khác nhau, việc tổ chức tốt sẽ giúp quản lý và phát triển hệ thống một cách rõ ràng và khoa học.
6. Đơn giản là tốt nhất. Luôn ghi nhớ nguyên tắc: giải pháp đơn giản nhất thường là giải pháp tốt nhất. Không phức tạp hóa thiết kế lớp hoặc hệ thống, đặc biệt ở giai đoạn ban đầu. Áp dụng nguyên lý này nếu có nhiều cách giải quyết, hãy chọn cách đơn giản và dễ hiểu nhất, bởi từ những phần tử đơn giản, chúng ta có thể xây dựng nên một hệ thống phức tạp nhưng ổn định và dễ bảo trì.

12.2. Biểu đồ lớp (Class diagram)

Biểu diễn lớp (Class) trong sơ đồ UML - Trong sơ đồ lớp của UML (Unified Modeling Language), mỗi lớp (class) được biểu diễn bằng một hình chữ nhật chia làm ba phần riêng biệt theo chiều dọc, từ trên xuống:



Hình 12.3 Biểu diễn lớp.

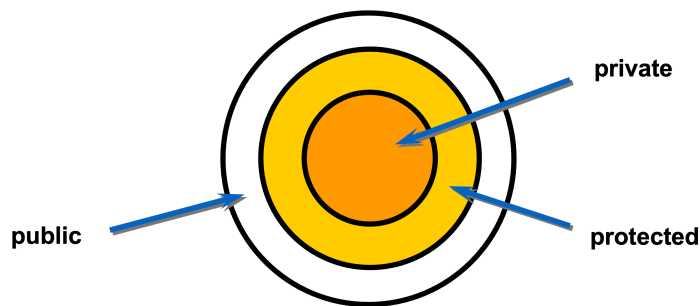
Tên lớp (Class Name): Nằm ở phần trên cùng của hình chữ nhật. Thể hiện tên của lớp, thường viết in đậm hoặc in nghiêng nếu là lớp trừu tượng. Tên lớp phải ngắn gọn, rõ ràng và thể hiện đúng vai trò hoặc đối tượng thực tế mà lớp mô tả. *Thuộc tính (Attributes):* Nằm ở phần giữa của hình chữ nhật. Mỗi thuộc tính thường được ghi theo định dạng: visibility name : type. Ví dụ: +name : String, -price : double. Ký hiệu quyền truy cập:

- + công khai (public)
- - riêng tư (private)
- # được bảo vệ (protected)

Phương thức (Methods): Nằm ở phần dưới cùng của hình chữ nhật. Mỗi phương thức cũng được mô tả theo định dạng: visibility methodName(parameterList) : returnType. Ví dụ: +getName() : String, +calculateTotal(price: double, tax: double) : double.

Phạm vi (Scope) của thuộc tính và thao tác trong lớp Khi thiết kế lớp trong mô hình hướng đối tượng, mỗi thuộc tính hoặc thao tác (phương thức) có thể có phạm vi (scope) khác nhau. Phạm vi giúp xác định số lượng thể hiện mà một thuộc tính hoặc thao tác sẽ tồn tại trong quá trình thực thi hệ thống.

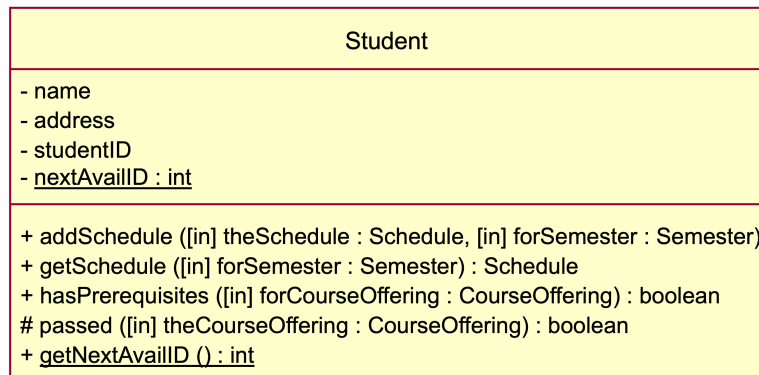
- Phạm vi thể hiện (Instance Scope): Mỗi thể hiện (đối tượng) của lớp sẽ có bản sao riêng của thuộc tính hoặc thao tác. Đây là kiểu mặc định trong hầu hết



Hình 12.4 Phạm vi truy cập.

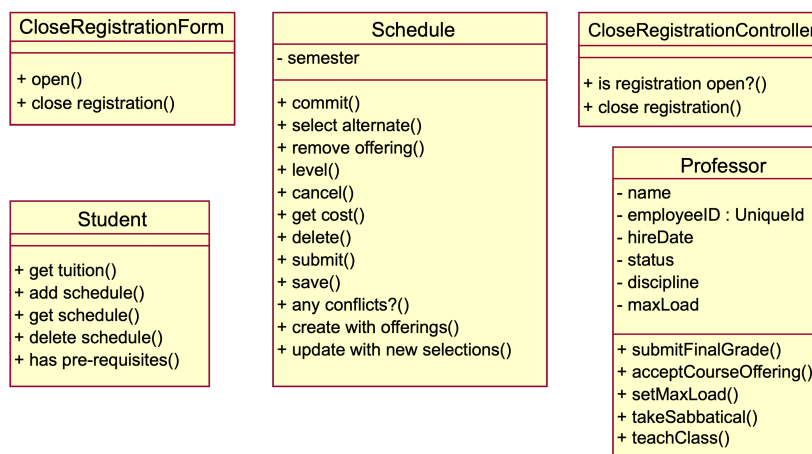
các hệ thống hướng đối tượng. Ví dụ: mỗi đối tượng Car sẽ có thuộc tính price và brand riêng.

- Phạm vi lớp (Classifier Scope): Thuộc tính hoặc thao tác chỉ có một thể hiện duy nhất cho toàn bộ lớp, được dùng chung bởi tất cả các đối tượng của lớp đó. Trong UML, phạm vi Classifier được biểu diễn bằng cách gạch dưới tên thuộc tính hoặc thao tác. Tương đương với thuộc tính tĩnh (static) trong các ngôn ngữ như Java, C++, C#. Ví dụ: một thuộc tính totalCars được khai báo dùng để đếm tổng số xe đã tạo – và được dùng chung cho tất cả đối tượng Car.



Hình 12.5 Phạm vi truy cập.

Biểu đồ lớp chỉ ra sự tồn tại của các lớp và mối quan hệ giữa chúng trong bản thiết kế logic của một hệ thống. Biểu đồ chỉ ra cấu trúc tĩnh của mô hình như lớp, cấu trúc bên trong của chúng và mối quan hệ với các lớp khác. Nó chỉ ra tất cả hoặc một phần cấu trúc lớp của một hệ thống. Tuy nhiên, không đưa ra các thông tin tạm thời. Đây là khung nhìn tĩnh của một hệ thống và chủ yếu hỗ trợ các yêu cầu chức năng của hệ thống.



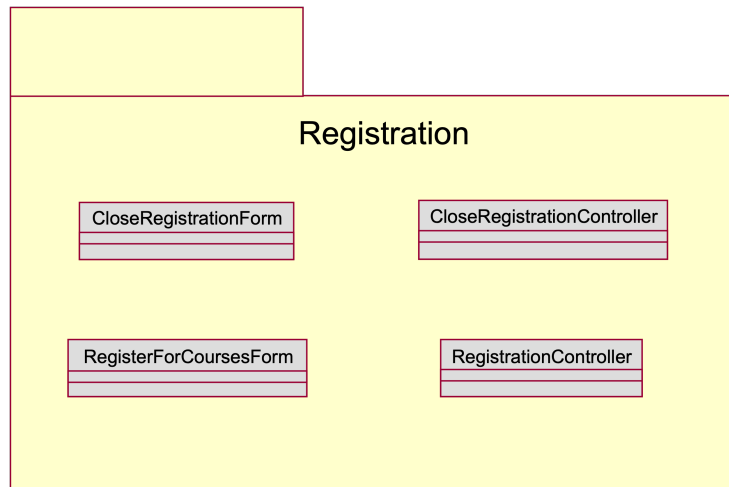
Hình 12.6 Phạm vi truy cập.

Biểu đồ lớp đóng vai trò quan trọng trong quá trình phân tích và thiết kế hệ thống hướng đối tượng. Việc sử dụng biểu đồ lớp mang lại nhiều lợi ích thiết thực, trong đó có ba lý do chính sau:

- **Xây dựng từ vựng của hệ thống (System Vocabulary):** Biểu đồ lớp giúp xác định các khái niệm trừu tượng liên quan đến hệ thống, đặc biệt là các thành phần ở biên, bên ngoài hoặc trong từng phần của hệ thống. Đây là cách thể hiện kết quả của quá trình trừu tượng hóa, thông qua việc mô tả các lớp và trách nhiệm (responsibilities) mà chúng đảm nhận. Từ đó hình thành nên một từ vựng chung giữa các nhà phát triển, nhà phân tích và khách hàng, giúp giao tiếp hiệu quả và nhất quán.
- **Mô tả cộng tác (Collaboration):** Biểu đồ lớp được dùng để nhóm các lớp và thành phần có liên quan, cho thấy cách chúng phối hợp với nhau để hoàn thành một nhiệm vụ cụ thể. Điều này rất hữu ích trong việc thiết kế quy trình nghiệp vụ, xác định vai trò và mối liên hệ giữa các đối tượng, giúp phát triển hệ thống có cấu trúc rõ ràng, dễ mở rộng và bảo trì.
- **Là cơ sở cho lược đồ cơ sở dữ liệu logic (Logical Database Schema)** Biểu đồ lớp cũng thường được sử dụng như một bản thiết kế khái niệm cho cơ sở dữ liệu (CSDL). Các lớp trong biểu đồ đại diện cho các thực thể (entities) mà hệ thống cần quản lý và lưu trữ lâu dài trong CSDL. Các thuộc tính của lớp tương ứng với các cột trong bảng dữ liệu, và các mối quan hệ giữa lớp phản ánh ràng buộc và liên kết giữa các bảng.

Gói (Package) là một cơ chế tổ chức quan trọng trong mô hình hướng đối

tượng, được sử dụng để nhóm các phần tử có liên quan lại với nhau thành một khối logic. Một phần tử trong mô hình có thể chứa các phần tử khác.



Hình 12.7 Registration Package.

12.3. Quan hệ giữa các lớp

Quan hệ liên kết (association): Là mối quan hệ ngữ nghĩa giữa hai hay nhiều lớp chỉ ra sự liên kết giữa các thể hiện của chúng. Mối quan hệ về mặt cấu trúc chỉ ra các đối tượng của lớp này có kết nối với các đối tượng của lớp khác.

Bội số quan hệ (Multiplicity) - Bội số quan hệ là số lượng thể hiện của một lớp liên quan tới một thể hiện của lớp khác. Với mỗi liên kết, có hai bội số quan hệ cho hai đầu của liên kết. Ví dụ: với mỗi đối tượng Professor có nhiều Course Offerings có thể được dạy. Với mỗi đối tượng Course Offering, có thể có 1 hoặc 0 Professor giảng dạy. Bội số rất quan trọng để thiết kế chính xác cấu trúc hệ thống, đặc biệt khi ánh xạ sang CSDL hoặc cài đặt mã nguồn.

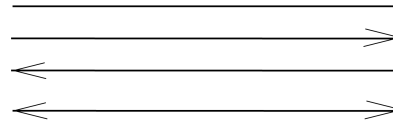
Bội số quan hệ giúp trả lời 2 câu hỏi: Liên kết là bắt buộc hay tùy chọn? Số lượng nhỏ nhất và lớn nhất các thể hiện của một lớp được liên kết với một thể hiện của lớp khác?

Bài tập về bội số quan hệ: Liên kết là bắt buộc hay tùy chọn? Số lượng nhỏ nhất và lớn nhất các thể hiện của một lớp được liên kết với một thể hiện của lớp khác.

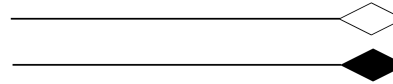
Cài đặt thực tế trên Java

```
//InsuranceCompany.java file
public class InsuranceCompany
```

- Association

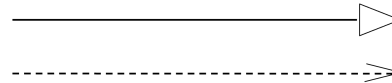


- Aggregation
 - Composition

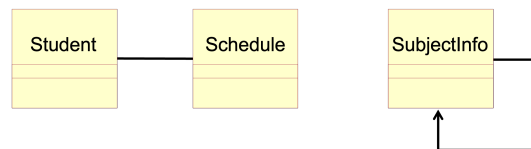


- Inheritance

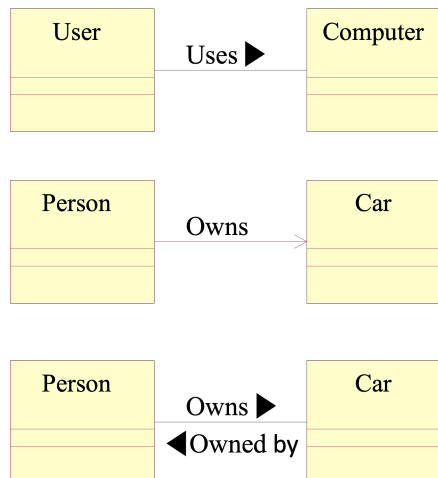
- Dependency



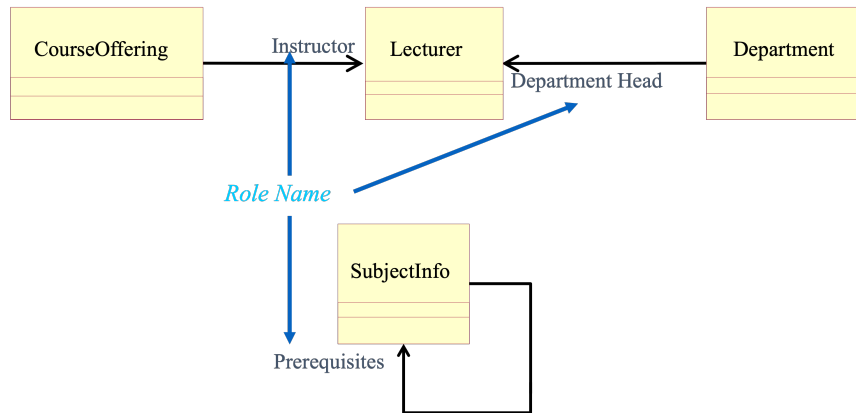
Hình 12.8 Quan hệ giữa các lớp.



Hình 12.9 Quan hệ liên kết.



Hình 12.10 Tên và hướng của liên kết.



Hình 12.11 Vai trò trong liên kết.

Unspecified	
Exactly One	1
Zero or More	0..*
Zero or More	*
One or More	1..*
Zero or One (optional value)	0..1
Specified Range	2..4
Multiple, Disjoint Ranges	2, 4..6

Hình 12.12 Biểu diễn bội số quan hệ.



Hình 12.13 Bội số quan hệ.



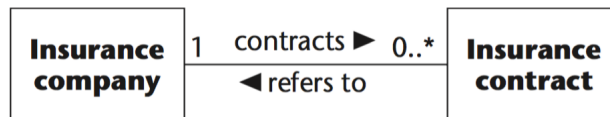
Hình 12.14 Ví dụ về bội số quan hệ.

```

{
    // Many multiplicity can be implemented using Collection
    private List<InsuranceContract> contracts;

    /* Methods */
}
// InsuranceContract.java file
public class InsuranceContract
{
    private InsuranceCompany refers_to;

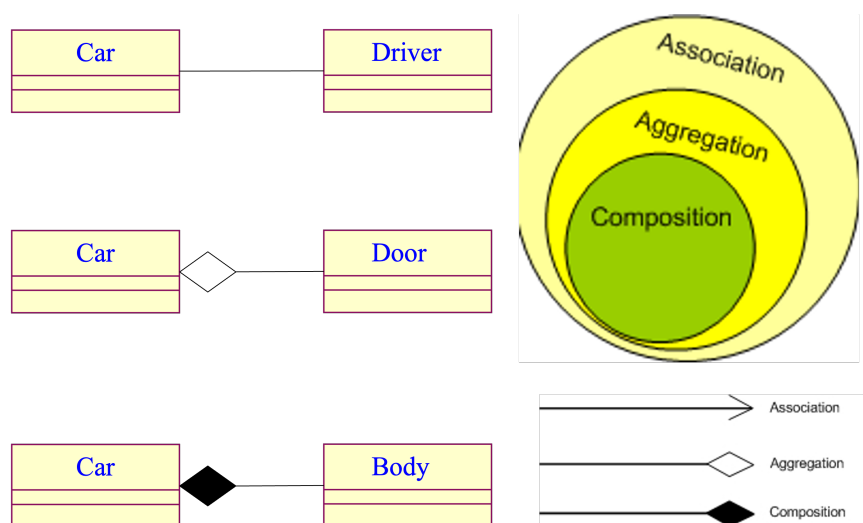
    /* Methods */
}
  
```



Hình 12.15 Cài đặt trên Java.

Các loại liên kết bao gồm:

- *Liên kết - Association (use-a)*: Các đối tượng của một lớp liên kết với các đối tượng của lớp khác
- *Kết tập - Aggregation (has-a/is-a-part)*: Liên kết mạnh-Strong association. Thể hiện của một lớp được tạo bởi (made up) các thể hiện của lớp khác
- *Hợp thành - Composition*: Kết tập mạnh-Strong aggregation. Đối tượng bộ phận không thể được chia sẻ, và cùng bị hủy với đối tượng tổng thể (Share life-time)



Hình 12.16 Các mối quan hệ.

Kết tập (aggregation): Là một dạng đặc biệt của liên kết mô hình hóa mối quan hệ toàn thể - bộ phận (whole-part) giữa đối tượng toàn thể và các bộ phận của nó. Kết tập là mối quan hệ "là một phần" (is a part-of). Bộ số quan hệ được biểu diễn giống như các liên kết khác.

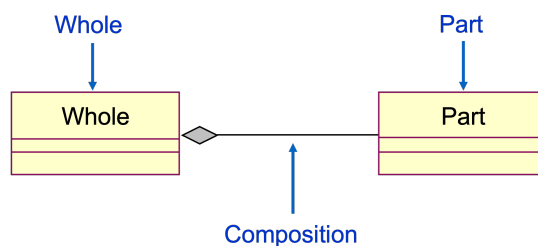
Ví dụ cài đặt thực tế trên Java

```
class Car {
    private List<Door> doors;
    Car(String name, List<Door> doors) {
        this.doors = doors;
    }

    public List<Door> getDoors() {
        return doors;
    }
}
```

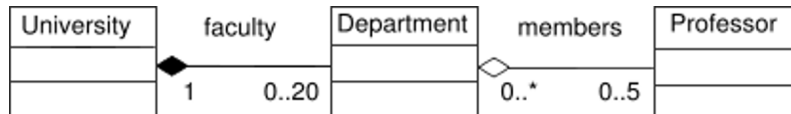
Hợp thành/Cấu thành (Composition): Là một dạng của kết tập với quyền sở hữu mạnh và các vòng đời trùng khớp giữa hai lớp. Trong đó, whole sở hữu part, tạo và hủy part; part bị bỏ đi khi whole bị bỏ, part không thể tồn tại nếu whole không tồn tại.

Ví dụ giữa kết tập và hợp thành:



Hình 12.17 Mối quan hệ hợp thành.

- University chứa (own) nhiều Department. Mỗi Department có 1 số các Professor
- Nếu hủy University: Các phòng ban cũng không còn tồn tại. Nhưng các Professor trong các Department vẫn còn tồn tại
- Dấu hiệu khác: 1 professor có thể làm trong nhiều Department; 1 Department chỉ thuộc về 1 University



Hình 12.18 Mối quan hệ kết tập và hợp thành.

Ví dụ về composition: Một thư mục có thể chứa nhiều tệp, trong khi một file được



Hình 12.19 Ví dụ composition.

đặt trong một thư mục. Nếu thư mục bị xóa, tất cả các file trong đó sẽ bị xóa theo. Bệnh viện có 1 hoặc nhiều phòng ban, mỗi phòng ban thuộc về chỉ một bệnh viện. Nếu bệnh viện đóng cửa thì tất cả các phòng ban cũng vậy.

Ví dụ về cài đặt trên Java của Composition:

```

final class Car {
    private final Engine engine;
    Car(EngineSpecs specs) {
        engine = new Engine(specs);
    }
}

```

```

    }
    void move() {
        engine.work();
    }
}

class Person {
    private final Brain brain;
    Person(Brain humanBrain) {
        brain = humanBrain;
    }
}
...
Brain b = new Brain();
// or we have an instance of Brain in other scopes
// not exactly in this scope
Person p1 = new Person(b);
Person p2 = new Person(b);

```

Cài đặt composition

```

public class House {
    private final Room room;
    public House() {
        room = new Room();
    }
}

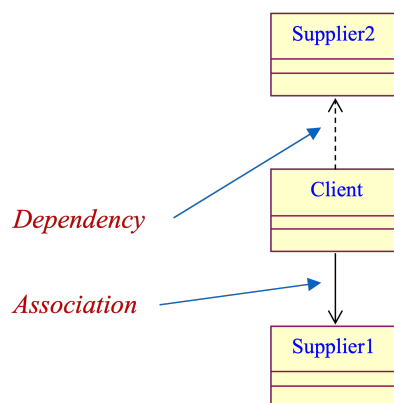
```

Mối quan hệ phụ thuộc: là quan hệ giữa hai đối tượng của hai lớp



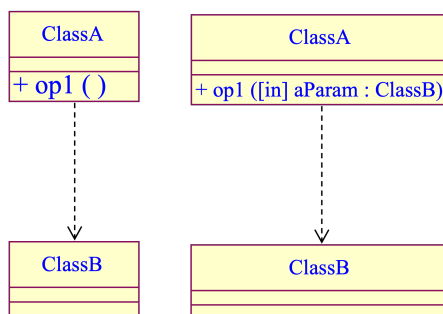
Hình 12.20 Quan hệ phụ thuộc.

Giữa phụ thuộc và liên kết, các đối tượng cần phải "biết nhau" để truyền thông điệp. Phụ thuộc: Local variable reference, Parameter reference, Global reference Liên kết: Field reference



Hình 12.21 Quan hệ phụ thuộc và liên kết.

Trong ví dụ bên dưới: Phương thức `op1()` tạo và sử dụng biến cục bộ tham chiếu tới đối tượng `ClassB`. Tham chiếu tới đối tượng lớp `ClassB` được truyền làm tham số trong phương thức `op1` của lớp `ClassA`.



Hình 12.22 Ví dụ quan hệ phụ thuộc.

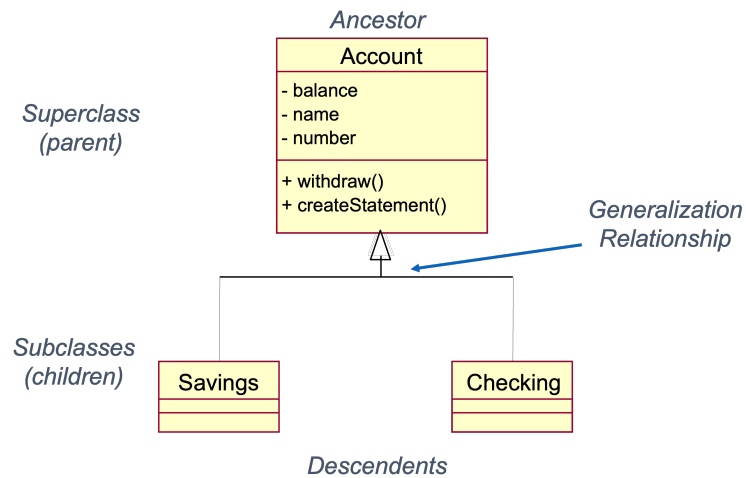
Tổng quát hóa (Generalization): Mỗi quan hệ giữa các lớp trong đó một lớp chia sẻ cấp trúc và/hoặc hành vi với một hoặc nhiều lớp khác. Được sử dụng để xác định sự phân cấp về mức độ trừu tượng hóa trong đó lớp con kế thừa từ một hoặc nhiều lớp cha. Có các hình thức: Đơn kế thừa (single inheritance) và đa kế thừa (multiple inheritance). Tổng quát hóa là mối liên hệ "là một loại" - is a kind of.

Đơn kế thừa: một lớp kế thừa từ một lớp khác:

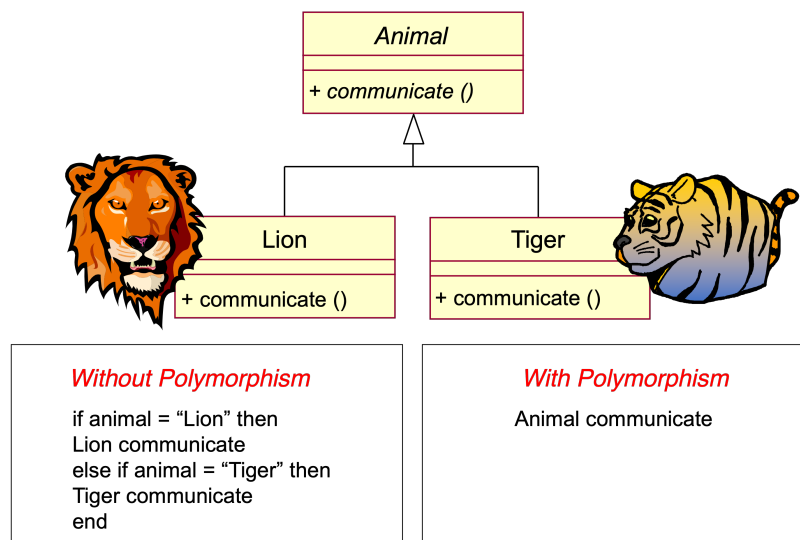
Đa hình (polymorphism) là khả năng che giấu các thực thi khác nhau dưới một giao diện duy nhất.

12.4. Bài tập

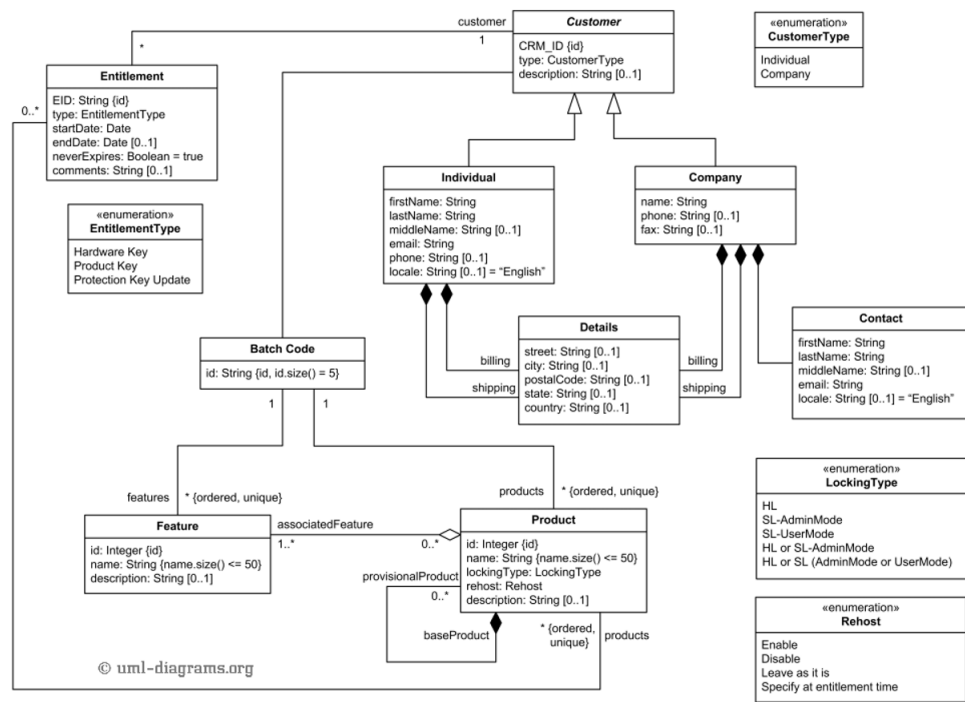
Phân tích biểu đồ lớp ở hình 12.25.



Hình 12.23 Đơn kế thừa.



Hình 12.24 Đa hình.



Hình 12.25 Bài tập biểu đồ lớp.

Tài liệu tham khảo

- [1] H. Rademacher, *Higher Mathematics from an Elementary point of view*, Birkhauser, 1983.
- [2] Martin Aigner and Gunter M. Ziegler, *Proofs from the book*, Springer, 1999.
- [3] T. Andreescu, R. Gelca, *Mathematical olympiad challenges*, Birkhauser, 2002.
- [4] Loren C. Larson, *Problem-Solving through problems*, Springer-Verlag, 1983.
- [5] L. E. Dickson, *New first course in the theory of equations* John Wiley & Sons, 1946.
- [6] Judita Cofman, *What to solve? Problems and Suggestions for Young Mathematicians*, Clarendon Press-Oxford, 1989.
- [7] Nguyễn Hữu Điển, *Phương pháp Dirichle và ứng dụng*, NXBKHK, 1999.
- [8] Nguyễn Hữu Điển, *Phương pháp Quy nạp toán học*, NXBGD, 2000.
- [9] Nguyễn Hữu Điển, *Những phương pháp điển hình trong giải toán phổ thông*, NXBGD, 2001.
- [10] Nguyễn Hữu Điển, *Những phương pháp giải bài toán cực trị trong hình học*, NXBKHK, 2001.
- [11] Nguyễn Hữu Điển, *Sáng tạo trong giải toán phổ thông*, NXBGD, 2002.
- [12] Nguyễn Hữu Điển, *Đa thức và ứng dụng*, NXBGD, 2003.
- [13] Nguyễn Hữu Điển, *Giải phương trình vô định nghiệm nguyên*, NXBDHQ, 2004.
- [14] Nguyễn Hữu Điển, *Giải toán bằng phương pháp đại lượng bất biến*, NXBGD, 2004.